

Groups as Diophantine Substrates

Arithmetic van Kampen certificates, three commutative phases, pair order, affine matrix inputs, a universal matrix semigroup and chronological matrix certificates

Low-degree certificates through the integral Heisenberg group and free matrix groups

AI-assisted research report prepared for Vladimir Reshetnikov from six manuscripts (batches 76, 78, 82 and 91 of ProveIt’s incoming reports), October 2–4, 2026

Abstract

Part I (source 03) treats a fixed two-generator finite presentation as a proof-producing substrate. Through Sanov’s arithmetic chart of a free subgroup of $\mathrm{SL}_2(\mathbb{Z})$, it builds explicit integer polynomials of degree at most four whose solvability is equivalent to filling a word with at most m conjugates of relators, with exactly $(2s + 13)m - 4$ witnesses for s relators, so that the compiler’s minimal allocated arity recovers filling area and the Dehn function exactly. Shared proof DAGs compile without expanding relator occurrences: for $[a^{2^k}, b^{2^\ell}]$ in $\langle a, b \mid [a, b] \rangle$ the area is $2^{k+\ell}$ but exactly $k + \ell$ product gates suffice and are necessary, and the quartic has $8(k + \ell) - 4$ witnesses.

Part II (source 05) compiles every finite system of integral quadratic equations into membership in an ordered product of three free abelian subgroups of a power of the discrete Heisenberg group, with an exact bijection between solutions and factorizations. With MRDP this gives three fixed subgroups whose product has computably enumerable complete membership, proposed as a negative answer to the three-subgroup question of König, Lohrey and Zetsche and of Roman’kov. Two commuting phases are decidable; for integer exponents this is a special case of known two-subgroup results, which the report credits.

Part III (source 06) proves that the pair counts of three-letter words leave no holes, and compiles exact quadratic and canonical quartic certificates of word realizability in three-generator Heisenberg submonoids. Part IV (source 07) proves that an affine unimodular input curve meets every fixed subgroup of $\mathrm{SL}_d(\mathbb{Z})$ in a finite or exactly periodic set, so that input degree two is sharp in every dimension, and compiles polynomial circuits into a fixed free-abelian unipotent subgroup when affine coordinates are existentially quantified.

Part V (source 08) gives 229 literal matrices in $\mathrm{SL}_4(\mathbb{Z})$ whose positive semigroup has the finite-tape halting problem of the Neary–Woods universal machine as its membership problem (r.e.-complete conditionally on the cited Neary–Woods chain), quartic certificates with $130r$ witnesses for each length r , and an exact generating function for the fibre of an accepted input.

Part VI (source 09) gives one fixed-arity polynomial of exact degree twelve (41,309 positive witnesses, 184,016 gates) recognizing, for histories of every length, the inputs accepted by a fixed-context matrix countdown of the research programme, whose semigroup descends from Part V’s.

The report is AI-assisted and unrefereed; nothing in it is formalized in Lean or Rocq, and priority is not certified for any Part.

Status. Proofs and executable finite checks; not kernel-checked. Part II’s main theorem is a proposed resolution of a published open problem. Section 1 states the common setup and what is not claimed; Appendix G gives the provenance.

Contents

1	Scope, common setup and conventions	11
1.1	The Parts	11
1.2	The integral Heisenberg group	13
1.3	Notions and letters	14
1.4	Relation to the repository and to the literature	18
1.5	Status and what is not claimed	23
1.6	Research questions	25
I	Arithmetic van Kampen certificates (source 03)	26
2	The problem addressed and the contribution boundary	27
2.1	What has been established here	27
2.2	Relationship to the repository	28
2.3	Conventions that prevent misleading conclusions	29
3	Finite presentations as proof-producing computational substrates	29
3.1	Why two generators do not remove undecidable examples	30
4	An exact quadratic arithmetic chart for the free group	30
4.1	What is compressed by this chart	31
4.2	Two indispensable hypotheses	31
5	A quartic compiler for a fixed relator itinerary	32
6	A uniform all-relator area-budget quartic	33
6.1	Sparsity, boundary typing, and presentation size	34
6.2	A complete one-cell example	34
7	A small-witness theorem and decidable area slices	35
7.1	Bit costs and the unary-budget qualification	36
8	The exact native arity profile and the cutoff obstruction	36
9	Shared relator-proof circuits	37
10	Quartic compilation of proof DAGs	39
10.1	Compression does not hide uncounted inverse histories	40
11	Sharp exponential compression for rectangular commutators	40

11.1	An elementary area detector	40
11.2	All boundary matrices are explicit	41
11.3	A twelve-variable area-four certificate	42
12	A more general addition-chain sandwich	42
13	Limits of compression and of witness-fiber claims	44
13.1	Sharing cannot give a computable complete proof budget	44
13.2	Semantic uniqueness is not single-fold Diophantineness	45
13.3	Where fixed-arity MRDP enters, and what it does not preserve	45
14	Executable evidence and reproducibility	45
14.1	Implemented components	46
14.2	The test ledger	46
14.3	Reproduction commands	47
14.4	What has not been tested or implemented	47
15	A formalization route into ProveIt	47
15.1	Free words and the arithmetic chart	48
15.2	Area witnesses and their semantics	48
15.3	Proof DAGs and their compilation	48
15.4	The external dependency ledger	48
16	Further research questions and proposed projects	49
17	Conclusion	50
II	Three commutative phases (source 05)	50
18	The question and the result	51
18.1	Why a new substrate is not enough	51
18.2	The published problem	52
18.3	A precise main statement	52
18.4	What is and is not obtained	52
19	Arithmetic normal forms	53
19.1	Integer binomials and polarization	53
19.2	A shared family of sparse linear forms	53
19.3	From general polynomials to quadratic systems	54

20 The three-phase compiler	54
20.1 Heisenberg coordinates	54
20.2 The three homomorphisms	55
20.3 Explicit finite generator lists	55
20.4 The exact product calculation	56
21 Embedding into a pure Heisenberg power	57
22 Universality and the three-subgroup problem	57
22.1 The imported arithmetic theorem	57
22.2 The fixed-instance argument	58
22.3 Why this is a resolution rather than an analogy	59
23 A matching two-phase decidability theorem	59
23.1 Integral logarithmic coordinates	59
23.2 The phase threshold	60
24 Nonnegative exponents and a computational interpretation	61
24.1 Free commutative monoids	61
24.2 Three-stage matrix reachability	61
25 Worked examples	62
25.1 A single multiplication gate	62
25.2 A circuit with repeated input use	62
25.3 Constants and impossible fibres	63
26 An explicit quartic Diophantine reverse compiler	63
27 Counting, weighted fibres, and height	64
27.1 Exact counting and the finite-fold boundary	64
27.2 Weights in the natural domain	64
27.3 Explicit witness heights	65
28 A small product invisible to every finite quotient	65
28.1 Four generators in $H \times \mathbb{Z}$	66
28.2 Interpretation and limits	67
29 Quantitative accounting and reproducible computation	67
29.1 Structural ledger	67
29.2 What the software implements	67
29.3 The verification receipt	68

29.4 Repository integration and trust boundary	68
30 Further research questions	69
30.1 Minimize the ambient group	69
30.2 Optimize the binomial form	69
30.3 Classify decidable triples	69
30.4 Machine-check the exact fibre theorem	69
30.5 Transfer counting without hidden multiplicity	70
30.6 Profinite closure of three-phase products	70
30.7 Other nilpotent substrates and higher degree	70
30.8 Quantitative hardness under restrictions	70
30.9 Stable versus exact acceptance	70
31 Conclusion	71
 III Pair order in class-two nilpotent words (source 06)	 71
32 The problem and the contribution	72
32.1 Why a word-realizability theorem is needed	72
32.2 Main results at a glance	72
32.3 Relation to the literature	73
33 Words, pair counts, and the binary base case	73
33.1 Conventions	73
33.2 Certificates and the meaning of uniqueness	74
34 An exact gap representation	74
34.1 Cutting at a distinguished letter	74
34.2 What has and has not been compressed	76
34.3 Nonemptiness of a moment fibre	76
35 The ternary no-holes theorem	76
35.1 A connected normalization graph	76
35.2 The theorem and exact endpoint optimization	77
35.3 Constructive recovery versus succinct decision	78
36 Two separators: four corners give the exact answer	78
36.1 Eliminating the gap equations	78
36.2 Worked example and the repository obstruction	79

36.3	Zero and one separator	80
36.4	Bit complexity	80
37	A canonical 24-witness quartic	80
37.1	A uniquely forced sign-split gadget	80
37.2	The construction as literal polynomial expressions	81
37.3	The exact ledger	81
37.4	Why more witnesses can be useful	83
37.5	Positive witnesses and signed arithmetic	83
37.6	The supplied polynomial export	83
38	Four letters: exact congruence holes	83
38.1	An infinite obstruction family	83
38.2	The smallest displayed hole and a convexity consequence	85
38.3	Why the ternary proof breaks	85
39	Transfer to Heisenberg computational substrates	85
39.1	The collection identity with every orientation paid	85
39.2	Projection to target-only membership	87
39.3	A matrix substrate recording complete pair profiles	87
40	Decidability with a supplied third multiplicity	87
41	Executable verification and reproducibility	89
41.1	Contents of the research package	89
41.2	Independent profile enumeration	90
41.3	Recorded exact checks	90
41.4	Code boundaries and error handling	91
42	Integration into ProveIt and further research	91
42.1	An incremental formalization route	91
42.2	Research questions with concrete next steps	92
42.3	A precise boundary of the contribution	94
IV	Affine matrix inputs (source 07)	94
43	The research question and the source boundary	95
43.1	What is being represented?	95
43.2	What was already in ProveIt?	95

43.3 Contribution and novelty status	96
44 Nilpotent normalization of an affine curve	97
45 An exact free basis and canonical integer coordinates	97
46 The all-dimensional finite-or-periodic theorem	99
47 The exact period formula	100
48 Sharp finite sets and diagnostic examples	102
48.1 The square-zero case	102
48.2 Dimension three: two points or two residue classes	102
48.3 Dimension four: a nonconsecutive sharp finite set	103
49 Explicit Diophantine certificates for the rigid languages	103
49.1 A two-dimensional lattice example	104
50 Why the classification is not a uniform decision algorithm	104
51 A commuting-polynomial extension	105
52 Input degree two is sharp, even with unrestricted dimension	106
52.1 The upper bound is prior repository work	106
52.2 Pointwise unipotence is not enough	109
53 Multiplication inside a jointly affine abelian pencil	109
53.1 A literal quartic certificate	110
54 A witness-preserving circuit compiler	110
54.1 The source format and the charged resources	110
54.2 A precise resource ledger	112
55 A completely worked circuit example	113
56 A decidable one-witness boundary	114
56.1 The quantifier boundary in one table	114
57 Effective normal forms when the lattice is supplied	115
58 Executable evidence and proof obligations	115
58.1 What the delivered program checks	115
58.2 What the tests do not establish	116

58.3	Reproduction	116
59	Further research questions and concrete continuation targets	117
60	Conclusion	119
V	A fixed universal matrix semigroup (source 08)	119
61	The two results and their exact interfaces	121
62	The fixed machine and the published dependency	122
62.1	The complete transition table	122
62.2	Exactly what universality supplies	123
63	A directed rewriting system with 93 rules	124
64	From rewriting to 114 fixed tiles	125
65	The numerical matrices and free embeddings	126
65.1	A faithful integer encoding	126
65.2	All generators and all target entries are numerical	127
66	The exact paired form of unrestricted semigroup words	127
67	The complete membership reduction	128
68	A literal quartic certificate for each numerical length	129
68.1	Every variable is allocated	129
68.2	Every residual is specified	129
69	Soundness completeness and the precise multiplicity theorem	130
69.1	Zero length and automatically excluded targets	131
69.2	Optional all-natural external inputs	131
70	Degrees coefficients and literal sparse size	131
70.1	Exact degree and fixed coefficient data	131
70.2	Monomial census with a declared convention	132
71	Resource ledgers with arithmetic and bit costs separated	132
71.1	Fixed matrices and finite-tape target encoding	132
71.2	Exact generic sparse evaluation charges	133
71.3	Certificate construction and value growth	134

71.4 Evaluating arbitrary assignments and constructing the family	134
72 Two concrete examples and what their certificates mean	135
72.1 An actual accepting finite tape	135
72.2 A length two target with two distinct roots	136
73 Exact factorization fibers and their asymptotic growth	136
73.1 Every accepted encoded word has an infinite unbounded fiber	136
73.2 An exact rational generating function for every halting live input	137
73.3 Support quasipolynomials and inversion	138
73.4 Exact growth for the released 94 tile example	139
74 Evidence architecture and portable reproduction	139
74.1 What has been checked and what each check establishes	139
74.2 Release contents and identity boundaries	141
74.3 Rebuilding and reading the artifacts	141
75 Consequences limitations and further questions	142
76 Conclusion	144
VI Chronological Diophantine matrix certificates (source 09)	144
77 The exact represented language	146
78 What is fixed and what was previously unpaid	147
78.1 Three separate levels of assertion	148
79 The inherited matrix interfaces and their exact use	149
79.1 Faithful alphabet and first-row injectivity	149
79.2 Context transfer and the lower marker	150
79.3 Synchronized rows recover the old product	150
79.4 The counted suffix and the direct input target	151
80 The explicit constructive exponentiation module	152
80.1 Positive leaves and complete equations	152
80.2 Why these clauses select the correct Pell index	153
80.3 Completeness with strictly positive leaves	153
81 Binary containment with complete extraction bounds	153

82 The signed radix and well-founded macro domains	154
82.1 No circular type reasoning	155
83 One common selector and paid whole-digit slices	156
84 Chronology with all three boundaries	157
85 Signed affine updates and a noncircular carry proof	157
86 Counter guards and the equivalence in both directions	158
86.1 Soundness of the positive polynomial zero	159
86.2 Completeness from a genuine accepting path	159
86.3 Boundary cases and nonclaims	159
87 The fully expanded source and exact resource ledger	160
87.1 Arithmetic gates	160
87.2 Module and witness ledger	161
87.3 Residual ledger	161
87.4 Exact total degree twelve	161
88 Independent verification and limits of the evidence	162
88.1 Complete symbolic reconstruction	162
88.2 Matrix and provenance reconstruction	163
88.3 Fresh finite challenges	163
89 Reproducibility and the immutable release	164
89.1 What a successful replay establishes	165
90 Future questions and present boundaries	165
A Part I: Scalar templates and an independent specification	167
B Part I: A compact theorem-dependency map	168
C Part I: Source and novelty audit	168
D Part II: A compact independent proof capsule	168
E Part II: Domain and claim audit	169
F Part II: Source audit and bibliography notes	170
G Provenance of this report	170

H Part III: Canonical witness names and a complete sample	175
I Part III: Repository provenance and dependency audit	176
J Part V: Deterministic specification of the literal matrix list	177
K Part V: The 94 tile accepting sequence	177
L Part VI: Notation and a complete outer-clause checklist	178
M Part VI: Principal source pins and dependency roles	178
References	180

1 Scope, common setup and conventions

[write] This section was written when the two manuscripts were merged. Part I and Part II print the two sources; text there without a [write] marker is the source’s own. [write 78] Parts III and IV were added on 2 October 2026 from two manuscripts of batch 78. Text written then is marked [write 78]; the sentences of this section that the new Parts made incomplete were extended in place, and Appendix G lists them. [write 82] Part V was added on 3 October 2026 from one manuscript of batch 82; text written then is marked [write 82], and the sentences of this section that it made incomplete were extended in place. [write 91] Part VI was added on 4 October 2026 from one manuscript of batch 91; text written then is marked [write 91], and the sentences of this section that it made incomplete were extended in place.

1.1 The Parts

This report is built from six AI-assisted research manuscripts (Appendix G gives the full provenance). Two were delivered together as batch 76 of ProveIt’s incoming reports (manuscripts 03 and 05) and are printed as Parts I and II. They are called *source 03* and *source 05* after the file prefixes of their shipped programs and data. Neither cites the other; they were written from different snapshots of the repository on the same day, and they share no theorem. [write 78] Two more, batch-78 manuscripts 09 and 13, are printed as Parts III and IV. They are called *source 06* and *source 07*, again after their file prefixes, which continue this report’s own sequence and are not batch-78 manuscript numbers. Both were written on 2 October 2026 from one later snapshot of the repository, before Parts I and II were written into this report; neither cites the other or an earlier source, and no two of the four share a theorem. All four are complementary work on one subject: *groups as Diophantine substrates*, with low-degree arithmetic certificates; the first three work through the integral Heisenberg group, and the fourth through unipotent matrix groups that contain it. [write 82] A fifth, batch-82 manuscript 22 (“Research Report 32” of the pipeline that delivered it), is printed as Part V and called *source 08*, again after its file prefix. It was written on 3 October 2026, cites no earlier source and no repository file, and works through free subgroups of $\mathrm{SL}_2(\mathbb{Z})$ placed in the two diagonal blocks of $\mathrm{SL}_4(\mathbb{Z})$. It shares no theorem with Parts I–IV, except that the first half of one lemma is the injectivity half of Part I’s Sanov theorem, proved differently. [write 91] A sixth, batch-91 manuscript 07 (“Research Report 55” of the same pipeline), is printed as Part VI and called *source 09*. It was written on 4 October 2026 against two research-programme commits of the previous evening, cites no earlier source, and works with a successor of Part V’s semigroup, a fixed semigroup of 195 singular integer 7×7 matrices; its subject is the Diophantine side, one polynomial of fixed arity for computations of every length. It shares no theorem with Parts I–V; its power module re-derives a lemma of another report of this category (Section 1.4).

- **Part I (source 03), *Arithmetic van Kampen certificates*.** The substrate is a finitely presented two-generator group; a word is an instance, and a product of conjugates of relators is an accepting proof. Sanov’s arithmetic chart of a free subgroup of $\mathrm{SL}_2(\mathbb{Z})$ turns each conjugator into four integers on one quadric, and the compilers are sums of squares of quadratic residuals (degree at most four) whose allocated arity records the filling area exactly (Theorems 5.1, 6.1 and 8.1). Shared proof DAGs compress the area exponentially, sharply for dyadic rectangular commutators (Theorem 11.2). Here the Heisenberg group is an *area detector*: a homomorphism from the free group whose central coordinate counts relator occurrences with sign (Section 11).
- **Part II (source 05), *Three commutative phases are Diophantine-universal*.** The

substrate is the ordered product $\mathcal{ABT}$ of three free abelian subgroups of a power H^d of the Heisenberg group. Every integral quadratic system is compiled into such a product with an exact bijection between solutions and factorizations (Theorem 20.2); with MRDP this gives three fixed subgroups with computably enumerable complete product membership (Theorem 18.1), proposed as a negative answer to the three-subgroup question of König, Lohrey and Zetsche [9] and Roman'kov [10]. Here the Heisenberg group is the *ambient group* of a membership problem.

- [write 78] **Part III (source 06), *Order is not a moment***. The substrate is a submonoid of H^r generated by three labelled elements, so a certificate must show that proposed letter and pair counts come from one common word. For three letters, fixing the multiplicities and two of the three scattered pair counts leaves a gapless interval of the third (Theorem 35.3). This gives a six-equation quadratic certificate with $2c + 4$ natural witnesses (Theorem 34.2), a four-corner formula for $c = 2$ (Theorem 36.2) and a canonical 24-witness quartic with exactly one witness tuple per accepted profile (Theorem 37.1). Four letters can leave arbitrarily large holes (Theorem 38.1). The profile certificates lift to three generators of H^r (Theorem 39.1), and membership is decidable when the multiplicity of one generator is supplied (Theorem 40.1). Here the Heisenberg group *records order*: its central coordinates are pair counts.
- [write 78] **Part IV (source 07), *Affine matrix inputs***. The substrate is membership of a matrix in a fixed subgroup of $SL_d(\mathbb{Z})$, and the question is how an ordinary integer input may be loaded. An entrywise affine unimodular curve meets every subgroup in a finite or exactly periodic set of inputs, in every dimension (Theorem 46.1), with an exact free basis (Theorem 45.2) and a sharp period (Theorem 47.4); with the repository's quadratic loader (Theorem 52.1) this makes input degree two sharp in every dimension (Corollary 52.2). With existentially quantified affine coordinates the obstruction disappears: a 6×6 multiplication gadget (Theorem 53.1) compiles every polynomial circuit into membership in a fixed free-abelian unipotent subgroup (Theorem 54.1). Here the Heisenberg group appears as the cyclic test subgroup $\langle h(1, 1, 0) \rangle$ (Section 1.2).
- [write 82] **Part V (source 08), *A fixed universal matrix semigroup***. The substrate is one fixed list of 229 literal matrices in $SL_4(\mathbb{Z})$ and the positive semigroup S they generate. A finite tape of the Neary–Woods machine $U_{15,2}$ is compiled into a target matrix, and the machine halts exactly when the target lies in S (Theorem 61.1); with the cited Neary–Woods chain this makes the membership problem of S many-one r.e.-complete. The proof passes through a directed rewriting system with 93 rules (Lemma 63.1), 114 correspondence tiles with a fresh separator (Lemma 64.1), a faithful encoding by conjugates of a Sanov shear (Lemma 65.1) and a free-group marker in the lower block that forces every positive word with the right lower block into paired form (Lemma 66.1). For each certificate length r a quartic sum of $17r + 4$ squares with $130r$ natural witnesses has roots in bijection with tile sequences (Theorem 61.2), and the factorization fibre of an accepted input has an exact rational generating function (Theorem 73.1). Here the matrices form a *semigroup*, not a group, and the free group is a *marker*.
- [write 91] **Part VI (source 09), *Chronological Diophantine matrix certificates***. The substrate is the research programme's five-register matrix countdown: two signed row vectors and a counter started at an ordinary positive input x , driven by one loader matrix and 96 pairs of tile matrices, whose acceptance is, under inherited interfaces, membership of a target $T(x)$ in a fixed semigroup of 195 singular integer 7×7 matrices. All time steps are packed in one radix, with one common selector per step, paid whole-digit slices, exact shift equalities

for the joins and a fixed coefficient bound that makes the signed updates carry-free; every power and every binary-containment mask is expanded through a fifteen-equation constructive Pell module. The result is one polynomial of exact degree twelve in x and 41,309 positive witnesses, with 184,016 gates, that has a positive zero exactly at the accepted inputs, whatever the length of the accepting history (Theorem 77.2). Here the matrices are a *fixture* with two fixed context words, not a universal alphabet, and the group-theoretic part is inherited.

Both sources use MRDP only as an imported classical theorem, at the last step, and both state explicitly that their constructions do not give finite-fold or single-fold Diophantine representations. [write 78] So does source 07 (Corollary 54.2); source 06 does not use MRDP. Both new sources state single-fold certificates only for restricted relations: source 06 over complete pair profiles, not over projected targets, and source 07 for languages already forced to be finite or periodic. Each Part keeps its own definitions, letters and numbering of statements; only the Heisenberg group itself is set up once, in Section 1.2. [write 82] Source 08 does not use MRDP: its universality comes from the Neary–Woods universal machine, and its Diophantine certificates form a family of polynomials, one for each certificate length, whose union over all lengths recovers membership. It states no fixed-arity, finite-fold or single-fold representation, and it proves that the family has infinitely many roots, over all lengths, on every accepted target. [write 91] Source 09 does not use MRDP either: it expands every exponential relation through mathlib’s constructive Pell theorems, and its polynomial is the only explicitly expanded one in this report with a fixed number of variables for computations of every length. It states no finite-fold, single-fold or real-witness representation (Section 86).

1.2 The integral Heisenberg group

Both sources use the same group with the same multiplication convention (source 03 in Section 11, source 05 in Sections 18 and 20). It is printed once here. Write

$$H = \text{UT}_3(\mathbb{Z}) = \left\{ h(a, b, c) = \begin{pmatrix} 1 & a & c \\ 0 & 1 & b \\ 0 & 0 & 1 \end{pmatrix} : a, b, c \in \mathbb{Z} \right\}.$$

Source 03 writes the element $h(x, y, z)$ as the triple $(x, y, z) \in \mathbb{Z}^3$ and calls the group H ; in this report that group is always written H , because Part I also uses the letter H for a height bound (Theorem 7.2). The two coordinate conventions coincide: $(x, y, z) = h(x, y, z)$, with no reordering, sign change or normalization. Direct matrix multiplication gives

$$h(a, b, c)h(a', b', c') = h(a + a', b + b', c + c' + ab'), \quad (1)$$

$$h(a, b, c)^{-1} = h(-a, -b, ab - c), \quad (2)$$

$$h(a, b, c)^r = h(ra, rb, rc + abb(r)) \quad (r \in \mathbb{Z}), \quad (3)$$

where $\mathbf{b}(r) = \binom{r}{2} = r(r-1)/2$ is the integer binomial of Part II (Section 19). In source 03’s notation, (1) reads $(x, y, z)(x', y', z') = (x + x', y + y', z + z' + xy')$ and (2) reads $(x, y, z)^{-1} = (-x, -y, -z + xy)$.

Associativity follows by expanding the third coordinate of a triple product (source 03). For nonnegative r , (3) follows by induction. For negative r , use (2) and the same formula, or verify that the proposed expression is a homomorphism of the exponent using (55) (source 05).

[write] Two consequences used by both Parts follow from (1) and (2): with $[g, k] = gkg^{-1}k^{-1}$,

$$[h(a, b, c), h(a', b', c')] = h(0, 0, ab' - a'b), \quad (4)$$

so every commutator is central, and the centre of H is $\{h(0, 0, c) : c \in \mathbb{Z}\}$. In a direct power H^d all formulas hold coordinatewise. (The identities (1)–(3) and (4) were rechecked symbolically at the write, with r from -4 to 4 in the power law.)

[write 78] Parts III and IV use the same group with the same convention. Source 06 writes the matrix $h(\alpha, \beta, \gamma)$ as $H(\alpha, \beta, \gamma)$ and the group as $\mathbb{H}$; this report writes $h(\alpha, \beta, \gamma)$ and H (Section 1.3). Source 07 works in $\mathrm{SL}_d(\mathbb{Z})$ and does not name the group, but its pencil $T(t, s) = I + tJ + sJ^2$ with $J = E_{12} + E_{23}$ is $h(t, t, s)$ and its generator $P = I + J$ is $h(1, 1, 0)$; by (3), $P^m = h(m, m, \mathbf{b}(m))$, which is its equation (145). Part II’s third phase uses the same binomial device in another cyclic subgroup: $h(-r, -r, \mathbf{b}(r)) = h(-1, -1, 0)^r$ in each coordinate of (66), whereas $h(1, 1, 0)^{-1} = h(-1, -1, 1)$, so $\langle h(1, 1, 0) \rangle \neq \langle h(-1, -1, 0) \rangle$ (Remark 53.2).

The two Parts use the group differently. Part I uses only the homomorphism $a \mapsto h(1, 0, 0)$, $b \mapsto h(0, 1, 0)$ from the free group and reads off the central coordinate (equation (44)). Part II uses the whole group: the three phase homomorphisms (64)–(66), and, for the two-phase theorem, the integral logarithmic coordinate $\lambda(h(a, b, c)) = 2c - ab$ of Section 23. [write 78] Part III uses three labelled elements of H^r and their words; its recording map (124) is Part I’s homomorphism applied to one pair of letters at a time, and its logarithmic map (125) has Part II’s coordinate λ as its third entry. Part IV uses only the cyclic subgroup $\langle h(1, 1, 0) \rangle$ of $\mathrm{SL}_3(\mathbb{Z})$. [write 82] Part V does not use H ; its free pair P, Q is Part I’s Sanov pair A, B of (10). [write 91] Part VI does not use H either; its lower marker P_0 and the matrix Q of Section 79 are Part V’s P, Q .

1.3 Notions and letters

Each Part keeps its source’s letters. Many letters carry different meanings in the two Parts; Table 1 lists every letter used in both, so that a statement of one Part is never read with the other Part’s meaning. Inside a Part, the meaning is always the one in that Part’s column. In Parts I and II the only renamed symbol is source 03’s H (the Heisenberg group), written H ; no normalization was changed. [write 78] Table 2 does the same for Parts III and IV, against each other and against Parts I and II. Two symbols of those sources are renamed, with no change of normalization: source 06’s Heisenberg element $H(\alpha, \beta, \gamma)$ and group $\mathbb{H}$ are written $h(\alpha, \beta, \gamma)$ and H , because H is a height bound in Parts I and II; and source 07’s interpolation coordinates $\lambda_i(t)$ and vector $\boldsymbol{\lambda}(t)$ are written $\theta_i(t)$ and $\boldsymbol{\theta}(t)$, because λ is Part II’s logarithmic coordinate $2c - ab$. [write 82] Table 3 does the same for Part V. No symbol of source 08 is renamed and no normalization changed; only its matrix

shorthand $\mathtt{mat}$ and its norm are printed with this report’s macros $\begin{pmatrix} \cdot & \cdot \\ \cdot & \cdot \end{pmatrix}$ and $\|\cdot\|_\infty$, the latter still

the maximum absolute row sum. [write 91] Table 5, at the start of Part VI so that no earlier table number moves, does the same for Part VI. Two symbols of source 09 are printed differently, with no change of normalization: its selector streams E_i are written Λ_i , because it also writes E_i for the lower-marker matrices and E_0 for the marker itself, and its digest macro is `\cms{sha}`. The source reuses some letters inside its own text, each within a stretch of sections: R is $\Psi(U)^{-1}$ in Section 79 and the repetition stream from Section 82 on; b is the base of a power in Sections 80 and 81 and the packing radix from Section 82 on; a is a Pell parameter in Section 80, a summation index in Section 81 and the initial row column in Section 82; H is the common offset, H_i a tile matrix and $H(s)$ a product. These are kept, as is the source’s own separation of C_0 from C_* and of D from D_c .

Table 1: Letters used in both Parts I and II.

Letter	Part I (source 03)	Part II (source 05)
H	the integral Heisenberg group (source: $H = \mathbb{Z}^3$)	the integral Heisenberg group $\text{UT}_3(\mathbb{Z})$
H	the height bound $H_{\mathcal{P}}(n, m)$ of (30), and H in its search cube	a height bound H on $\text{ht}(x)$ in Section 27
h	number of free conjugation nodes; an integer exponent in proofs	a Heisenberg element $h(a, b, c)$
A, B	the shear matrices $\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$ of (10); in software words, A, B mean a^{-1}, b^{-1}	the phase homomorphisms $A(x, u), B(y)$ of (64), (65) (and their small versions (82), (83))
T, $\mathcal{T}$	—	the third phase $T(r)$ (66) and its image $\mathcal{T}$; not a time bound
C	the selected relator matrix C_i (19); the constant $C = n + mL$ (30); circuit nodes $C_0, \dots, C_3$ and $C_t = [a^t, b]$	the normal-form matrix $C \in \mathbb{Z}^{m \times N}$ (60)
D	a proof DAG; a fixed conjugator $D \in \mathcal{S}$; the words $D_t = [a^u, b^t]$	the linear part $D \in \mathbb{Z}^{m \times n}$ (60)
E	the chart equation $E = x + t + 4xt - yz$ (11)	the reverse quartic E_t (77); $\mathcal{E}$ the set of mixed pairs
L	the maximal relator length	the $N \times n$ matrix of linear forms
N, M	witness and residual counts $N_{\text{it}}, N_{\mathcal{P}}(m), N_D, M_{\text{it}}, M_{\mathcal{P}}(m), M_D$	$N = n + q$ binomial coordinates; M a modulus (Section 28)
q	$q = 2s + 1$, the number of labels	$q = \mathcal{E} $; a formal variable in $Z_t(q)$
m	the area budget	the number of equations
n	the word length $ w $	the number of variables
s	the number of relators	an index of binomial coordinates
t	a chart coordinate; label words t_j	the target $t \in \mathbb{Z}^m$
r, u	relators r_j ; conjugators u_i ; exponents in $[a^u, b^v]$	phase parameters $r, u \in \mathbb{Z}^N$
p	the number of product nodes	generators p_i ; $p = h(1, 0, 0)$ in a proof
λ, ℓ	label indices λ_i ; chain lengths $\ell_+, \ell_{\pm}$; an exponent ℓ	the logarithmic coordinate λ ; coefficients ℓ_{ji} ; the weight $\ell(x)$
c, d, e	cost $c(D)$; $d = k + \ell$; selectors e_{ij}	constants c ; the dimension d of $\mathbb{H}^d$, coefficients d_{ji} ; coordinate vectors e_i and the rank e of $\mathbb{Z}^e$

Table 2: [write 78] Letters of Parts III and IV that are used differently elsewhere in the report.

Letter	Part III (source 06)	Part IV (source 07)	Parts I and II
H, h, H	$\mathbb{H}^r$ the direct power; $h(\alpha, \beta, \gamma)$ an element (source: $H(\alpha, \beta, \gamma)$ in $\mathbb{H}$)	H, H_0 finitely presented groups in Theorem 52.1	H height bounds; in Part I h is also a node count
λ, μ, ℓ	ℓ the doubled logarithmic map (125), with Part II's λ as third entry; $\lambda_j^{\pm}, \mu_j^{\pm}$ sign-split witnesses	$\theta_i, \boldsymbol{\theta}$ interpolation coordinates (source: $\lambda_i, \boldsymbol{\lambda}$); ℓ a linear functional	II: $\lambda = 2c - ab$; I: label indices

Letter	Part III (source 06)	Part IV (source 07)	Parts I and II
a, b, c	multiplicities n_A, n_B, n_C of the letters; c is also the number of separators. Part II's $2c - ab$ reads $2\gamma - \alpha\beta$ here, never $2n_C - n_A n_B$	free letters a, b and $a_i = b^{-i} a b^i$ (Theorem 52.1); $a_i(t) = k! \theta_i(t)$	II: coordinates of $h(a, b, c)$
k, p, q	pair counts K_{AB}, K_{AC}, K_{BC}	$k + 1$ the nilpotency index; p a prime; $q(t), q_i$ polynomials	I: $q = 2s + 1$; II: $q = \mathcal{E} $
A, B, C, D	letters of the alphabet; A_s, B_s, C_s target coordinates	the curve $C + nD$; $A = VBV^{-1}$; B a shear and a lattice basis; A_N the evaluation group; A_i, B_i affine operands; A_j coefficients; D also a degree	I: shears A, B , matrix C , DAG D ; II: phases A, B , matrices C, D
$B(\cdot)$	—	$B(t) = \binom{t}{2}$, Part II's $\mathfrak{b}(t)$	II: the phase $B(y)$
L, U, D	gap functionals (92); corner values L_i, U_i ; endpoints $\mathcal{L}_c, \mathcal{U}_c$	the curve $L(n)$ and pencil $\mathcal{L}$; $U(n) = I + nN$	I: relator length L ; II: matrices L, D
T	T_{ij} doubled pair counts	the pencil $T(t, s) = h(t, t, s)$; the hit set T	II: the third phase $T(r) = h(-r, -r, \mathfrak{b}(r))$
E	—	the shear $E(t)$; $E = D(d - 1)$; matrix units E_{ij}	I: chart equation; II: E_t
K	scattered pair counts K_{XY}	a subgroup (K, K_g, K_S, K', K_0)	—
P	the canonical quartic P	$P = I + J = h(1, 1, 0)$; the period $P_k(e)$	I: p product nodes
N	profile count $N(a, b, c)$	$N = C^{-1}D$, nilpotent	I: witness counts; II: $N = n + q$
r, d, g	r Heisenberg factors; d alphabet size; g, h, j or g_1, g_2, g_3 generators	$r = \text{rank } \Lambda$; d matrix size; g multiplication gates	II: r phase parameters, d the dimension of $\mathbf{H}^d$
Φ, Δ, e	potential Φ_A ; determinants Δ_s ; a parity remainder e	$\Phi : \mathbb{Z}^k \rightarrow A_N$; finite difference Δ ; quotient exponent e	II: e the rank of $\mathbb{Z}^e$
S	gap-index sum $ac - p$	a c.e. set	II: a c.e. set

Table 3: [write 82] Letters of Part V that are used differently elsewhere in the report.

Letter	Part V (source 08)	Parts I–IV
A_i, B_i, C	the 4×4 generators (V.10)–(V.12); $A, \dots, O$ are also the machine states	I: the Sanov shears A, B (Part V's P, Q); II: phases; III: letters; IV: the curve $C + nD$, $A = VBV^{-1}$
P, Q	the Sanov shears $\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$; P is also the lower block of C and of every target	I: these are A, B ; III: the quartic P ; IV: $P = h(1, 1, 0)$, the period $P_k(e)$
E_j, E	conjugates $E_j = Q^{-j} P Q^j$; the target map $E(\ell, \rho)$	I: the chart equation E ; II: E_t ; IV: the shear $E(t)$, matrix units E_{ij}
D	the upper block $\begin{pmatrix} 157 & 4 \\ -6712 & -171 \end{pmatrix}$ of C	I: a proof DAG; II: a linear part; IV: the curve $C + nD$

Letter	Part V (source 08)	Parts I–IV
$T, T_w, \mathcal{T}$	a target upper block; $\mathcal{T}(w)$ the target of a word	II: the third phase $T(r)$; IV: the pencil $T(t, s)$, a hit set
U, U_i, V_i	the machine $U = U_{15,2}$; the upper blocks $U_i = \Phi(h_i)$, $V_i = \Phi(g_i)$	I: the chart matrix $U(x, y, z, t)$; III: corner values U_i ; IV: $U(n) = I + nN$
H, H_r, G_r	H machine moves before halting; H_U the halting set; prefix products H_r, G_r	I, II: height bounds H ; IV: finitely presented groups H
S	the semigroup; S_s selection residuals	III: a gap-index sum; IV: a c.e. set
K	the kernel word K ; norm bounds K_H, K_G ; canonicity residuals K^H, K^G	III: pair counts; IV: a subgroup
r, ρ	r the certificate (inner) length; ρ the right tape half	II: phase parameters r ; III: the number of Heisenberg factors
t, x_i, h_i, g_i	lower free generators $t = E_0 = P$, $x_i = E_i$; target entries t_{ab} ; tile sides (h_i, g_i)	I: a chart coordinate t ; II: the target t ; h the Heisenberg element
Φ	the word encoding Φ	III: a potential; IV: $\Phi : \mathbb{Z}^k \rightarrow A_N$
B, d, L, A	in Section 73: the number B of erasure schedules, d genuine rewrites, L an lcm, A a weight product	see the rows above and Tables 1–2

Five words need the same care.

- **Phase** (Part II) means one factor of an ordered product of commuting subgroups or submonoids: a *commutative phase* is a family of pairwise commuting matrices that may be used in any order and number before the product moves to the next phase. It is not a complex phase, not a phase of the Fabius or Rvachev functions, and not a stage of the “four-phase zero test” in Part XIII of the report *canonical-diophantine-certificates*.
- **History-free** (the subtitle of Part I) means that the certificate stores no conjugator *word*: no word length, list of letters or matrix-multiplication history, only four chart integers per conjugator (Section 4). The tempting reading is that of Part XIV of *canonical-diophantine-certificates*, “history-free routing certificates”, where the certified object is an outcome (firing counts) rather than a time-ordered execution of a chip-routing network. The two senses are related, but they concern different histories.
- **Compiler** in both Parts means an explicit map from a group-theoretic or arithmetic problem instance to polynomials or matrices. It is unrelated to the sparse-machine “universal compiler” with arithmetic-operation counts of the repository’s Hilbert-tenth-problem research notes; neither Part claims or improves an arithmetic-operation record.
- **Slice**: Corollary 7.4 of Part I concerns a bounded-*area* slice $\text{Area}_{\mathcal{P}}(w) \leq m$; Corollary 20.3 of Part II concerns a *central* slice of the group $H^N \times \mathbb{Z}^m$.
- **Exponents**: in Part II, integer exponents make the phases subgroups, and natural exponents make them submonoids; matrix entries may be negative in both cases.

[write 78] Parts III and IV add six more.

- **Moment** (Part III’s title, “moment fibre”) means a gap-index sum $\sum_i (c - i)x_i$, the number of later separators summed over the occurrences of a letter. It is not a probabilistic moment, and not Part IV’s moment curve $(t, t^2, \dots, t^k)$ (Lemma 45.3).

- **Phase** does not occur in Parts III and IV. One step of a proof of source 06 is called “the second phase” there; it is printed as “the second stage” (proof of Lemma 35.2).
- **Pair count** in Part III is the central coordinate of Part I’s area detector on a positive word; it is not a filling area.
- **Single-fold**: Part III’s canonical quartic has exactly one witness tuple over each complete pair profile, not over each projected target (Section 39.2); Part IV’s certificates are single-fold over each accepted input of a language already forced to be finite or periodic (Theorem 49.1). Neither is a single-fold representation of c.e. sets.
- **Ordinary input and loader** (Part IV) are used as in the Hilbert-tenth-problem research notes: one integer n enters a fixed matrix curve $L(n)$, with no further quantified coordinates; an *existential* coordinate is a different interface (128).
- **Exactly periodic** (Part IV) means a union of residue classes on all of $\mathbb{Z}$, not eventually periodic; Spec_c (Part III) is a set of pair counts, not a spectrum of a ring or an operator.

[write 82] Part V adds three more.

- [write 82] **Universal** (Part V’s title) means one fixed list of matrices whose membership problem simulates a universal Turing machine; it is not a universal polynomial and carries no arithmetic-operation count, and the r.e.-completeness it names is conditional on the cited Neary–Woods chain.
- [write 82] **Certificate length r** (Part V) is the number of tiles; the polynomial F_r has $130r$ witnesses, so the family has no fixed arity.
- [write 82] **Fibre** (Part V) counts factorization words, equivalently tile sequences or canonical roots, of one fixed target, not distinct matrices; it is finite at each r and infinite over all r .

[write 91] Part VI adds three more.

- [write 91] **Chronological** (Part VI’s title) means that the certificate stores the states of one history in time order, with every join between consecutive steps enforced, not a multiset of locally valid steps.
- [write 91] **Fixed arity** (Part VI) means one polynomial with a fixed list of 41,309 witnesses for histories of every length, whereas Part V’s number of witnesses grows with the certificate length r . It does not mean a universal polynomial: the polynomial is for one fixture.
- [write 91] **Gates** (Part VI) are binary additions, subtractions and multiplications, with fixed numerals free; this is the convention of the research programme’s “operations”, so Part VI’s 184,016 gates and the programme’s 84-operation bound are counted the same way.

1.4 Relation to the repository and to the literature

Neither source cites the related material below, which was identified at placement; this subsection supplies the cross-references (pins and dates in Appendix G). [write 78] The second list concerns Parts III and IV.

- **Report *probabilistic-quantum-and-continuous-computation*, Part VI [13]** (sources 11 and 16 there; its Theorem `pqc:dr:thm:main`). It compiles finite group presentations into

rational rotation gates in $\text{SO}(4, \mathbb{Z}[1/5])$ by Mihailova’s fibre-product construction, obtains c.e.-complete membership for a fixed finitely generated subgroup, and certifies bounded *words* in that matrix group by quartic synthesis certificates. Part [I](#) has the same input (finite presentations and their relators) but a different substrate and resource: words of a free group in $\text{SL}_2(\mathbb{Z})$, and relator *area* and proof-DAG size. Part [II](#) has a statement of the same kind (a fixed c.e.-complete membership problem for fixed integer matrices) for a different object, a product of three abelian subgroups of a unitriangular group, by a different mechanism.

- **The research note `group_commutator_universal_substrate.md`** [\[14\]](#) of the Hilbert-tenth-programme. For every c.e. set it computes a finitely generated subgroup of $\text{SL}(4, \mathbb{Z})$ whose membership along a fixed quadratic matrix curve is that set, by commutator relators, Higman’s embedding theorem and Mihailova’s construction. This is the same kind of universality statement as Theorem [22.1](#), for subgroup membership in $\text{SL}(4, \mathbb{Z})$ rather than for a product of three abelian subgroups of H^d . Neither it nor Part VI of [\[13\]](#) measures relator area or compiles proof DAGs, which is what Part [I](#) does.
- **The research note `group_unipotent_input_loaders.md`** [\[15\]](#). It proves by ping-pong that $U = \begin{pmatrix} 1 & 4 \\ 0 & 1 \end{pmatrix}$ and $B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ form a free pair in $\text{SL}_2(\mathbb{Z})$ and uses it to load ordinary inputs into conjugates. Part [I](#) uses Sanov’s different pair [\(10\)](#) and, crucially, its exact arithmetic *image*, the integral points of one quadric (Theorem [4.1](#)); the note gives no description of the image of its pair, and Part [I](#) warns that the congruence description must not be assumed for other shear parameters (the end of Section [4](#)).
- **The research note `heisenberg_two_generator_membership.md`** [\[16\]](#). It decides membership in every submonoid of H^r generated by two elements, and in the commuting case it uses the injective coordinates $\ell(a, b, c) = (2a, 2b, 2c - ab)$, the same device as Part [II](#)’s $\lambda = 2c - ab$ (Section [23](#)). Its commuting case is the special case of Theorem [23.1](#) with one generator in each phase and natural exponents. The note ends: three or more generators are necessary for undecidable membership in this substrate, “this statement does not claim that three suffice”. *Part II does not answer that question.* The note concerns a submonoid generated by three elements, which contains arbitrary interleaved words; Part [II](#) concerns an ordered product of three subgroups or submonoids, each commutative and generated by many elements, with no interleaving across phases. The note’s primary source is a different paper of Roman’kov (arXiv:2209.14786, Skolem systems compiled into submonoids), not the paper [\[10\]](#) cited in Part [II](#). [\[write 78\]](#) (2 October 2026.) *Part III answers this open point in part.* It pays word realizability exactly for three generators: for a ternary alphabet the third pair count fills an interval (Theorem [35.3](#)), with exact quadratic and quartic certificates (Theorems [34.2](#) and [37.1](#)), and membership in a three-generator submonoid of H^r is decidable when the number of occurrences of one generator is supplied (Theorem [40.1](#)). It also derives the exact conditional spectrum behind the note’s ternary false family [\(105\)](#). The unrestricted three-generator question stays open; source 06 does not decide it (question 9 of Section [42.2](#)). Source 06 cites that paper of Roman’kov as [\[33\]](#).
- **The research note `group_projective_label_aligned_lanes.md`** [\[8\]](#) is source 03’s own comparison (Section [2](#)); it is unchanged from source 03’s pin to the write.
- **Report *canonical-diophantine-certificates***. Its compilers go from bounded executions to canonical arithmetic zeros; Part [I](#)’s native compiler is explicitly not canonical (Proposition [13.3](#)), and Part [II](#) compiles in the reverse direction, from arithmetic to a group, preserving fibres. Its

remark that a four-squares conversion “can introduce many witnesses for one natural value” (in its research questions) is the same caveat as Part II’s four-square paragraph in Section 22.

- **Bounded search.** Corollary 8.3, Theorem 13.2 and Proposition 27.1 use the argument “a computable bound would give a bounded search, hence a decision procedure”, as do Corollary `ppc:qm:cor:nobound` of [13] and Proposition `cdc:bd:prop:nobound` of *canonical-diophantine-certificates*. The statements are different (Dehn and sharing profiles, factorization exponents, mortal-word lengths, witness heights), so all are printed.

[write 78] Parts III and IV relate to the following material; sources 06 and 07 cite it only where said.

- **Part I’s area detector and Part II’s coordinate.** Part III’s recording map (124) is the homomorphism (44), one Heisenberg factor per pair of letters; on positive words its central coordinate is the scattered pair count, whereas Part I uses it on arbitrary words to bound van Kampen area. Part III’s map (125) is $(2a, 2b, \lambda)$ with Part II’s λ , and its correction (126) is (73) summed over pairs: the third independent derivation of that device in the repository, after the note [16] and Section 23. No theorem of Parts I and II is re-proved in Part III.
- **Parikh matrices** [17]. For an ordered binary alphabet the Parikh matrix is the factor (124); source 06 compares its profiles with 2-binomial equivalence and cites Parikh matrices only through [32] (note in Section 32.3).
- **The research note `group_affine_input_obstruction.md`** [35], which source 07 cites and generalizes: its paired- SL_2 classification (empty, a singleton or one coset) is the case $k = 1$ of Theorem 46.1, and its SL_3 example $\{1, 2\}$ is re-derived as a case $k = 2$. That note, the commutator note [14] and the programme’s README restrict the degree-two conclusion to that interface, or say that it does not extend to general SL_3 or SL_4 curves; after Part IV the restriction applies to the single-coset shape only, not to decidability or to the degree threshold (Corollary 52.2). Those notes belong to another part of the repository and are not edited here.
- **The commutator note** [14] (source 07’s [36]). Theorem 52.1 is its construction, reconstructed and attributed by source 07; what Part IV adds is the synthesis of Corollary 52.2.
- **The unipotent-loader note** [15]. Part IV’s curve $Q(n)$ (143) is the note’s swapped-basis curve $L_*(x)$ with $c = 12$, and Step 4 of Theorem 52.1 re-derives its free pair; source 07 does not cite the note.
- **The complete matrix-membership compiler** [18]: a fully charged positive Diophantine certificate for membership of $\mathrm{diag}(L_r, L_r)$ in a fixed subgroup of $\mathrm{SL}_2(\mathbb{Z}) \times \mathrm{SL}_2(\mathbb{Z})$ given by a supplied macro table. It does part of Part IV’s question 10 (note there).
- **The programme’s affine mortality notes** (for example `group_affine_bipartite_mortality5.md`). They load one affine input matrix as a generator of a matrix semigroup and ask for a zero product, an interface with singular matrices and no subgroup test; they are consistent with Part IV and do not overlap it.
- **Part VI of [13]** prints Mihailova’s generators (`ppc:dr:prop:mihailova`, `ppc:rr:lem:mihailova`); Step 3 of Theorem 52.1 prints them again, as a marked second route.
- **Part II and Part IV.** Theorem 22.1 shows that Part IV’s subgroup hypothesis is necessary, and that its rigidity fails for products of three subgroups even along a central affine

line (Remark 46.3); the two Parts share a binomial device in different cyclic subgroups (Remark 53.2).

- **The programme’s reviews of both archives**, written on the day of their arrival and before this placement: [19] for source 06 and [20] for source 07. They found no mathematical defect, replayed both programs, and prove two reductions printed as dated notes, a cheaper finalizer for Part III’s quartic (after Theorem 37.1) and one block fewer for each square gate of Part IV’s compiler (after Theorem 54.1).

[write 82] Part V relates to the following material, none of which source 08 cites.

- **Report *quadratic-orthant-certificates*** [21]. Its Parts III and V print the same $U_{15,2}$ table and the same erratum; Table 4 is the collection’s third printing (note after it), and source 08’s serialized table is that report’s data file, shipped there and not here. That report uses the machine for clocks, membranes and reset Petri nets and shares no theorem with Part V.
- **The research note *neary_woods_explicit_universal_tm.md*** [22] and **the notes *gpcp_complete_fixed_program.md* and *gpcp_fixed_program_input_bridge.md*** [23]. Lemmas 63.1 and 64.1 re-derive their rewriting and fresh-delimiter tile construction for the same machine (credit after Lemma 64.1). The note continues to a fixed-arity universal polynomial; Part V continues to a matrix semigroup.
- **Part I.** Part V’s free pair P, Q is Part I’s Sanov pair, and the first half of Lemma 65.1 is the injectivity half of Theorem 4.1, printed as a second proof (note after the lemma).
- **Part IV, Part VI of [13] and the commutator note** [14] construct fixed finitely generated *subgroups* of $SL_4(\mathbb{Z})$ or of $SO(4, \mathbb{Z}[1/5])$ with c.e.-complete membership; Part V’s object is a fixed positive *semigroup*. Neither kind of result implies the other. Part V answers Part IV’s question 10 only in part (note there).
- **The complete matrix-membership compiler** [18], whose numerical universal alphabet the programme’s README calls uninstantiated. Part V’s 229 matrices are a literal universal alphabet, but for a positive semigroup in the block form $\text{diag}(SL_2(\mathbb{Z}), \text{free group})$, not for that compiler’s inverse-closed interface $\text{diag}(L_r, L_r)$, and with bounded-length certificates rather than a fixed-arity polynomial; the programme’s review [24] says the same.
- **Report *stochastic-and-thermal-exactness***, Part I (its Theorem `ste:ee:thm:pcp`), reduces Post correspondence to integer matrix *mortality*, with the instance as input; Part V has one fixed semigroup and the target as input. They share the classical correspondence-to-matrices idea, not a theorem.
- **The programme’s review of the archive** [24] (commit 32da04296) and **its successor note** [25] (commit 0f7d618f4), both of 3 October 2026 and earlier than this write. The review read the archive as data, executed none of its programs, reconstructed the 93 rules, 114 tiles and 229 matrices, the coefficient census and the 189-factor accepting product, recounted the six exported polynomials and their evaluator charges, reported no discrepancy, and derived a 197-generator subset; the successor reduces the count to 193. Both are summarized after question 4 of Section 75. The shipped programs are the delivered ones; neither note patches them.
- **Reports *fixed-universal-polynomials***, Part I, and ***five-particle-binary-automata***, Part II (Reports 23 and 16 of the same pipeline, placed in the same batch): source 08’s

dependency audit cites them for the same finite $U_{15,2}$ input convention, Report 16 for its loader and machine table and Report 23 for an explicit ordinary-input frame. Part V uses nothing else from them, and the audit calls Report 23’s conditional theorem unnecessary for the input hardness used here.

- [write] Added 3 October 2026 (batch 83, reciprocal note). **Report *canonical-diophantine-certificates***, Part XX: the same fixed machine $U_{15,2}$ (Neary–Woods, Table 16), with the same imported program-to-tape encoder, drives a literal periodic sandpile loader there (its manuscript 22, the pipeline’s Report 35, Theorem `cdc:lp:thm:loader`); its delivered provenance record (`data/22-literal-sandpiles-evidence-loader-PROVENANCE.json` of that report) says the pinned transition data are “supplied from the prior matrix-semigroup construction”. Neither report implements the encoder, and no theorem is shared.

[write 91] Part VI relates to the following material; source 09 cites the research-programme notes of its pins and nothing else of the repository.

- **Part V and its successors.** Part VI’s semigroup descends from Part V’s through the research programme’s 197- and 193-generator reductions [24, 25] (note at Part V’s question 4), the faithful recoding in $\Gamma_1(5)$ [59], the absorption of two fixed contexts [57], the five-register countdown [55] and the counted suffix [58]. Part VI answers Part V’s question 7 in part, and supplies for one fixture the fixed-arity unbounded checker that Part IV’s question 10 asks for (notes at both questions).
- **The research programme’s uniform matrix compilers** [27], written on the night between the source’s pins and the archive’s arrival: the same ordinary-input histories, paid uniformly over valid program recipes, from 176,586 operations down to 1,393 operations at exact degree 71,105 (1,399 at degree 35,587). Neither they nor source 09 cite the other; the note in Section 78 compares them.
- **The programme’s review of the archive** [26] (commit a9ab9a698), written on the day it arrived and before this write: it confirms the DAG’s 184,016 gates, 41,309 witnesses, 23,618 squared residuals and exact degree 12 independently, reports no defect, finds no lower paid unbounded compiler in the archives it read, and leaves the 84-operation universal bound unchanged (note in Section 88).
- **Report *periodic-turmite-first-revisits*** [28]: the POWER module of Section 80 is its Lemma `ptr:ai:lem:exp`, printed here as a marked second route (note there).
- **Report *signal-machine-collision-certificates***, Part X [29] (the pipeline’s Report 58, batch 91, written at the same time as this Part). Its five-signal certificate uses this Part’s POWER module, and its delivered package bundles byte copies of three files shipped here: `code/09-chrono-matrix-science-build_certificate.py`, `09-chrono-matrix-science-SOURCENOTES.md` and `data/09-chrono-matrix-independent_audit-pell-dependency.json` (there `science/sources/prior-build_certificate.py.txt`, `POWER_SOURCE_NOTES.md` and `pell-dependency.json`). The collection ships them only here. [write] Added 4 October 2026 (batch 91, reciprocal note): that Part is now written (commit 61c9e3eb2). It prints the module once, as Report 58’s Lemma `smc:fd:lem:power`, names this Part as the origin of the three files, and uses the module again in Report 59 (Part X), Report 60 (Part XI) and, as its base-two instance with positive adapters, in the native-gap certificates of Reports 65 and 66 (Parts XII and XIII); Report 67 shrinks it to 22, 16, 14, 13 and 12 leaves (Theorem `smc:pp:thm:modules` there).

- **Report *canonical-diophantine-certificates***, Part XXI (the pipeline’s Reports 50 and 52–54, batch 91, written at the same time): fixed-arity packed certificates for sandpile stabilization and target firing, with the same expanded binary-containment masks (note after Lemma 81.1). No theorem is shared. [write] Added 4 October 2026 (batch 91, reciprocal note): except the containment lemma itself, Lemma 81.1, which that Part proves for the same equation system as its Lemma `cdc:fx:lem:subset`; a dated note there now records this Part.

No result of the three neighbouring reports of this category (*canonical-diophantine-certificates*, *probabilistic-quantum-and-continuous-computation*, *liveness-beyond-halting*) is re-proved in this report, so no proof was replaced by a pointer. [write 78] This remains true of Parts III and IV with one exception: Step 3 of Theorem 52.1 re-proves the finite generation of Mihailova’s fibre product, which is `pqc:dr:prop:mihailova` of [13]; it is printed as a marked second route with that pointer. [write 82] Part V re-proves no result of a neighbouring report. Its machine table is the third printing of a table of *quadratic-orthant-certificates*, and its rewriting and tile lemmas re-derive research-programme constructions; both are credited where they occur. [write 91] Part VI re-proves one result of a neighbouring report: its POWER module (Section 80) is Lemma `ptr:ai:lem:exp` of *periodic-turmite-first-revisits*, printed as a marked second route with that pointer. Its other re-proofs are of research-programme notes that it cites (note after Lemma 79.1).

Literature credit for the two-phase theorem. For *integer* exponents, Theorem 23.1 is a special case of known results: a product of two subgroups of a polycyclic group is closed in the profinite topology and therefore has decidable membership [9, Remark 6.7], and Roman’kov proves that the product of two subgroups membership problem is decidable in every finitely generated nilpotent group of class two [10]. The group $H^d \times \mathbb{Z}^e$ is both. Source 05 does not cite these results for its two-phase theorem; the credit and a precise statement of what source 05 adds are in Remark 23.2 after the theorem. The three-subgroup statement, Theorem 18.1, is different: both sources [9, 10] state that case as open, and no earlier resolution was found at placement (a check of the arXiv text of [9] and of the abstract of [10]); its priority is not certified.

1.5 Status and what is not claimed

The report is AI-assisted and unrefereed, and **nothing in it is formalized in Lean or Rocq**. Both sources give conventional proofs together with finite exact-integer checks that are illustrations, not proofs. Part II’s main theorem is a *proposed* resolution of a published open problem, not a peer-reviewed or proof-assistant-checked theorem. Every limitation stated by either source is kept in its Part; collected here, neither Part claims

- historical priority for its constructions (both sources say so; Part I, Appendix C; Part II, Appendix E and Appendix F);
- a finite-fold or single-fold Diophantine representation of c.e. sets, or of group word problems; the native fibres of Part I are generally infinite (Proposition 13.3), and Part II transfers fold bounds only conditionally;
- a universal arithmetic-operation record, an improvement of the universal degree-four bound, an optimized ambient dimension, or a numerically instantiated universal presentation, polynomial or matrix table;
- that its counts are minimal among all Diophantine representations: they are the literal counts of the stated compilers;

- one fixed-arity polynomial with a varying number of variables: in Part I the budget m and the circuit shape are external data, and the passage to a fixed arity is MRDP's, which preserves neither fibres nor the area ledger (Section 13);
- any new Lean or Rocq verification.

[write 78] Parts III and IV have the same status: proofs and finite exact checks, AI-assisted, unrefereed, not formalized, priority not established, as each source says. Every limitation of sources 06 and 07 is kept in its Part; collected here, neither claims

- an improvement of the repository's universal arithmetic-operation bound (87 operations at the sources' pin), or a numerically instantiated universal monoid, Higman subgroup or matrix table; source 06 does not instantiate Roman'kov's undecidable submonoid;
- a single-fold or finite-fold representation of c.e. sets: Part III's canonical quartic is single-fold over a complete pair profile, not over a projected matrix target (Section 39.2), and Part IV's single-fold certificates concern only languages already forced to be finite or periodic, while its compiler's added coordinates are unique only given the source witnesses, which a four-square substitution can multiply;
- one fixed-arity polynomial for all separator counts c (Part III), or optimality of Part IV's dimension $6g + 2$ or of its multiplication gadget;
- that Part III decides three-generator membership without a supplied multiplicity, or that three generators give undecidable membership;
- a uniform algorithm computing Part IV's finite or periodic description from subgroup generators: Proposition 50.1 shows that there is none;
- that Part IV's degree threshold bounds anything but the input degree in its stated interface; its degree-two half rests on the external effective Higman embedding [43], through the repository construction it reconstructs.

[write 82] Part V has the same status: proofs and finite exact checks, AI-assisted, unrefereed, not formalized; source 08 makes no novelty or optimality claim. Its many-one r.e.-completeness is conditional on the cited Neary–Woods simulation chain, which it neither reproves nor implements; the local equivalence, the polynomial family and the fibre formulas are proved. Every limitation of source 08 is kept in its Part; collected here, it does not claim

- inverse closure, subgroup membership, or an instantiated universal subgroup, presentation or Higman alphabet: its semigroup is positive;
- an implemented loader from arbitrary programs to finite $U_{15,2}$ tapes: its target encoder starts from a finite tape;
- one fixed-arity polynomial for all certificate lengths, or a finite-fold representation: its certificates are a family F_r with $130r$ witnesses whose union over r recovers membership, and every accepted target has infinitely many roots over all r (Section 73);
- unique targets: canonical signed parts make the roots correspond to tile sequences, not to targets (Section 72);
- an optimal generator count, dimension or certificate size, or an improvement on the matrix-undecidability literature; the research programme has since lowered the generator count to

193 (Section 75);

- an arithmetic-operation bound: the evaluator charges of Section 71 belong to one unoptimized evaluator;
- a decision procedure: its fibre formulas presuppose an accepted input.

[write 91] Part VI has the same status: proofs and finite exact checks, AI-assisted, unrefereed, not formalized. Its audits and reviews were written by the pipeline that delivered it; the research programme’s review [26], the one check from outside, certifies the arithmetic of its DAG, not the inherited theorems. Its numerical theorem is relative to two pinned mathlib theorems, and its matrix reading to research-programme theorems that it summarizes and does not re-prove. Every limitation of source 09 is kept in its Part; collected here, it does not claim

- a universal polynomial or an arbitrary-program compiler: the contexts [110 and A0] are one fixture, not claimed universal, and program-dependent matrices would need a newly computed radix factor;
- an improvement of the repository’s 84-operation universal bound, a shortest circuit, or optimality of any count;
- a finite-fold or unique-witness representation, a real-witness equivalence (the positive-integer domain is part of the theorem), a bound on witness size, or a computable bound on the packing offset from the input;
- a mortality theorem, inverse-closed group membership, or a result about the 193-generator Pell391 loader;
- a materialized zero: no complete numerical nested-Pell witness was generated, and neither Lean nor the upstream programs were run.

1.6 Research questions

Both agendas are printed in full: the twelve questions of Part I (Section 16) and the nine research directions of Part II (Section 30). Neither Part answers a question of the other, and the placement survey of the repository found none of them answered there. Three pairs are related and should be read together: higher nilpotent invariants as lower-bound detectors for proof DAGs (Question 16.4) and higher unitriangular groups as substrates for higher-degree compilers (Section 30.7); the first formal milestones, the Sanov chart and fixed-itinerary compiler (Question 16.1) and the exact fibre theorem (Section 30.4), each independent of MRDP; and passing the repository’s existing certificate compilers through the new constructions with fair metrics or intact fibres (Question 16.10 and Section 30.5).

[write 78] Part III adds twelve research questions (Section 42.2) and Part IV ten (Section 59). Two of Part IV’s are partly answered elsewhere in the repository: Part II answers the negative half of question 9 (subgroups replaced by rational subsets) for products of three subgroups or of three free commutative submonoids (Remark 46.3), and question 10 (a fully charged universal matrix certificate) is partly done by the research note [18]; both are re-scoped at the questions. Part III’s question 11 (higher-step nilpotent groups) belongs with Question 16.4 and Section 30.7, and its question 9 (three generators without a supplied multiplicity) is what remains open of the research note’s question (Section 1.4). The placement survey found no other question of Parts III and IV answered in the repository.

[write 82] Part V adds seven questions (Section 75). Its question 4 (generator and dimension counts) is answered in part by the research programme, which lowered the count from 229 to 197 and then to 193 on the day the archive arrived; the note there re-scopes it. Its question 7 (a uniform unbounded polynomial) is what remains open of Part IV’s question 10, which Part V answers only in part: it instantiates a universal alphabet, but for a positive semigroup and with bounded-length certificates (note at that question). It does not answer Part IV’s question 9 for positive-word semigroups, because its target map is not an affine curve. Its question 6 (formalization) belongs with Question 16.1 and Section 30.4. No other question of Part V is answered in the repository. [write 91] (4 October 2026.) Part VI answers Part V’s question 7 in part, for a successor semigroup and one fixture (note at that question).

[write 91] Part VI adds six questions (Section 90). Questions 1, 2, 4 and 6 (selection cost, shared range structure, an explicit program-family compiler, honest comparison of ledgers) bear on the research programme’s uniform compilers of the same night, which pay the program family in 1,393 operations but at exact degree at least 35,587. Question 3 (a smaller coefficient bound) is answered in part by the source’s own packing review, and question 5 (formalizing the interface chain) belongs with Question 16.1 and Section 30.4. The note after the questions says how far each is answered. Together with the programme’s compilers, Part VI also supplies the fixed-arity unbounded checker of Part IV’s question 10, for a semigroup rather than a subgroup (note there).

Part I

Arithmetic van Kampen certificates (source 03)

[write] Part I prints batch-76 manuscript 03, *Arithmetic van Kampen Certificates: Quartic equations, exact filling area, and optimal proof-DAG compression*, subtitled *A history-free Diophantine compiler for group-theoretic computation* (“history-free” in the sense of Section 1.3), research report prepared for Vladimir Reshetnikov, October 2, 2026. Its labels carry the prefix **gts:vk:**. Its abstract and the artifact boundary of its title page follow.

Abstract (source 03). For a fixed two-generator finite group presentation with s relators, we construct an explicit family of integer polynomials of degree at most four whose solvability at a boundary word is equivalent to filling that word with at most m conjugates of defining relators. The compiler allocates exactly $(2s + 13)m - 4$ integer witnesses for $m \geq 1$. Fixing the ordered relator choices reduces this to $8m - 4$. A classical arithmetic description of a free subgroup of $\mathrm{SL}_2(\mathbb{Z})$ eliminates conjugator-word histories: each conjugator is represented by four integers satisfying a single quadratic equation.

We give a small-witness bound, identify the minimum allocated arity of the area compiler with the presentation’s filling area, and derive the precise obstruction to a computable complete area cutoff. We then compile shared group-proof circuits into quartics without expanding their relator occurrences. For $[a^{2^k}, b^{2^\ell}]$ in $\langle a, b \mid [a, b] \rangle$, the area is $2^{k+\ell}$, whereas the minimum number of product gates in the specified proof calculus is exactly $k + \ell$. The resulting fixed-conjugator quartic uses only $8(k + \ell) - 4$ witnesses when $k + \ell \geq 1$. A more general addition-chain sandwich bounds rectangular commutator proof complexity.

The report provides complete proofs of the compiler and compression statements, exact-arithmetic software, adversarial examples, and a formalization and research agenda. Classical inputs are identified explicitly. Priority for the particular synthesis is not established; no unrestricted

finite-fold MRDP theorem, universal arithmetic-operation record, or new Lean verification is claimed.

Artifact boundary. The mathematical development is a research draft with executable checks. It is not a kernel-checked extension of ProveIt. The repository snapshot used for comparison is `c58206ca101d4744a015a0f0104646109357d943`.

2 The problem addressed and the contribution boundary

The broad assertion that an effectively simulable computational substrate has a Diophantine acceptance relation is already covered by MRDP. Repeating that assertion for another model of computation is usually less useful than preserving a resource that the model makes mathematically meaningful. Here that resource is *filling area*: the number of relator applications needed to establish a word identity.

Our computational substrate is a finitely presented group. The finite presentation is the program, a word is the instance, and an expression of that word as a product of conjugates of defining relators is an accepting proof. This viewpoint includes systems with undecidable word problem through classical constructions. Its advantage for Diophantine compilation is that free-group elements admit a particularly economical arithmetic representation.

The central distinction is between three different resources:

$$\text{conjugator word length,} \quad \text{expanded relator area,} \quad \text{shared proof-circuit size.} \quad (5)$$

The first compiler charges for the second resource but not the first. The second compiler charges for the third, without expanding the second. Integer magnitude and coefficient bit length remain separate costs throughout.

2.1 What has been established here

The following table is a contribution ledger, not a claim of historical priority. All numerical variable counts refer to the literal compilers specified in this report, before optional elimination or simplification.

Result	Content	Status
Arithmetic free-group chart	Four integral coordinates and one quadratic determinant equation	Classical Sanov theorem; proof included
Fixed-itinerary compiler	$8m - 4$ variables, $5m$ quadratic residuals	Explicit construction and proof here
Area-budget compiler	$(2s + 13)m - 4$ variables, $(2s + 11)m$ residuals	Explicit construction and proof here
Witness height	An elementary height bound for at least one witness at every accepting budget	Uses a cited classical conjugator bound
Native arity profile	Exact affine recovery of filling area and Dehn function	Consequence for the defined compiler
Proof-DAG compiler	Linear allocation in the number of circuit nodes, not expanded leaves	Explicit construction and proof here
Optimal dyadic grids	Area $2^{k+\ell}$; exactly $k + \ell$ product gates	Sharp theorem in the specified calculus
Addition-chain bounds	A lower signed-chain bound and an upper ordinary-chain bound	Proved here as a refinement

The main question resolved within this program is therefore concrete: *can one give an explicit low-degree Diophantine certificate whose visible resource is group-theoretic proof area, and can shared proofs compress that resource with a sharp, provable gain?* The answer is yes, with the formulas and exact counts above. This is not presented as a resolution of a famous previously named open problem. The broader novelty and publication significance require further specialist comparison.

2.2 Relationship to the repository

ProveIt’s documented MRDP endpoint produces a fixed finite polynomial representation for a recursively enumerable subset of the natural numbers; its public statement does not itself specify a practical compiler or a numerical degree or witness bound [7]. The repository also contains an extensive native-stream/queue compiler program. In particular, the inspected group-projective note optimizes the storage positions of signed-shear controller lanes while preserving chronological macro execution [8]. That is a packed-history construction, not the relator-area compiler developed below.

The present work is a complementary semantic front end. We do not transfer the repository’s formal-verification status to the new theorems, and we do not claim an improvement to its arithmetic-operation records. The exact chart, local matrix equations, and proof-circuit semantics below are proved directly. Only the final fixed-arity existence discussion invokes the MRDP interface at an abstract level.

[write] This repository relationship is incomplete; Section 1.4 supplies the missing comparisons. The collection’s report on probabilistic, quantum and continuous computation (Part VI of [13]) compiles finite presentations into rational rotation gates by Mihailova’s construction and certifies bounded *words* in that matrix group; the research programme’s group-commutator note [14] obtains universal subgroup membership in $\mathrm{SL}(4, \mathbb{Z})$, and its unipotent-loader note [15] proves a different free pair in $\mathrm{SL}_2(\mathbb{Z})$ faithful without describing its image arithmetically. None of them measures relator area or compiles proof DAGs. Both repository files cited here are unchanged from the pin c58206ca101d4744a015a0f0104646109357d943 to this report’s write.

2.3 Conventions that prevent misleading conclusions

All existential variables in the displayed compilers range over $\mathbb{Z}$, not over positive integers. The budget m and the shape of a proof circuit are external finite data used to construct a polynomial; they do not index a variable number of quantified variables inside a single formula. Equality of boundary and proof words is equality in the free group, not merely in the quotient group.

A bijection between polynomial zeros and *semantic proof witnesses* does not mean that an input has only one proof. Indeed, the direct compilers generally have infinitely many zeros over a single accepted input. We prove that explicitly in Section 13. Finally, the sharp circuit lower bound is a lower bound in our stated group-proof calculus, not a lower bound for arbitrary Diophantine representations or arbitrary arithmetic circuits.

3 Finite presentations as proof-producing computational substrates

Let $F(a, b)$ be the free group on a, b . A word uses the alphabet $\{a, a^{-1}, b, b^{-1}\}$; $|w|$ denotes the length of its freely reduced representative. Fix a finite presentation

$$\mathcal{P} = \langle a, b \mid r_1, \dots, r_s \rangle, \quad G_{\mathcal{P}} = F(a, b) / \langle\langle r_1, \dots, r_s \rangle\rangle. \quad (6)$$

The relators are fixed words. Repeated or freely trivial relators are permitted, though they are normally removed in applications.

Definition 3.1 (Area and its zero case). For $w \in F(a, b)$, let $\text{Area}_{\mathcal{P}}(w)$ be the least $m \in \mathbb{N}$ such that

$$w = \prod_{i=1}^m u_i r_{j_i}^{\epsilon_i} u_i^{-1} \quad \text{in } F(a, b), \quad u_i \in F(a, b), \quad \epsilon_i \in \{1, -1\}. \quad (7)$$

Set the area to $+\infty$ if no such expression exists. The empty product is 1, so the only element of area zero is the identity of $F(a, b)$.

Thus $w = 1$ in $G_{\mathcal{P}}$ if and only if $\text{Area}_{\mathcal{P}}(w) < \infty$. This is the algebraic formulation of van Kampen area. It is particularly convenient here because it specifies exactly what the arithmetic witnesses must recover.

To encode a budget rather than an exact nonpadded length, introduce the fixed label table

$$t_0 = 1, \quad t_{2j-1} = r_j, \quad t_{2j} = r_j^{-1} \quad (1 \leq j \leq s), \quad q = 2s + 1. \quad (8)$$

A *padded factorization of length m* is an expression $w = \prod_{i=1}^m u_i t_{\lambda_i} u_i^{-1}$ with $0 \leq \lambda_i < q$. Such a factorization exists exactly when $\text{Area}_{\mathcal{P}}(w) \leq m$. Identity padding is intentional and will also explain some witness multiplicities.

Definition 3.2 (Presentation-dependent Dehn function).

$$\delta_{\mathcal{P}}(n) = \max\{\text{Area}_{\mathcal{P}}(w) : |w| \leq n, w = 1 \text{ in } G_{\mathcal{P}}\}. \quad (9)$$

The maximum is over finitely many reduced words and includes 1. It is a finite natural number for every n , even when it is not a computable function of n .

3.1 Why two generators do not remove undecidable examples

Classical work on the group word problem supplies finitely presented groups with undecidable word problem [4]. The Higman–Neumann–Neumann embedding construction effectively embeds a finitely presented group into a two-generator finitely presented group [5]; a detailed account with an explicit substitution appears in [6, Section 2.3]. Consequently, the present two-generator framework includes fixed presentations with undecidable word problem. The finite substitution also transfers the relevant word-problem reduction.

This observation is an applicability result, not an explicit machine-to-presentation compiler in the delivered software. We do not instantiate a particular universal machine’s full relator table here. Nor do we claim that the embedding preserves the original presentation’s area exactly. All exact area identities in this report refer to the specified two-generator presentation itself.

One may think of the group substrate as an unconventional proof machine. Its accepting computations are normal-closure certificates rather than time-ordered configurations. This changes which structural resource can survive arithmetic compilation.

4 An exact quadratic arithmetic chart for the free group

Set

$$A = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}, \quad \rho : F(a, b) \longrightarrow \mathrm{SL}_2(\mathbb{Z}), \quad \rho(a) = A, \quad \rho(b) = B. \quad (10)$$

For four integers x, y, z, t , write

$$U(x, y, z, t) = \begin{pmatrix} 1 + 4x & 2y \\ 2z & 1 + 4t \end{pmatrix}, \quad E(x, y, z, t) = x + t + 4xt - yz. \quad (11)$$

A direct expansion gives

$$\det U = 1 + 4E. \quad (12)$$

Theorem 4.1 (Sanov’s chart). *The map ρ is injective, and its image is exactly*

$$\mathcal{S} = \{U(x, y, z, t) : (x, y, z, t) \in \mathbb{Z}^4, E(x, y, z, t) = 0\}. \quad (13)$$

Consequently, the integral points of $E = 0$ are in bijection with elements of $F(a, b)$.

These are classical theorems of Sanov [1]; the two statements, with precisely these congruence conditions, are also recorded in [2, Theorems 1–2]. We include proofs so that the compiler’s key semantic dependency is visible.

Proof of injectivity. On nonzero vectors in $\mathbb{R}^2$, define

$$X_A = \{(x, y) : |x| > |y|\}, \quad X_B = \{(x, y) : |y| > |x|\}.$$

For every nonzero integer h ,

$$A^h X_B \subseteq X_A, \quad B^h X_A \subseteq X_B.$$

For example, if $|y| > |x|$, then $|x + 2hy| \geq 2|y| - |x| > |y|$.

A nonempty reduced word consisting of a single syllable is a nontrivial power of A or B . An alternating syllable word whose first and last syllables are both powers of A sends X_B into X_A ; one

beginning and ending in powers of B sends X_A into X_B . Such a word cannot act as the identity. If its first and last syllables use different generators, conjugate it by a nonzero power of the first generator chosen not to cancel the first syllable. The resulting reduced syllable word begins and ends with that generator, and so is nontrivial by the preceding argument. Conjugation preserves whether a matrix is the identity. This proves that no nonempty reduced word maps to the identity. $\square$

Proof of the image description. Matrices of the stated congruence form with determinant one are closed under multiplication and inverse: the off-diagonal entries remain even and the diagonal entries remain 1 modulo 4. Both A and B have that form. Thus $\rho(F(a, b)) \subseteq \mathcal{S}$.

For the converse take $M = \begin{pmatrix} \alpha & \beta \\ \gamma & \eta \end{pmatrix} \in \mathcal{S}$. Its first-column entries satisfy $\alpha \equiv 1 \pmod{4}$ and $\gamma \equiv 0 \pmod{2}$. In particular $\alpha \neq 0$, and $|\alpha| \neq |\gamma|$. If $\gamma \neq 0$ and $|\alpha| > |\gamma|$, choose an integer h nearest to $\alpha/(2\gamma)$ and left-multiply by A^{-h} . This replaces α by $\alpha - 2h\gamma$, whose absolute value is at most $|\gamma|$. Equality is impossible because the former is odd and the latter even. If instead $|\gamma| > |\alpha|$, left multiplication by B^{-h} with h nearest to $\gamma/(2\alpha)$ replaces γ by a value of absolute value strictly less than $|\alpha|$.

Each step decreases the positive integer $\max(|\alpha|, |\gamma|)$ and preserves membership in $\mathcal{S}$. The process terminates with $\gamma = 0$. Determinant one then forces $\alpha\eta = 1$; since both diagonal entries are 1 modulo 4, they are both 1. The remaining matrix is $A^{\beta/2}$. Reversing the elementary left multiplications expresses M as a product of powers of A and B .

Finally, the four chart coordinates are uniquely read from the matrix entries, and injectivity of ρ identifies a unique free-group element. This proves the asserted bijection. $\square$

4.1 What is compressed by this chart

An arbitrary conjugator $u \in F(a, b)$ is now represented by four integers and the equation $E = 0$. No word length, list of letters, or matrix multiplication history for u is needed in the existential certificate. This is an arity saving, not a claim that all conjugators have short binary encodings. For instance, A^N has a compact matrix even when the literal word a^N is long; other words can produce exponentially large entries in their length.

The descent in the proof is implemented by `decode_sanov`. It returns powers as compressed syllables rather than expanding them into letters. No claim about an optimal bit-complexity bound for decoding is needed below.

4.2 Two indispensable hypotheses

Example 4.2 (Omitting determinant one produces false proofs). Consider $\langle a, b \mid a^5 \rangle$, the boundary word a , and $U = \begin{pmatrix} 1 & 0 \\ 0 & 5 \end{pmatrix}$. This matrix has the congruence form in (11), with coordinates $(0, 0, 0, 1)$, and satisfies

$$UA^5 = AU.$$

Thus a compiler using the cross-multiplied conjugacy equation but omitting $E = 0$ would accept a with one use of a^5 . That is false: mapping a to a generator of the cyclic group of order 5 and b to the identity proves that a is nontrivial in the presented group. Here $\det U = 5$ and $E = 1$; the determinant residual rejects the spurious certificate.

Example 4.3 (Arbitrary unimodular conjugators also produce false proofs). Let $J = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$. It has determinant one, but

$$JAJ^{-1} = B^{-1}.$$

For $\langle a, b \mid a \rangle$, this would falsely give a one-relator proof of b^{-1} . The quotient is infinite cyclic generated by b , so that boundary is nontrivial. The defect is that $J \notin \mathcal{S}$: it is not the image of a free-group conjugator under ρ .

The exact arithmetic image, rather than mere matrix invertibility, is therefore essential. The same congruence shortcut must not be assumed for other shear parameters; the situation for parameters at least 3 is different [2, Theorem 3].

5 A quartic compiler for a fixed relator itinerary

Fix a list of words $(t_{\lambda_1}, \dots, t_{\lambda_m})$ from (8); the choices and signs are part of the compiler input, not existential variables. Put

$$R_i = \rho(t_{\lambda_i}), \quad W = \rho(w).$$

The boundary W may first be regarded as four polynomial parameters. For $m \geq 1$, allocate a chart matrix U_i for each factor, and unrestricted matrices $P_1, \dots, P_{m-1}$. Set $P_0 = I$ and $P_m = W$ without allocating variables for them.

Use the equations

$$E(U_i) = 0, \quad 1 \leq i \leq m, \quad (14)$$

$$P_{i-1}U_iR_i - P_iU_i = 0, \quad 1 \leq i \leq m. \quad (15)$$

A matrix equation denotes four scalar equations. The notation $E(U_i)$ means E applied to the four chart coordinates of U_i .

Theorem 5.1 (Fixed-itinerary compiler). *For $m \geq 1$, the sum of squares of all scalar residuals in (14)–(15) is an explicitly computable integer polynomial $Q_\lambda(W; \mathbf{z})$ of degree at most four, with exactly*

$$N_{\text{it}}(m) = 8m - 4 \quad \text{integer witnesses}, \quad M_{\text{it}}(m) = 5m \quad \text{scalar residuals}. \quad (16)$$

For every $w \in F(a, b)$,

$$\exists \mathbf{z} \in \mathbb{Z}^{8m-4} \quad Q_\lambda(\rho(w); \mathbf{z}) = 0 \iff \exists u_1, \dots, u_m \in F(a, b) \quad w = \prod_{i=1}^m u_i t_{\lambda_i} u_i^{-1}. \quad (17)$$

For a fixed boundary, its zero tuples are in bijection with the ordered tuples of conjugator group elements satisfying the right-hand side.

Proof. Each R_i is constant, and each U_i is affine-linear in its four coordinates. Thus each entry of (15) has degree at most two, as does E . Their squared sum has degree at most four. It vanishes over the integers exactly when every residual vanishes, since squares are nonnegative.

At a zero, Theorem 4.1 gives a unique $u_i \in F(a, b)$ with $U_i = \rho(u_i)$. In particular U_i is invertible. Equation (15) implies

$$P_i = P_{i-1}U_iR_iU_i^{-1}.$$

Induction starting at $P_0 = I$ yields $W = \prod_i U_i R_i U_i^{-1}$. Injectivity of ρ gives the word identity in (17).

Conversely, given the conjugator elements, take their unique chart coordinates and let P_i be the successive matrix products. Every residual vanishes. The P_i are uniquely determined, so the correspondence is bijective. The allocated count is $4m + 4(m - 1) = 8m - 4$, and each factor contributes one determinant residual and four matrix residuals. $\square$

Remark 5.2 (Why the cross-multiplied equation matters). Writing $P_i = P_{i-1} U_i R_i \text{adj}(U_i)$ directly would generally introduce cubic entries and hence a degree-six squared sum. Equation (15) moves the inverse across the equality, retaining degree two without any new inverse variables. The determinant equation justifies that move. Determinant conditions for P_i are unnecessary: they follow inductively from the recovered factorization.

For $m = 0$, use

$$Q_0(W) = (W_{11} - 1)^2 + W_{12}^2 + W_{21}^2 + (W_{22} - 1)^2. \quad (18)$$

It has no existential variables and accepts precisely the free-group identity at boundary inputs of the form $\rho(w)$.

6 A uniform all-relator area-budget quartic

An itinerary may be unknown. We now encode its choices by integer one-hot selectors without letting the polynomial degree exceed four. Let $R_j = \rho(t_j)$ for $0 \leq j < q$, where $q = 2s + 1$.

For each $i = 1, \dots, m$, allocate a chart matrix U_i , an unrestricted matrix V_i , and selector integers $e_{i0}, \dots, e_{i,q-1}$. Also allocate $P_1, \dots, P_{m-1}$, and again set $P_0 = I$, $P_m = W$. The selected matrix is the expression

$$C_i = \sum_{j=0}^{q-1} e_{ij} R_j; \quad (19)$$

its entries are not additional variables.

The complete residual list is

$$E(U_i) = 0, \quad (20)$$

$$e_{ij}(e_{ij} - 1) = 0 \quad (0 \leq j < q), \quad (21)$$

$$\sum_{j=0}^{q-1} e_{ij} - 1 = 0, \quad (22)$$

$$V_i - P_{i-1} U_i = 0, \quad (23)$$

$$V_i C_i - P_i U_i = 0, \quad (24)$$

for each $1 \leq i \leq m$. Let $Q_{\mathcal{P},m}$ be the sum of squares of the scalar residuals. The matrix V_i is the degree-control device: it separates the potentially cubic expression $P_{i-1} U_i C_i$ into two quadratic constraints.

Theorem 6.1 (Area-budget equivalence and exact ledger). *For every fixed presentation (6) and $m \geq 1$, the construction gives an integer polynomial of degree at most four with exactly*

$$N_{\mathcal{P}}(m) = (q + 12)m - 4 = (2s + 13)m - 4, \quad (25)$$

$$M_{\mathcal{P}}(m) = (q + 10)m = (2s + 11)m \quad (26)$$

allocated integer witnesses and scalar residuals, respectively. For every word w ,

$$\exists \mathbf{z} \in \mathbb{Z}^{N_{\mathcal{P}}(m)} \quad Q_{\mathcal{P},m}(\rho(w); \mathbf{z}) = 0 \quad \Longleftrightarrow \quad \text{Area}_{\mathcal{P}}(w) \leq m. \quad (27)$$

The integer zero tuples are in bijection with ordered padded factorizations of length m , where a semantic factorization records both its label indices and its conjugator group elements.

Proof. The equations $e(e-1) = 0$ over $\mathbb{Z}$ force each selector to be 0 or 1. Their sum is 1, so there is a unique selected index λ_i and $C_i = R_{\lambda_i}$. The chart equation supplies a unique group element u_i with matrix U_i .

Substitution of (23) into (24) gives $P_{i-1}U_iR_{\lambda_i} = P_iU_i$. The proof of Theorem 5.1 now recovers the padded factorization and hence area at most m .

Conversely, start with any padded factorization. Set its selector vectors to the associated standard basis vectors, use the unique chart coordinates of the conjugators, set the P_i to partial products, and take $V_i = P_{i-1}U_i$. These choices satisfy every residual and are uniquely determined. Repeated relators in the table do not destroy the claimed bijection because the semantic witness records label indices, not just selected matrices.

All residuals are at most quadratic. Each cell allocates four chart variables, four V_i entries, and q selectors; the $m-1$ internal P_i matrices add $4(m-1)$. This gives (25). Per cell there are $1 + q + 1 + 4 + 4 = q + 10$ residuals, giving (26). The sum-of-squares argument completes the proof. $\square$

6.1 Sparsity, boundary typing, and presentation size

No promise about W is necessary to define the polynomial. In fact, any zero forces $W \in \mathcal{S}$ by the soundness induction. The word interpretation uses $W = \rho(w)$ so that the accepted input is unambiguous.

For fixed $\mathcal{P}$, the number of residuals and the number of monomials in their expanded squared sum are $O_{\mathcal{P}}(m)$. A local residual uses only

$$P_{i-1}, P_i, U_i, V_i, e_{i0}, \dots, e_{i,q-1}.$$

Treating the boundary entries as fixed parameters, these sets form bags in a path decomposition of the variable-interaction graph. Each bag has at most $q + 16$ variables, so the pathwidth is at most $q + 15$. Squaring a residual does not introduce variables from a different bag. This is a structural description, not a tractability theorem for integer feasibility on such graphs.

Let $L = \max(1, |r_1|, \dots, |r_s|)$. Since the matrix norm bound proved below gives relator and boundary entry bit lengths $O(L)$ and $O(|w|)$, the coefficients after substituting $W = \rho(w)$ have bit length $O(|w| + L + \log(m+1))$, with presentation-dependent constants. The logarithmic term allows coefficients such as the sum of the m selector-normalization constants. Arity, coefficient height, and the cost of computing the boundary are therefore all explicitly separated.

6.2 A complete one-cell example

For $\mathcal{P}_{ab} = \langle a, b \mid [a, b] \rangle$, use $[x, y] = xyx^{-1}y^{-1}$ and

$$R = \rho([a, b]) = \begin{pmatrix} 21 & -8 \\ 8 & -3 \end{pmatrix}, \quad R^{-1} = \begin{pmatrix} -3 & 8 \\ -8 & 21 \end{pmatrix}. \quad (28)$$

The $m = 1$ budget system has 11 variables and 13 residuals: U contributes four coordinates, V four entries, and (e_0, e_1, e_2) three selectors. In matrix notation the non-Boolean equations are

$$E(U) = 0, \quad V = U, \quad V(e_0 I + e_1 R + e_2 R^{-1}) = WU.$$

The polynomial is exactly the squared sum of these residuals and the four selector residuals. For $w = [a, b]$, the zero with $U = V = I$ and $(e_0, e_1, e_2) = (0, 1, 0)$ is immediate. The fixed-itinerary compiler for the single label R uses only four witnesses and five residuals.

For comparison, allocating the budget compiler at $m = 2$ gives 26 variables, and at $m = 4$ gives 56. Section 11 will encode the area-four boundary $[a^2, b^2]$ with a shared proof using only 12 variables.

7 A small-witness theorem and decidable area slices

An existential polynomial representation does not automatically give a decision procedure. In this compiler, however, a specified area budget also gives an explicit bound on at least one accepting witness. The needed geometric input is classical.

Lemma 7.1 (Bounded conjugators; classical). *Let w have length n and finite area α in a presentation whose relators have length at most L . There is a factorization of the form (7) with α factors such that every conjugator has length at most $n + L\alpha$.*

This is Lemma 2.D.2 of Cornulier–Tessera [3]. Its diagrammatic argument bounds basepoint-to-face paths in a van Kampen diagram by boundary length plus the total face-perimeter budget. We use this statement as an identified external lemma; it is not a new filling-length theorem. The ensuing arithmetic height estimate is derived explicitly.

For a real 2×2 matrix, let $\|M\|_\infty$ be the maximum absolute row sum. It is submultiplicative. Each of $A^{\pm 1}, B^{\pm 1}$ has this norm equal to 3, so

$$\|\rho(v)\|_\infty \leq 3^{|v|} \quad (v \in F(a, b)). \quad (29)$$

Theorem 7.2 (One bounded witness exists). *Let $m \geq 1$, $|w| = n$, and $L = \max(1, |r_1|, \dots, |r_s|)$. Define*

$$C = n + mL, \quad H_{\mathcal{P}}(n, m) = 3^{m(2C+L)+C+1}. \quad (30)$$

If $\text{Area}_{\mathcal{P}}(w) \leq m$, then $Q_{\mathcal{P},m}(\rho(w); \mathbf{z}) = 0$ has an integer solution with

$$|z_j| \leq H_{\mathcal{P}}(n, m) \quad \text{for every allocated coordinate } j. \quad (31)$$

Proof. Let $\alpha = \text{Area}_{\mathcal{P}}(w) \leq m$. If $\alpha > 0$, apply Lemma 7.1; if $\alpha = 0$, start with the empty factorization. Pad to length m using the identity label and identity conjugators. We obtain conjugators of length at most C and selected relators of length at most L .

Equation (29) gives $\|U_i\|_\infty \leq 3^C$ and $\|U_i R_{\lambda_i} U_i^{-1}\|_\infty \leq 3^{2C+L}$. Consequently,

$$\|P_i\|_\infty \leq 3^{i(2C+L)}, \quad \|V_i\|_\infty \leq 3^{(i-1)(2C+L)+C}.$$

Every matrix entry is bounded by its row-sum norm. The chart coordinates recovered by dividing the off-diagonal entries by 2 or subtracting 1 and dividing the diagonal entries by 4 have absolute values at most 3^C . The selectors are 0 or 1. The exponent in (30) safely dominates all these bounds. Completeness of Theorem 6.1 supplies the corresponding zero tuple. $\square$

Remark 7.3 (Scope of the height assertion). The theorem bounds *some* solution, not all solutions. It is stated for the all-label budget compiler, whose itinerary can be chosen from a short-conjugator factorization. It is not a claimed bound for an arbitrarily prescribed itinerary. Restricting the order of relators can change the available factorizations, so the classical lemma must not silently be applied after such a restriction.

Corollary 7.4 (Effective bounded-area decision). *For a fixed finite presentation, the predicate $\text{Area}_{\mathcal{P}}(w) \leq m$ is decidable uniformly in w, m . In fact, the displayed compiler and height bound give a primitive-recursive finite search procedure.*

Proof. Freely reduce w , compute $W = \rho(w)$, and handle $m = 0$ by $W = I$. For $m \geq 1$, compute the finite list of residuals and $H_{\mathcal{P}}(|w|, m)$, then enumerate the integer cube $[-H, H]^{N_{\mathcal{P}}(m)}$ and test every tuple. A zero is sound by Theorem 6.1; if the area is at most m , Theorem 7.2 places a zero in that cube. All bounds are explicit finite arithmetic expressions. $\square$

This decision procedure is not proposed as an efficient solver. It is useful because it exposes exactly why an area cutoff is enough to decide the word problem. It also shows that the possible noncomputability of (9) does not arise within a fixed finite area slice.

7.1 Bit costs and the unary-budget qualification

For fixed presentation and $m \geq 1$, the logarithm of the height bound is

$$O(m(n + mL + 1)).$$

Multiplying by the $O_{\mathcal{P}}(m)$ coordinates gives a total witness bit bound

$$O_{\mathcal{P}}(m^2(n + mL + 1)). \quad (32)$$

The integer arithmetic needed to check the residuals is polynomial in their written bit size. Therefore this construction supplies polynomial-size arithmetic certificates when m is measured as a unary budget and words are supplied explicitly. It does *not* give polynomial-size certificates in $n + \log m$ for the uncompressed budget family. The family has $\Theta(m)$ allocated coordinates before their magnitudes are even considered.

8 The exact native arity profile and the cutoff obstruction

For an accepted word w , let $\nu_{\mathcal{P}}(w)$ be the smallest number of variables *allocated by the all-label compiler* at a budget accepting w . The zero-budget case allocates zero variables. This defines a deliberately compiler-relative resource, not an invariant of all possible Diophantine descriptions.

Theorem 8.1 (Area is exactly visible in allocated arity). *For a word of finite area α ,*

$$\nu_{\mathcal{P}}(w) = \begin{cases} 0, & \alpha = 0, \\ (2s + 13)\alpha - 4, & \alpha \geq 1. \end{cases} \quad (33)$$

If $\nu_{\mathcal{P}}^{\max}(n)$ is the maximum of this quantity over null words of length at most n , then

$$\nu_{\mathcal{P}}^{\max}(n) = \begin{cases} 0, & \delta_{\mathcal{P}}(n) = 0, \\ (2s + 13)\delta_{\mathcal{P}}(n) - 4, & \delta_{\mathcal{P}}(n) \geq 1. \end{cases} \quad (34)$$

Proof. The accepting budgets are exactly those with $m \geq \alpha$ by Theorem 6.1. The allocation $(2s + 13)m - 4$ is strictly increasing for $m \geq 1$. Thus the smallest accepting allocation occurs at $m = \alpha$, except for the separately treated zero case. Taking the maximum over the finite set of null words gives the second formula. $\square$

The content is not that Diophantine variables intrinsically measure area. The content is that this particular transparent compiler preserves that resource exactly, whereas a general MRDP compression need not.

Theorem 8.2 (Computable cutoff equivalence). *For a fixed finite presentation $\mathcal{P}$, the following are equivalent:*

- (i) *The word problem of $G_{\mathcal{P}}$ is decidable.*
- (ii) *$\delta_{\mathcal{P}}$ has a total computable upper bound.*
- (iii) *$\delta_{\mathcal{P}}$ itself is a total computable function.*
- (iv) *The maximum native allocation $\nu_{\mathcal{P}}^{\max}$ has a total computable upper bound.*

Proof. Assume (i). Enumerate the finitely many reduced words of length at most n and use the word-problem decider to select precisely the null words. For each such word, test budgets $m = 0, 1, 2, \dots$ by Corollary 7.4 until the first success. Each of these searches terminates and computes its exact area. Their maximum computes $\delta_{\mathcal{P}}(n)$, proving (iii). Clearly (iii) implies (ii).

If $f(n)$ is a computable upper bound as in (ii), decide whether w is null by deciding the single bounded-area question $\text{Area}_{\mathcal{P}}(w) \leq f(|w|)$. This proves (i). Finally, (34) converts a computable bound in either (ii) or (iv) to a bound in the other. For example, a bound on $\nu_{\mathcal{P}}^{\max}$ also bounds $\delta_{\mathcal{P}}$ after adding a harmless constant. $\square$

This equivalence is a classical feature of Dehn functions expressed through the new compiler. Its proof here makes no appeal to a solver for unrestricted Diophantine equations.

Corollary 8.3 (No recursive complete area or native-arity budget). *Choose a fixed two-generator finite presentation with undecidable word problem. For every total computable $f : \mathbb{N} \rightarrow \mathbb{N}$, there are infinitely many n such that*

$$\delta_{\mathcal{P}}(n) > f(n).$$

The same absence of a computable upper bound holds for $\nu_{\mathcal{P}}^{\max}$.

Proof. A computable bound would contradict Theorem 8.2. If there were only finitely many exceptions to $\delta_{\mathcal{P}}(n) \leq f(n)$, their finite maximum could be incorporated as a constant into a total computable upper bound. Therefore exceptions occur infinitely often. $\square$

The corollary asserts infinitely many exceedances, not eventual domination of every computable function. Confusing those two statements would strengthen the conclusion without justification.

9 Shared relator-proof circuits

An ordinary van Kampen factorization repeats every relator occurrence. A proof may instead reuse an established subexpression. We formalize that possibility with an acyclic circuit over the normal closure.

Definition 9.1 (Relator-proof DAG). A relator-proof DAG for $\mathcal{P}$ is a finite directed acyclic graph in topological order, with a distinguished output node and all nodes reachable from it by following dependency edges. Its permitted node types are:

- (a) a leaf equal to 1 or to one of the fixed words $r_j^{\pm 1}$;
- (b) a product node $Z = XY$, referring to two earlier nodes, which may coincide;
- (c) an inverse node $Z = X^{-1}$;
- (d) a conjugation node $Z = uXu^{-1}$, where $u \in F(a, b)$ is a semantic witness;
- (e) a fixed conjugation node $Z = dXd^{-1}$, where $d \in F(a, b)$ is supplied as part of the circuit.

Its value is computed in the free group. Its product complexity is the number p of product nodes. The other nodes are not charged in this particular complexity measure, but are charged by the arithmetic compiler below.

A fixed conjugator may be supplied by its Sanov matrix, provided membership in $\mathcal{S}$ is checked. Such a matrix identifies a unique free-group element. Supplying an arbitrary element of $\text{SL}_2(\mathbb{Z})$ instead would repeat the error in Example 4.3.

Definition 9.2 (Expanded relator cost). Assign cost 0 to an identity leaf and cost 1 to a relator leaf. A product node has the sum of the two child costs; an inverse or either kind of conjugation inherits the cost of its child. If both input edges of a product refer to the same child, its cost is counted twice. Write $c(D)$ for the output cost.

Cost is a structural integer determined by the DAG. It can be evaluated in topological order without expanding a proof tree. No auxiliary arithmetic witnesses are needed merely to state that integer.

Lemma 9.3 (Expansion soundness). *If a proof DAG D evaluates to w , then*

$$\text{Area}_{\mathcal{P}}(w) \leq c(D). \quad (35)$$

Proof. Induct on the nodes. A relator has a one-factor expression and the identity has the empty expression. Products concatenate factorizations. Inverting a product of conjugates reverses their order and inverts each relator, without changing the number of factors. Conjugating a product by u replaces every factor conjugator v by uv , again preserving the number of factors. A repeated dependency is copied when the DAG is unfolded, exactly as required by the additive cost definition. $\square$

Lemma 9.4 (Doubling bound). *A proof DAG with p product nodes satisfies*

$$c(D) \leq 2^p. \quad (36)$$

This includes $p = 0$, when the possible costs are 0 and 1.

Proof. Before any product node, all costs are at most 1. After the first $j - 1$ product nodes in a topological ordering, all costs are at most 2^{j-1} : intervening unary nodes do not increase cost. The j th product adds two previously available costs, so it is at most 2^j . Induction gives the bound. $\square$

Remark 9.5 (Acyclicity is a semantic condition). One cannot replace a DAG by an arbitrary cyclic system of equations and retain Lemma 9.3. A self-reference such as $Z = Z$ imposes no proof obligation and could admit any boundary without a relator leaf. The compiler and software validate topological order, permissible leaf labels, and reachability. These are not optional metadata checks.

10 Quartic compilation of proof DAGs

For each nonleaf node, allocate an unrestricted 2×2 output matrix, except at the distinguished output, whose matrix is the boundary parameter W . Leaves use their fixed matrices. For a free conjugation node also allocate a chart matrix U .

The matrix residuals for the four nonleaf types are

$$Z - XY = 0 \quad (\text{product}), \quad (37)$$

$$Z - \text{adj}(X) = 0 \quad (\text{inverse}), \quad (38)$$

$$E(U) = 0, \quad ZU - UX = 0 \quad (\text{free conjugation}), \quad (39)$$

$$ZD - DX = 0 \quad (\text{fixed conjugation}). \quad (40)$$

Here $D \in \mathcal{S}$ is a fixed matrix. For a 2×2 matrix the adjugate is linear in its entries, so (38) is linear. The assertion that it is the inverse is recovered by induction, rather than assumed for unrestricted matrices.

Theorem 10.1 (Proof-DAG quartic compiler). *Let a valid proof DAG have p product nodes, h free conjugation nodes, j inverse nodes, and h_c fixed conjugation nodes. Suppose it has at least one nonleaf node. The squared sum of residuals (37)–(40) is an integer polynomial of degree at most four with exactly*

$$N_D = 4p + 8h + 4j + 4h_c - 4, \quad (41)$$

$$M_D = 4p + 5h + 4j + 4h_c \quad (42)$$

allocated integer witnesses and scalar residuals. At a boundary $W = \rho(w)$ it has a zero if and only if some assignment of its free conjugator group elements makes the DAG evaluate to w in $F(a, b)$.

Zero tuples are in bijection with those semantic conjugator assignments. Every accepted boundary satisfies $\text{Areap}(w) \leq c(D) \leq 2^p$.

Proof. Proceed in topological order. Leaf matrices are in $\mathcal{S}$. At a product node the residual forces the output to be the product of already recovered matrices, hence again in $\mathcal{S}$. At an inverse node, the child has determinant one by induction, so its adjugate is its inverse and lies in $\mathcal{S}$.

At a free conjugation node, $E(U) = 0$ gives a unique free-group conjugator and an invertible matrix U . The residual $ZU = UX$ forces $Z = UXU^{-1}$, which is the intended conjugate and is in $\mathcal{S}$. A fixed conjugation is the same argument with a verified constant D . Thus every node has exactly the required semantic value, and the boundary equality is equality in the free group by injectivity of ρ .

Conversely, given an assignment to the free conjugators, evaluate the DAG and insert its matrices into the allocated coordinates. All residuals vanish. Each chart coordinate and each internal output is unique once the semantic assignment is fixed, establishing the bijection.

There are $p + h + j + h_c$ nonleaf output matrices, with four boundary coordinates supplied rather than allocated. This contributes $4(p + h + j + h_c) - 4$ variables. The h free conjugators add $4h$. All nonleaf nodes contribute four matrix residuals, and each free conjugator contributes one more. These are the stated counts. Residual degree is at most two, so the squared sum has degree at most four. The area and doubling bounds follow from Lemmas 9.3 and 9.4. $\square$

If the circuit consists only of a leaf, use the four entry residuals $W - R$ and no auxiliary variables. The formula (41) is not applied to this exceptional case.

10.1 Compression does not hide uncounted inverse histories

The construction relies specifically on 2×2 matrices and the exact chart. The adjugate of a 2×2 matrix is linear; no determinant-solving subroutine or sequence of inverse operations is concealed in (38). Likewise, an arbitrary free conjugator is represented by its chart rather than by a guessed length and a word trace. However, its integer coordinates can be large, and the cost of searching for them is not included in the node count.

The algebra also shows why an arbitrary satisfying tuple cannot contain a nonunimodular internal output: the first such node in a topological ordering would have only genuine earlier inputs and hence a genuine output. This is the precise inductive argument that permits us to omit redundant determinant equations on internal matrices.

11 Sharp exponential compression for rectangular commutators

Consider

$$\mathcal{P}_{ab} = \langle a, b \mid r \rangle, \quad r = [a, b] = aba^{-1}b^{-1}. \quad (43)$$

Its quotient is $\mathbb{Z}^2$. We will measure the area and proof-DAG product complexity of a word that is trivial in this quotient but nontrivial in the ambient free group.

11.1 An elementary area detector

Use the integral Heisenberg group H of Section 1.2 in the coordinates $(x, y, z) = h(x, y, z)$, with the product $(x, y, z)(x', y', z') = (x + x', y + y', z + z' + xy')$ of (1). ([write] Source 03 calls this group $H = \mathbb{Z}^3$ and proves associativity and the inverse formula $(x, y, z)^{-1} = (-x, -y, -z + xy)$ at this point; both are printed once, in Section 1.2.) The elements $(0, 0, z)$ are central. Map a to $(1, 0, 0)$ and b to $(0, 1, 0)$. This defines a homomorphism from the free group, not from the quotient in (43). Direct computation gives

$$r \mapsto (0, 0, 1), \quad [a^u, b^v] \mapsto (0, 0, uv). \quad (44)$$

Every conjugate of $r^{\pm 1}$ therefore contributes ± 1 to the third coordinate. A product with m relator occurrences has third coordinate of absolute value at most m .

Proposition 11.1 (Exact rectangular area). *For positive integers u, v ,*

$$\text{Area}_{\mathcal{P}_{ab}}([a^u, b^v]) = uv. \quad (45)$$

Proof. The Heisenberg homomorphism gives the lower bound uv . For the upper bound, use the free-group identities

$$[xy, z] = x[y, z]x^{-1}[x, z], \quad (46)$$

$$[x, yz] = [x, y]y[x, z]y^{-1}. \quad (47)$$

Both follow by multiplying out and cancelling adjacent inverse pairs. First iterate (46) to express $[a^u, b]$ as a product of u conjugates of r . Then iterate (47) to express $[a^u, b^v]$ as a product of v conjugates of that word. Expanding gives exactly uv conjugates of r . $\square$

This elementary area calculation is a familiar central-extension argument. The sharp compression theorem adds a circuit-optimality statement and an explicit quartic allocation.

Theorem 11.2 (Optimal dyadic proof compression). *Let $k, \ell \geq 0$, put $d = k + \ell$, and set*

$$w_{k,\ell} = [a^{2^k}, b^{2^\ell}].$$

In presentation (43):

- (i) $\text{Area}(w_{k,\ell}) = 2^d$.
- (ii) *Among all relator-proof DAGs of Definition 9.1 evaluating to $w_{k,\ell}$, the minimum number of product nodes is exactly d .*
- (iii) *There is such a DAG with d product nodes and d fixed conjugation nodes and no free conjugation or inverse nodes.*
- (iv) *For $d \geq 1$, its quartic compiler allocates exactly $8d - 4$ integer witnesses and $8d$ scalar residuals. In contrast, the minimum accepting allocation in the direct all-label area compiler is $15 \cdot 2^d - 4$.*

Proof. Part (i) follows from Proposition 11.1. For the construction, start with $C = [a, b]$. If the current word is $C = [a^u, b^v]$, equation (46) gives

$$[a^{2^u}, b^v] = a^u C a^{-u} C. \quad (48)$$

This uses one fixed conjugation node and one product node referring to the old node twice. After k such steps the value is $[a^{2^k}, b]$. Likewise, (47) gives

$$[a^u, b^{2^v}] = C b^v C b^{-v}. \quad (49)$$

After ℓ additional steps the required output is reached. Each step doubles expanded cost; the starting cost is one. There are exactly d products and d fixed conjugations.

For optimality, any competing DAG with p product nodes satisfies

$$2^d = \text{Area}(w_{k,\ell}) \leq c(D) \leq 2^p$$

by Lemmas 9.3 and 9.4. Hence $p \geq d$, even when arbitrary free conjugations and inverses are allowed. The construction attains equality.

Theorem 10.1 with $p = h_c = d$, $h = j = 0$ gives $8d - 4$ witnesses and $8d$ residuals. The direct compiler has $s = 1$ and minimum budget 2^d , hence allocation $15 \cdot 2^d - 4$. At $d = 0$ the DAG is just the relator leaf and uses no auxiliary variables. $\square$

11.2 All boundary matrices are explicit

For arbitrary integers u, v , direct multiplication gives

$$\rho([a^u, b^v]) = \begin{pmatrix} 1 + 4uv + 16u^2v^2 & -8u^2v \\ 8uv^2 & 1 - 4uv \end{pmatrix}. \quad (50)$$

The fixed conjugators in (48) are $A^{2^i} = \begin{pmatrix} 1 & 2^{i+1} \\ 0 & 1 \end{pmatrix}$; those in (49) are $B^{2^j} = \begin{pmatrix} 1 & 0 \\ 2^{j+1} & 1 \end{pmatrix}$. Thus neither the boundary nor the constant conjugators require an expanded word representation.

Every intermediate entry in the constructed dyadic proof has $O(d + 1)$ bits. There are $O(d + 1)$ matrices and coefficients to write, so the total written data and the exhibited witness have $O((d + 1)^2)$ bits. We assert linear *variable count*, not linear total bit length.

$d = k + \ell$	Exact area	Optimal products	Direct variables	Shared variables
0	1	0	11	0
1	2	1	26	4
2	4	2	56	12
5	32	5	476	36
10	1,024	10	15,356	76
20	1,048,576	20	15,728,636	156

Remark 11.3 (The parameter in “exponential”). The gap is exponential in d , the binary-exponent description parameter and the optimal product count. The literal boundary word has length $2(2^k + 2^\ell)$. Thus this is not a claim of exponential area in literal word length for $\mathbb{Z}^2$; for balanced exponents its area is quadratic in that length. The result compares proof representations of the same boundary, while making their input encodings explicit.

11.3 A twelve-variable area-four certificate

Take $k = \ell = 1$. The circuit is

$$C_0 = R, \quad C_1 = AC_0A^{-1}, \quad C_2 = C_1C_0, \quad C_3 = BC_2B^{-1}, \quad W = C_2C_3.$$

Only C_1, C_2, C_3 are allocated matrices. Its sixteen residuals are the entries of

$$\begin{aligned} C_1A - AR, & \quad C_2 - C_1R, \\ C_3B - BC_2, & \quad W - C_2C_3. \end{aligned} \tag{51}$$

For avoidance of doubt, the first residual block in (51) is also a 2×2 matrix; each of the four blocks contributes four scalar equations. The fixed boundary is

$$W = \rho([a^2, b^2]) = \begin{pmatrix} 273 & -64 \\ 64 & -15 \end{pmatrix}.$$

A complete zero is obtained with

$$C_1 = \begin{pmatrix} 37 & -88 \\ 8 & -19 \end{pmatrix}, \quad C_2 = \begin{pmatrix} 73 & -32 \\ 16 & -7 \end{pmatrix}, \quad C_3 = \begin{pmatrix} 137 & -32 \\ 304 & -71 \end{pmatrix}.$$

The squared sum of the sixteen entries is a quartic in twelve variables. This is an explicit, fully numerical polynomial specification; the accompanying sparse JSON file lists every residual coefficient. The area lower bound is four, but no list of four independent factor-conjugator witnesses occurs in this encoding.

12 A more general addition-chain sandwich

The dyadic theorem is exact because its area is an exact power of two. For other rectangular boundaries, ordinary and signed addition chains give a useful refinement rather than an unjustified guess at the optimum.

Definition 12.1 (Chain lengths). For $n \geq 1$, let $\ell_+(n)$ be the least number of additions needed to produce n starting from 1, each addition using two earlier available values, with reuse allowed. Thus $\ell_+(1) = 0$.

Let $\ell_\pm(n)$ be the least number of binary additions needed when sign changes are free: a new value can be $\epsilon x + \eta y$ for earlier values x, y and signs $\epsilon, \eta \in \{1, -1\}$. The values 0 and 1 are available initially. Reuse and intermediate negative values are allowed. This explicit convention avoids relying on differing conventions for addition–subtraction chains.

Write $\text{pc}_{\mathcal{P}}(w)$ for the minimum product-node count among all relator-proof DAGs evaluating to a null word w . No bound on the number of unary nodes is built into this definition; the minimum is still a well-defined natural number because ordinary finite factorizations give proof DAGs.

Theorem 12.2 (Rectangular addition-chain bounds). *For positive integers u, v ,*

$$\ell_\pm(uv) \leq \text{pc}_{\mathcal{P}_{\text{ab}}}([a^u, b^v]) \leq \ell_+(u) + \ell_+(v). \quad (52)$$

Given ordinary addition chains of lengths L_u and L_v for u and v , there is a proof using at most $L_u + L_v$ products and at most $L_u + L_v$ fixed conjugations. Unless $u = v = 1$, it gives a quartic with at most $8(L_u + L_v) - 4$ allocated witnesses.

Proof. For the lower bound, apply the Heisenberg map of (44) to every node of any candidate proof. Every node image is central: this is true for the relator leaves and is preserved by products, inverses, and conjugations. Record only its third coordinate. Leaves supply 0 or ± 1 ; a product adds two earlier coordinates; inversion changes a sign; conjugation leaves the coordinate unchanged. Deleting copied values and free sign changes therefore gives a signed chain for uv using at most the number of product nodes in the proof. This proves the left inequality.

For the upper bound, suppose an addition-chain step computes $r + s$ from earlier positive values. Maintain the already proved words $C_t = [a^t, b]$. Equation (46) yields

$$C_{r+s} = a^r C_s a^{-r} C_r,$$

which uses one fixed conjugation and one product. Starting from the relator C_1 , the chain for u produces $[a^u, b]$.

Next maintain $D_t = [a^u, b^t]$, starting with that final word. Equation (47) yields

$$D_{r+s} = D_r b^r D_s b^{-r}.$$

The addition chain for v thus adds one fixed conjugation and one product per step. Any unused nodes can be discarded; for minimal chains they can already be chosen to contribute to the target. The retained circuit has $K \leq L_u + L_v$ products and the same number of fixed conjugations. For $K \geq 1$, Theorem 10.1 gives $8K - 4 \leq 8(L_u + L_v) - 4$ allocated witnesses. If $u = v = 1$, the relator leaf suffices. $\square$

Remark 12.3 (Exact versus upper-bound wording). For ordinary addition chains conventionally written with increasing positive entries, every step used in the target derivation can be retained and extraneous steps omitted. The exact allocation $8K - 4$ applies to the retained K -step construction. Supplying unneeded chain entries does not improve the upper bound. The delivered software implements the dyadic specialization, not an optimizer for minimal addition chains.

The elementary growth argument for a signed chain gives $\ell_{\pm}(n) \geq \lceil \log_2 n \rceil$. For a power of two, repeated doubling meets this bound. Hence (52) independently recovers Theorem 11.2's exact product count.

There is also a completely constructive upper bound without solving a chain-minimization problem. Reading the binary digits of n from left to right gives an ordinary chain of length

$$\lfloor \log_2 n \rfloor + \text{popcount}(n) - 1.$$

Therefore the product complexity of a positive rectangular commutator is at most the sum of these two expressions for u and v . This bound and the Heisenberg signed-chain lower bound need not coincide. Determining when they do is a natural, nontrivial continuation of the project.

13 Limits of compression and of witness-fiber claims

13.1 Sharing cannot give a computable complete proof budget

For a fixed presentation define the worst-case compressed product profile

$$\gamma_{\mathcal{P}}(n) = \max\{\text{pc}_{\mathcal{P}}(w) : |w| \leq n, w = 1 \text{ in } G_{\mathcal{P}}\}. \quad (53)$$

Proposition 13.1 (Comparison with the Dehn profile). *If $\delta_{\mathcal{P}}(n) \geq 1$, then*

$$\lceil \log_2 \delta_{\mathcal{P}}(n) \rceil \leq \gamma_{\mathcal{P}}(n) \leq \max(\delta_{\mathcal{P}}(n) - 1, 0). \quad (54)$$

If $\delta_{\mathcal{P}}(n) = 0$, then $\gamma_{\mathcal{P}}(n) = 0$.

Proof. Choose a word attaining $\delta_{\mathcal{P}}(n)$. Any proof of it with p products has area at most 2^p , so its product count is at least the logarithmic lower bound. For the upper bound, a word of positive area α has an ordinary factorization with α factors. Represent each factor by a relator leaf and a conjugation, then combine them with $\alpha - 1$ products. The free-group identity needs no product. $\square$

Theorem 13.2 (No computable general sharing bound). *If $G_{\mathcal{P}}$ has undecidable word problem, then $\gamma_{\mathcal{P}}$ has no total computable upper bound. More explicitly, for every total computable $f : \mathbb{N} \rightarrow \mathbb{N}$, there are infinitely many n with $\gamma_{\mathcal{P}}(n) > f(n)$.*

Proof. If all null words of length at most n had proofs with at most $f(n)$ products, Lemmas 9.3 and 9.4 would give $\delta_{\mathcal{P}}(n) \leq 2^{f(n)}$. This is a computable Dehn bound, contradicting Theorem 8.2. Finitely many exceptions to a proposed upper bound can again be absorbed into a constant, proving infinitely many exceedances. $\square$

This theorem remains true although conjugator length, conjugator magnitude, and the number of unary nodes are not charged in pc. It is therefore a genuine limitation on this proof-sharing calculus, not merely a complaint that an implementation has insufficient memory. On the other hand, it does not diminish the exact compression on the rectangular family: selected structured instances and worst-case completeness are different questions.

13.2 Semantic uniqueness is not single-fold Diophantineness

Proposition 13.3 (Infinite fibers in the native compiler). *Suppose the relator table includes a nontrivial element $r \in F(a, b)$. The fixed-itinerary compiler with one selected factor r and boundary r has infinitely many integer zero tuples. The same is true of the all-label compiler at that boundary and budget one.*

Proof. For every integer h , take the conjugator $u = r^h$. Then $uru^{-1} = r$. Nontrivial elements of a free group have infinite order: write such an element as a conjugate of a nonempty cyclically reduced word; its nonzero powers have nonempty freely reduced representatives. Hence the elements r^h are distinct. By Theorem 4.1, they have distinct chart tuples, producing infinitely many distinct polynomial zeros. $\square$

More generally, right multiplication of a conjugator u by any power of the selected nontrivial relator leaves uru^{-1} unchanged. Identity padding creates an even larger freedom: every conjugator acts identically on the identity factor.

The bijections in Theorems 5.1, 6.1, and 10.1 mean that no extra *arithmetic* ambiguity is added once all semantic conjugator choices are fixed. They do not make semantic proofs unique. Thus this construction does not establish finite-fold or single-fold representability of unrestricted group word problems, let alone the general finite-fold MRDP conjecture.

13.3 Where fixed-arity MRDP enters, and what it does not preserve

Encode words by natural numbers in any explicit effective way. The relation

$$w = 1 \text{ in } G_{\mathcal{P}} \iff \exists m \geq 0 \exists \mathbf{z} \, Q_{\mathcal{P},m}(\rho(w); \mathbf{z}) = 0$$

is a countable union of effectively decidable budget slices. This display abbreviates a computably enumerable relation; it is *not* one first-order polynomial formula with m controlling the number of variables.

MRDP then supplies a separate fixed polynomial with a fixed finite witness tuple for the encoded unbounded relation. The repository’s documented endpoint states exactly that fixed-arity existence principle for natural inputs and witnesses [7]. It can be used after proving the relevant enumerable-predicate adapter. Nothing in the present explicit family identifies the resulting fixed polynomial, preserves its witness fibers, or transfers the exact area/arity ledger to it.

The native compilers here use signed integer witnesses. A simple existential conversion to natural witnesses replaces each z by $z^+ - z^-$, with $z^+, z^- \in \mathbb{N}$; substitution preserves the quartic degree but doubles the allocated coordinates and introduces nonunique decompositions. This elementary conversion is sufficient for discussing the interface and emphatically does not preserve the zero-set bijections or improve fold properties.

14 Executable evidence and reproducibility

The accompanying package contains an exact-integer implementation and symbolic residual generators. Its validation record is in `verification/receipt.json`; the tests were actually executed with Python 3.13.5 and SymPy 1.14.0. They all passed. These are finite computations, not a substitute for the parametric proofs or a claim of Lean/Rocq verification.

[write] In this report the package files are shipped under prefixed names: `code/van_kampen.py` and `code/verify.py` as `code/03-van-kampen-van_kampen.py` and `code/03-van-kampen-verify.py`; `verification/receipt.json`, `verification/render_check.json`, `verification/export_check.json`, the three `examples/*.json` files and `requirements.txt` as `data/03-van-kampen-*`; and `SOURCES.md` as `03-van-kampen-SOURCES.md`. Every count in the ledger below equals the shipped receipt. The suite was rerun at the write on a copy with the delivered names, under Python 3.13.5 and SymPy 1.14.0: it passed, and its receipt and both exported polynomials equal the shipped files apart from line endings.

14.1 Implemented components

The module `code/van_kampen.py` implements matrix arithmetic, word evaluation and free reduction, Sanov chart encoding and even-Euclidean decoding, direct numeric budget residuals, the fixed-itinerary symbolic compiler, the all-label symbolic compiler, a general validated proof-DAG compiler, and the optimal dyadic grid construction.

Symbolic output is a sparse JSON description of the quadratic residuals together with the instruction to sum their squares. This is an exact polynomial specification, not a call to an unspecified solver. Each residual is stored as integer coefficients and exponent vectors relative to an explicit ordered parameter-and-variable list. Keeping the squared-sum form avoids a needless expansion of a large quartic while retaining every coefficient needed for reproducibility.

The generic DAG implementation checks legal relator leaves, acyclic topological references, and output reachability. Fixed conjugating matrices must pass the exact chart test. Integer arithmetic uses Python’s arbitrary-precision integers; no floating-point tolerance is used in the tests.

14.2 The test ledger

Check	Count
Freely reduced words through length 8: distinct matrices, chart and decoder round trips	13,121
Chart-coordinate tuples in $[-8, 8]^4$ examined	83,521
Integral points among those tuples, all decoded	417
Additional deterministic random word round trips	1,000
Valid area-budget certificates with budgets 1, 2, 3	3,615
Corrupted product, selector, and boundary certificates rejected	10,845
Symbolic all-label systems checked for degree and exact ledger	9
Symbolic/numeric budget evaluations cross-checked	72
Fixed-itinerary symbolic systems checked	5
Omitted-hypothesis false-positive constructions verified	2
Dyadic grid certificates for $0 \leq k, \ell \leq 20$	441
Grid symbolic systems checked, including the leaf-only case	6
Mixed free-conjugation/inverse/product DAG zero tuples checked	100
Malformed DAGs rejected	3
Symbolic rectangular boundary identity verified	1

An additional large grid with $(k, \ell) = (100, 100)$ was checked without expanding its 2^{200} relator occurrences. Its expanded cost has 201 binary digits. This illustrates the implemented sharing

behavior; it does not constitute an exhaustive lower-bound search. The proof of optimality is Theorem 11.2.

For the small-budget certificate family, the 15 choices per cell combine three relator labels with five conjugators $1, a, a^{-1}, b, b^{-1}$. The number of genuine certificates tested is therefore $15 + 15^2 + 15^3 = 3615$. For each one, three deterministic corruptions were tested. The numerical evaluator and symbolic generator are separate implementations in the same delivered codebase, not independently audited external software.

14.3 Reproduction commands

From the package root, install the declared symbolic dependency and run:

```
python -m pip install -r requirements.txt
python code/verify.py --receipt verification/receipt.json
python code/van_kampen.py budget 1 --relator abAB \
  --output examples/commutator_budget_1.json
python code/van_kampen.py grid 1 1 \
  --output examples/grid_1_1.json
pdflatex -interaction=nonstopmode -halt-on-error arithmetic_van_kampen.tex
pdflatex -interaction=nonstopmode -halt-on-error arithmetic_van_kampen.tex
```

The capital letters A and B in software words mean a^{-1} and b^{-1} ; they are not additional generators. The matrix constants named A and B in the Python module are the positive shear generators, matching (10).

[write] These commands use the delivered layout. `verify.py` imports `van_kampen` by its delivered name, the second, third and fourth commands overwrite recorded files, and `arithmetic_van_kampen.tex` is the delivered manuscript, printed here as Part I and not shipped separately. The README of this report gives a recipe that runs on a copy and leaves the shipped files untouched.

14.4 What has not been tested or implemented

The software does not search the enormous cube in Theorem 7.2 on substantial examples, construct a universal machine’s explicit finite group presentation, optimize arbitrary proof DAGs, find minimum addition chains, or call a general integer-polynomial solver. It generates and verifies concrete systems and supplied witnesses. There is no Lean source in the archive because no new Lean development was built or checked for this report.

15 A formalization route into ProveIt

The natural trusted boundary is the compiler correctness theorem, rather than the Python compiler or the numerical examples. A useful implementation would return both a finite polynomial syntax object and a theorem connecting its evaluation to the semantic proof relation. The following modules separate the obligations.

15.1 Free words and the arithmetic chart

Start with a normal-form representation of the free group, its evaluation into 2×2 integer matrices, and the subgroup specified by the congruences in (11). Prove closure and the determinant identity by ring normalization. Formalize ping-pong injectivity and the even-Euclidean termination measure separately. The decoder correctness theorem should return equality of matrix evaluations and a chart-membership equivalence; it need not expose an expanded word of a huge power.

The descent proof requires attention to signed division, nearest-integer selection, and parity-based strictness. These are small but essential obligations. Replacing nearest-integer division by truncation toward zero without rechecking the inequalities would invalidate the proof.

15.2 Area witnesses and their semantics

Define a padded factorization as a finite list of pairs consisting of a label and a free-group element. Prove that its existence at length m is equivalent to area at most m . Keep identity padding explicit, because it affects both completeness and the description of fibers.

The fixed-itinerary and budget compilers can then be formalized as finite vectors of quadratic polynomials. A generic sum-of-squares lemma over integers connects simultaneous residual vanishing to the single quartic. The central induction reconstructs P_i and proves their determinant and chart membership; extra hypotheses on those intermediate matrices should not be inserted accidentally.

15.3 Proof DAGs and their compilation

Use a topologically indexed node datatype with child indices strictly smaller than the current index. This makes acyclicity structural rather than an unproved Boolean side condition. Label legality and chart membership of supplied constants are proof-carrying fields. Prove the cost bound and the semantic area inequality independently of the arithmetic compiler.

Next prove the local matrix semantics of each gate and compose them by the same topological induction. The product-count optimum for dyadic grids can be formalized without the full general Dehn-function machinery: it needs the Heisenberg homomorphism, the commutator identities, the circuit construction, and the doubling lemma.

15.4 The external dependency ledger

The only substantive geometric input not reproved fully here is Lemma 7.1; it is needed for the displayed height bound and the direct bounded-search procedure. A formal project should either prove it from a van Kampen diagram library or record it as an explicit temporary assumption. It is not needed for the chart theorem, either quartic soundness/completeness theorem, DAG semantics, the dyadic optimum, or the addition-chain sandwich.

Classical undecidable-group existence and the two-generator embedding theorem are needed only to instantiate the noncomputability corollaries. The core compiler is valid for every supplied two-generator finite presentation without those existence inputs. MRDP is needed only to pass from the unbounded enumerable relation to a separate fixed-arity polynomial; it is not used to justify any displayed quartic.

This separation permits useful formally checked endpoints well before the deepest surrounding group-theoretic libraries are completed. In particular, a kernel-checked twelve-variable polynomial

for $[a^2, b^2]$, a proved generic fixed-itinerary compiler, and a proved parametric dyadic optimality theorem are realistic independent milestones.

16 Further research questions and proposed projects

The following questions are proposed as continuations, not as claims that the associated problems have been settled or that every item is a recognized open problem in the literature.

Question 16.1 (Verified arithmetic free-group chart). Can the full Sanov chart equivalence, including a terminating decoder to compressed syllables, be added to the repository with no new mathematical axioms? The immediate target is a theorem equating integral points of $x + t + 4xt - yz = 0$ with free-group elements, followed by certified compilation of the fixed-itinerary system.

Question 16.2 (Lower allocation at fixed quartic degree). Can the all-label construction beat $(2s + 13)m - 4$ variables while retaining an explicit semantic bijection and degree at most four? A successful improvement must account for selector typing, the triple-product obstruction, and the determinant condition. This is an upper-bound improvement problem; the present counts are not claimed optimal among quartics.

Question 16.3 (Exact rectangular proof complexity). When does $\text{pc}([a^u, b^v]) = \ell_{\pm}(uv)$ hold, and when is a larger value forced by free-group information invisible to the Heisenberg quotient? Theorem 12.2 leaves a specific interval. Small non-dyadic rectangles are an accessible source of candidate identities and counterexamples, but a search over a bounded set of conjugators alone cannot certify a global lower bound.

Question 16.4 (Stronger arithmetic lower-bound detectors). Can higher nilpotent quotients, several central coordinates, or suitably controlled invariants give proof-DAG lower bounds stronger than the scalar area/doubling bound? One would want an invariant whose update under each allowed node is explicitly bounded, so that a certificate for the lower bound can itself be checked mechanically.

Question 16.5 (Proof-sharing profiles beyond the abelian presentation). For presentations with large Dehn functions, how close can $\gamma_{\mathcal{P}}(n)$ be to the logarithmic lower bound in (54)? Which families admit near-maximal reuse, and which force close to the unshared number of products? The metric must continue to state whether arbitrary conjugations, inverse nodes, and coefficient magnitude are charged.

Question 16.6 (Centralizer-normalized witnesses). Can one quotient conjugators by the evident right-centralizer symmetry and impose a canonical representative in a comparably small arithmetic language? Even a solution for one-relator bounded-area slices would be useful. It would not automatically settle unrestricted finite-fold MRDP: normalizing individual factors does not make whole factorizations finite or unique.

Question 16.7 (A native higher-rank chart). Can a useful finite-generator free group be represented by a similarly exact low-dimensional arithmetic chart without first embedding its presentation into a two-generator group? An embedding may preserve the word problem while changing area substantially. A higher-rank chart with a quantitative area-preserving interface would allow more direct comparisons with machine-to-group constructions.

Question 16.8 (Concrete universal substrate front end). Select a published, fully explicit finite presentation with an effective machine-simulation theorem, instantiate every relator, and connect

the simulator’s accepting condition to the quartic compiler. The deliverable should include the actual relator table, the input-word map, and the two directions of correctness. Merely citing the existence of an undecidable presentation is not that deliverable.

Question 16.9 (Automatic discovery of reusable group proofs). Can an optimizer propose repeated conjugate patterns in a known proof and produce a validated DAG with a substantially smaller polynomial? A safe design treats heuristic discovery as untrusted and has a small semantic checker certify the transformed proof. The area and product-count annotations should be checked along with the boundary.

Question 16.10 (Compiler interoperability and fair metrics). Can the repository’s packed-history representations and the present area/DAG representations share a common verified input boundary? Comparisons should report variables, maximum degree, arithmetic operations, coefficient bit size, witness height, and whether a numerical universal instance is actually instantiated. A saving in one metric need not improve the others.

Question 16.11 (Fixed-arity compression retaining area information). Can a fixed-arity Diophantine representation retain a recognizable area parameter or a controllable relationship between witness size and filling area? The present family does not do this: its arity changes with m . Any proposed packing must prove both integer soundness and completeness, and state separately what happens to witness fibers.

Question 16.12 (Structure-aware certificate solving). Can practical search exploit the Sanov chart, central invariants, and the path decomposition rather than treating the final squared sum as an opaque polynomial? Useful benchmarks include finding a one-factor conjugacy proof, recovering small area budgets, and verifying or improving shared rectangular circuits. Solver performance should be measured separately from certificate size.

17 Conclusion

The principal construction is explicit: a free-group conjugator is a point on one integral quadric; a relator application is a small matrix identity; summing squares produces a quartic. For a fixed itinerary, the allocated count is $8m - 4$, and for an unknown itinerary drawn from s relators it is $(2s + 13)m - 4$. The resulting family preserves filling area exactly as a compiler-relative arity resource.

The second construction exposes a different resource. A proof DAG can reuse a whole normal-closure proof, and its quartic need not duplicate the reused relator occurrences. The dyadic rectangular words give an exact theorem: area $2^{k+\ell}$, optimal product count $k + \ell$, and a fixed-conjugator quartic with $8(k + \ell) - 4$ witnesses outside the leaf-only case. The Heisenberg detector and addition-chain refinement make the lower-bound mechanism as explicit as the upper construction.

These results do not evade undecidability or the distinction between a variable-arity family and a fixed polynomial. There is no computable complete area or sharing cutoff for presentations with undecidable word problem, and the native witness fibers are generally infinite. The productive next step is therefore not an overbroad universality claim, but a verified compiler with an honest resource ledger, followed by a concrete machine-to-presentation instance and sharper proof-sharing theorems.

Part II

Three commutative phases (source 05)

[write] Part II prints batch-76 manuscript 05, *Three Commutative Phases Are Diophantine-Universal: A fibre-preserving Heisenberg compiler and a proposed resolution of the three-subgroup problem*, research manuscript prepared for Vladimir Reshetnikov, October 2, 2026. Its labels carry the prefix `gts:tp:`. Its abstract and its statement of status and scope follow. A *phase* is one commuting factor of an ordered product (Section 1.3). For integer exponents the “matching two-phase decidability result” of the abstract is a special case of known results, credited in Remark 23.2.

Abstract (source 05). We give an explicit construction which represents every finite system of integral quadratic equations by membership in a product of three free abelian subgroups of a finite direct power of the discrete Heisenberg group. The construction preserves the entire solution fibre: arithmetic solutions are in bijection with ordered triples of subgroup elements whose product is the target. The key device combines the integer-valued polynomial $\mathfrak{b}(t) = t(t-1)/2$ with three commuting phases of upper-unitriangular matrices. Polarization converts mixed products into linear combinations of binomials of linear forms, while central auxiliary generators record these combinations.

Applying classical MRDP and circuit flattening yields three fixed free abelian subgroups with computably enumerable complete product membership, even when targets lie on a fixed central affine cyclic line. This provides a negative answer to the three-subgroup decidability question explicitly left open in the literature. We also prove a matching two-phase decidability result for commutative phases, a nonnegative-exponent version, an explicit quartic reverse representation, quantitative witness bounds, and a small three-subgroup product that is not profinitely closed. An executable compiler and 190,826 deterministic exact-integer checks accompany the proofs.

Status and scope. This is a proposed research result with full mathematical proofs, not a peer-reviewed or proof-assistant-checked theorem. The literature search identified the published open question but does not certify historical priority. The universal fixed matrices are proved to exist; the attached software instantiates arbitrary supplied quadratic systems and explicit examples, not a numerically expanded universal MRDP polynomial. None of the repository’s recent arithmetic-operation records is assumed.

18 The question and the result

18.1 Why a new substrate is not enough

Classical MRDP states that every computably enumerable set of natural numbers has a Diophantine representation. Consequently, observing that a finitely specified computational substrate can be simulated by a Turing machine usually yields a Diophantine existence theorem immediately. Such a statement alone does not explain the substrate’s structure, control witness multiplicity, or improve a known representation boundary.

The construction here addresses all three issues. Its substrate is a product of only three commutative families of integer matrices. Every matrix within one phase commutes with every other matrix in that phase; arbitrary interleaving is forbidden. Arithmetic appears solely from interaction between phases. The construction proves both directions by an explicit coordinate calculation and recovers every exponent from the original arithmetic witness. It also crosses a specific published decidability boundary.

18.2 The published problem

König, Lohrey, and Zetsche proved that a fixed product of four abelian subgroups of a sufficiently large direct power of the discrete Heisenberg group can have undecidable membership. Their Remark 6.7 asks about three subgroups [9]. Roman'kov's later paper, published online on September 20, 2024 and in print on March 1, 2025, again explicitly leaves the three-subgroup case open for nilpotent groups of class two [10].

We resolve the stated existence question in the negative by constructing three free abelian subgroups. The new argument is the binomial compiler below, not the appeal to MRDP at the final step. All claims of resolution refer to this proof; independent review and a broader priority investigation remain appropriate before publication.

18.3 A precise main statement

Write $H = UT_3(\mathbb{Z})$ for the discrete Heisenberg group of the matrices $h(a, b, c)$ ([write] source 05's defining display is printed in Section 1.2). Natural numbers include zero. A subset of an effectively presented group is *computably enumerable complete* when it is computably enumerable and every computably enumerable set is many-one reducible to it.

Theorem 18.1 (Three fixed subgroups). *There are a fixed positive integer d , three fixed finitely generated free abelian subgroups $\mathcal{A}, \mathcal{B}, \mathcal{T} \leq H^d$, and fixed central elements $g_0, g_1 \in Z(H^d)$, with g_1 of infinite order, such that*

$$\{n \in \mathbb{N} : g_0 g_1^n \in \mathcal{A}\mathcal{B}\mathcal{T}\}$$

is computably enumerable complete. The three subgroups have pairwise trivial intersections. Thus membership in the fixed set $\mathcal{A}\mathcal{B}\mathcal{T}$ is undecidable.

The quantifiers are important. The ambient dimension, every generator, and both target elements are chosen once. They do not depend on n . Only the exponent in $g_0 g_1^n$ changes. The assertion concerns a fixed subset of a fixed group, not merely a decision problem where the group or the number of phases is part of the input.

A stronger structural statement is proved first. Given a quadratic map $F : \mathbb{Z}^n \rightarrow \mathbb{Z}^m$, its fibre $F^{-1}(t)$ is exactly the set of ordered three-phase factorizations of a corresponding target, under an explicit bijection. This includes empty, finite, and infinite fibres. The same construction works over $\mathbb{N}$ with free commutative submonoids.

18.4 What is and is not obtained

The main new output is a three-phase, fibre-preserving representation, and its consequence for the stated open problem. The two-phase theorem gives a sharp phase count for these commutative matrix systems. The quartic representation gives an explicit reverse compiler rather than a new universal degree record. Counting statements concern subgroup elements or basis-exponent vectors, not all spelling variants of words. Nothing here proves that all computably enumerable sets have finite-fold or single-fold Diophantine representations.

The computational interpretation is exact target reachability. This is not a step-by-step simulation theorem for a physical matrix device, nor does it assert robust computation with bounded precision. Unbounded integer exponents are essential.

19 Arithmetic normal forms

19.1 Integer binomials and polarization

Define

$$\mathfrak{b}(t) = \binom{t}{2} = \frac{t(t-1)}{2} \quad (t \in \mathbb{Z}).$$

This is an integer for every integer t , including negative t . Moreover, $\mathfrak{b}(t) \geq 0$ for every integer t : either $t \geq 1$, or $t \leq 0$, and the two consecutive factors have the same sign or one is zero.

Lemma 19.1. *For all $s, t \in \mathbb{Z}$,*

$$\mathfrak{b}(s+t) = \mathfrak{b}(s) + \mathfrak{b}(t) + st, \quad (55)$$

$$t^2 = 2\mathfrak{b}(t) + t, \quad (56)$$

$$st = \mathfrak{b}(s+t) - \mathfrak{b}(s) - \mathfrak{b}(t). \quad (57)$$

Proof. Multiplying each equality by two and expanding gives identical integral polynomials on the two sides. The second equality is immediate from the definition, and the third is a rearrangement of the first. $\square$

The use of integer binomials rather than division by two in a lattice is important. All generator coordinates below will be integers. The coefficients of the normal form will be integers, and negative exponents will require no extra parity constraint.

19.2 A shared family of sparse linear forms

Let $F = (F_1, \dots, F_m) : \mathbb{Z}^n \rightarrow \mathbb{Z}^m$ have degree at most two, where $n, m \geq 1$. Write it uniquely as

$$F_j(x) = c_j + \sum_{i=1}^n \ell_{ji} x_i + \sum_{i=1}^n d_{ji} x_i^2 + \sum_{1 \leq i < k \leq n} o_{jik} x_i x_k. \quad (58)$$

All coefficients are integers. Let $\mathcal{E}$ be the set of unordered pairs $\{i, k\}$ whose mixed coefficient is nonzero in at least one row. Put $q = |\mathcal{E}|$ and $N = n + q$.

The N rows of an $N \times n$ matrix L are the n coordinate forms x_i , followed by the forms $x_i + x_k$ for $\{i, k\} \in \mathcal{E}$, in a fixed order. The first n rows form the identity matrix. In particular,

$$L \text{ is injective,} \quad L(\mathbb{N}^n) \subseteq \mathbb{N}^N, \quad \|Lx\|_\infty \leq 2\|x\|_\infty. \quad (59)$$

Proposition 19.2 (Integral binomial normal form). *One can compute integral matrices $C \in \mathbb{Z}^{m \times N}$, $D \in \mathbb{Z}^{m \times n}$ and $c \in \mathbb{Z}^m$ such that*

$$F(x) = C\mathfrak{b}(Lx) + Dx + c \quad (x \in \mathbb{Z}^n), \quad (60)$$

where $\mathfrak{b}$ is applied coordinatewise. The number of binomial coordinates is

$$N = n + q \leq n + \binom{n}{2}.$$

An explicit choice is

$$D_{ji} = \ell_{ji} + d_{ji}, \quad (61)$$

$$C_{j,i} = 2d_{ji} - \sum_{k < i} o_{jki} - \sum_{k > i} o_{jik}, \quad (62)$$

$$C_{j,\{i,k\}} = o_{jik}. \quad (63)$$

Proof. Replace each square in (58) by (56), and each mixed term by (57). A mixed term contributes its coefficient to the row indexed by $x_i + x_k$, and the negative of that coefficient to each of the rows indexed by x_i and x_k . A square contributes $2d_{ji}$ to $\mathfrak{b}(x_i)$ and d_{ji} to x_i . These contributions are exactly (61)–(63). No assertion about the existence of solutions is used in deriving this identity. $\square$

Keeping all coordinate forms, even unused ones, costs at most n rows but makes witness recovery injective. The same pair form is shared across all equations. This avoids compiling every occurrence of a mixed product into a separate matrix coordinate.

19.3 From general polynomials to quadratic systems

An arithmetic circuit over $\mathbb{Z}$ is a finite acyclic list of constant, addition, subtraction and multiplication gates. Introduce one variable for every wire, including the freely chosen input wires. The gate creating wire z gives one equation of one of the forms

$$z - a = 0, \quad z - x - y = 0, \quad z - x + y = 0, \quad z - xy = 0.$$

A prescribed output is imposed by one additional linear equation.

Lemma 19.3 (Unique circuit extension). *For any assignment to the free wires, there is exactly one assignment to all gate-output wires satisfying the gate equations. It is the ordinary circuit evaluation. Therefore imposing output equations preserves the solution fibre on the free wires bijectively.*

Proof. Order the gates topologically. A constant gate has its output fixed by its equation. At every subsequent gate, the values on its input wires have already been determined, and its equation determines its output uniquely. Induction proves uniqueness and also constructs the assignment. $\square$

Parameters can be treated as free wires with final linear pinning equations. Thus the coefficients of the quadratic system are independent of parameter values: parameters appear only in the right-hand-side vector t . This will supply the fixed-subgroup quantifier in Theorem 18.1.

Over $\mathbb{N}$, write a polynomial equation as $P^+ = P^-$ with nonnegative coefficients, evaluate both sides using nonnegative constants, addition and multiplication, and equate the outputs. All gate values are then natural. Subtraction of two natural quantities is used only in the polynomial residual, not as a natural-number gate operation.

20 The three-phase compiler

20.1 Heisenberg coordinates

Direct matrix multiplication gives the multiplication, inverse and power formulas (1), (2) and (3). ([write] These three formulas and their proof are printed once, in Section 1.2, with their labels.)

The commutator is central. In particular, elements with $b = 0$ commute with one another, as do elements with $a = 0$. A fixed cyclic direction is also commutative. These are exactly the three types of directions we shall use.

Work first in

$$G = \mathbf{H}^N \times \mathbb{Z}^m.$$

Products, inverses and powers in the $\mathbf{H}$ coordinates are componentwise; the last m coordinates add. The latter coordinates form a central direct factor. They are not additional phases.

20.2 The three homomorphisms

Fix the normal form $F(x) = C\mathbf{b}(Lx) + Dx + c$ of Proposition 19.2. For $x, y \in \mathbb{Z}^n$ and $u, r \in \mathbb{Z}^N$, define

$$A(x, u) = \left((h((Lx)_s, 0, -u_s))_{s=1}^N, Dx + Cu \right), \quad (64)$$

$$B(y) = \left((h(0, (Ly)_s, 0))_{s=1}^N, 0 \right), \quad (65)$$

$$T(r) = \left((h(-r_s, -r_s, \mathbf{b}(r_s)))_{s=1}^N, 0 \right). \quad (66)$$

The letter T denotes the third phase, not a time bound. Set

$$\mathcal{A} = \text{im } A, \quad \mathcal{B} = \text{im } B, \quad \mathcal{T} = \text{im } T.$$

Lemma 20.1 (Commuting phases, ranks and intersections). *The maps $A : \mathbb{Z}^{n+N} \rightarrow G$, $B : \mathbb{Z}^n \rightarrow G$ and $T : \mathbb{Z}^N \rightarrow G$ are injective group homomorphisms. Consequently $\mathcal{A}, \mathcal{B}, \mathcal{T}$ are free abelian subgroups of ranks $n + N, n, N$, respectively. Their pairwise intersections are trivial.*

Proof. In a product of two A -elements, each Heisenberg cross term is zero because the right factor has $b = 0$. The remaining coordinates are linear in (x, u) . Therefore $A(x, u)A(x', u') = A(x + x', u + u')$. For B , the cross term is zero because the left factor has $a = 0$.

For the third phase, (1) gives central coordinate $\mathbf{b}(r_s) + \mathbf{b}(r'_s) + r_s r'_s$, which equals $\mathbf{b}(r_s + r'_s)$ by (55). Thus $T(r)T(r') = T(r + r')$; equivalently, the s th coordinate is the r_s th power of $h(-1, -1, 0)$.

If $A(x, u) = 1$, then $Lx = 0$, whence $x = 0$, and its Heisenberg central coordinates give $u = 0$. The injectivity of B follows from that of L , and the horizontal coordinates of $T(r)$ recover r directly.

If $A(x, u) = B(y)$, horizontal coordinates give $Lx = Ly = 0$, so $x = y = 0$; central coordinates then give $u = 0$. If $A(x, u) = T(r)$, the b -coordinates give $r = 0$, and injectivity gives $x = u = 0$. If $B(y) = T(r)$, the a -coordinates give $r = 0$ and then $y = 0$. $\square$

20.3 Explicit finite generator lists

Let e_i denote a coordinate vector in $\mathbb{Z}^n$ and e_s one in $\mathbb{Z}^N$. The generator bases are

$$\begin{aligned} \mathcal{A} : & A(e_1, 0), \dots, A(e_n, 0), A(0, e_1), \dots, A(0, e_N), \\ \mathcal{B} : & B(e_1), \dots, B(e_n), \\ \mathcal{T} : & T(e_1), \dots, T(e_N). \end{aligned} \quad (67)$$

For clarity, $A(e_i, 0)$ has Heisenberg entries $h(L_{si}, 0, 0)$ and abelian coordinate $D_{\cdot i}$. The generator $A(0, e_s)$ has one Heisenberg central entry -1 and abelian coordinate $C_{\cdot s}$. The corresponding B

generator has entries $h(0, L_{si}, 0)$. The s th T generator has $h(-1, -1, 0)$ in coordinate s and identities elsewhere.

These lists contain no hidden dependence on t . The potentially nonlinear formula for $T(r)$ is simply a formula for powers of these fixed matrices. All noncentral entries of the displayed Heisenberg generators lie in $\{-1, 0, 1\}$.

20.4 The exact product calculation

For vectors, $\odot$ denotes componentwise multiplication. Put $a = Lx$ and $b = Ly$. Using (1) twice gives

$$A(x, u)B(y)T(r) = \left(\left(h(a_s - r_s, b_s - r_s, -u_s + a_s(b_s - r_s) + \mathfrak{b}(r_s)) \right)_{s=1}^N, Dx + Cu \right). \quad (68)$$

The sign and order in the central coordinate matter. After the first two phases the central term is $-u_s + a_s b_s$. Multiplication by the third phase adds $\mathfrak{b}(r_s) - a_s r_s$, yielding the expression shown. Reversing two phases would define a different compiler.

For $t \in \mathbb{Z}^m$ let

$$g(t) = (1_{\mathbb{H}^N}, t - c). \quad (69)$$

This is central. Define

$$\text{Fact}_t = \{(a_0, b_0, t_0) \in \mathcal{A} \times \mathcal{B} \times \mathcal{T} : a_0 b_0 t_0 = g(t)\}.$$

Theorem 20.2 (Exact fibre equivalence). *For every $t \in \mathbb{Z}^m$, the map*

$$\Phi_t : F^{-1}(t) \longrightarrow \text{Fact}_t, \quad x \longmapsto (A(x, \mathfrak{b}(Lx)), B(x), T(Lx)) \quad (70)$$

is a bijection. In particular,

$$F(x) = t \text{ has an integer solution} \iff g(t) \in \mathcal{ABT}.$$

Proof. For a factorization of $g(t)$, injectivity in Lemma 20.1 gives unique parameters $(x, u), y, r$ for its three group elements. The first two coordinates in every Heisenberg factor of (68) must vanish. Hence

$$r = Lx = Ly.$$

Since L is injective, $y = x$. The Heisenberg central coordinates in (68) now reduce to $-u + \mathfrak{b}(Lx)$, so they force

$$u = \mathfrak{b}(Lx).$$

The last m coordinates consequently say

$$Dx + C\mathfrak{b}(Lx) = t - c,$$

which is equivalent to $F(x) = t$ by (60). Every factorization is therefore exactly the displayed image of its unique x .

Conversely, if $F(x) = t$, take $y = x$, $r = Lx$ and $u = \mathfrak{b}(Lx)$. Each Heisenberg coordinate in (68) vanishes, and the abelian coordinate is $t - c$. Thus the three displayed elements multiply to $g(t)$. The two procedures are inverse, proving bijectivity. $\square$

Corollary 20.3 (An exact quadratic image in a central slice). *For $Z_0 = \{1_{\mathbf{H}^N}\} \times \mathbb{Z}^m$,*

$$(AB\mathcal{T}) \cap Z_0 = \{(1_{\mathbf{H}^N}, F(x) - c) : x \in \mathbb{Z}^n\}.$$

This is an equality of sets; Theorem 20.2 additionally describes the multiplicity of every point as an ordered factorization.

The theorem has no existential “slack” beyond the original x : the additional parameters u, y, r are uniquely recovered. This feature will allow counting, weighted and height statements without a separate normalization theorem.

21 Embedding into a pure Heisenberg power

The direct abelian factor is convenient for arithmetic but unnecessary for the main group-theoretic conclusion.

Proposition 21.1 (Two faithful embeddings). *There are explicit injective homomorphisms*

$$\begin{aligned} \mathbf{H}^N \times \mathbb{Z}^m &\hookrightarrow \mathbf{H}^{N+m}, \\ \mathbf{H}^N \times \mathbb{Z}^m &\hookrightarrow \mathbf{H}^{N+\lceil m/2 \rceil}. \end{aligned}$$

The first takes the whole abelian direct factor to the center of the ambient group. Both preserve the exact factorization fibres among the images of the three subgroups.

Proof. For the first map, send each extra integer z_j to $h(0, 0, z_j)$ in a new Heisenberg factor. For the second, pack a pair (p, q) into $h(p, 0, q)$; when m is odd, append a zero to the last pair. Since $b = 0$, the group law on each packed pair is addition. The maps are injective by inspection. If three image elements multiply to the image target, injectivity implies the same equality before applying the embedding. This proves preservation in both directions and preserves the tuple bijection. $\square$

Remark 21.2 (Centrality is not preserved by arbitrary packing). The packed element $h(p, 0, q)$ need not be central in the larger Heisenberg factor. Therefore the smaller ambient dimension is not used when claiming that targets lie in $Z(\mathbf{H}^d)$. The central-target version consistently uses $N + m$ factors. Confusing centrality in an embedded subgroup with centrality in the ambient group would give an incorrect strengthened statement.

If one wants literal matrices rather than tuples of 3×3 blocks, take a block-diagonal matrix with these blocks. This embeds $\mathbf{H}^d$ in $\text{UT}_{3d}(\mathbb{Z})$. It changes neither the number of phases nor the factorization relation.

22 Universality and the three-subgroup problem

22.1 The imported arithmetic theorem

We use classical MRDP in the following standard form. For every computably enumerable $S \subseteq \mathbb{N}$, there exist $k \in \mathbb{N}$ and an integer polynomial $P \in \mathbb{Z}[a, w_1, \dots, w_k]$ such that

$$\forall n \in \mathbb{N} : \quad n \in S \iff \exists w \in \mathbb{N}^k \ P(n, w) = 0. \quad (71)$$

The number of variables and P are fixed for S . This is the inherited arithmetic foundation, not a theorem newly proved here. A primary formal reference is the DPRM entry by Bayer, David, Stock, Pal, Matiyasevich and Schleicher [11]. The ProveIt repository has its own MRDP endpoint, with a finite polynomial and natural witnesses [12].

For the subgroup version, remove restrictions on arithmetic witness signs. Replace each w_i by

$$z_{i1}^2 + z_{i2}^2 + z_{i3}^2 + z_{i4}^2.$$

The classical four-square theorem shows that existence over the new integer variables is equivalent to existence of the original natural witnesses. This preliminary step is existence-preserving, but is not claimed to be fibre-preserving relative to w : a natural number can have several four-square representations. Fibre preservation begins with the resulting integer polynomial, and is exact from that point onward. The nonnegative submonoid construction in Section 24 avoids this preliminary multiplicity change.

22.2 The fixed-instance argument

Theorem 22.1 (Every enumerable set on a central line). *For every computably enumerable $S \subseteq \mathbb{N}$, there exist a fixed d , fixed free abelian subgroups $\mathcal{A}, \mathcal{B}, \mathcal{T} \leq H^d$, and fixed central g_0, g_1 with g_1 of infinite order such that*

$$n \in S \iff g_0 g_1^n \in \mathcal{ABT} \quad (n \in \mathbb{N}). \quad (72)$$

The three subgroups have pairwise trivial intersections.

Proof. Choose P as in (71), and perform the four-square substitution to obtain an integer polynomial $Q(a, z)$. Choose an arithmetic circuit for Q . Its free wires are a and the coordinates of z . Flatten the circuit as in Lemma 19.3, include the equation setting its final output to zero, and add the input equation $a = n$.

Let x be the tuple of all wire variables. Put the gate residuals, the output wire and the input wire into one fixed integral quadratic map $F : \mathbb{Z}^v \rightarrow \mathbb{Z}^m$, placing the input pinning equation last. The required right-hand side is

$$t(n) = ne_m.$$

The constant terms of F already include all literal gate constants; they are not varying input data. Thus

$$\exists x \in \mathbb{Z}^v \ F(x) = ne_m \iff n \in S.$$

Apply Theorem 20.2. Its three subgroups depend on F only. Its target is $(1, ne_m - c)$, which is $g_0 g_1^n$ for

$$g_0 = (1, -c), \quad g_1 = (1, e_m).$$

Both are central, and g_1 has infinite order. Use the central embedding of Proposition 21.1 into a pure power of H . Injectivity preserves ranks, pairwise intersections, and membership. This proves all claims with data fixed independently of n . $\square$

Proof of Theorem 18.1. Take S to be a fixed computably enumerable complete halting set. The map $n \mapsto g_0 g_1^n$ is computable, so (72) gives many-one hardness. Enumerating all integer exponent tuples in the three finite bases enumerates $\mathcal{ABT}$; each equality test is exact integer matrix arithmetic. Thus this fixed product is computably enumerable, and therefore computably enumerable complete. The central-line restriction is complete as well, since it is exactly S . $\square$

22.3 Why this is a resolution rather than an analogy

The published question asks whether a product of three finitely generated subgroups in a nilpotent group of class two always has decidable membership. Theorem 18.1 supplies a counterexample inside the particularly concrete class H^d , with each subgroup free abelian. It therefore directly answers that question. It does not substitute a rational subset with arbitrarily many factors, a subgroup generated by the three phases, a variable-dimensional input family, or a semigroup word problem.

The argument supplies an algorithm from any given quadratic system to its three generator lists. The attached code implements that algorithm. The existence of a suitable fixed universal starting system is supplied by MRDP, not by a numerical matrix table in this package. Consequently no specific optimized numerical value of d is claimed for the universal instance.

[write] The question answered here concerns ordered products of three *subgroups*. It is not the question left open by the repository's two-generator Heisenberg note [16], whether *submonoids* generated by three elements of a Heisenberg power can have undecidable membership; Theorem 18.1 does not settle that question (Section 1.4). [write 78] Part III settles part of it (Theorem 40.1). Theorem 22.1 also shows that the subgroup hypothesis of Part IV's rigidity theorem is necessary (Remark 46.3).

23 A matching two-phase decidability theorem

The third phase is essential for this type of universality. We give a direct proof for commuting phases in $H^d \times \mathbb{Z}^e$; it also handles nonnegative and mixed exponent restrictions.

23.1 Integral logarithmic coordinates

Define

$$\lambda(h(a, b, c)) = 2c - ab.$$

Then direct expansion of (1) gives

$$\lambda(hh') = \lambda(h) + \lambda(h') + ab' - ba'. \quad (73)$$

Two Heisenberg elements commute exactly when $ab' - ba' = 0$. Moreover,

$$a(h^r) = ra(h), \quad b(h^r) = rb(h), \quad \lambda(h^r) = r\lambda(h) \quad (r \in \mathbb{Z}). \quad (74)$$

In a direct product all these formulas apply coordinatewise.

For a phase generated by pairwise commuting $p_1, \dots, p_k$, the triple (a, b, λ) of $p_1^{r_1} \cdots p_k^{r_k}$ is therefore a linear function of the exponent vector. Generator independence is unnecessary for this observation. The extra abelian coordinates are linear as well.

Theorem 23.1 (Two commuting phases are decidable). *Let $G = H^d \times \mathbb{Z}^e$. Given two finite lists of generators, with each list pairwise commuting, and a target $g \in G$, it is decidable whether*

$$p_1^{r_1} \cdots p_k^{r_k} q_1^{s_1} \cdots q_\ell^{s_\ell} = g$$

for exponents in $\mathbb{Z}$, in $\mathbb{N}$, or with any prescribed mixture of these two domains. This remains true when d, e and the lists are part of the finite input.

Proof. Write the two products as u and v . For each Heisenberg coordinate, let (a_g, b_g, λ_g) be the target coordinates. The horizontal conditions are

$$a_u + a_v = a_g, \quad b_u + b_v = b_g. \quad (75)$$

Substituting $a_v = a_g - a_u$ and $b_v = b_g - b_u$ into (73), the central condition becomes

$$\lambda_g = \lambda_u + \lambda_v + a_u b_g - b_u a_g. \quad (76)$$

Each term on the right is linear in the phase exponents: a_g, b_g are fixed target integers, not unknowns. The extra e coordinates give further linear equations. Thus the original problem is equivalent to a finite integer linear system in the allowed exponent domains.

Conversely, if these linear equations hold, (75) gives the correct horizontal coordinates of uv , and (76) gives the correct λ . Since $2c = \lambda + ab$, this gives equality of the central coordinates as well. There is no lost parity condition: both u, v and g are already integral group elements.

An integer linear system over $\mathbb{Z}$ is decidable by Smith normal form. With nonnegative restrictions it is an existential Presburger formula, hence decidable; unrestricted integer variables can be split into differences of nonnegative variables if desired. For completeness, this last decidability can also be obtained by finite automata reading binary digits with bounded carries for fixed linear coefficients. The finite state space records carries and verifies each linear equation; intersection handles the finitely many rows, and automaton emptiness handles the existential variables. These are decision procedures, not claims of a uniform polynomial running time for all domain choices. $\square$

Remark 23.2 ([write] Credit, and what this theorem adds). For integer exponents in both phases, Theorem 23.1 is a special case of known results, which source 05 does not cite for it. König, Lohrey and Zetsche observe that a product of two subgroups of a polycyclic group is closed in the profinite topology and therefore has decidable membership [9, Remark 6.7]; Roman’kov proves that the product of two subgroups membership problem is decidable in every finitely generated nilpotent group of class two [10]. The group $H^d \times \mathbb{Z}^e$ is polycyclic and nilpotent of class at most two, and two pairwise commuting lists generate two abelian subgroups, so the integer case is covered by either result. What source 05 adds is: (i) a direct and elementary decision procedure for commuting lists, through the integral logarithmic coordinate λ , which reduces membership to one explicit integer linear system without an appeal to profinite closure; and (ii) natural and mixed exponent domains, that is, ordered products of two finitely generated commutative submonoids, or of such a submonoid and an abelian subgroup. These are not products of subgroups, so neither cited result applies to them as stated. For natural exponents the question is a restricted instance of the knapsack problem, which is decidable in H but undecidable in sufficiently large direct powers of H [9]. No literature on products of submonoids beyond [9, 10] was searched, so no priority is claimed for (ii). The coordinate λ is also the device of the repository’s two-generator Heisenberg note [16], whose commuting case is the case of (ii) with one generator in each phase (Section 1.4).

23.2 The phase threshold

For this family of groups, one commuting phase gives only a linear membership problem. Two commuting phases still linearize because the target fixes the sum of their horizontal coordinates. A third phase creates a closed horizontal triangle whose central coordinate is a quadratic function of its size. The central direct-factor generators let different such quadratic values be combined without introducing another phase.

Together, Theorems 18.1 and 23.1 give the sharp statement: *two commutative phases always have decidable membership, whereas three fixed commutative phases can have computably enumerable complete membership*. The direct proof above is specifically for commuting phases; we do not silently apply it to internally nonabelian subgroup lists.

[write] For integer exponents the decidable half of this sharp statement is not new (Remark 23.2). What source 05 adds to the comparison is the undecidable half, Theorem 18.1, and the decidable half for natural and mixed exponent domains.

24 Nonnegative exponents and a computational interpretation

24.1 Free commutative monoids

Restrict the three maps to nonnegative parameters:

$$\begin{aligned}\mathcal{A}_+ &= \{A(x, u) : x \in \mathbb{N}^n, u \in \mathbb{N}^N\}, \\ \mathcal{B}_+ &= \{B(y) : y \in \mathbb{N}^n\}, \\ \mathcal{T}_+ &= \{T(r) : r \in \mathbb{N}^N\}.\end{aligned}$$

These are monoids generated by the same finite lists (67), using only nonnegative powers. Injectivity makes them free commutative monoids. Matrix entries may be negative; it is the exponents that are nonnegative.

Theorem 24.1 (Natural fibre equivalence). *For every $t \in \mathbb{Z}^m$, restriction of (70) gives a bijection*

$$\{x \in \mathbb{N}^n : F(x) = t\} \longleftrightarrow \{(a, b, t_0) \in \mathcal{A}_+ \times \mathcal{B}_+ \times \mathcal{T}_+ : abt_0 = g(t)\}.$$

Proof. If $x \in \mathbb{N}^n$, then $Lx \in \mathbb{N}^N$, and $\mathbf{b}(Lx) \in \mathbb{N}^N$. Hence the canonical tuple lies in the three monoids. Conversely every factorization in the monoids has nonnegative parameters. The coordinate recovery in Theorem 20.2 applies without change and recovers a natural x . $\square$

Corollary 24.2 (Three fixed free commutative monoids). *There are a fixed d and three fixed free commutative submonoids of H^d whose ordered product has computably enumerable complete membership, even on a fixed central affine cyclic line.*

Proof. Use the natural-witness polynomial in (71) directly. Split it into two nonnegative-coefficient polynomials, flatten their natural arithmetic circuits, equate the outputs, and pin the input to n . Apply Theorem 24.1 and the central embedding. Completeness and computable enumeration follow as in Theorem 18.1. $\square$

24.2 Three-stage matrix reachability

One can view the monoid result as a matrix-register substrate with three control states. The first state permits the generators of $\mathcal{A}_+$, in any order and any nonnegative number. A switch moves to the second phase, which permits the generators of $\mathcal{B}_+$. A second switch moves to the last phase, which permits the generators of $\mathcal{T}_+$. Acceptance is exact equality of the final register with the supplied target. Repetition inside a phase is just addition of its exponent counters.

For a fixed set of such operations, this target-reachability problem can represent every halting input of a fixed universal machine. It uses only unitriangular integer matrices, a fixed block

dimension, and two phase switches. It does not require a machine that interprets an arbitrary input program by dynamically selecting new matrix generators.

This is a reduction-theoretic form of computational universality. There is no claim that successful exponents are efficiently obtainable from a halting time, that failures have bounded certificates, or that replacing exact equality by an error tolerance preserves the theorem. Ordinary bounded search remains possible, but no computable uniform sufficient bound exists for the universal instance; see Section 27.

25 Worked examples

25.1 A single multiplication gate

Take the quadratic equation $xy - z = t$, with variable vector $v = (x, y, z)$. A shared normal form is

$$\begin{aligned} Lv &= (x, y, z, x + y), \\ C &= \begin{pmatrix} -1 & -1 & 0 & 1 \end{pmatrix}, \\ D &= \begin{pmatrix} 0 & 0 & -1 \end{pmatrix}, \quad c = 0. \end{aligned}$$

Thus $N = 4$ and the ambient group is $H^4 \times \mathbb{Z}$. The ranks are 7, 3, 4. The first phase is

$$A(v, u) = (h(x, 0, -u_1), h(y, 0, -u_2), h(z, 0, -u_3), h(x + y, 0, -u_4); -z - u_1 - u_2 + u_4).$$

The other phases are $B(v)$ with horizontal b -coordinates $(x, y, z, x + y)$, and $T(r)$ as defined above.

For $v = (2, 3, 6)$, the canonical data are

$$r = (2, 3, 6, 5), \quad u = (1, 3, 15, 10).$$

The abelian coordinate is $-6 - 1 - 3 + 10 = 0$, and every Heisenberg coordinate vanishes after the three phases. Therefore the canonical triple multiplies to the identity. For $v = (2, 3, 5)$ it instead multiplies to target $t = 1$. The variable u_3 has coefficient zero in the abelian coordinate, but is not a free slack: the third Heisenberg block still forces $u_3 = \mathfrak{b}(z)$ exactly.

The attached `examples/multiplication.json` lists all fourteen basis matrices by their exact coordinates. The file can be regenerated from the smaller `multiplication_input.json`. ([write] Shipped as `data/05-three-phases-multiplication.json` and `data/05-three-phases-multiplication_input.json`.)

25.2 A circuit with repeated input use

Consider a circuit with free wires a, b and gates

$$w_2 = ab, \quad w_3 = a^2, \quad w_4 = w_2 + w_3.$$

The quadratic vector map, including two pinned wires, is

$$F(a, b, w_2, w_3, w_4) = (w_2 - ab, w_3 - a^2, w_4 - w_2 - w_3, a, w_4).$$

For prescribed input a_0 and output v_0 , set the target to $(0, 0, 0, a_0, v_0)$. The three-phase factorization fibre is exactly the set of integer b satisfying $a_0 b + a_0^2 = v_0$, with all gate values and auxiliary phase

coordinates uniquely determined. This illustrates the separation between fixed program coefficients and varying pinned input/output values.

For this circuit $n = 5$, only the pair $\{a, b\}$ contributes a mixed form, so $N = 6$, $m = 5$ and the phase ranks are 11, 5, 6. The central-target embedding uses eleven Heisenberg blocks; the more compact embedding uses nine, without an ambient centrality guarantee. The software supplies both the quadratic specification and the resulting generator coordinates.

25.3 Constants and impossible fibres

Constants need no new matrix interaction. For instance $F(x) = 3$ has $C = D = 0$, $c = 3$. The target is $(1, t - 3)$. All canonical triples have abelian coordinate zero, so the target is attainable exactly when $t = 3$. When it is attainable, every integer x is a distinct arithmetic witness and a distinct factorization. This simple case illustrates why the theorem preserves fibres, rather than always manufacturing a unique factorization.

26 An explicit quartic Diophantine reverse compiler

A representation should expose its arithmetic certificates in both directions. The matrix compiler has a particularly simple reverse map. For $a = Lx$ and $b = Ly$ define the integral polynomial

$$\begin{aligned} E_t(x, u, y, r) = \sum_{s=1}^N & \left[(a_s - r_s)^2 + (b_s - r_s)^2 \right. \\ & \left. + (2u_s - 2a_s(b_s - r_s) - r_s(r_s - 1))^2 \right] \\ & + \sum_{j=1}^m ((Dx + Cu)_j - (t_j - c_j))^2. \end{aligned} \quad (77)$$

Every expression being squared has degree at most two. Thus E_t has integer coefficients and total degree at most four, including when t is regarded as an input variable rather than a fixed value.

Theorem 26.1 (Quartic exactness). *For integer parameters, and also after restricting every parameter to be natural,*

$$E_t(x, u, y, r) = 0 \iff A(x, u)B(y)T(r) = g(t).$$

The zero set projects bijectively to $\{x : F(x) = t\}$ in the corresponding domain. It uses precisely $2n + 2N$ parameter coordinates and no further auxiliary variables.

Proof. Let the coordinates of the product in (68) be $(\alpha_s, \beta_s, \gamma_s)$ and $v \in \mathbb{Z}^m$. Equation (77) is the literal identity

$$E_t = \sum_s (\alpha_s^2 + \beta_s^2 + 4\gamma_s^2) + \sum_j (v_j - (t_j - c_j))^2. \quad (78)$$

Indeed the quadratic central residual in (77) is $-2\gamma_s$. A sum of nonnegative integer squares is zero exactly when every summand is zero. This gives the group equality. The fibre statement then follows from Theorems 20.2 and 24.1. $\square$

For the fixed universal instance, replace t by ne_m to obtain one fixed quartic polynomial in the ordinary input n and a fixed finite set of witness variables. This is not the assertion that one can encode all unbounded computations by a variable-free or time-bounded quartic. The fixed witness number comes from starting with one fixed MRDP polynomial and one fixed arithmetic circuit.

The construction does not improve the classical universal degree-four boundary. Its contribution is the explicit inverse interpretation, the three-phase structural restriction, and the absence of additional witness multiplicity after the chosen quadratic system.

27 Counting, weighted fibres, and height

27.1 Exact counting and the finite-fold boundary

By Theorem 20.2, for any target t ,

$$|F^{-1}(t)| = |\text{Fact}_t|,$$

including infinite cardinality. Each subgroup map is injective, so counting ordered triples of subgroup elements is identical to counting the three basis-exponent vectors. It is not identical to counting unrestricted words: commuting generators can be permuted, and group words can insert cancelling pairs without changing their elements.

If a chosen Diophantine representation has at most k witnesses for each input, the corresponding factorizations have at most k triples after a unique circuit extension and this compiler. If a chosen representation is single-fold, its compiled fibres are singletons on positive instances. These are conditional transfers. Ordinary MRDP does not supply the finite-fold hypothesis. The four-square substitution can also increase multiplicity, which is why the strongest counting statement begins after that substitution, or uses the natural monoid version directly.

27.2 Weights in the natural domain

For a canonical natural factorization, its basis-exponent length is

$$\begin{aligned} \ell(x) &= \|x\|_1 + \|\mathbf{b}(Lx)\|_1 + \|x\|_1 + \|Lx\|_1 \\ &= 2 \sum_{i=1}^n x_i + \sum_{s=1}^N \frac{(Lx)_s((Lx)_s + 1)}{2}. \end{aligned} \tag{79}$$

Thus the ordinary count by total phase length is represented by

$$Z_t(q) = \sum_{\substack{x \in \mathbb{N}^n \\ F(x)=t}} q^{\ell(x)}. \tag{80}$$

This is a well-defined formal power series: since $\ell(x) \geq 2 \sum_i x_i$, only finitely many vectors contribute to any fixed coefficient. There is no assumption of analytic convergence here. The multivariate series weighted only by the recovered source vector is exactly

$$\sum_{(a,b,t_0) \in \text{Fact}_t^+} z^{\text{source}(a,b,t_0)} = \sum_{\substack{x \in \mathbb{N}^n \\ F(x)=t}} z^x.$$

Every exponent vector has coefficient zero or one in the latter series.

For a fixed universal monoid instance, the coefficient of a specified power of q is computable by bounded enumeration. Nevertheless, testing whether the entire series is identically zero as its target varies is undecidable. This follows directly from Corollary 24.2; it does not require a claim about transcendence or asymptotics of the series.

27.3 Explicit witness heights

For an integer vector v , write $\text{ht}(v) = \|v\|_\infty$. If $\text{ht}(x) \leq H$ with integer $H \geq 1$, then

$$\text{ht}(y) \leq H, \quad \text{ht}(r) \leq 2H, \quad \text{ht}(u) \leq 2H^2 + H$$

for its canonical factorization. The last bound follows because for $-2H \leq t \leq 2H$, the maximum of $t(t-1)/2$ is $2H^2 + H$, attained at $t = -2H$. Therefore

$$\text{ht}(x) \leq \text{ht}(x, u, y, r) \leq 2H^2 + H. \quad (81)$$

The compiler itself has only quadratic height overhead relative to the chosen arithmetic witness. In the natural domain, the sharper bound $\text{ht}(u) \leq 2H^2 - H$ holds.

The coordinates of the individual group elements have comparable explicit bounds once C, D are fixed. For example, the abelian coordinate of A has absolute value at most

$$H \max_j \sum_i |D_{ji}| + (2H^2 + H) \max_j \sum_s |C_{js}|.$$

Such estimates are not bounds in the external input alone: an arithmetic witness may be vastly larger than the input.

Proposition 27.1 (No computable universal search bound). *For the fixed complete instance of Theorem 18.1, there is no total computable function $b : \mathbb{N} \rightarrow \mathbb{N}$ such that every attainable target $g_0 g_1^n$ has a phase factorization with all basis exponents of absolute value at most $b(n)$. The analogous assertion holds for the natural monoid instance.*

Proof. Such a function would give a decision procedure: compute $b(n)$, enumerate all exponent tuples in its finite box, and test their exact products. The promised bound guarantees acceptance of every attainable target, and the finite search rejects every unattainable target. This decides a computably enumerable complete set, a contradiction. $\square$

One may form a Busy-Beaver-type envelope of minimum successful factorization heights up to input n . It has no total computable majorant. No eventual domination claim is needed: the absence of any computable majorant is the precise statement established by the proof.

28 A small product invisible to every finite quotient

The universal construction is large when based on a universal polynomial. A different application gives a very small explicit example with a local-global failure.

28.1 Four generators in $H \times \mathbb{Z}$

Let $G = H \times \mathbb{Z}$ and define

$$A(x, u) = (h(x, 0, -u), x + 12u), \quad (82)$$

$$B(y) = (h(0, y, 0), 0), \quad (83)$$

$$T(r) = (h(-r, -r, \mathfrak{b}(r)), 0). \quad (84)$$

These give free abelian subgroups of ranks 2, 1, 1, generated respectively by

$$\mathcal{A} = \langle (h(1, 0, 0), 1), (h(0, 0, -1), 12) \rangle, \quad \mathcal{B} = \langle (h(0, 1, 0), 0) \rangle, \quad \mathcal{T} = \langle (h(-1, -1, 0), 0) \rangle.$$

The pairwise intersections are trivial. These are the compiler data for

$$L = (1), \quad C = (12), \quad D = (1), \quad c = 0, \quad F(x) = 12\mathfrak{b}(x) + x = 6x^2 - 5x.$$

Take $g = (h(0, 0, 0), -1)$.

Theorem 28.1 (Explicit nonclosed three-subgroup product). *The element g is not in $\mathcal{ABT}$, but its image belongs to the image of $\mathcal{ABT}$ in every finite quotient of G . Consequently $\mathcal{ABT}$ is not closed in the profinite topology.*

Proof. By Theorem 20.2, a factorization of g would require

$$6x^2 - 5x = -1,$$

or equivalently

$$(2x - 1)(3x - 1) = 0. \quad (85)$$

In the integral domain $\mathbb{Z}$, this would force $x = 1/2$ or $x = 1/3$, neither of which is an integer. Thus no factorization exists.

Next fix any integer modulus $M \geq 1$ and write $M = 2^a b$ with b odd. Choose an integer x satisfying

$$x \equiv 3^{-1} \pmod{2^a}, \quad x \equiv 2^{-1} \pmod{b},$$

where a condition modulo one is omitted. Both inverses exist. The Chinese remainder theorem gives such an x , because 2^a and b are coprime. The first factor of (85) vanishes modulo b , while the second vanishes modulo 2^a . Their product is therefore zero modulo M . Thus $F(x) \equiv -1 \pmod{M}$.

The canonical triple $A(x, \mathfrak{b}(x))B(x)T(x)$ is exactly $(h(0, 0, 0), F(x))$. It is congruent to g modulo M in every coordinate. This proves the required inclusion for the coordinate reduction quotient $G \rightarrow \text{UT}_3(\mathbb{Z}/M\mathbb{Z}) \times \mathbb{Z}/M\mathbb{Z}$.

Finally let $\varphi : G \rightarrow Q$ be any homomorphism to a finite group. Choose M divisible by the orders of all elements of $\varphi(G)$, for example $M = |\varphi(G)|$. Let

$$p = h(1, 0, 0), \quad q = h(0, 1, 0), \quad z = h(0, 0, 1).$$

The identity

$$h(a, b, c) = p^a q^b z^{c-ab}$$

shows that an element with all Heisenberg coordinates and its extra integer coordinate divisible by M lies in $\ker \varphi$. Hence the kernel of coordinate reduction modulo M is contained in $\ker \varphi$. The canonical triple constructed for this M has the same image as g under φ , proving the assertion for every finite quotient. $\square$

28.2 Interpretation and limits

The obstruction is elementary: the rational roots $1/2$ and $1/3$ can be chosen differently at different primes, although neither is an integer. The CRT construction patches these local choices for every finite modulus. The matrix compiler turns this arithmetic failure into an exact failure of profinite separability for a product of three subgroups.

This small example is not claimed to be universal. The known decidability of knapsack in $\mathbf{H} \times \mathbb{Z}^e$ is consistent with it [9]: a decision procedure need not decide nonmembership by finding a separating finite quotient. Embedding the extra integer centrally in another Heisenberg block gives the same explicit nonclosed product in $\mathbf{H}^2$.

29 Quantitative accounting and reproducible computation

29.1 Structural ledger

For a quadratic system with n variables, m equations and q distinct mixed-variable pairs, the construction has the following exact ledger.

Quantity	Value
Binomial coordinates N	$n + q$
Ambient group before packing	$\mathbf{H}^N \times \mathbb{Z}^m$
Central-target pure Heisenberg power	$\mathbf{H}^{N+m}$
Compact pure Heisenberg power	$\mathbf{H}^{N+[m/2]}$
First phase rank	$n + N$
Second phase rank	n
Third phase rank	N
Total basis exponents	$2n + 2N$
Degree of reverse SOS polynomial	at most 4
New freely varying witness coordinates over each x	0

The last row means that the auxiliary values are forced, not that their coordinate slots disappear from the polynomial.

For a circuit whose multiplication gates supply q distinct off-diagonal input pairs, the same formula applies after including all wire variables. Gate-output variables are uniquely determined but still count toward n . Literal constants affect coefficients and target offsets, not the phase count. The dimension bound is not asserted to be optimal.

29.2 What the software implements

The standard-library Python module `code/heisenberg_compiler.py` implements exact Heisenberg group arithmetic, the integral binomial normal form, all three phase maps, canonical factorizations, the quartic residual, generator bases, JSON serialization, and circuit flattening. It accepts signed coefficients and negative exponents. Its public JSON loader rejects floating-point values, Boolean values masquerading as integers, malformed dimensions, repeated cross pairs, and invalid pair indices.

A compact specification describes each quadratic row by a constant, linear coefficients, diagonal square coefficients, and a sparse list of cross coefficients. Given a specification, the program emits all generator coordinates. For example:

```
python code/heisenberg_compiler.py \
  examples/multiplication_input.json multiplication_regenerated.json
python code/verify.py
```

The verification program also regenerates the included example files.

[write] In this report the programs are shipped as `code/05-three-phases-heisenberg_compiler.py` and `code/05-three-phases-verify.py`, and the six examples and `verification_receipt.json` as `data/05-three-phases-*`. The verification program imports the compiler by its delivered name and rewrites the receipt and all six example files in place, relative to its own directory's parent; it must therefore be run on a copy with the delivered layout (see the README). It was so run at the write, under Python 3.14.4: it passed with 190,826 assertions, and all seven regenerated files equal the shipped ones apart from line endings. There is no floating-point computation, numerical tolerance, online query, or external proof-assistant dependency.

29.3 The verification receipt

The supplied deterministic run uses random seed 20261002 and records **190,826 successful exact-integer assertions**. The main test families include independent 3×3 matrix multiplication and signed powers, 80 randomly generated quadratic systems with up to five variables and four equations, generator commutation, homomorphism identities, canonical factorizations, and the off-zero quartic identity (78).

The tests also enumerate all $15^4 = 50,625$ assignments of (x, u, y, r) in $[-7, 7]^4$ for a one-variable system. Every assignment is checked both against the exact fibre converse and against the quartic equivalence; these are not tests restricted to canonical solutions. Perturbation tests change one purported binomial witness or one synchronized copy and verify rejection. Additional tests check circuit flattening, natural-domain closure and weights, two-phase linearization, and the finite-quotient example for every modulus $1 \leq M \leq 2000$.

For the nonclosed product, the unbounded nonexistence proof is (85) in the integral domain $\mathbb{Z}$, not the finite check on $[-5000, 5000]$. Likewise, finite tests cannot prove undecidability or universality. Their purpose is to expose sign, indexing, domain and implementation errors in a construction whose proof is given above.

29.4 Repository integration and trust boundary

The relevant ProveIt source interface is the MRDP development under `Computability/HilbertTenthProblem/Lean/Diophantine/`. The inspected core file describes its finite-polynomial, natural-witness endpoint and its arithmetization route [12]. The repository was changing during the audit. A confirmed commit reference used for the source read is

433df1be37188224dd00d0561bfb157949cd6320.

No full repository build was performed.

[write] The repository relationship in this section names only the MRDP core. Section 1.4 adds the related repository material: Part VI of the report [13] (Mihailova's compiler into rotation-

group membership), the research note on group commutators [14] (universal subgroup membership in $\mathrm{SL}(4, \mathbb{Z})$) and the note on two-generator Heisenberg submonoids [16] (the same coordinate $\lambda = 2c - ab$, decidability for two generators, and an open question for three generators that this Part does not answer). `Diophantine/Common/MRDPCore.lean` is unchanged from the pin to this report's write. The project's formal endpoint is the theorem `Diophantine.mrdp` with its variants `Diophantine.mrdp_iff` and `Diophantine.mrdp_dioph_iff` in `Computability/HilbertTenthProblem/Lean/Diophantine/MRDP.lean`; none of this Part's statements is formalized.

A clean integration would add a separate module for Heisenberg coordinates, a module for the three subgroup homomorphisms, and a theorem consuming an arbitrary quadratic representation. The final MRDP corollary would import the existing endpoint. Such a development would not need to reprove the repository's optimized native-stream, Pell, or arithmetic-operation constructions. It would also not inherit their numerical claims.

The attached code is an executable compiler and test suite, not a Lean formalization. The theorem statements in this article should not be marked as kernel-checked until the missing formalization is actually written and built.

30 Further research questions

30.1 Minimize the ambient group

Determine the least d for which some fixed product of three free abelian subgroups of H^d has undecidable membership. Our proof gives a constructive finite upper bound from a supplied universal quadratic system, but does not optimize it. The distinction between retaining central targets and using the compact packing is relevant. A first concrete task is to expand one well-audited universal polynomial, count its distinct mixed pairs, and emit a literal universal matrix table.

30.2 Optimize the binomial form

Our family uses coordinate forms and pair sums. More general integral forms could reduce N , but one must retain injectivity for fibre recovery. In the monoid setting, their coefficients should also preserve nonnegativity, or an alternative domain encoding must be proved. The optimization problem is a simultaneous integral decomposition of several quadratic forms with explicit injectivity and positivity constraints, not merely diagonalizing one form over the rationals.

30.3 Classify decidable triples

Not every triple generated by the compiler is universal. Identify useful conditions on C, D, L under which its quadratic fibres are decidable or have semilinear descriptions. Rank restrictions, common definiteness, disjoint interaction graphs and bounded numbers of nonlinear equations are natural candidates. Any claimed classification should include both integer and natural domains and distinguish target-dependent from fixed coefficient questions.

30.4 Machine-check the exact fibre theorem

The first formal milestone should be Theorem 20.2, independently of MRDP. A proof assistant must check integer binomial identities for negative arguments, injective homomorphisms, the ordered

product formula, and both directions of the solution bijection. Only then should one add circuit flattening and the fixed universal specialization. The profinite example provides a smaller end-to-end target including a universal statement about finite quotients.

30.5 Transfer counting without hidden multiplicity

Investigate which existing repository certificate compilers can be passed through this construction with their exact fibres intact. Some have unique bounded-history witnesses; others merely have surjective projections. Their distinction persists here. A worthwhile result would identify a large class of natural certificates with computable weighted generating functions, while preserving the boundary between computable coefficients and undecidable global nonemptiness.

30.6 Profinite closure of three-phase products

Theorem 28.1 proves nonclosure already in $H \times \mathbb{Z}$. Classify the triples of ranks $(2, 1, 1)$ for which closure does hold. More generally, determine how local solvability of the associated quadratic map controls profinite closure of its central slice. Closure of the entire product is a stronger question than closure of one chosen slice and should not be conflated with it.

30.7 Other nilpotent substrates and higher degree

The arithmetic is carried by a class-two central commutator and an integral quadratic refinement. It is natural to seek an abstract criterion on a class-two lattice that supports the same three-phase compiler. Higher unitriangular groups have higher binomial terms in powers; can they give fibre-preserving compilers for higher-degree systems with lower dimension, or improve rank bounds without increasing the number of phases?

30.8 Quantitative hardness under restrictions

For externally bounded exponents, the problem becomes finite. Determine precise complexity under binary versus unary bounds, coefficient bounds, and fixed versus variable generator lists. Fibre preservation makes exact counting reductions plausible, but a separate reduction and size analysis are required before asserting a counting-complexity classification. Similarly, study whether useful instance-specific witness bounds can be certified without contradicting Proposition 27.1.

30.9 Stable versus exact acceptance

The present result is exact and arithmetical. Replacing matrix equality by a congruence makes every fixed quotient finite; replacing it by an analytic tolerance changes the problem in a different way. Characterize which acceptance modifications preserve a useful part of the arithmetic information. The nonclosed product shows that all finite congruence tests together can still miss an obstruction, so modular approximation alone cannot be treated as a complete decision procedure.

31 Conclusion

The three-phase construction converts a quadratic system into a product of three free abelian subgroups without changing its solution fibre. The critical mechanism is visible in one Heisenberg coordinate: two phases store matching horizontal values, a third cancels both, and its binomial power term forces a unique quadratic witness. Central auxiliary generators then enforce arbitrary linear combinations of these values without a fourth phase.

Classical MRDP makes one fixed instance universal and yields a negative answer to the stated three-subgroup decidability question. Two commuting phases, in contrast, reduce to integer linear equations. The same compiler also supplies free commutative monoid versions, explicit quartic certificates, exact counting and weight transfers, and a small profinite nonclosure example.

The proof is concrete enough to audit line by line and the generator compiler is executable. The remaining validation tasks are independent mathematical review, formal verification, and a more exhaustive priority check. These tasks concern confidence and attribution; the main construction itself is given explicitly in the manuscript rather than left as an unproved reduction lemma.

Part III

Pair order in class-two nilpotent words (source 06)

[write 78] Part III prints batch-78 manuscript 09, *Order Is Not a Moment: Exact Pair-Count Geometry and Diophantine Certificates for Heisenberg Computation*, a research report prepared for Vladimir Reshetnikov, 2 October 2026, under the header “ProveIt research extension”; it names no author and is AI-assisted. Its labels carry the prefix `gts:po:`. Its Section n is Section $n + 31$ here, and its Theorem $n.m$ is Theorem $(n + 31).m$; its Appendices A and B are Appendices H and I. It was written from repository commit `4cccfa068` and could not know Parts I and II, which were written into this report after that commit; Section 1.4 and the [write 78] notes below supply the cross-references. Its Heisenberg element $H(\alpha, \beta, \gamma)$ is written $h(\alpha, \beta, \gamma)$, and one proof step it calls a “phase” is called a “stage” (Section 1.3). Its abstract and status box follow.

Abstract (source 06). A low-degree formula for the product of a word in a class-two nilpotent group is not yet a sound membership certificate: its proposed pair-order counts must come from a common word. We address this missing relation directly. For a ternary alphabet, fixing letter multiplicities and two of the three independent pair counts leaves a gapless interval of possible values of the third count. A constructive normalization argument proves this for arbitrary multiplicities. Cutting at the occurrences of one letter gives an exact six-equation quadratic certificate with $2c + 4$ natural witnesses, where c is that letter’s externally fixed multiplicity.

When $c = 2$, the interval endpoints are the minimum and maximum of four explicit bilinear corner values. Compiling these endpoints gives a single quartic polynomial with 24 natural witnesses and exactly one witness tuple on each accepted pair profile. Four letters admit the opposite behavior: after all other pair counts are fixed, the remaining spectrum can be an arithmetic progression with arbitrarily large gaps. We lift the certificates to arbitrary finite direct powers of the integer Heisenberg group, prove a supplied-multiplicity decidability result, and provide executable exact checks and a complete machine-readable quartic.

Status and scope (source 06). This is a research draft with complete mathematical arguments

and reproducible finite tests. It extends a specific ordering obstruction in the audited repository. Independent priority is not established; the results are not Lean-kernel-checked or peer-reviewed. No improvement of a universal Diophantine operation bound, and no resolution of the general single-fold representation problem, is claimed.

32 The problem and the contribution

32.1 Why a word-realizability theorem is needed

In a class-two nilpotent group, the value of a product can be collected using letter counts and ordered pair counts. This makes such groups attractive arithmetic substrates: multiplication inside an upper-unitriangular matrix already produces a bilinear term. The danger is that individually plausible pair counts need not be jointly realizable by a word. Replacing a computation by unconstrained moments can therefore introduce false accepting instances.

The immediate starting point is the repository note *Heisenberg submonoids: a paid two-generator certificate and an order obstruction* [30]. It supplies an exact binary certificate and demonstrates failures of pair bounds and weighted triangle inequalities for three generators. Its remaining issue is global word realizability. We retain that issue rather than repackage a generic bounded computation tableau. The contribution here is an exact treatment of ternary profiles, a particularly small closed treatment when one multiplicity is two, and a precise obstruction to extending the interval method to four letters.

The distinction between an arithmetic *collection identity* and a *realizability theorem* runs through the report. The identity says what a real word would evaluate to. The realizability theorem says that a proposed tuple of statistics actually has a word behind it. Only their combination is a sound membership certificate.

32.2 Main results at a glance

Let a, b, c be the multiplicities of A, B, C . Let k, p, q count scattered occurrences of AB, AC, BC , respectively. Write

$$\text{Spec}_c(a, b; p, q) = \{k \in \mathbb{N} : \text{some word has these six statistics}\}.$$

The following results are proved below, including zero-multiplicity boundary cases.

Result	Precise content
Ternary interval theorem	If $0 \leq p \leq ac$ and $0 \leq q \leq bc$, then $\text{Spec}_c(a, b; p, q)$ is a nonempty integer interval.
Gap certificate	For each externally fixed c , six quadratic equations and $2c + 4$ natural witnesses characterize the complete profile relation.
Two-separator formula	For $c = 2$, four explicitly given corners determine both endpoints; no word-length search is necessary.
Canonical quartic	A fixed polynomial in five profile parameters and 24 natural witnesses has exactly one witness tuple for each accepted $c = 2$ profile.
Four-letter obstruction	One-coordinate spectra may be $\{0, b, 2b, \dots, b \lfloor n/2 \rfloor\}$ with all five other pair counts fixed.
Heisenberg transfer	Adding $3r$ quadratic coordinate equations lifts a profile certificate to three prescribed generators in H^r .

The quartic export has 297 nonzero monomials. This is the size of one literal expanded polynomial, not an optimized arithmetic-operation bound.

32.3 Relation to the literature

The relevant statistics are established objects in combinatorics on words. Rigo and Salimov define m -binomial equivalence by equality of scattered-subword counts of length at most m [31]. Thus a complete pair profile, together with letter counts, is exactly the data of 2-binomial equivalence. Our question is realizability of partially prescribed such data, not the complexity of an infinite word.

Parikh matrices and counter-based rewriting systems provide related ways to retain order information [32]. We do not identify the present profile with one fixed ordered-alphabet Parikh matrix: the latter may also retain longer subwords. The invariants in every theorem below are explicitly listed.

[write 78] The Parikh-matrix comparison can be made exact. Parikh matrices were introduced by Mateescu, Salomaa, Salomaa and Yu [17]. For an ordered binary alphabet $X < Y$ the Parikh matrix of a word is $h(n_X, n_Y, K_{XY})$, so each factor of the recording substrate (124) is a binary Parikh matrix, and Lemma 33.1 is the description of binary Parikh matrices: $h(a, b, k)$ with natural entries is one exactly when $k \leq ab$. The Parikh matrix of an ordered ternary alphabet $A < B < C$ records $n_A, n_B, n_C, K_{AB}, K_{BC}$ and the number of scattered subwords ABC , but not K_{AC} . Theorem 35.3 therefore concerns 2-binomial data, as the paragraph above says, and not the entries of one Parikh matrix. The literature on Parikh-matrix ambiguity and ternary M -equivalence (see [32]) was not searched for the interval theorem at this write; no priority is claimed for it here either.

On the group side, Roman'kov proves undecidable membership for a fixed finitely generated submonoid of a sufficiently large finite direct power of the Heisenberg group [33]. This gives a substantive reason to study these arithmetic substrates. In a different direction, Shafrir proves bounded generation when the commutator subgroup has Hirsch length one, and obtains membership decidability for commutator Hirsch length two via a reduction of Bodart [34]. Those ambient-group hypotheses do not cover arbitrary H' . Our results restrict word statistics or the multiplicity of one generator instead.

The literature search underlying this draft did not establish a priority claim for the interval theorem, the four-corner formulation, or the particular compiled polynomial. They are presented as self-contained derivations addressing the repository's stated gap, not as certified first discoveries. The mathematical statements do not depend on their historical novelty.

33 Words, pair counts, and the binary base case

33.1 Conventions

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$. Every word is finite; the empty word is allowed. For distinct letters X, Y , define

$$K_{XY}(w) = \#\{(i, j) : 1 \leq i < j \leq |w|, w_i = X, w_j = Y\}.$$

These are *scattered pairs*, not adjacent bigrams. If n_X is the multiplicity of X , then

$$K_{XY} + K_{YX} = n_X n_Y, \quad K_{XX} = \binom{n_X}{2}. \quad (86)$$

Consequently, for d letters it is enough to specify their multiplicities and the $\binom{d}{2}$ pair counts for one orientation of each unordered pair.

For the alphabet $\{A, B, C\}$, use the coordinate order

$$(a, b, c; k, p, q) = (n_A, n_B, n_C; K_{AB}, K_{AC}, K_{BC}). \quad (87)$$

Relabeling the alphabet transfers every ternary statement to any choice of the one unspecified pair count.

Lemma 33.1 (Binary realization). *A word with a copies of A , b copies of B , and k scattered AB pairs exists if and only if $0 \leq k \leq ab$.*

Proof. The bounds are necessary. If $a = 0$, necessarily $k = 0$, and B^b works. For $a > 0$, write $k = ta + s$ with $0 \leq s < a$. If $t = b$, the upper bound forces $s = 0$, and $A^a B^b$ works. Otherwise $t < b$, and the word

$$B^{b-t-1} A^s B A^{a-s} B^t \quad (88)$$

has the required multiplicities and pair count $s(t+1) + (a-s)t = ta + s$. All exponents are nonnegative. $\square$

This lemma is also the basic realization fact used in the repository note [30]. We include the construction because it will supply the completeness direction of the new gap compiler.

33.2 Certificates and the meaning of uniqueness

A natural-number Diophantine certificate for a relation $R(\mathbf{t})$ is a finite collection of integer-coefficient polynomial equations

$$F_i(\mathbf{t}, \mathbf{z}) = 0, \quad \mathbf{z} \in \mathbb{N}^s,$$

whose solvability is equivalent to R . A sum of squares converts the collection to one equation without changing its integer zero set. When every F_i is at most quadratic, that equation has degree at most four.

A certificate is called *single-fold* here when, for each fixed admissible parameter tuple, exactly one complete natural witness tuple solves it, and no tuple solves it on a rejected input. This is a property of the particular relation and its auxiliary variables. It does not say that only one word realizes the statistics. Nor does uniqueness survive an arbitrary existential projection of some relation parameters.

34 An exact gap representation

34.1 Cutting at a distinguished letter

Write a ternary word uniquely as

$$w = w_0 C w_1 C \cdots C w_c, \quad w_i \in \{A, B\}^*. \quad (89)$$

Let $x_i = n_A(w_i)$ and $y_i = n_B(w_i)$. An A in gap i precedes exactly $c - i$ copies of C ; the same is true for B . Hence

$$\sum_{i=0}^c x_i = a, \quad \sum_{i=0}^c y_i = b, \quad (90)$$

$$\sum_{i=0}^c (c - i)x_i = p, \quad \sum_{i=0}^c (c - i)y_i = q. \quad (91)$$

For such gap vectors define

$$L(x, y) = \sum_{0 \leq i < j \leq c} x_i y_j, \quad D(x, y) = \sum_{i=0}^c x_i y_i, \quad U(x, y) = L(x, y) + D(x, y). \quad (92)$$

The first quantity counts forced cross-gap AB pairs. The second is the capacity for within-gap AB pairs.

Proposition 34.1 (Exact interval for fixed gaps). *For fixed nonnegative vectors x, y , the possible values of K_{AB} in words of the form (89) with these gap vectors are precisely*

$$[L(x, y), U(x, y)] \cap \mathbb{Z}.$$

Proof. If $k_i = K_{AB}(w_i)$, then

$$K_{AB}(w) = L(x, y) + \sum_{i=0}^c k_i, \quad 0 \leq k_i \leq x_i y_i.$$

By Lemma 33.1, each k_i independently fills its entire interval. The sum of integer intervals $[0, d_i]$ is $[0, \sum d_i]$. Explicitly, given u in that interval, allocate $k_i = \min(u_{\text{remaining}}, d_i)$ successively. The remaining amount reaches zero, and (88) realizes each allocated value. $\square$

Theorem 34.2 (Six-equation compiler). *Fix $c \in \mathbb{N}$ externally. The profile $(a, b, c; k, p, q)$ is realizable if and only if there exist*

$$x_0, \dots, x_c, y_0, \dots, y_c, u, v \in \mathbb{N}$$

satisfying the following six equations:

$$\sum x_i - a = 0, \quad \sum y_i - b = 0, \quad (93)$$

$$\sum (c - i)x_i - p = 0, \quad \sum (c - i)y_i - q = 0, \quad (94)$$

$$k - L(x, y) - u = 0, \quad D(x, y) - u - v = 0. \quad (95)$$

There are $2c + 4$ witnesses. All residuals have degree at most two, and their sum of squares is a quartic polynomial.

Proof. A real word supplies its gap vectors. The difference $u = k - L$ is its total within-gap pair count, and $v = D - u$ is the unused capacity, so every witness is natural. Conversely, the first four equations recover the prescribed multiplicities and moments. The last two say $k = L + u$ with $0 \leq u \leq D$. Proposition 34.1 produces a word with the entire requested profile.

For the degree statement, L, D are quadratic. Their square-sum has degree exactly four: the last residual contains the nonzero degree-two term $x_0 y_0$, and a sum of squares of real homogeneous forms cannot vanish identically unless every form does. This also covers $c = 0$. $\square$

34.2 What has and has not been compressed

The compiler does not unroll all $a + b + c$ letter positions. Its arity depends only on the supplied number of separators. Given the gap witnesses, L can be evaluated with a prefix sum:

$$s_0 = 0, \quad L = \sum_{j=0}^c s_j y_j, \quad s_{j+1} = s_j + x_j.$$

Thus an evaluation circuit can use $O(c)$ arithmetic operations, even though the expanded expression for L has $O(c^2)$ potential terms. The prefix values are computed registers, not extra supplied witnesses in Theorem 34.2. No optimized literal gate count is asserted.

The theorem is a family of finite polynomials indexed by c . When c varies, the number of variables varies. Merely writing c in the notation does not turn this family into one fixed-arity polynomial. Also, different gap vectors can represent the same pair profile. The six-equation certificate is not single-fold in general.

34.3 Nonemptiness of a moment fibre

For $a, c \in \mathbb{N}$, any integer $p \in [0, ac]$ admits an A -gap vector satisfying (90)–(91). To see this for $a > 0$, put $S = ac - p$ and divide $S = ar + s$, $0 \leq s < a$. Give $a - s$ occurrences gap index r and s occurrences gap index $r + 1$. If $r = c$, then $s = 0$, so no index exceeds c . The case $a = 0$ has $p = 0$ and the zero vector. Construct y independently. Hence the bounds on p, q are sufficient for the existence of at least one word with those two pair counts, although they do not determine which k values are possible.

35 The ternary no-holes theorem

35.1 A connected normalization graph

Fix a, b, c, p, q with $p \leq ac$ and $q \leq bc$. Let $\mathcal{W}$ be the finite set of words with those multiplicities and those two pair counts. We define transformations of $\mathcal{W}$ that preserve p, q and change K_{AB} by at most one.

The first transformation swaps adjacent A, B letters inside a gap. It preserves all gap multiplicities and changes K_{AB} by exactly one. The second performs a pair of opposite crossings of a distinguished letter past two different copies of C . One crossing moves the letter from gap i to $i + 1$, and the other from gap j to $j - 1$, where $j - i \geq 2$. The two crossings are treated as one transformation; they preserve the letter's gap-index sum. They also preserve the entire A/B projection of the word, so K_{AB} is unchanged.

The second transformation can always be enabled when the relevant extreme gaps contain the chosen letter. Move the last such letter in gap i to the right boundary using adjacent A/B swaps, and the first such letter in gap j to the left boundary. The two boundary copies of C are distinct. Perform the two letter- C swaps and take the resulting word as the next vertex. The intermediate word after only the first crossing is not asserted to belong to $\mathcal{W}$.

Lemma 35.1 (Balancing one letter). *Keeping the other letter's gap multiplicities fixed, these transformations connect any A -gap vector with total a and gap-index sum $S = ac - p$ to a uniquely determined balanced vector. For $a > 0$, if $S = ar + s$, $0 \leq s < a$, that vector has $a - s$ occurrences in gap r and s in gap $r + 1$; all other entries vanish. The same statement holds for B .*

Proof. If occupied gap indices $i < j$ differ by at least two, use the paired crossing described above. On the A -gap vector it acts as

$$x \mapsto x - e_i - e_j + e_{i+1} + e_{j-1},$$

where coincident middle indices are counted twice. It preserves total count and first moment. The nonnegative integer potential

$$\Phi_A(x) = \sum_{i=0}^c i^2 x_i$$

decreases by

$$i^2 + j^2 - (i+1)^2 - (j-1)^2 = 2(j-i-1) > 0.$$

Consequently the process terminates. At termination the occupied indices are contained in two adjacent positions. The total and first moment then force exactly the vector stated. If $a = 0$, the zero vector is already unique. All preparation swaps take place within gaps, and crossing an A past C does not change any B -gap index. Thus the other letter's gap vector stays fixed throughout. $\square$

Lemma 35.2 (Connectedness). *The graph on $\mathcal{W}$ generated by within-gap A/B swaps and paired crossings is connected. Its edges change K_{AB} by at most one.*

Proof. Balance the A occurrences using Lemma 35.1, then balance the B occurrences. The second stage preserves the balanced A -gap vector. Finally sort each binary gap to $A^{x_i}B^{y_i}$ by adjacent swaps. The resulting word depends only on a, b, c, p, q : both balanced vectors are uniquely determined. Every word therefore has a path to the same vertex. Reverse paths are allowed, proving connectedness. The changes in K_{AB} were computed when defining the transformations. $\square$

The normal form is a connectivity device, not an extremizer. In particular, no inference that it minimizes or maximizes K_{AB} is used.

35.2 The theorem and exact endpoint optimization

Define the finite sets

$$\mathcal{G}_c(a, p) = \left\{ x \in \mathbb{N}^{c+1} : \sum x_i = a, \quad \sum (c-i)x_i = p \right\}.$$

For feasible p, q , put

$$\mathcal{L}_c(a, b; p, q) = \min_{\substack{x \in \mathcal{G}_c(a, p) \\ y \in \mathcal{G}_c(b, q)}} L(x, y), \quad \mathcal{U}_c(a, b; p, q) = \max_{\substack{x \in \mathcal{G}_c(a, p) \\ y \in \mathcal{G}_c(b, q)}} U(x, y). \quad (96)$$

Theorem 35.3 (Ternary interval theorem). *For all natural a, b, c, p, q with $p \leq ac$ and $q \leq bc$,*

$$\text{Spec}_c(a, b; p, q) = [\mathcal{L}_c(a, b; p, q), \mathcal{U}_c(a, b; p, q)] \cap \mathbb{Z}. \quad (97)$$

Outside those two moment bounds, the spectrum is empty. In particular, fixing all letter counts and any two independent pair counts over a ternary alphabet leaves no internal holes in the third pair count.

Proof. The moment bounds are necessary, and their sufficiency for nonemptiness was proved above. Since $\mathcal{W}$ is finite and nonempty, the integer function $w \mapsto K_{AB}(w)$ has a minimum and maximum. Choose words attaining them and a graph path between those words, using Lemma 35.2. A sequence of integers changing by at most one cannot pass from its minimum to its maximum while skipping an intervening integer. Hence its full image is an interval.

By Proposition 34.1, the minimum over all words is exactly the minimum of L over all feasible gap vectors, and the maximum is the maximum of U . This identifies the endpoints with (96). Relabeling A, B, C gives the last assertion. $\square$

Remark 35.4 (What the theorem does not say). The full three-dimensional set of feasible (k, p, q) need not consist of every lattice point passing elementary box or triangle inequalities. The theorem concerns one-dimensional slices after two pair counts are fixed. It does not assert convexity of the full profile set, a finite universal inequality description, or efficient computation of (96) when c is large and binary-encoded.

35.3 Constructive recovery versus succinct decision

The proof gives more than existential connectedness. It gives an explicit normalization path. An extremal pair of words can be connected through the common normal form, and the first time the path reaches a desired intermediate pair count produces a realizing word. Alternatively, the gap compiler provides a direct construction through binary blocks.

Both constructions can be expensive in the bit length of the numeric input: a word itself has $a + b + c$ letters. A polynomial-time decision procedure on binary-encoded counts should not be confused with a procedure printing that entire word in polynomial time. The next section gives succinct decision for $c = 2$ without making the latter claim.

36 Two separators: four corners give the exact answer

36.1 Eliminating the gap equations

Set $c = 2$ and assume

$$0 \leq p \leq 2a, \quad 0 \leq q \leq 2b. \quad (98)$$

There are three gaps. Put $x = x_0$ and $y = y_0$. Solving the two total and moment equations for each letter gives

$$(x_0, x_1, x_2) = (x, p - 2x, a - p + x), \quad (y_0, y_1, y_2) = (y, q - 2y, b - q + y). \quad (99)$$

Nonnegativity is equivalent to the independent integer intervals

$$x_- := \max(0, p - a) \leq x \leq x_+ := \lfloor p/2 \rfloor, \quad (100)$$

$$y_- := \max(0, q - b) \leq y \leq y_+ := \lfloor q/2 \rfloor. \quad (101)$$

The domain bounds ensure that these intervals are nonempty, including when a or b is zero.

Substitution into (92) produces the bilinear polynomials

$$L_2(x, y) = bp - bx - pq + py + 2qx - 3xy, \quad (102)$$

$$U_2(x, y) = ab - aq + ay + pq - 2py - qx + 3xy. \quad (103)$$

For example, the lower expression follows immediately from

$$x(y_1 + y_2) + x_1 y_2 = x(b - y) + (p - 2x)(b - q + y).$$

The upper expression is ab minus the number of forced cross-gap BA pairs.

Lemma 36.1 (Bilinear rectangle extrema). *The minimum and maximum of an integer bilinear function $F(x, y) = \alpha xy + \beta x + \gamma y + \delta$ on a nonempty integer rectangle occur at corners of that rectangle.*

Proof. For fixed y , F is affine in x , so one endpoint of the x interval is at least as good as a chosen point for minimization. With that endpoint fixed, choose a minimizing endpoint of the y interval. The same reasoning applies to maximization. Degenerate intervals cause no difficulty. $\square$

Theorem 36.2 (Four-corner characterization). *For natural parameters a, b, p, q, k , a word with profile $(a, b, 2; k, p, q)$ exists if and only if (98) holds and*

$$\min_{\substack{x \in \{x_-, x_+\} \\ y \in \{y_-, y_+\}}} L_2(x, y) \leq k \leq \max_{\substack{x \in \{x_-, x_+\} \\ y \in \{y_-, y_+\}}} U_2(x, y). \quad (104)$$

Proof. Equations (99)–(101) parametrize every feasible pair of gap vectors without loss. Lemma 36.1 computes the minimum lower endpoint and maximum upper endpoint in (96). Theorem 35.3 fills every integer between these possibly differently attained extrema. Necessity of all conditions follows from the gap representation. $\square$

The appeal to the interval theorem is essential. Optimizing the lower and upper endpoint separately does not, by itself, prove that the union of all intermediate gap intervals has no holes.

36.2 Worked example and the repository obstruction

Take $(a, b, p, q) = (3, 2, 2, 1)$. Then $x \in [0, 1]$ and $y = 0$. Equations (102)–(103) reduce to

$$L_2(x, 0) = 2, \quad U_2(x, 0) = 5 - x.$$

The exact spectrum is $\{2, 3, 4, 5\}$, with the following explicit representatives:

k	Word	K_{AC}	K_{BC}
2	CBAACBA	2	1
3	CABACBA	2	1
4	CAABCBA	2	1
5	CAABCAB	2	1

More generally, for $a = m \geq 3$, $b = 2$, $p = 2$, $q = 1$, the same calculation yields

$$\text{Spec}_2(m, 2; 2, 1) = \{2, 3, \dots, m + 2\}. \quad (105)$$

In the cyclic convention (K_{AB}, K_{BC}, K_{CA}) , the repository's strict-interior false family is $(1, 1, 2m - 2)$ with counts $(m, 2, 2)$ [30]. Since $K_{AC} = 2m - K_{CA} = 2$, it asks for $k = 1$ in (105) and is rejected by the exact lower bound 2. This derives an entire conditional spectrum, not just a contradiction for that one proposed value.

36.3 Zero and one separator

For $c = 0$, the exact conditions are $p = q = 0$ and $0 \leq k \leq ab$. One natural slack s gives a canonical three-equation presentation

$$p = 0, \quad q = 0, \quad ab - k - s = 0.$$

For $c = 1$, there is no remaining gap-vector choice:

$$(x_0, x_1) = (p, a - p), \quad (y_0, y_1) = (q, b - q).$$

Thus the conditions are

$$p \leq a, \quad q \leq b, \quad p(b - q) \leq k \leq ab - (a - p)q. \quad (106)$$

Assigning a natural slack to each of these four inequalities gives four quadratic equations with four uniquely determined witnesses. The qualitative change at $c = 2$ is the appearance of nontrivial endpoint optimization; it is still only a four-corner calculation.

36.4 Bit complexity

For $c = 2$, domain checks, two divisions by two, four lower values, four upper values, and two comparisons decide the relation. The number of arithmetic operations is constant, independently of $a + b$. If the binary input length is B , every intermediate value has $O(B)$ bits: all expressions involve only a fixed number of additions and products of at most two input-sized quantities. Schoolbook arithmetic therefore gives a deterministic $O(B^2)$ bit-time bound, with no attempt to optimize constants. Constructing the canonical witnesses below has the same bound.

The general gap enumeration in the companion program is deliberately different: it is a small-instance construction and diagnostic routine, not a polynomial-in-bit-length algorithm for variable c .

37 A canonical 24-witness quartic

37.1 A uniquely forced sign-split gadget

For an integer-valued expression d , introduce natural variables s^+, s^- . The equations

$$s^+ - s^- - d = 0, \quad s^+ s^- = 0 \quad (107)$$

have exactly one solution:

$$s^+ = \max(d, 0), \quad s^- = \max(-d, 0).$$

Indeed at least one variable is zero, and the difference determines the other. If $d = 0$, both are zero. When d is at most quadratic, both residuals remain at most quadratic. This gadget can compare quadratic expressions without introducing an inequality or leaving unconstrained selector choices at ties.

For a natural integer t , a canonical floor-half gadget uses natural f, e with

$$t - 2f - e = 0, \quad e(e - 1) = 0. \quad (108)$$

The second equation forces $e \in \{0, 1\}$, and the first then forces $f = \lfloor t/2 \rfloor$ and the remainder e .

37.2 The construction as literal polynomial expressions

All relation parameters (a, b, p, q, k) and all supplied witnesses are natural. First use two copies of (107) to force

$$x_- = (p - a)_+, \quad y_- = (q - b)_+,$$

retaining both the positive and negative parts in each copy. Use (108) twice to force $x_+ = \lfloor p/2 \rfloor$ and $y_+ = \lfloor q/2 \rfloor$. Add two natural bound slacks s_p, s_q with

$$2a - p - s_p = 0, \quad 2b - q - s_q = 0. \quad (109)$$

At this point there are ten witnesses and ten residuals.

List the corners in the fixed order

$$(x_-, y_-), (x_-, y_+), (x_+, y_-), (x_+, y_+),$$

and let $L_1, \dots, L_4$ and $U_1, \dots, U_4$ be their values under (102) and (103). These are quadratic expressions in existing parameters and witnesses, not additional variables.

Set $m_0 = L_1$, $h_0 = U_1$. For $j = 1, 2, 3$, use sign-split witnesses λ_j^+, λ_j^- and μ_j^+, μ_j^- satisfying

$$\lambda_j^+ - \lambda_j^- - (m_{j-1} - L_{j+1}) = 0, \quad \lambda_j^+ \lambda_j^- = 0, \quad (110)$$

$$\mu_j^+ - \mu_j^- - (h_{j-1} - U_{j+1}) = 0, \quad \mu_j^+ \mu_j^- = 0, \quad (111)$$

and define expressions

$$m_j = m_{j-1} - \lambda_j^+, \quad h_j = h_{j-1} + \mu_j^-. \quad (112)$$

On every natural solution,

$$m_j = \min(L_1, \dots, L_{j+1}), \quad h_j = \max(U_1, \dots, U_{j+1}).$$

The symbols m_j, h_j in these formulas denote substitutions into expressions. They are not existentially quantified quantities and do not add equations. For example, $m_2 = L_1 - \lambda_1^+ - \lambda_2^+$ remains quadratic.

Finally introduce $s_-, s_+ \in \mathbb{N}$ and require

$$k - m_3 - s_- = 0, \quad h_3 - k - s_+ = 0. \quad (113)$$

This completes the construction. No minimum, maximum, floor operation, or implicit order predicate appears after the specified substitutions into its 24 polynomial residuals.

37.3 The exact ledger

Component	Natural witnesses	Residuals
Positive/negative parts of $p - a$ and $q - b$	4	4
Quotients and remainders for $p/2, q/2$	4	4
Moment domain slacks	2	2
Three comparisons for the minimum of four L_i	6	6
Three comparisons for the maximum of four U_i	6	6
Lower and upper endpoint slacks	2	2
Total	24	24

The corners are computed expressions using existing symbols. The ledger counts the complete supplied witness tuple, including negative parts, parity remainders, and domain slacks. It does not count arithmetic operations, and should not be compared directly to the repository's operation ledgers.

Theorem 37.1 (Explicit single-fold quartic). *Let $F_1, \dots, F_{24}$ be the residuals just specified, with (112) substituted rather than existentially quantified, and put*

$$P(a, b, p, q, k; \mathbf{z}) = \sum_{i=1}^{24} F_i^2. \quad (114)$$

Then P is an integer-coefficient polynomial of total degree exactly four. For every $(a, b, p, q, k) \in \mathbb{N}^5$,

$$\exists! \mathbf{z} \in \mathbb{N}^{24} : P(a, b, p, q, k; \mathbf{z}) = 0$$

if the profile $(a, b, 2; k, p, q)$ is realizable, and no such tuple exists otherwise. The accompanying export gives the complete expanded polynomial and each residual separately.

Proof. A zero of P over the integers is exactly a simultaneous zero of all F_i , since integer squares are nonnegative. The initial split gadgets and floor gadgets uniquely recover the four corners, and the domain slacks enforce (98). Each comparison gadget then forces its two split variables uniquely, in the displayed order. Induction in j proves the asserted values of m_j, h_j . The final equations are equivalent to $m_3 \leq k \leq h_3$, with uniquely forced endpoint slacks. Theorem 36.2 proves soundness.

Conversely, a realizable profile satisfies the domain conditions and the exact endpoint bounds. Assign all initial split and floor variables their stated integer values, then evaluate the six comparison gadgets in order, and finally assign the nonnegative endpoint gaps. Every residual vanishes. The same forced order proves uniqueness, including tied corner values and zero bounds.

Each initial residual is at most quadratic. Each corner value is quadratic in parameters and the already introduced variables. Subtracting linear combinations of split variables does not increase this degree, so all comparison and final residuals remain at most quadratic. Thus P has degree at most four. Its degree-four homogeneous part is a sum of squares of the degree-two homogeneous parts of the residuals. At least one such part is nonzero, for example the product of the two variables in an initial split gadget. A sum of squares of nonzero real polynomial forms cannot be the zero polynomial. Hence the degree is exactly four. $\square$

[write 78] (2 October 2026.) The research programme's review of this manuscript [19] confirms the construction and proves a cheaper finalizer for the same residuals, numbered $F_0, \dots, F_{23}$ there. The eight sign-split products uv (indices 1, 3, 11, 13, 15, 17, 19, 21) take natural values on natural inputs. Keeping them unsquared and squaring the other sixteen preserves the complete natural zero set and its unique witness tuple, stays nonnegative on the whole real nonnegative orthant, keeps degree four, has 289 monomials and saves eight squaring multiplications. Leaving the two parity residuals $e(e-1)$ (indices 5, 7) unsquared as well gives 287 monomials and saves ten, but then only natural-zero equivalence survives: with $a = 1/4$, $p = 1/2$, $\text{xlower_plus} = 1/4$, $\text{p_rem} = 1/2$ and all other coordinates zero, that polynomial equals $-1/4$. Neither variant removes a witness or changes Theorem 37.1; the shipped export `data/06-pair-order-canonical_c2_quartic.json` is the unreduced polynomial (114).

37.4 Why more witnesses can be useful

For $c = 2$, the gap certificate already uses only eight witnesses and six residuals. The canonical compiler deliberately uses more. Its benefits are deterministic witness generation from binary-encoded parameters, elimination of gap-vector search, and exactly one full arithmetic witness on each accepted profile. It is not a witness-count improvement over the six-equation construction.

Profile certificate	Witnesses	Residuals	Multiplicity
General c gap certificate	$2c + 4$	6	Possibly many
$c = 0$ direct slacks	1	3	Exactly one
$c = 1$ direct slacks	4	4	Exactly one
$c = 2$ gap specialization	8	6	Possibly many
$c = 2$ endpoint compiler	24	24	Exactly one

The witness tuple is canonical only as an arithmetic tuple. Several words may share its profile, and the tuple need not choose one of them. This avoids concealing a separate canonical-word problem inside the single-fold assertion.

37.5 Positive witnesses and signed arithmetic

The natural witness convention is not optional: (107) is not a correct sign-decomposition gadget over unrestricted integer witnesses. To use the repository's strictly positive convention, replace each z_i by $\hat{z}_i - 1$ and require $\hat{z}_i > 0$. This gives a bijection between the complete zero sets and preserves total degree and single-foldness. Any runtime subtractions in a counted circuit must still be charged.

Replacing each natural witness by a sum of four integer squares would give an unrestricted-integer presentation but generally destroy uniqueness through the nonunique square representations. No such substitution is used in the advertised 24-witness result.

37.6 The supplied polynomial export

The file `data/canonical_c2_quartic.json` lists parameter and witness names, all 24 residuals, and the expanded P . Each monomial is represented by its sorted list of variable names, with repetitions denoting powers, and an integer coefficient. The empty list denotes the constant monomial. Coefficients are exact integers, not floating-point approximations.

The standard-library implementation combines equal monomials, discards zero coefficients, and makes an immutable owned copy of its polynomial data. Its export has degree four and 297 nonzero terms. Independent signed off-zero evaluations in the test suite compare the expanded output to the sum of the residual squares; these checks supplement the symbolic construction rather than replace it.

38 Four letters: exact congruence holes

38.1 An infinite obstruction family

The ternary interval theorem is sharp in alphabet size. The following family fixes every pair count except K_{AB} .

Theorem 38.1 (Arbitrarily large pair-count gaps). *Let $n \in \mathbb{N}$ and $b \geq 1$. Consider words with multiplicities*

$$(n_A, n_B, n_C, n_D) = (n, b, 2, 2)$$

and with the fixed pair counts

$$(K_{AC}, K_{AD}, K_{BC}, K_{BD}, K_{CD}) = (n, n, b, 2b, 3). \quad (115)$$

The exact spectrum of the remaining count is

$$\{K_{AB}\} = \{bj : 0 \leq j \leq \lfloor n/2 \rfloor\}. \quad (116)$$

For every such j , a realizing word is

$$A^j C B^b D A^{n-2j} C D A^j. \quad (117)$$

Proof. Since $K_{BD} = 2b$ is maximal, every B precedes both copies of D . The C/D projection has two copies of each letter and $K_{CD} = 3$; its unique form is $CDCD$. For completeness, the six projections $CCDD, CDCD, CDDC, DCCD, DCDC, DDCC$ have $K_{CD} = 4, 3, 2, 2, 1, 0$, respectively.

All B letters precede the first D and therefore also precede the second C . These latter pairs already contribute b to K_{BC} . The requirement $K_{BC} = b$ leaves no B before the first C . After deleting A , the word is consequently forced to be

$$C B^b D C D. \quad (118)$$

For an occurrence of A , let h_C, h_D be the numbers of C, D before it. Along this skeleton the possible height pairs are

$$(0, 0), \quad (1, 0), \quad (1, 1), \quad (2, 1), \quad (2, 2),$$

where the $(1, 0)$ height includes the gaps inside the B block. All satisfy $h_C \geq h_D$.

Since A has two possible following C letters,

$$K_{AC} = 2n - \sum_A h_C, \quad K_{AD} = 2n - \sum_A h_D.$$

The two equalities in (115) therefore force $\sum_A h_C = \sum_A h_D = n$. Every individual difference $h_C - h_D$ is nonnegative, so each must be zero. Thus A can occur only in the three equal-height gaps $(0, 0), (1, 1), (2, 2)$.

Let z_0, z_1, z_2 count the A letters in those three gaps. Then

$$z_0 + z_1 + z_2 = n, \quad z_1 + 2z_2 = n,$$

so $z_0 = z_2 = j$ and $z_1 = n - 2j$, with $0 \leq j \leq \lfloor n/2 \rfloor$. Only the first group of A letters precedes any B , so $K_{AB} = bj$. The forced form is precisely (117); direct counting proves the converse. $\square$

$$\begin{array}{ccccccc} A^j & C & B^b & D & A^{n-2j} & CD & A^j \\ \hline & & & & & & \\ \text{height } (0, 0) & & & \text{height } (1, 1) & & \text{height } (2, 2) & \end{array}$$

Only the leftmost A block contributes to K_{AB} ; the two outer block sizes must agree. This is the source of the arithmetic progression.

38.2 The smallest displayed hole and a convexity consequence

With $n = b = 2$, all four letters occur twice and the exact K_{AB} spectrum is $\{0, 2\}$. Representatives are

$$\text{CBBDAACD} \quad (k = 0), \quad \text{ACBBDCDA} \quad (k = 2).$$

There is no representative with $k = 1$ and the same other five pair counts. The claim here is a small explicit example, not a proof of global minimality by total word length.

Corollary 38.2 (An integral hole in the convex hull). *For the fixed multiplicities $(2, 2, 2, 2)$, the realizable complete pair profiles cannot be characterized by any conjunction of affine real inequalities and equalities in the six pair coordinates alone, followed only by restricting those coordinates to integers.*

Proof. The feasible profiles with $k = 0$ and $k = 2$ and other coordinates $(2, 2, 2, 4, 3)$ have an integral midpoint with $k = 1$. Every affine inequality or equality valid at both endpoints is valid at their midpoint. The midpoint is not realizable by Theorem 38.1. The argument even allows an infinite conjunction. $\square$

This does not exclude nonlinear constraints, congruence conditions, auxiliary integer variables, or a disjunctive description. It pinpoints exactly why a convex pair-profile relaxation is insufficient.

38.3 Why the ternary proof breaks

With one separator species, a letter's order relative to the separators is encoded by a single height. Paired crossings preserve that one moment and can balance its distribution. With two separator species, an occurrence has a height vector, and fixing all other pair counts imposes several simultaneous restrictions. In (118), the inequality $h_C \geq h_D$ and equality of their sums force every occurrence onto the diagonal heights. Transfers that would fill an intermediate K_{AB} value leave that allowed diagonal. The new obstruction is not a loose endpoint estimate; the missing values are genuinely absent.

39 Transfer to Heisenberg computational substrates

39.1 The collection identity with every orientation paid

Use coordinates

$$h(\alpha, \beta, \gamma) = \begin{pmatrix} 1 & \alpha & \gamma \\ 0 & 1 & \beta \\ 0 & 0 & 1 \end{pmatrix}, \quad \alpha, \beta, \gamma \in \mathbb{Z}.$$

Matrix multiplication gives

$$h(\alpha, \beta, \gamma)h(\alpha', \beta', \gamma') = h(\alpha + \alpha', \beta + \beta', \gamma + \gamma' + \alpha\beta'). \quad (119)$$

[write 78] This is the multiplication law (1) of Section 1.2 in source 06's Greek coordinates; it is kept here because this Part refers to it. Fix three labelled generators $g_1, g_2, g_3 \in \mathbf{H}^r$. Equal group elements may carry different labels; multiplicities count labels. In factor s , their coordinates are

$(\alpha_{si}, \beta_{si}, \gamma_{si})$, $i = 1, 2, 3$. These are fixed signed integer coefficients, not nonnegative witnesses. For a word in the generators, let

$$(n_1, n_2, n_3) = (a, b, c), \quad K_{12} = k, \quad K_{13} = p, \quad K_{23} = q.$$

Define the reverse and diagonal pair counts by (86). If the target's factor- s coordinates are (A_s, B_s, C_s) , repeated use of (119) gives

$$A_s = \sum_{i=1}^3 \alpha_{si} n_i, \tag{120}$$

$$B_s = \sum_{i=1}^3 \beta_{si} n_i, \tag{121}$$

$$C_s = \sum_{i=1}^3 \gamma_{si} n_i + \sum_{i,j=1}^3 \alpha_{si} \beta_{sj} K_{ij}. \tag{122}$$

The cross term in a product records one contribution for every earlier/later pair of generator occurrences, which proves the last identity directly. No sorting of the actual word is assumed.

For a polynomial presentation, avoid divisions by setting

$$T_{ii} = n_i(n_i - 1), \quad T_{ij} = 2K_{ij} \ (i < j), \quad T_{ji} = 2n_i n_j - 2K_{ij} \ (i < j).$$

Then the central equation is the quadratic residual

$$2C_s - \sum_i 2\gamma_{si} n_i - \sum_{i,j} \alpha_{si} \beta_{sj} T_{ij} = 0. \tag{123}$$

The horizontal equations are linear. Doubling the central coordinate is exact over the integers and introduces no witness.

Theorem 39.1 (Profile-preserving matrix lift). *Fix $g_1, g_2, g_3 \in \mathbf{H}^r$ and externally fix $c \in \mathbb{N}$. A target $t \in \mathbf{H}^r$ is the value of a word with prescribed statistics $(a, b, c; k, p, q)$ if and only if the gap certificate of Theorem 34.2 and the $3r$ equations (120), (121), (123) hold.*

This is a certificate with $2c + 4$ natural witnesses, $6 + 3r$ quadratic-or-linear residuals, and a quartic sum of squares. For $c = 2$, the canonical profile certificate can instead be used, giving 24 natural witnesses and $24 + 3r$ residuals, with exactly one complete witness tuple for each accepted target-and-profile parameter tuple.

Proof. An actual word gives a valid pair profile and satisfies the collection identities, so it supplies a solution. Conversely, the profile certificate produces one actual word with all specified counts. The collection identities show that this word has exactly the target coordinates in every factor. Thus one common word, not a separate word for each factor, realizes the target.

All added residuals have degree at most two and use only the supplied profile statistics, fixed generator coefficients, and target coordinates. They add no auxiliary variables. For the canonical specialization, the profile witnesses are unique by Theorem 37.1, and the matrix equations neither create nor multiply those witnesses. $\square$

39.2 Projection to target-only membership

If the interface supplies only the target and requires exactly two occurrences of g_3 , then a, b, p, q, k become five additional natural witnesses. The gap specialization therefore gives 13 witnesses and $6 + 3r$ residuals; the canonical specialization gives 29 witnesses and $24 + 3r$ residuals. All remain quadratic systems and hence yield quartics.

The 29-witness target-only version need not be single-fold. Distinct realizable profiles can evaluate to the same group element, and every such profile has its own unique 24-tuple. This distinction between uniqueness *over a profile* and uniqueness *over a target* is essential.

The target coordinates may be signed integers, which is compatible with the natural witness convention: the polynomial parameters and fixed coefficients may be signed while its existentially quantified profile and gadget variables remain natural. The provided Python core implements the natural profile interface; the signed-target lift is specified and proved here, rather than advertised as a separate implemented exporter.

39.3 A matrix substrate recording complete pair profiles

There is a particularly transparent way to realize pair statistics as group coordinates. For an alphabet of $d \geq 2$ letters, take one factor of H for each unordered pair $i < j$. In that factor assign

$$g_i = h(1, 0, 0), \quad g_j = h(0, 1, 0),$$

and assign every other letter the identity. In the product over all $\binom{d}{2}$ factors, a word evaluates in factor (i, j) to

$$h(n_i, n_j, K_{ij}). \tag{124}$$

This follows directly from (119). Consequently equality of these group products is exactly equality of complete pair profiles: the horizontal coordinates record the letter counts and the centers record all independent pair counts.

This map is not injective on words. It is injective on their pair-profile classes. For $d = 3$ it realizes the interval theorem as a statement about central-coordinate slices of a fixed three-generator submonoid of H^3 . For $d = 4$, Theorem 38.1 gives exact central-coordinate holes in a fixed four-generator submonoid of H^6 . Thus the combinatorial obstruction has an explicit group-theoretic substrate and is not only a relaxation artifact in abstract statistics.

[write 78] In each factor this map is Part I's area detector $a \mapsto h(1, 0, 0)$, $b \mapsto h(0, 1, 0)$ of (44), applied to the positive word in two of the letters. On such words its central coordinate is the scattered pair count. It is not van Kampen area, for which Part I uses the detector only as a lower bound, and it is not Part II's coordinate λ . For an ordered binary alphabet the factor (124) is the Parikh matrix of the two-letter projection (Section 32.3).

In the cyclic three-factor presentation used by the repository, the central coordinates are $(K_{AB}, K_{BC}, K_{CA}) = (k, q, ac - p)$. This is just a change of pair orientation; the same realizability theorem applies.

40 Decidability with a supplied third multiplicity

The certificate family separates one finite control parameter, c , from the potentially unbounded numbers of the other two generators. The following consequence makes that boundary precise.

Theorem 40.1 (Supplied-multiplicity decision). *Given a finite r , three integer Heisenberg tuples $g, h, j \in \mathbf{H}^r$, a target $t \in \mathbf{H}^r$, and a natural integer c , it is decidable whether a word over the three generator labels g, h, j with exactly c occurrences of j evaluates to t . No bound on the numbers of g and h is supplied or required.*

Proof. For each factor s , let

$$\Delta_s = \alpha_{sg}\beta_{sh} - \alpha_{sh}\beta_{sg}.$$

First suppose some $\Delta_s \neq 0$. Subtract the known horizontal contribution of c occurrences of j from that factor's target. The two remaining horizontal equations form a nonsingular 2×2 linear system for the multiplicities a, b of g, h . Compute its unique rational solution and reject unless both values are natural. The possible pair counts now lie in finite ranges

$$0 \leq k \leq ab, \quad 0 \leq p \leq ac, \quad 0 \leq q \leq bc.$$

Enumerate these ranges. For each profile, finite gap enumeration decides realizability, and the collection equations check the target in every factor. This finite search is exact.

Now suppose every $\Delta_s = 0$. Thus g, h commute. For one factor define the injective integer-coordinate map

$$\ell h(\alpha, \beta, \gamma) = (2\alpha, 2\beta, 2\gamma - \alpha\beta). \quad (125)$$

It is injective because the first two coordinates recover α, β , after which the last recovers γ . For two elements of one Heisenberg factor, its central failure of additivity is

$$\alpha\beta' - \alpha'\beta.$$

In particular, $\ell(g^a) = a\ell(g)$. For a full word, the central correction relative to the sum of the individual ℓ coordinates is, in each factor,

$$\Delta_{gh}(2k - ab) + \Delta_{gj}(2p - ac) + \Delta_{hj}(2q - bc). \quad (126)$$

The first term vanishes. Since c is an already supplied integer, the remaining target equations in a, b, p, q are linear. The exact constraints on the latter two statistics are simply

$$a, b, p, q \geq 0, \quad p \leq ca, \quad q \leq cb.$$

Every such p, q is jointly realizable by the moment-fibre nonemptiness argument, and k need not be specified because it has coefficient zero in every factor. Therefore the desired membership question is a finite system of integer linear equations and inequalities over nonnegative variables.

Introduce nonnegative slacks for the two inequalities and decide the resulting linear system by Lemma 40.2. Injectivity of (125) ensures that solving its equations is equivalent to the original group-coordinate equalities. Both branches terminate and are correct. $\square$

[write 78] The map (125) is $(2\alpha, 2\beta, \lambda)$ with Part II's integral logarithmic coordinate $\lambda = 2\gamma - \alpha\beta$, and (126) is (73) summed over the pairs of a word. This is the third independent derivation of that device in the repository, after the research note [16] (its Section 4) and Section 23. For $c = 0$ the commuting branch is the note's commuting two-generator case, which in turn is the special case of Theorem 23.1 with one generator in each phase and natural exponents.

Lemma 40.2 (Elementary linear-feasibility algorithm). *For a given integer matrix M and integer vector d , the existence of $z \in \mathbb{N}^s$ with $Mz = d$ is decidable.*

Proof. Encode all coordinates of z in base two, padding them to a common length and reading least-significant bits first. Start with carry vector $v_0 = -d$. At each step choose a bit vector $b \in \{0, 1\}^s$ and permit a transition precisely when every coordinate of $v + Mb$ is even; then set

$$v' = (v + Mb)/2.$$

For row i , put $B_i = \max(|d_i|, \sum_j |M_{ij}|)$. The finite box $|v_i| \leq B_i$ contains the initial state and is preserved by every transition. A path of length t ending at zero gives

$$0 = v_t = \frac{-d + M \sum_{u=0}^{t-1} 2^u b_u}{2^t},$$

so the finite bits yield a nonnegative solution. Conversely, the padded bits of any nonnegative solution give exactly such a path; all its intermediate carries are integral and remain in the box. Finite graph reachability from $-d$ to zero therefore decides the question. Empty paths cover the zero solution when appropriate. $\square$

Theorem 40.1 is uniform in the supplied c , even though one branch may use very large finite searches. It does not decide the problem with an unrestricted existentially quantified c : searching all possible values of c would not terminate on every negative instance. It also does not prove undecidable membership with only three generators; no such assertion is needed here.

[write 78] (2 October 2026.) With the interval theorem, Theorem 40.1 answers in part the question that the research note [16] leaves open and that Part II does not answer (Section 1.4): for three generators of a submonoid of $\mathbf{H}^r$, word realizability is now paid exactly, and membership is decidable once the multiplicity of one generator is supplied. Whether three generators can give undecidable submonoid membership is not decided here (question 9 of Section 42.2).

41 Executable verification and reproducibility

41.1 Contents of the research package

The package is self-contained apart from a standard LaTeX installation for rebuilding the PDF. The Python code requires Python 3.10 or later and uses only the standard library.

File	Purpose
article.tex, article.pdf	This report and its editable source.
code/pair_geometry.py	Gap constructions, normalization paths, exact corner decision, polynomial gadgets, and quartic export.
code/verify.py	Independent finite checks and receipt replay.
data/canonical_c2_quartic.json	Complete residual list and expanded quartic.
data/verification_receipt.json	Recorded exact test counts and compiler-source digest.
README.md, Makefile	Build and execution instructions, interfaces, and limitations.

Run from the package root:

```
python3 code/verify.py
python3 code/pair_geometry.py decide 3 2 2 1 4
python3 code/pair_geometry.py export /tmp/canonical_c2_quartic.json
latexmk -pdf -interaction=nonstopmode article.tex
```

The second command accepts the worked profile and emits the complete natural witness tuple. Replacing the last argument by 1 rejects it. Arguments are in the order a, b, p, q, k , with $c = 2$ fixed by the command's interface.

[write 78] In this report the package is shipped under the prefix 06-pair-order-: code/06-pair-order-pair_geometry.py, code/06-pair-order-verify.py and code/06-pair-order-Makefile, and data/06-pair-order-canonical_c2_quartic.json, data/06-pair-order-verification_receipt.json and data/06-pair-order-worked_example.json; the last, which the table above does not list, is the captured output of the command `decide 3 2 2 1 4`. Source 06's `article.tex`, `article.pdf`, `README.md` and checksum ledger are not shipped. The verifier imports the geometry module by its delivered name and finds the data through the delivered layout, and both `verify.py -write` and the Makefile's `export` target overwrite recorded files; run the programs only on a copy with the delivered layout (see the README). When the manuscript was placed, such a run passed in 59 seconds under Python 3.14.4, comparing every count with the bundled receipt, and the `decide` and `export` commands reproduced the two other data files apart from line endings.

41.2 Independent profile enumeration

The reference enumerator does not use the corner formula or gap constraints. It computes realizable triples (k, p, q) by a last-letter recurrence. Appending A creates no new forward-oriented pairs; appending B adds the current number of A letters to k ; appending C adds the current numbers of A and B to p, q . Starting from the empty word, this recurrence exhausts the profile set for each bounded multiplicity tuple.

For every resulting (p, q) slice, the tests compare the full profile set with the union of all gap intervals, verify the no-holes property, and, when $c = 2$, compare both endpoints with the four-corner formula. A separate enumeration of every ternary word of length at most eight checks the normalization path, including its invariant preservation and every change in K_{AB} .

41.3 Recorded exact checks

The bundled receipt records the following results:

Check	Count
Multiplicity boxes, $0 \leq a, b \leq 5, 0 \leq c \leq 4$	180
Nonempty ternary slices checked	8,730
Realizable profiles in those boxes	60,549
Two-separator endpoint checks	1,296
Ternary words in normalization-path tests, lengths 0–8	9,841
Normalization edges checked	22,619
Complete bounded profile decisions, including rejections	23,128
Accepted canonical witness samples	2,371
Rejected one-coordinate witness mutations	4,460
Signed off-zero expanded-polynomial checks	250
Independent direct 3×3 matrix products	2,000
Four-letter words with two of each letter	2,520

The quartic tests also evaluate endpoint witnesses for inputs built from powers 10^{20} , 10^{100} , and 10^{300} , without generating words of those lengths. The four-letter enumeration finds eight noninterval fibres for the displayed coordinate orientation in the all-two multiplicity box; the theorem uses one of them and proves its infinite extension. The repository’s strict-interior false family is checked for $m = 3, \dots, 99$.

These numbers describe finite computations, not the domains of validity of the theorems. In particular, single-foldness is proved by the triangular forcing argument, not by mutating a few witness coordinates. The tests do not exhaust all 24-dimensional candidate zero sets. Matrix comparisons check the collection formula independently of the scalar product routine, but do not instantiate a universal monoid or a halting reduction.

41.4 Code boundaries and error handling

Numeric input validators reject negative naturals, nonintegers, and Boolean values used as integers. Polynomial construction copies its input into an immutable canonical representation, preventing later mutation of an input list from changing a supposedly frozen expression.

The functions producing explicit words or enumerating gap vectors are intended for modest instances. They may take time proportional to large counts and may allocate long words. The two-separator decision command and canonical witness evaluation use exact big-integer arithmetic instead and do not enumerate words.

The compiler also permits evaluating polynomial residuals on signed arbitrary assignments for algebraic testing. Such an assignment is not a valid natural witness just because its residuals vanish. On rejected instances, the triangular candidate assignment may have a negative bound or endpoint slack; the natural-domain check is therefore part of the public decision procedure.

42 Integration into ProveIt and further research

42.1 An incremental formalization route

The most useful first formalization is the profile theorem, not a monolithic expansion of the quartic. A reasonable dependency chain is:

1. Define letter multiplicities and scattered pair counts on lists. Prove append formulas and the

binary realization lemma.

2. Define gap decomposition and prove the total, moment, and cross-gap identities. Derive the six-equation soundness and completeness theorem.
3. Formalize paired crossings, potential descent, and the unique balanced gap vector. Prove the connectedness and interval theorems.
4. Verify the two-separator substitutions by polynomial normalization, then derive the corner criterion and canonical gadget uniqueness.
5. Add Heisenberg coordinates and prove the collection formula. Compose the profile and matrix certificates with a zero-set theorem for the sum of squares.

Proposed modules might be named `WordPairProfile`, `TernaryGapRealization`, `TernaryPairInterval`, and `CanonicalTwoSeparator`. These are suggested names, not claims that such modules already exist or have been checked. The Python receipt is computational evidence; it is not a trusted Lean proof object.

A later compiler integration should produce explicit natural-to-positive coordinate maps, charge every subtraction and coefficient multiplication, and preserve the stated interface. The present construction intentionally separates polynomial degree, witness count, and runtime arithmetic count; optimizing one should not be reported as an improvement of another.

42.2 Research questions with concrete next steps

1. General endpoint complexity. Determine the bit complexity of evaluating (96) with binary-encoded a, b, c, p, q . The interval theorem removes internal membership gaps, but it leaves a structured bilinear optimization over integer moment fibres. A useful first target is an algorithm polynomial in c and the bit lengths of a, b, p, q , rather than in the word length.

2. A closed formula for three separators. For $c = 3$, each gap vector has two free coordinates after the total and moment equations are eliminated. Work out its integer polygon and determine whether finitely many residue-sensitive boundary candidates compute the extrema. The $c = 2$ formula suggests a concrete test case for an arithmetic compiler more efficient than generic enumeration.

3. Small-support extremizers. Over real moment polytopes, linear objectives have sparse extremizers. Integer fibres can require additional support or residue corrections. Prove a correct support bound for a pair of extremizing integer gap vectors, or exhibit families ruling out an attractive constant bound. This could be the bridge from a separator-dependent compiler to a succinct all- c endpoint description.

4. Uniform arity without hiding the separator count. Find a fixed-arity Diophantine certificate for the full ternary profile relation with c as an ordinary parameter, accompanied by an explicit bound on witnesses and degree. General representability results alone would not supply the desired small construction or uniqueness. Encoding the gap list must be paid rather than treated as a variable-length primitive.

5. Canonical versus smallest certificates. Can the 24-witness endpoint construction be reduced while retaining uniqueness, especially on tied corners? Can both extrema share comparison work? A valid improvement should supply a complete zero-set bijection or a fresh triangular forcing proof, rather than discard apparently unused negative parts without checking all branches.

6. Succinct realizing words. The binary construction uses only a few runs. Determine whether every realizable ternary profile has a representative with a number of runs bounded independently of its multiplicities, or bounded by a useful function of c . A constructive result would convert the current decision certificate into a compact explicit word certificate. It would also clarify how our restricted invariants interact with group-theoretic bounded-generation results [34].

7. Classifying four-letter spectra. Theorem 38.1 realizes pure arithmetic progressions. Which finite sets, unions of intervals, or residue patterns can occur when one pair count varies and all other counts are fixed? Seek structural restrictions before attempting a universal catalogue. The all-two enumeration provides a finite initial test set, not evidence of a complete classification.

8. Nonconvex profile constraints for matrix compilers. Corollary 38.2 excludes affine constraints in the profile coordinates alone. Develop exact nonlinear or disjunctive constraints that detect separator-skeleton obstructions while keeping a manageable Diophantine interface. Compare their witness cost with simply supplying a gap decomposition.

9. Three-generator membership without a supplied multiplicity. Theorem 40.1 treats c as input, not as an unbounded existential variable. Identify additional geometric conditions on three generators in H^r that bound the necessary c , permit its elimination, or reduce the problem to a decidable arithmetic class. The present report does not assert that the unrestricted three-generator problem is currently open in every relevant formulation; its exact relation to existing classifications requires a dedicated literature audit.

[write 78] Part II's three-subgroup theorem does not bear on this question: it concerns ordered products of subgroups or submonoids, with no interleaving of letters across phases (Section 1.4).

10. From profile existence to unambiguous target membership. Find generator families for which the target coordinates determine (a, b, p, q, k) uniquely, so that the 24-witness single-fold property survives projection to targets. Conversely, characterize the affine and quadratic kernels responsible for multiple accepted profiles over one target. This question separates arithmetic witness ambiguity from word ambiguity.

11. Higher-step nilpotent groups. Upper-unitriangular matrices of larger size retain longer scattered subwords. Develop a version of the separator method for length-three counts and test whether connected-slice phenomena persist under a carefully specified collection of fixed invariants. The two-separator moment argument cannot simply be assumed to extend to those higher-order statistics.

[write 78] Compare Question 16.4 of Part I and Section 30.7 of Part II.

12. Counting profiles and connecting to sequence research. Let $N(a, b, c)$ count distinct ternary pair profiles. The interval theorem gives the exact identity

$$N(a, b, c) = \sum_{p=0}^{ac} \sum_{q=0}^{bc} (\mathcal{U}_c(a, b; p, q) - \mathcal{L}_c(a, b; p, q) + 1).$$

For $c = 2$, substitute the corner expressions and seek piecewise quasipolynomial formulas, generating functions, and asymptotics along natural diagonals. This connects the current compiler problem to the repository’s enumerative and asymptotic interests. No OEIS identification or asymptotic formula is claimed in this report.

42.3 A precise boundary of the contribution

The missing order relation is now completely characterized for ternary profiles, with an exact finite gap certificate for each separator count and a closed canonical quartic for the two-separator slice. The four-letter family proves a genuine failure of one-dimensional interval interpolation beyond that alphabet size. These statements are independent of any computational universality claim.

Nothing here instantiates Roman’kov’s entire undecidable monoid, lowers the repository’s advertised universal Diophantine-operation bound, or proves a global single-fold representation theorem for enumerable sets. The results are most useful as sound compiler components and as a sharply delimited account of what pair-count compression preserves, what it loses, and where further arithmetic work is needed.

Part IV

Affine matrix inputs (source 07)

[write 78] Part IV prints batch-78 manuscript 13, *Affine Matrix Inputs and Diophantine Universality: Sharp rigidity, exact period bounds, and computation with existential witnesses*, a research report prepared for Vladimir Reshetnikov, October 2, 2026; it names no author and is AI-assisted. Its labels carry the prefix `gts:ai:`. Its Section n is Section $n + 42$ here, and its Theorem $n.m$ is Theorem $(n + 42).m$; it has no appendices. Like source 06 it was written from commit `4cccfa068` and could not know the other Parts. Its interpolation coordinates λ_i are written θ_i (Section 1.3), and its automatic cross-references are printed with the statement type spelled out. Its abstract follows.

Abstract (source 07). A matrix subgroup can have an undecidable membership problem, but the way an ordinary integer input is loaded into that subgroup matters. We prove that, for every dimension d , every entrywise affine curve $L(n) = C + nD$ in $\mathrm{SL}_d(\mathbb{Z})$, and every fixed subgroup K , the set of integers for which $L(n) \in K$ is either finite or exactly periodic. If $N = C^{-1}D$ has nilpotency index $k + 1$, the finite case has at most k elements, and every prescribed set of at most k integers is realizable. A canonical free-abelian coordinate system gives the optimal universal period

$$P_k(e) = \prod_{p|e} p^{v_p(e) + \lceil \log_p k \rceil},$$

where e is the exponent of the relevant finite lattice quotient. This extends the paired- SL_2 obstruction in the audited `ProveIt` sources to arbitrary dimension. Combined with the repository’s

quadratic subgroup-input construction, it proves that input degree two is sharp even when the matrix dimension is unrestricted.

The obstruction changes completely when extra integer coordinates are existentially quantified. We give a six-dimensional, jointly affine, unimodular multiplication gadget with one uniquely determined auxiliary coordinate. It compiles a polynomial circuit with g multiplication gates into membership in a fixed free-abelian unipotent subgroup of $\mathrm{SL}_{6g+2}(\mathbb{Z})$, with exactly $2g$ uniquely determined added integer coordinates. The resulting certificate is an explicit quartic sum of squares. We also prove a commuting-polynomial extension, single-fold certificates for the rigid languages, a uniformity obstruction, and a one-witness decidability result for the block-cyclic model. The package includes an exact-arithmetic implementation and 91,142 finite verification cases. The results are developed as a repository-relative extension; global publication priority is not established, and no improved fixed universal-polynomial operation count or proof-assistant verification is claimed.

43 The research question and the source boundary

43.1 What is being represented?

A computational substrate is useful for Diophantine work only after its input convention, acceptance condition, and witnesses have been specified. The statement that a model is Turing-complete does not by itself give a small polynomial, a bounded number of variables, or a canonical certificate. In the present report the substrate is matrix subgroup membership. The first interface is

$$n \in S \iff L(n) \in K, \quad (127)$$

where K is fixed, n is the ordinary numerical input, and every entry of $L(n)$ is a prescribed polynomial in n . There are no additional input coordinates in this interface. Membership in K may, of course, mean that some arbitrarily long word in its generators evaluates to $L(n)$. We do not count that word as a free arithmetic operation.

The second interface permits existential integer coordinates:

$$n \in S \iff \exists \mathbf{w} \in \mathbb{Z}^m : \mathcal{L}(n, \mathbf{w}) \in K. \quad (128)$$

Here $\mathcal{L}$ will be jointly affine in all its coordinates and unimodular for every assignment. The difference between (127) and (128) is decisive. An affine ordinary-input curve cannot recognize a nonrecursive set in any dimension, whereas a fixed affine pencil with finitely many existential coordinates can do so even when K is abelian and its direct membership test is elementary.

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$, but the rigidity theorems use all of $\mathbb{Z}$. A set is *exactly periodic* if it is a union of residue classes modulo some positive integer on all of $\mathbb{Z}$, not merely eventually periodic. Restriction to positive inputs is performed only after the classification. Integer and natural witnesses are kept distinct.

43.2 What was already in ProveIt?

The source audit was anchored to the repository revision

4cccfa06866b6b81b2467e1cf7514ea85ed0216d.

The accompanying `SOURCE_AUDIT.md` records the inspected paths and available blob hashes. Four source interfaces are important.

First, *Affine paired- SL_2 input cannot give universal subgroup membership* proves that one affine SL_2 block, or two independently chosen such blocks, has a membership preimage that is empty, a singleton, or one full arithmetic progression [35]. The proof uses $N^2 = 0$. Its SL_3 example realizes $\{1, 2\}$, correctly showing that the stronger single-coset classification does not persist in higher dimension. It does not settle whether a weaker all-dimensional obstruction survives.

Second, *An ordinary-input universal substrate from group commutators* constructs a fixed quadratic curve in $SL_4(\mathbb{Z})$ whose membership in a suitable finitely generated subgroup represents any given computably enumerable subset of the positive integers [36]. Its group-theoretic upper bound is prior repository work, not a new universality construction in this report.

Third, the October 2 imported-substrate review distinguishes bounded certificates from one fixed polynomial handling unbounded computations, and distinguishes existence of witnesses from preservation of unique witnesses. It reports an 87-operation fixed universal-polynomial benchmark, but does not attribute such a bound to the matrix input loader [37]. We use that review as a scope constraint, not as a benchmark independently reverified here.

Finally, the repository contains an MRDP theorem whose polynomial and number of natural witnesses are independent of the input [38]. We invoke MRDP only when deriving the general computably enumerable consequence of our circuit compiler. The main affine rigidity theorem does not use MRDP.

[write 78] The repository already contained further material that source 07 does not list (Section 1.4): the research note [15], whose curve source 07 reprints as $Q(n)$ in Section 52; the complete matrix-membership compiler [18], which bears on question 10 of Section 59; the programme’s affine mortality notes, which load an affine input matrix as a semigroup generator for a zero-product problem, a different interface; and Part VI of the report [13], which prints Mihailova’s generators. Part II of this report, written after source 07’s pin, shows that the subgroup hypothesis of Theorem 46.1 cannot be dropped (Remark 46.3). The four repository files cited above are unchanged from the pin to this write.

43.3 Contribution and novelty status

The principal extension is the all-dimensional finite-or-periodic theorem, strengthened by an exact free basis, a sharp finite-hit bound, and a sharp period formula. The explicit multiplication gadget and compiler explain a contrasting mechanism for Diophantine universality. They are not presented as a new proof of MRDP or as the discovery that multiplication can be encoded in unipotent matrices.

Integer-valued polynomials and polynomial sequences in groups have a substantial established theory [39, 40, 41]. Our proofs are elementary and self-contained at the points where the sharp bounds are obtained. The literature check did not establish whether an identical combination of statements has previously been published. Accordingly, “new” below means a result developed here beyond the inspected repository boundary, not a certified claim of global priority. Research questions in Section 59 are proposed continuation problems; none is advertised as a famous previously open conjecture solved here.

44 Nilpotent normalization of an affine curve

Lemma 44.1 (Affine unimodular normal form). *Suppose $C, D \in M_d(\mathbb{Z})$ and $\det(C + nD) = 1$ for infinitely many integers n . Then $C \in \mathrm{SL}_d(\mathbb{Z})$ and*

$$L(n) = C(I + nN), \quad N = C^{-1}D \in M_d(\mathbb{Z}), \quad N^d = 0.$$

Proof. The polynomial $\det(C + tD) - 1$ has infinitely many roots and is therefore zero. At $t = 0$ this gives $\det C = 1$, so C^{-1} has integer entries. Consequently $\det(I + tN) = 1$ identically. For an indeterminate z ,

$$\det(zI - N) = z^d \det(I - z^{-1}N) = z^d.$$

The identity may be read first in $\mathbb{Q}(z)$ and then in $\mathbb{Q}[z]$. Cayley–Hamilton gives $N^d = 0$. $\square$

If $N = 0$, the hit set is either empty or all of $\mathbb{Z}$. Henceforth suppose $N \neq 0$, and let $k + 1$ be its nilpotency index. Thus

$$N^{k+1} = 0, \quad N^k \neq 0, \quad 1 \leq k \leq d - 1. \quad (129)$$

Set $U(n) = I + nN$. All the $U(n)$ commute, but they need not form a cyclic group: $U(s)U(t) = U(s + t)$ holds only when $N^2 = 0$. This is exactly why the old single-coset conclusion cannot simply be repeated in dimension three.

Lemma 44.2 (Faithful truncated algebra). *The homomorphism*

$$\mathbb{Q}[z]/(z^{k+1}) \longrightarrow \mathbb{Q}[N], \quad z \longmapsto N,$$

is an isomorphism. On the nilpotent ideal (z) , the finite series $\log(1 + x)$ and $\exp(x)$ are mutually inverse, and $\log(ab) = \log a + \log b$ for $a, b \in 1 + (z)$.

Proof. The minimal polynomial of N is z^{k+1} . This proves the first assertion, including the linear independence of $N, N^2, \dots, N^k$. The formal logarithm and exponential identities hold in $\mathbb{Q}[[z]]$ and descend modulo z^{k+1} , where every relevant sum is finite. The multiplicative identity uses commutativity, which holds in this algebra. $\square$

In particular,

$$\log U(n) = \sum_{j=1}^k \frac{(-1)^{j+1}}{j} n^j N^j. \quad (130)$$

Thus the logarithmic image of the affine matrix curve is an invertible linear image of the ordinary moment curve $(n, n^2, \dots, n^k)$. The central issue is to retain its *integral* lattice, rather than work only with its rational span.

45 An exact free basis and canonical integer coordinates

For $1 \leq i \leq k$ define the rational polynomial

$$\theta_i(t) = \binom{t}{i} \binom{k-t}{k-i}, \quad \boldsymbol{\theta}(t) = (\theta_1(t), \dots, \theta_k(t))^{\top}. \quad (131)$$

Binomial coefficients here mean their polynomial extensions. In particular, negative arguments are allowed. The identity $\binom{-a}{j} = (-1)^j \binom{a+j-1}{j}$ for $a > 0$ shows that each θ_i takes integer values on all integers.

Lemma 45.1 (Integral interpolation). *For every polynomial $f \in \mathbb{Q}[t]$ of degree at most k with $f(0) = 0$,*

$$f(t) = \sum_{i=1}^k \theta_i(t) f(i). \quad (132)$$

The polynomials $\theta_1, \dots, \theta_k$ form a $\mathbb{Z}$ -basis of the group of integer-valued polynomials of degree at most k that vanish at zero.

Proof. At $t = j \in \{1, \dots, k\}$ the value of θ_i is δ_{ij} ; at zero all are zero. Both sides of (132) have degree at most k and agree at $k+1$ points. If f is integer-valued, its coefficients in this interpolation basis are the integers $f(1), \dots, f(k)$. Conversely every integer combination of the θ_i is integer-valued. Linear independence follows by evaluation at $1, \dots, k$. $\square$

Theorem 45.2 (The affine evaluation lattice). *Let*

$$A_N = \langle I + nN : n \in \mathbb{Z} \rangle \leq \mathrm{SL}_d(\mathbb{Z}).$$

Then A_N is free abelian of rank k , with basis $U(1), U(2), \dots, U(k)$. For every integer n ,

$$U(n) = \prod_{i=1}^k U(i)^{\theta_i(n)}. \quad (133)$$

Thus the map $\Phi : \mathbb{Z}^k \rightarrow A_N$ defined by $\Phi(a_1, \dots, a_k) = \prod_i U(i)^{a_i}$ is an isomorphism and $\Phi^{-1}(U(n)) = \theta(n)$.

Proof. Apply Lemma 45.1 to the vector-valued polynomial (130). It gives $\log U(n) = \sum_i \theta_i(n) \log U(i)$. Exponentiating proves (133). The exponents are integers, so the assertion is genuinely about the group A_N , not a divisible enlargement of it.

The coefficient matrix of $\log U(1), \dots, \log U(k)$ relative to $N, \dots, N^k$ is obtained from $(i^j)_{1 \leq j, i \leq k}$ by nonzero row rescalings. Its determinant is nonzero: factor i from column i and use the Vandermonde determinant on $1, \dots, k$. Hence the logarithms are rationally independent. A relation $\prod_i U(i)^{a_i} = I$ therefore forces all $a_i = 0$. Together with (133), this proves the theorem. $\square$

The exact basis removes two possible ambiguities. The rank is the nilpotency index minus one, not necessarily the matrix dimension minus one. Also, no subgroup of $\mathbb{Q}^k$ with unbounded denominators is being introduced: every evaluation is in the explicit lattice generated by k fixed matrices.

Lemma 45.3 (Affine independence). *Any $s \leq k+1$ distinct points among $\theta(n)$, $n \in \mathbb{Z}$, are affinely independent.*

Proof. The θ_i are a rational basis of the polynomials of degree at most k with zero constant term. Thus $\theta(t)$ differs from $(t, t^2, \dots, t^k)$ by an invertible linear transformation. The vectors $(1, n, n^2, \dots, n^k)$ at s distinct integers have rank s , since their first s coordinates form a Vandermonde matrix. This is equivalent to affine independence. $\square$

46 The all-dimensional finite-or-periodic theorem

Theorem 46.1 (Affine subgroup-input rigidity). *Let $L(n) = C + nD \in \mathrm{SL}_d(\mathbb{Z})$ for every integer n , and let $K \leq \mathrm{SL}_d(\mathbb{Z})$ be any subgroup. Set*

$$T = \{n \in \mathbb{Z} : L(n) \in K\}.$$

If $D = 0$, then T is empty or all of $\mathbb{Z}$. Otherwise let $k + 1$ be the nilpotency index of $C^{-1}D$.

If $T \neq \emptyset$, choose $n_0 \in T$, and let

$$\Lambda = \Phi^{-1}(K \cap A_N) \leq \mathbb{Z}^k, \quad r = \mathrm{rank} \Lambda. \quad (134)$$

Then

$$n \in T \iff \theta(n) - \theta(n_0) \in \Lambda. \quad (135)$$

If $r < k$, the set T has at most $r + 1$ elements. If $r = k$, it is exactly periodic. Consequently every T is finite with at most $d - 1$ elements, or is exactly periodic.

Proof. By Lemma 44.1, $L(n) = CU(n)$. Since $L(n_0) \in K$, subgroup closure gives

$$L(n) \in K \iff L(n_0)^{-1}L(n) \in K \iff U(n_0)^{-1}U(n) \in K \cap A_N.$$

The factor C cancels in the indicated order; it is not moved across $U(n)$. Applying Φ^{-1} proves (135).

When $r < k$, every point $\theta(n)$ with $n \in T$ belongs to the affine rational space $\theta(n_0) + \mathrm{span}_{\mathbb{Q}}(\Lambda)$ of dimension r . If there were $r + 2$ distinct such points, Lemma 45.3 would make them affinely independent, a contradiction.

When $r = k$, the quotient $\mathbb{Z}^k/\Lambda$ is finite. Let e be its exponent, so $e\mathbb{Z}^k \subseteq \Lambda$. Each θ_i is integer-valued of degree at most k and is periodic modulo e . For the latter fact, a crude direct bound is $e k!$: the polynomial $k!\theta_i$ has integer coefficients, so its value at $n + ek!$ minus its value at n is divisible by $ek!$. Division by $k!$ shows that $\theta_i(n + ek!) \equiv \theta_i(n) \pmod{e}$. Thus (135) is unchanged by this positive shift. The sharp period bound is proved in Section 47. $\square$

Corollary 46.2 (No affine ordinary-input universality). *For every fixed dimension, affine unimodular curve, and subgroup, the positive-input language (127) is recursive. No noncomputable computably enumerable set has such a representation, even if the dimension is allowed to vary when choosing the representation.*

Proof. A finite set and a union of finitely many residue classes are recursive. This statement concerns each fixed set: it does not assert a uniform algorithm extracting its finite description from arbitrary subgroup generators. Such a uniform algorithm is impossible in general; see Section 50. $\square$

The theorem excludes more than undecidability. An infinite nonperiodic set, such as the positive squares, cannot occur. Indeed, an infinite periodic set has bounded gaps between successive sufficiently large elements, whereas the square gaps $2n + 1$ are unbounded.

Remark 46.3 ([write 78] The subgroup hypothesis is necessary). The proof of Theorem 46.1 uses closure of K under $L(n_0)^{-1}L(n)$, and the conclusion fails for products of subgroups even on the simplest curves. In Theorem 22.1 the hit set $\{n \in \mathbb{N} : g_0 g_1^n \in \mathcal{ABT}\}$ is an arbitrary prescribed c.e. set, where $\mathcal{A}, \mathcal{B}, \mathcal{T}$ are free abelian subgroups of $H^d \leq \mathrm{SL}_{3d}(\mathbb{Z})$ and g_1 is central of infinite order. As

a block-diagonal matrix, $g_1 = I + E$ with E a sum of matrix units in the corner positions of the diagonal blocks, so $E^2 = 0$, and $g_0 g_1^n = g_0 + n g_0 E$ is an entrywise affine unimodular curve with $N = E$ square-zero: the case $k = 1$. Hence, first, an ordered product of three free abelian subgroups, and by Corollary 24.2 one of three free commutative submonoids, can have a nonrecursive hit set along an affine line, so Theorem 46.1 does not extend to such rational subsets; this answers the negative half of question 9 of Section 59 for three-factor products. Second, when the prescribed set is not recursive, $\mathcal{ABT}$ is not a subgroup. For two commuting factors, Theorem 23.1 reduces membership of $g_0 g_1^n$ to an integer linear system whose right side depends polynomially on n ; whether hit sets of affine curves are then always finite or periodic is proved in neither Part and is left open.

47 The exact period formula

The expression $\lfloor \log_p k \rfloor$ below is the largest integer b with $p^b \leq k$. It is an integer definition and does not require floating-point logarithms.

Definition 47.1. For $k, e \geq 1$, define

$$P_k(e) = \prod_{p|e} p^{v_p(e) + \lfloor \log_p k \rfloor} = e \prod_{p|e} p^{\lfloor \log_p k \rfloor}. \quad (136)$$

The empty product gives $P_k(1) = 1$.

Lemma 47.2 (Newton basis). *Every integer-valued $f \in \mathbb{Q}[t]$ of degree at most k has a unique expansion*

$$f(t) = \sum_{j=0}^k a_j \binom{t}{j}, \quad a_j \in \mathbb{Z}.$$

If $f(0) = 0$, then $a_0 = 0$. Thus the bases $(\theta_i)_{i=1}^k$ and $((\binom{t}{j})_{j=1}^k$ are related by a unimodular integer change of coordinates.

Proof. Let $\Delta f(t) = f(t+1) - f(t)$. The identity $\Delta \binom{t}{j} = \binom{t}{j-1}$ gives the Newton formula $f(t) = \sum_{j=0}^k \Delta^j f(0) \binom{t}{j}$. It follows either by induction on degree or by agreement at $0, 1, \dots, k$. All finite differences are integers. Uniqueness follows from the different degrees. The last assertion follows because both families are $\mathbb{Z}$ -bases of the same group, by Lemma 45.1. $\square$

Lemma 47.3 (Simultaneous binomial divisibility). *For $k, e \geq 1$ and $n \in \mathbb{Z}$,*

$$e \mid \binom{n}{j} \text{ for every } 1 \leq j \leq k \iff P_k(e) \mid n. \quad (137)$$

Proof. The case $e = 1$ is immediate. First take $n > 0$, fix $p^a \parallel e$, and put $b = \lfloor \log_p k \rfloor$.

For sufficiency suppose $v_p(n) \geq a + b$. The identity $j \binom{n}{j} = n \binom{n-1}{j-1}$ gives

$$v_p \left(\binom{n}{j} \right) \geq v_p(n) - v_p(j) \geq a + b - b = a$$

whenever the coefficient is nonzero; a zero coefficient is divisible by every power of p . This works for all $1 \leq j \leq k$.

For necessity let $s = v_p(n)$. If $s \leq b$, choose $j = p^s \leq k$. For each $1 \leq \ell < p^s$ we have $v_p(n - \ell) = v_p(\ell)$. Consequently

$$\binom{n-1}{p^s-1} = \prod_{\ell=1}^{p^s-1} \frac{n-\ell}{\ell}$$

is a p -adic unit. Since $v_p(n/p^s) = 0$, the identity $\binom{n}{p^s} = (n/p^s)\binom{n-1}{p^s-1}$ contradicts divisibility by p . Thus $s > b$. Applying the same argument with $j = p^b$ now gives

$$v_p\left(\binom{n}{p^b}\right) = s - b.$$

Divisibility by p^a forces $s \geq a + b$. Taking all primes dividing e proves (137) for positive n ; $n = 0$ is immediate.

For negative n , work in $(\mathbb{Z}/e\mathbb{Z})[z]/(z^{k+1})$. The left side of (137) says $(1+z)^n = 1$ in this ring. The element $1+z$ is a unit, so this is equivalent to $(1+z)^{-n} = 1$. The positive case applies to $-n$ and completes the proof. $\square$

Theorem 47.4 (Optimal universal period). *Every integer-valued polynomial of degree at most k is periodic modulo e with period $P_k(e)$. The vector $\theta(n)$ has least positive period $P_k(e)$ modulo e , and*

$$\theta(n) \equiv 0 \pmod{e} \iff P_k(e) \mid n. \quad (138)$$

Consequently, in the full-rank case of Theorem 46.1, $P_k(e)$ is a period of the hit set, where e is the exponent of $\mathbb{Z}^k/\Lambda$.

Proof. Put $P = P_k(e)$. The binomial identity

$$\binom{n+P}{j} = \sum_{\ell=0}^j \binom{P}{\ell} \binom{n}{j-\ell}$$

holds as a polynomial identity in n , including negative integers. By Lemma 47.3, its terms with $\ell \geq 1$ are divisible by e for $j \leq k$. Hence every Newton basis polynomial, and therefore every integer-valued polynomial of degree at most k , has period P modulo e .

The unimodular basis change in Lemma 47.2 shows that $\theta(n) \equiv 0 \pmod{e}$ is equivalent to the simultaneous binomial divisibility condition. This proves (138). If $q > 0$ is any period of θ modulo e , then $\theta(q) = \theta(0) = 0$ modulo e , so $P \mid q$. Since P itself is a period, it is the least one. Finally $e\mathbb{Z}^k \subseteq \Lambda$ makes congruence modulo e sufficient to preserve (135). $\square$

Corollary 47.5 (Sharpness over all full-rank lattices). *For every $k, e \geq 1$, let N be a nilpotent Jordan block of size $k+1$, and let*

$$K = \langle U(1)^e, \dots, U(k)^e \rangle.$$

Then $\{n : U(n) \in K\} = P_k(e)\mathbb{Z}$. In particular, no smaller universal period depending only on k and e can replace (136).

Proof. Here $\Lambda = e\mathbb{Z}^k$, whose quotient has exponent e . Apply (138). The subgroup $P_k(e)\mathbb{Z}$ has least positive period $P_k(e)$. $\square$

A bound such as $e \operatorname{lcm}(1, \dots, k)$ is valid but can be wasteful: only primes already dividing e need their additional powers. For example,

$$P_3(2) = 4, \quad P_8(12) = 2^{2+3} 3^{1+1} = 288.$$

The period of an individual hit set may be smaller than the universal bound; the sharpness assertion is a worst-case statement with fixed k, e .

48 Sharp finite sets and diagnostic examples

Theorem 48.1 (Every permitted finite set is realizable). *Fix $k \geq 1$ and a nonempty finite set $F \subseteq \mathbb{Z}$ with $|F| \leq k$. There exist an affine curve $L(n) \in \operatorname{SL}_{k+1}(\mathbb{Z})$ and a finitely generated abelian subgroup K such that*

$$\{n \in \mathbb{Z} : L(n) \in K\} = F.$$

The subgroup can be chosen of rank $|F| - 1$.

Proof. Let J be the nilpotent Jordan block of size $k + 1$, put $U(n) = I + nJ$, and choose $n_0 \in F$. Define

$$L(n) = U(n_0)^{-1}U(n), \quad K = \langle U(n_0)^{-1}U(f) : f \in F \setminus \{n_0\} \rangle.$$

The inverse of $U(n_0)$ is an integral finite geometric series, so L is an affine integral unimodular curve. Every element of F is accepted. The vectors $\theta(f) - \theta(n_0)$, $f \in F \setminus \{n_0\}$, are independent by Lemma 45.3; hence K has rank $|F| - 1$. A further accepted integer would give $|F| + 1 \leq k + 1$ moment-curve points in an affine space of dimension $|F| - 1$, contradicting the same lemma. $\square$

This strengthens numerical sharpness: the finite exceptions can be placed at arbitrarily prescribed integers, not only at a convenient initial interval. For a fixed unshifted curve $U(n)$, however, zero is always accepted because every subgroup contains the identity. The translation by $U(n_0)^{-1}$ is therefore substantive.

48.1 The square-zero case

When $k = 1$, we have $\theta_1(n) = n$. Every subgroup of $\mathbb{Z}$ is $a\mathbb{Z}$, and (135) gives an empty set, a singleton, or one coset $n_0 + a\mathbb{Z}$. This recovers the exact conclusion of the paired- SL_2 source [35]. The general theorem is not obtained by asserting that higher-dimensional affine curves are powers of one matrix; they usually are not.

48.2 Dimension three: two points or two residue classes

For $k = 2$,

$$\theta_1(n) = n(2 - n), \quad \theta_2(n) = \frac{n(n - 1)}{2}. \quad (139)$$

Let $J = E_{12} + E_{23}$ and $U(n) = I + nJ$. The subgroup $\langle U(1) \rangle$ corresponds to $\Lambda = \mathbb{Z}(1, 0)$, so its hit set is $\{0, 1\}$. Replacing $U(n)$ by $U(n - 1)$ yields $\{1, 2\}$, the higher-dimensional boundary example in the repository.

There is also an infinite example that is not one coset. Let

$$K = \langle U(1), U(2)^2 \rangle.$$

Its lattice is $\mathbb{Z} \times 2\mathbb{Z}$. Therefore

$$U(n) \in K \iff \frac{n(n-1)}{2} \equiv 0 \pmod{2} \iff n \equiv 0 \text{ or } 1 \pmod{4}.$$

This set is exactly periodic but is not a coset of a subgroup of $\mathbb{Z}$. Both the finite and periodic alternatives in the all-dimensional theorem are genuinely broader than the old one-coset classification.

48.3 Dimension four: a nonconsecutive sharp finite set

For a Jordan block of size four, let $K = \langle U(2), U(5) \rangle$. Then

$$\{n : U(n) \in K\} = \{0, 2, 5\}.$$

For an independent check, logarithmic coordinates relative to J, J^2, J^3 are $(n, -n^2/2, n^3/3)$. The linear functional $(v_1, v_2, v_3) \mapsto 10v_1 + 14v_2 + 3v_3$ vanishes on both generator logarithms and has value

$$10n - 7n^2 + n^3 = n(n-2)(n-5)$$

on $\log U(n)$. Hence any accepted integer must be one of the three printed roots, all of which are indeed accepted. This avoids treating a finite search window as evidence for the unbounded assertion.

49 Explicit Diophantine certificates for the rigid languages

The classification can itself be compiled into a polynomial without MRDP, once the relevant lattice is supplied. This is useful as a certificate format, although the lattice need not be effectively recoverable from an arbitrary subgroup presentation.

Theorem 49.1 (A single-fold lattice certificate). *In the nonconstant, nonempty situation of Theorem 46.1, suppose an integer column basis $B \in M_{k \times r}(\mathbb{Z})$ of Λ and an accepted input n_0 are given. Put*

$$a_i(t) = k! \theta_i(t) \in \mathbb{Z}[t].$$

Then the polynomial

$$F(n, \mathbf{y}) = \sum_{i=1}^k \left(a_i(n) - a_i(n_0) - k! \sum_{j=1}^r B_{ij} y_j \right)^2 \quad (140)$$

has degree at most $2k$ and satisfies

$$L(n) \in K \iff \exists \mathbf{y} \in \mathbb{Z}^r : F(n, \mathbf{y}) = 0.$$

For every accepted input, the integer witness tuple $\mathbf{y}$ is unique.

Proof. From the product expression in (131), multiplication by $k!$ clears its denominators: the remaining factor is $\binom{k}{i}$ times a product of integral linear factors. A sum of integer squares vanishes exactly when each summand vanishes. Dividing each residual equation by the nonzero integer $k!$ yields $\theta(n) - \theta(n_0) = B\mathbf{y}$. This is equivalent to (135). A column basis has independent columns, so the solution $\mathbf{y}$, when it exists, is unique. Each residual has degree at most k , giving the degree bound. $\square$

For $r = 0$, the sum over j is empty and there are no witnesses. The empty language is represented by the constant polynomial 1; the full language by 0. These cases do not require choosing n_0 for an empty set.

Corollary 49.2 (Canonical natural witnesses). *The certificate in Theorem 49.1 has a version with $2r$ natural witnesses, still unique over every accepted integer input, and degree at most $\max(2k, 4)$.*

Proof. Replace y_j by $u_j - v_j$, with $u_j, v_j \in \mathbb{N}$, and add the squared residuals $(u_j v_j)^2$. Their vanishing forces $u_j v_j = 0$. Every integer y_j has exactly one such decomposition: $u_j = \max(y_j, 0)$ and $v_j = \max(-y_j, 0)$. The resulting sum of squares has the stated degree and exact witness correspondence. $\square$

This is a single-fold representation of a restricted class of recursive languages. It says nothing about whether arbitrary computably enumerable sets have finite-fold or single-fold Diophantine representations. Nor is the numerical witness bound a new universal-polynomial record: it applies precisely to languages already forced to be finite or periodic.

49.1 A two-dimensional lattice example

For the period-four example in Section 48, take $n_0 = 0$ and $B = \text{diag}(1, 2)$. With $k = 2$, (140) becomes

$$(2n(2 - n) - 2y_1)^2 + (n(n - 1) - 4y_2)^2 = 0.$$

Its integer solution is unique when it exists, namely $y_1 = n(2 - n)$ and $y_2 = n(n - 1)/4$. Existence is equivalent to $n \equiv 0, 1 \pmod{4}$. This example exposes all coefficients and witnesses; there is no implicit matrix-word decoder.

50 Why the classification is not a uniform decision algorithm

It is important to distinguish a theorem about the form of every fixed hit set from an algorithm computing that form from arbitrary subgroup data. Finitely generated subgroups of a direct product of two free groups can have undecidable membership; the fibre-product mechanism used in the repository and in the classical literature supplies such examples [44, 36].

Proposition 50.1 (A nonconstant all-or-empty obstruction). *There is a fixed finitely generated subgroup $K' \leq \text{SL}_6(\mathbb{Z})$ such that no algorithm, given the coefficients of an affine unimodular curve, always decides whether its hit set in K' is empty. This remains true for a family of nonconstant curves whose hit sets are promised to be either empty or all of $\mathbb{Z}$.*

Proof. Fix a finitely generated $K_0 \leq \text{SL}_4(\mathbb{Z})$ with undecidable membership, and let

$$E(t) = \begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix}, \quad K' = \{\text{diag}(h, E(m)) : h \in K_0, m \in \mathbb{Z}\}.$$

For a supplied $C \in \text{SL}_4(\mathbb{Z})$, define

$$L_C(n) = \text{diag}(C, E(n)).$$

This is nonconstant and entrywise affine; its coefficient of n has rank one. Every $E(n)$ lies in $\langle E(1) \rangle$, so

$$\{n : L_C(n) \in K'\} = \begin{cases} \mathbb{Z}, & C \in K_0, \\ \emptyset, & C \notin K_0. \end{cases}$$

An algorithm deciding which case holds would decide membership in K_0 . $\square$

In particular, Theorem 49.1 is effective *from an accepted point and an intersection-lattice basis*. It is not a claim that those data can always be extracted from arbitrary matrix generators. The ordinary input in the rigidity theorem is the varying scalar n , not the entries of the whole fixed matrix C . Confusing these two input conventions would incorrectly turn the classification into a solution of an undecidable problem.

51 A commuting-polynomial extension

Affinity forces all normalized coefficients to lie in the algebra generated by one nilpotent matrix. A related argument works for higher-degree matrix polynomials provided their normalized coefficient matrices commute.

Theorem 51.1 (Commuting polynomial inputs). *Let*

$$L(t) = CM(t), \quad M(t) = I + \sum_{j=1}^D t^j A_j,$$

where $C \in \mathrm{SL}_d(\mathbb{Z})$, the matrices $A_j \in M_d(\mathbb{Z})$ commute pairwise, and $\det M(t) = 1$ identically. For every subgroup $K \leq \mathrm{SL}_d(\mathbb{Z})$, the hit set $T = \{n : L(n) \in K\}$ is finite or exactly periodic.

If M is nonconstant, put $E = D(d-1)$. The finite case has at most E elements. More precisely, $A = \langle M(n) : n \in \mathbb{Z} \rangle$ is free abelian of some rank $h \leq E$. In the nonempty case, if $K \cap A$ has full rank in A and the quotient has exponent e , then $P_E(e)$ is a period of T . If the intersection has smaller rank, $|T| \leq E$.

Proof. Commuting matrices over an algebraically closed field can be simultaneously upper triangularized. For completeness, find a common eigenvector by restricting successive commuting operators to an eigenspace of a preceding one; these restrictions preserve that eigenspace. Induction on the quotient dimension then gives a common triangular basis.

In this basis, the diagonal entries of $M(t)$ are scalar polynomials with constant term one. Their product is the constant polynomial one. Since a polynomial ring over a field has no nonconstant units, each diagonal polynomial is one. Thus each A_j is strictly upper triangular in the common basis. Every product of d such matrices is zero. Consequently the commutative algebra without identity generated by the A_j is nilpotent of index at most d .

The finite logarithm

$$f(t) = \log M(t) = \sum_{\ell=1}^{d-1} \frac{(-1)^{\ell+1}}{\ell} (M(t) - I)^\ell$$

is a matrix-valued rational polynomial of degree at most E and has $f(0) = 0$. By integral interpolation, its integer evaluations generate the same additive group as the finite list $f(1), \dots, f(E)$. All their entries are rational. That generated group is finitely generated and torsion-free, hence free abelian, of rank $h \leq E$. Logarithm and exponential identify it with A .

Choose an abstract integer basis of this additive group and express $f(t)$ in its coordinates. The resulting vector $\mathbf{F}(t) \in \mathbb{Q}[t]^h$ has degree at most E , vanishes at zero, takes values in $\mathbb{Z}^h$ at integers, and its integer evaluations span $\mathbb{Q}^h$. If n_0 is accepted, cancellation as in Theorem 46.1 gives a lattice $\Lambda \leq \mathbb{Z}^h$ with

$$n \in T \iff \mathbf{F}(n) - \mathbf{F}(n_0) \in \Lambda.$$

For full-rank Λ , each coordinate is periodic modulo the quotient exponent e with period $P_E(e)$, by Theorem 47.4 and the Newton basis proof.

If $\text{rank } \Lambda < h$, choose a nonzero rational linear functional ℓ annihilating Λ . Every accepted n is a root of $q(t) = \ell(\mathbf{F}(t) - \mathbf{F}(n_0))$. This polynomial is not identically zero: otherwise evaluation at zero first gives $\ell(\mathbf{F}(n_0)) = 0$, and then ℓ vanishes on all evaluations of $\mathbf{F}$, contradicting their spanning property. Hence q has at most E roots. $\square$

The affine case has the stronger rank-sensitive finite bound $r + 1$ because its logarithmic curve is a full moment curve. An arbitrary commuting-polynomial curve can have algebraic dependencies, so the proof above uses the degree of a separating scalar polynomial instead.

The hypothesis is *pairwise commutativity of the normalized coefficients*. It must not be replaced by the assertion that every individual evaluated matrix is unipotent. A family of unipotent matrices can vary through noncommuting directions. The quadratic universal curves in the next section do exactly this.

52 Input degree two is sharp, even with unrestricted dimension

52.1 The upper bound is prior repository work

The following upper bound is the group-commutator construction already present in **ProveIt** [36]. We reconstruct its mathematical interfaces because the conclusion about a sharp threshold depends on more than a bare statement that some matrix membership problem is undecidable. The proof uses the effective form of Higman's embedding theorem [42, 43]. It does not instantiate a particular universal finite presentation or provide a fixed-size Diophantine checker for arbitrary matrix words.

Theorem 52.1 (Quadratic upper bound; repository construction). *There is a fixed degree-two curve $\Gamma : \mathbb{Z} \rightarrow \text{SL}_4(\mathbb{Z})$ such that, for each computably enumerable $S \subseteq \mathbb{Z}_{>0}$, one can construct a finitely generated subgroup K_S with*

$$n \in S \iff \Gamma(n) \in K_S \quad (n > 0).$$

One choice is $\Gamma(n) = \text{diag}(R(n), R(n))$, where, with $t = 4n - 1$,

$$R(n) = \begin{pmatrix} 1 - 4t & -16 \\ t^2 & 1 + 4t \end{pmatrix}. \quad (141)$$

Proof. We separate four steps so that the input curve is not confused with the quotient group in which the computation is encoded.

Step 1: an exact recursively presented group. Write $[g, h] = ghg^{-1}h^{-1}$, $a_i = b^{-i}ab^i$, and

$$G_S = \langle a, b \mid [a_n, a_0] = 1 \text{ for } n \in S \rangle.$$

Consider also the graph group on generators c_i , $i \in \mathbb{Z}$, with a commuting relation between c_i, c_j exactly when $|i - j| \in S$. Translation of the indices defines an automorphism. Adjoining a stable generator b with $b^{-1}c_i b = c_{i+1}$ gives a semidirect product in which the graph group embeds. Eliminating all c_i except c_0 gives the presentation of G_S , with $a = c_0$.

If $n \in S$, the commutator $[a_n, a_0]$ is a defining relation. If $n > 0$ and $n \notin S$, map c_n, c_0 to the two free generators of F_2 and every other c_i to the identity. Every graph relation is respected, since the only pair that could fail to commute is the missing edge. The image of $[c_n, c_0]$ is a nontrivial reduced free commutator. Thus

$$[a_n, a] = 1 \text{ in } G_S \iff n \in S. \quad (142)$$

The positive-input convention excludes $n = 0$, whose commutator is always trivial.

Step 2: a finite presentation with named free letters. An enumerator for S enumerates the relators of G_S . Effective Higman embedding gives a finitely presented group H_0 and an injective homomorphism $G_S \hookrightarrow H_0$, together with words for the images of a, b [42, 43]. Adjoin fresh presentation letters named a, b and relations identifying them with these image words. The resulting finite presentation H still contains G_S injectively. Its ambient free group F has finitely many basis letters, two of them the named a, b .

The distinction is essential: a, b are independent letters in F , though their images in the quotient H may satisfy relations. A faithful matrix representation is assigned to F , not directly to H . No bound on the number of presentation letters is needed for the matrix curve below.

Step 3: a fibre-product subgroup. Let $H = F / \langle\langle R_1, \dots, R_m \rangle\rangle$ and set

$$M(H) = \{(u, v) \in F \times F : u = v \text{ in } H\}.$$

This subgroup is generated by (z, z) for the finitely many free basis letters z , together with $(1, R_j)$ for $1 \leq j \leq m$. [write 78] (This is Proposition `pqc:dr:prop:mihailova` of [13], also its Lemma `pqc:rr:lem:mihailova` and Section 3 of [14]; the verification that follows is source 07's, printed as a second route.) Indeed, if $u^{-1}v$ is a product of conjugates $gR_j^{\pm 1}g^{-1}$, each factor is represented by

$$(1, gR_j^{\pm 1}g^{-1}) = (g, g)(1, R_j)^{\pm 1}(g, g)^{-1},$$

and multiplication after (u, u) gives (u, v) . This is the usual Mihailova fibre-product mechanism [44].

Conjugate once by $(a, 1)$:

$$M_a = (a, 1)M(H)(a^{-1}, 1).$$

Then

$$\begin{aligned} (a_n, a_n) \in M_a &\iff (a^{-1}a_n a, a_n) \in M(H) \\ &\iff a^{-1}a_n a = a_n \text{ in } H \\ &\iff [a_n, a] = 1 \text{ in } H \\ &\iff n \in S. \end{aligned}$$

Injectivity of the embedding in Step 2 is what justifies the last equivalence.

Step 4: a faithful fixed matrix input. Let

$$V = \begin{pmatrix} 1 & 4 \\ 0 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}.$$

These matrices freely generate a rank-two free group. One elementary verification starts with the matrices having off-diagonal entries 2, 2. On the real projective line, nonzero powers of the upper shear send $|x| < 1$ into $|x| > 1$, and nonzero powers of the lower shear send $|x| > 1$ into $|x| < 1$. The ping-pong argument proves freeness. Conjugating by $\text{diag}(2, 1)$ changes the off-diagonal entries to 4, 1 and preserves freeness. This is a standard free-matrix construction; see also [45].

In the free group on V, B , the elements $V^i B V^{-i}$ at distinct integer indices are freely independent. To check this directly, combine adjacent occurrences with the same index. A nonempty reduced word in the remaining conjugates has the form

$$V^{i_1} B^{e_1} V^{i_2 - i_1} B^{e_2} \dots V^{i_s - i_{s-1}} B^{e_s} V^{-i_s},$$

with nonzero e_j and distinct adjacent indices. Its internal alternating blocks cannot cancel, so it is nontrivial.

Assign the free basis letter b to B , a to

$$A = V B V^{-1} = \begin{pmatrix} 5 & -16 \\ 1 & -3 \end{pmatrix},$$

and any remaining letters to distinct $V^i B V^{-i}$ with $i \geq 2$. This gives a faithful representation $\rho : F \hookrightarrow \text{SL}_2(\mathbb{Z})$ for every finite rank, with the same images of a, b . A direct multiplication gives

$$\rho(a_n) = B^{-n} A B^n = R(n)$$

with R as in (141). The image of M_a under the faithful block-diagonal map $(u, v) \mapsto \text{diag}(\rho(u), \rho(v))$ is the required finitely generated subgroup K_S . All its generators are computable from the finite presentation, while the displayed curve is independent of S . $\square$

The two-generator-output refinement of effective Higman embedding is not needed in this reconstruction. It would permit a different convenient finite-rank embedding, but the infinite free family in Step 4 already handles arbitrary finite presentation rank.

[write 78] Theorem 52.1 and its proof reconstruct Sections 1–4 of the research note [14], as source 07 says; the theorem is not a result of this report. Step 4 also re-derives the free pair and conjugate family of [15] (its Section 1).

Corollary 52.2 (Sharp degree threshold). *Among ordinary-input representations (127) with integral unimodular matrix curves and a fixed finitely generated subgroup, the least possible entrywise polynomial degree capable of representing a noncomputable computably enumerable set is two. This remains true when the matrix dimension is unrestricted; degree two suffices in dimension four.*

Proof. Degree zero or one is ruled out in every dimension by Corollary 46.2. Degree two is supplied by Theorem 52.1 after choosing a noncomputable computably enumerable S . $\square$

This is a lower bound on *input degree in a specified subgroup interface*. It is not a lower bound on the degree of Diophantine polynomials, arithmetic operations, number of witnesses, or matrix dimension for other computational interfaces.

52.2 Pointwise unipotence is not enough

The same repository gives another quadratic loader,

$$Q(n) = \begin{pmatrix} 1 + 12n & 1 \\ -144n^2 & 1 - 12n \end{pmatrix}. \quad (143)$$

[write 78] This curve is $L_*(x)$, equation (12) of the research note [15], with $c = 12$; it is also printed in [14]. Source 07 does not cite the former note. Every evaluated matrix satisfies $\det Q(n) = 1$, $\operatorname{tr} Q(n) = 2$, and $(Q(n) - I)^2 = 0$. Nevertheless its normalized coefficient matrices do not commute. In fact

$$Q(0)^{-1}Q(n) = I + nX + n^2Y,$$

where

$$X = \begin{pmatrix} 12 & 12 \\ 0 & -12 \end{pmatrix}, \quad Y = \begin{pmatrix} 144 & 0 \\ -144 & 0 \end{pmatrix}.$$

For example $(XY)_{11} = 0$ but $(YX)_{11} = 1728$. Thus Theorem 51.1 does not apply. A restriction stated merely as “unipotent matrix inputs” would not express the required obstruction.

53 Multiplication inside a jointly affine abelian pencil

The negative theorem does not survive unrestricted existential coordinates. The following small construction makes the reason explicit.

Let

$$J = E_{12} + E_{23}, \quad P = I + J, \quad B(t) = \begin{pmatrix} t \\ 2 \end{pmatrix} = \frac{t(t-1)}{2},$$

and define a jointly affine matrix pencil

$$T(t, s) = I + tJ + sJ^2 = \begin{pmatrix} 1 & t & s \\ 0 & 1 & t \\ 0 & 0 & 1 \end{pmatrix}. \quad (144)$$

It is an integer unimodular matrix for every integer assignment, and

$$P^m = I + mJ + B(m)J^2 \quad (m \in \mathbb{Z}). \quad (145)$$

The binomial identity for negative powers follows either from the finite nilpotent expansion or by checking the multiplication law and inverse. The entry (1, 2) makes the exponent unique. Consequently

$$T(t, s) \in \langle P \rangle \iff s = B(t). \quad (146)$$

Here subgroup membership is easy: it is one quadratic equality. The computational effect will come from existentially choosing coordinates that must satisfy many such equalities together.

Theorem 53.1 (Six-dimensional multiplication with one auxiliary). *For all integers x, y, z, u ,*

$$\operatorname{diag}(T(x + y, u), T(x - y, u - 2z + y)) \in \langle P \rangle \times \langle P \rangle \quad (147)$$

holds if and only if

$$z = xy \quad \text{and} \quad u = B(x + y).$$

The matrix in (147) is jointly affine in x, y, z, u , lies in $\operatorname{SL}_6(\mathbb{Z})$ for every assignment, and has exactly one auxiliary solution u for every true multiplication triple (x, y, xy) .

Proof. By (146), membership is equivalent to

$$u = B(x + y), \quad u - 2z + y = B(x - y).$$

The identity

$$B(x + y) - B(x - y) = 2xy - y \tag{148}$$

turns their difference into $2z = 2xy$. Conversely, if $z = xy$ and $u = B(x + y)$, (148) gives the second equation. All matrix entries in (147) are affine expressions. Both blocks have determinant one for every assignment, not just on their zero set. Uniqueness of u follows from the first equation. $\square$

The auxiliary u is nonnegative even for signed x, y , because $B(t) \geq 0$ for every integer t . No nonnegativity assumption is needed for the other variables or for the two cyclic exponents.

Remark 53.2 ([write 78] The binomial device of Part II). In the coordinates of Section 1.2, $T(t, s) = h(t, t, s)$ and $P = h(1, 1, 0)$, so (146) says that $h(t, t, s)$ lies in $\langle h(1, 1, 0) \rangle$ exactly when $s = \mathfrak{b}(t)$. Part II's third phase uses the same binomial term in another cyclic subgroup, $\langle h(-1, -1, 0) \rangle$ (equation (66)); the two subgroups differ, since $h(1, 1, 0)^{-1} = h(-1, -1, 1)$. The polarizations differ as well: (148) here, $st = \mathfrak{b}(s + t) - \mathfrak{b}(s) - \mathfrak{b}(t)$ in (57). So do the mechanisms. Part II puts the existential witnesses into the exponents of three commutative phases and keeps the target on a fixed central line; Part IV puts them into the target coordinates of a jointly affine pencil tested against one fixed free abelian subgroup $\langle P \rangle^{2g} \times \{I_2\}$. Part II had already shown that commutative unipotent data can carry universal existential arithmetic (Theorem 18.1); source 07, which could not know that Part, reaches the same conclusion independently in Corollary 54.2, and its Proposition 56.1 plays the role of Part II's phase threshold for its own resource, the number of quantified coordinates.

53.1 A literal quartic certificate

Define the quadratic residuals

$$R_1 = 2u - (x + y)(x + y - 1), \tag{149}$$

$$R_2 = 2(u - 2z + y) - (x - y)(x - y - 1). \tag{150}$$

Then

$$R_1 - R_2 = 4(z - xy). \tag{151}$$

Thus the single polynomial $R_1^2 + R_2^2$ vanishes over the integers precisely on the complete witness graph in Theorem 53.1. Its degree is at most four. The identity also provides a simple corruption diagnostic: changing the product coordinate while retaining the two residual equations is impossible.

There is no assertion that dimension six or one auxiliary is optimal under every possible matrix encoding. These are explicit achieved bounds for this particular jointly affine, independent-block construction. The use of two different affine arguments, $x + y$ and $x - y$, is what recovers their product while leaving all matrix entries affine.

54 A witness-preserving circuit compiler

54.1 The source format and the charged resources

Take an integer polynomial given by a straight-line arithmetic circuit using addition, subtraction, integer constants, and multiplication. Absorb every purely affine computation between multiplication

gates into the operands of the next such gate. After ordering the multiplication gates topologically, the source has the form

$$z_i = A_i(\mathbf{v}, z_1, \dots, z_{i-1})B_i(\mathbf{v}, z_1, \dots, z_{i-1}), \quad 1 \leq i \leq g, \quad (152)$$

with final output $A_0(\mathbf{v}, \mathbf{z})$, where all A_i, B_i, A_0 are integral affine forms. This normalization is an algebraic reorganization, not a claim that large coefficients or long affine forms have zero implementation cost. The parameter g counts multiplication gates only, and the resulting coefficient sizes still belong to the output data.

For each gate, introduce one additional coordinate u_i . Its two blocks are

$$T(A_i + B_i, u_i), \quad T(A_i - B_i, u_i - 2z_i + B_i). \quad (153)$$

Append the two-dimensional block $E(A_0)$ from Section 50. Define

$$K_g = \langle P \rangle^{2g} \times \{I_2\} \leq \mathrm{SL}_{6g+2}(\mathbb{Z}). \quad (154)$$

Each cyclic factor occupies its own three-dimensional diagonal block. This is a fixed free-abelian subgroup of rank $2g$, depending only on g , not on the coefficients of the source polynomial or on its input.

Theorem 54.1 (Affine abelian compiler). *For the source circuit (152), let $\mathcal{L}(\mathbf{v}, \mathbf{z}, \mathbf{u})$ be the block diagonal matrix consisting of (153) for all gates and the final block $E(A_0)$. Then:*

- (i) $\mathcal{L}$ is jointly affine in every source, register, and auxiliary coordinate, and is in $\mathrm{SL}_{6g+2}(\mathbb{Z})$ for every integer assignment.
- (ii) For every fixed source tuple $\mathbf{v}$, the source polynomial vanishes if and only if there exist $\mathbf{z}, \mathbf{u} \in \mathbb{Z}^g$ with $\mathcal{L}(\mathbf{v}, \mathbf{z}, \mathbf{u}) \in K_g$.
- (iii) Whenever the source polynomial vanishes, this tuple of $2g$ added integer coordinates is unique.
- (iv) All normalized coefficient matrices of $\mathcal{L}$ commute. An explicit quartic sum of squares defines precisely the same integer witness set.

Proof. All entries in (153) and $E(A_0)$ are affine combinations of the coordinates. Each diagonal block is upper triangular with diagonal entries one, so the determinant of the entire matrix is one identically.

By Theorem 53.1, membership of gate i in its two cyclic factors is equivalent to the multiplication equation in (152), together with

$$u_i = B(A_i + B_i).$$

Because A_i, B_i depend only on source coordinates and earlier registers, induction on i shows that all z_i are exactly the source circuit values and are uniquely determined. The u_i are then uniquely determined too. The last block belongs to $\{I_2\}$ precisely when $A_0 = 0$. This proves the equivalence and uniqueness.

Within a three-dimensional block, every matrix is a polynomial in the same J of degree at most two; within the last block every matrix is a polynomial in E_{12} of degree at most one. Different blocks commute. The constant block matrix is invertible in this same commutative algebra, so multiplying by its inverse to normalize the constant term preserves coefficient commutativity.

For the polynomial certificate, take A_0^2 plus the squares of the two residuals (149)–(150) with $(x, y, z, u) = (A_i, B_i, z_i, u_i)$ for each gate. As the operands are affine in the full coordinate list, all gate residuals are quadratic. Their squared sum is an integral polynomial of degree at most four, and zero exactly when every block has the required membership. $\square$

The compiler preserves the source witness set by a bijection after adjoining its uniquely determined computational coordinates. It does not reduce the number of original solutions of the source equation. The group-word exponents are also determined by the displayed first-superdiagonal entries, so this construction does not hide uncontrolled word multiplicities inside an undecidable membership oracle.

54.2 A precise resource ledger

If the source has m existential integer coordinates and g multiplication gates, the output uses $m + 2g$ existential integer coordinates and dimension $6g + 2$. It has $2g$ independent cyclic subgroup generators and $2g + 1$ residual equations, combined into one quartic polynomial. The matrix itself has $6g + 2$ diagonal entries equal to one and only a linear number of nonconstant off-diagonal positions.

These are structural counts, not a total arithmetic-operation count for evaluating the final polynomial. Expanding long affine forms can increase that count, as can converting natural domains to integer domains. Neither the six-dimensional multiplication gadget nor the dimension formula is a comparison with the repository’s fixed universal-polynomial operation benchmark.

[write 78] (2 October 2026.) The research programme’s review of this manuscript [20] confirms Theorems 53.1 and 54.1 and proves a reduction for square gates. For a gate $z = x^2$ the second block $T(0, u - 2z + x)$ lies in $\langle P \rangle$ exactly when $u = 2z - x$. Deleting that block and coordinate and keeping $T(2x, 2z - x)$, whose membership equation is $4(z - x^2) = 0$, gives a bijection of the complete integer zero sets, and the old sum of squares equals the new one on the restoration graph. With s squares among the g gates the bounds become dimension $6g - 3s + 2$, $2g - s$ added coordinates and $2g - s + 1$ residuals; the example of Section 55, whose second gate is a square, goes from dimension 20 to 17, from six added coordinates to five and from seven residuals to six. No operation count is asserted there either.

Corollary 54.2 (Computably enumerable relations through affine abelian inputs). *Every computably enumerable subset of $\mathbb{N}^a$ has a representation*

$$\mathbf{n} \in S \iff \exists \mathbf{w} \in \mathbb{Z}^m : \mathcal{L}(\mathbf{n}, \mathbf{w}) \in K,$$

where K is a fixed finitely generated free-abelian unipotent matrix subgroup and $\mathcal{L}$ is a fixed jointly affine integral unimodular pencil with commuting normalized coefficients.

Proof. MRDP gives an integer-coefficient polynomial with existential natural witnesses [46, 38]. Replace each natural witness by a sum of four integer squares. Lagrange’s four-square theorem gives an equivalent equation with unrestricted integer witnesses. Compile a circuit for this substituted polynomial using Theorem 54.1. The resulting g , matrix dimension, subgroup, and affine pencil are all fixed once S is fixed. $\square$

The four-square substitution can introduce multiple representations of one original natural witness. The uniqueness assertion of Theorem 54.1 concerns only its additional gate and triangular-number

coordinates, conditional on all source coordinates. Consequently Corollary 54.2 supplies no single-fold MRDP theorem. It also supplies no numerical universal dimension until a particular universal source polynomial and a particular circuit have actually been chosen and counted.

This corollary uses MRDP as an external theorem; it is not an alternative derivation of MRDP from abelian subgroup membership. Its value is an explicit interface translation and a sharp conceptual separation: even completely commutative matrix data can carry universal existential arithmetic, although no one-parameter commuting-polynomial input curve can do so without those existential coordinates.

55 A completely worked circuit example

Consider

$$f(a, b, c) = (ab + c)^2 - (a + c)b - 7. \quad (155)$$

A source circuit uses three multiplication gates:

$$z_1 = ab, \quad z_2 = (z_1 + c)(z_1 + c), \quad z_3 = (a + c)b, \quad A_0 = z_2 - z_3 - 7.$$

The compiler therefore produces a matrix in $\text{SL}_{20}(\mathbb{Z})$, a fixed subgroup of rank six, and six added integer coordinates $(z_1, z_2, z_3, u_1, u_2, u_3)$. Its seven blocks are

$$\begin{aligned} T(a + b, u_1), & & T(a - b, u_1 - 2z_1 + b), \\ T(2z_1 + 2c, u_2), & & T(0, u_2 - 2z_2 + z_1 + c), \\ T(a + c + b, u_3), & & T(a + c - b, u_3 - 2z_3 + b), \\ E(z_2 - z_3 - 7). \end{aligned}$$

Every displayed entry is affine in the full coordinate list. The source polynomial has not been expanded into high-degree matrix entries.

At $(a, b, c) = (2, 2, -1)$, the unique added coordinates are

$$(z_1, z_2, z_3) = (4, 9, 2), \quad (u_1, u_2, u_3) = (6, 15, 3).$$

The resulting blocks are

$$T(4, 6), T(0, 0), T(6, 15), T(0, 0), T(3, 3), T(-1, 1), E(0).$$

Their cyclic exponents are respectively 4, 0, 6, 0, 3, -1, and the final block is the identity. The negative exponent is intentional: $B(-1) = 1$ and $P^{-1} = I - J + J^2$. A nonnegative-word interpretation without inverse generators would be a different substrate and would invalidate this example.

The delivered implementation tests all $11^3 = 1331$ source triples in $[-5, 5]^3$. Exactly

$$(-4, 2, 5), \quad (2, 2, -5), \quad (2, 2, -1)$$

are accepted in this box. For each of these three solutions, each of its six added coordinates is perturbed independently by +1 and -1; all 36 corruptions are rejected. The theorem proves rejection of every incorrect witness tuple, not merely these perturbations.

56 A decidable one-witness boundary

It is tempting to read Corollary 54.2 as saying that a single auxiliary input coordinate already suffices. For the explicit block-cyclic family used here, that is false.

Proposition 56.1 (One quantified coordinate is insufficient). *Fix finitely many integral affine forms $\ell_i(n, y), m_i(n, y)$ and optional affine identity-test forms $a_j(n, y)$. The predicate*

$$\exists y \in \mathbb{Z} : \quad T(\ell_i(n, y), m_i(n, y)) \in \langle P \rangle \text{ for every } i, \quad E(a_j(n, y)) = I \text{ for every } j$$

is recursive as a predicate of n . The same holds with $y \in \mathbb{N}$.

Proof. For a fixed integer input n , the condition is a finite conjunction of polynomial equations in one variable y :

$$2m_i(n, y) - \ell_i(n, y)(\ell_i(n, y) - 1) = 0, \quad a_j(n, y) = 0.$$

Their degrees in y are at most two. If every polynomial is identically zero, a witness exists. Otherwise select one nonzero polynomial. A nonzero constant has no root; a linear polynomial has at most one candidate integer root; a quadratic has at most two candidates, computable by its discriminant and exact divisibility tests. Test these finitely many candidates in every remaining equation, imposing $y \geq 0$ in the natural case. This always terminates and decides the predicate. $\square$

Thus a universal existential projection in this particular normal form needs at least two quantified coordinates. We do not claim that two suffice, or that this elementary lower bound is close to optimal. It also says nothing about affine pencils tested against arbitrary nonabelian subgroups with undecidable direct membership.

56.1 The quantifier boundary in one table

Input interface	Subgroup assumptions	Proven boundary
One affine integer input	Any fixed subgroup, any dimension	Finite or exactly periodic
One polynomial integer input; commuting normalized coefficients	Any fixed subgroup	Finite or exactly periodic
One quadratic integer input	Suitable fixed finitely generated subgroup of $\mathrm{SL}_4(\mathbb{Z})$	Can represent every c.e. language by the repository construction
Jointly affine inputs with sufficiently many existential coordinates	Fixed free-abelian unipotent subgroup; commuting coefficients	Can represent every c.e. language, using MRDP and the explicit compiler
Jointly affine input with one quantified coordinate in the printed block-cyclic form	Independent copies of $\langle P \rangle$ and identity tests	Recursive

The first two negative results and the fourth positive result are compatible because projecting an integer algebraic relation can radically change its complexity. Low-degree loading and an easy group-membership test do not imply a decidable existential theory.

57 Effective normal forms when the lattice is supplied

The hidden intersection problem in Section 50 should not obscure the effective content once the intersection lattice is available. Suppose n_0 and a column basis B have been supplied, as in Theorem 49.1. Smith normal form gives unimodular matrices W, V such that

$$WBV = \begin{pmatrix} d_1 & & 0 \\ & \ddots & \\ 0 & & d_r \\ \hline & & 0 \end{pmatrix}, \quad d_i > 0, \quad d_i \mid d_{i+1},$$

with the evident rectangular interpretation. Put $\mathbf{q}(n) = W(\boldsymbol{\theta}(n) - \boldsymbol{\theta}(n_0))$. Then the hit predicate is precisely

$$d_i \mid q_i(n) \quad (1 \leq i \leq r), \quad q_i(n) = 0 \quad (r < i \leq k). \quad (156)$$

Each q_i is an explicitly computable integer-valued rational polynomial of degree at most k .

If $r = k$, the quotient exponent is $e = d_k$. Compute $P = P_k(e)$ and test the residues $0, \dots, P - 1$ in (156); the accepted residues are the full exact description of the hit set. This is a finite procedure, not a polynomial-time claim: a binary-encoded period can be numerically large.

If $r < k$, at least one equality polynomial q_i with $i > r$ is nonzero. Otherwise the moment curve would lie in a proper affine space, contrary to Lemma 45.3. Clear denominators, compute the finite list of integer roots of such a polynomial, and filter it through (156). A root algorithm can use a standard effective rational-root bound followed by finite search; no unresolved integer-polynomial problem remains in one variable. The theorem guarantees that the final list has at most $r + 1$ elements.

This gives a practical separation of responsibilities for a future formal implementation: subgroup reasoning supplies n_0 and B ; elementary integer linear algebra and one-variable arithmetic then produce the final language description and the polynomial certificate.

58 Executable evidence and proof obligations

58.1 What the delivered program checks

The file `verify.py` uses only Python's standard library. Integer computations are exact; rational linear algebra uses `fractions.Fraction`. The deterministic test seed is 20261002. The included receipt `verification.json` records the following successful cases.

Test family	Cases
Truncated-ring factorizations and independent moment identities	405
Exact periods and simultaneous zero-residue kernels	43,452
Prescribed finite hit sets, checked by exact lattice membership	2,232
Full-rank lattice hit sets and their predicted periods	10,452
The explicitly printed finite and periodic membership examples	603
Multiplication gadget: correct and corrupted assignments	30,625
Compiled source-circuit assignments in $[-5, 5]^3$	1,331
Corrupted accepted-circuit witnesses rejected	36
Algebraic checks of the repository's quadratic loader	2,001
Rejection of noninteger input types	5
Total recorded cases	91,142

The totals count test cases, not all individual assertions: several cases check more than one identity. In particular, the period suite tests $1 \leq k \leq 8$ and $1 \leq e \leq 24$, including negative integers, and checks that no smaller positive period has the required zero residue. The finite-set suite constructs the prescribed lattice and uses exact rational elimination rather than searching only for short subgroup words. The arbitrary full-rank examples use independent integral lattices, not only diagonal ones.

The multiplication suite tests $x, y \in [-12, 12]$ and independent errors in z and u from -3 to 3 . The circuit suite compares the matrix condition, its quartic residual, and direct source evaluation. The quadratic-loader checks establish determinant and pointwise nilpotence identities on the tested range; they do not test its computability-theoretic universality.

58.2 What the tests do not establish

Finite testing cannot prove the all-dimensional theorem, the exact formula for all moduli, or the universal-group embedding. Those are established by the mathematical arguments and their explicitly stated external dependencies. Conversely, a correct proof does not automatically certify every line of its implementation. The two forms of evidence are therefore kept separate.

No new Lean or Rocq development was compiled for this report. The existence of an MRDP theorem in the repository does not formally verify our nilpotent-lattice arguments or our matrix gadget. Nor was an actual universal Higman presentation materialized and converted to a numerical matrix generating list. A proposed use of that list in an operation-count competition would still have to charge the presentation, input loading, and complete unbounded membership-certificate construction.

The executable circuit objects represent affine operands, causal multiplication registers, witness generation, block evaluation, and the quartic checker. They are a reference implementation and test harness, not a general computer-algebra simplifier or a certified arithmetic-operation optimizer.

58.3 Reproduction

From the extracted package directory, run

```
python3 verify.py --output verification.replayed.json
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

Python 3.10 or later and an ordinary LaTeX installation with the packages named in the preamble suffice. The supplied receipt can be compared with a fresh run. All checks are local; no network access or external solver is required.

[write 78] In this report the program is shipped as `code/07-affine-inputs-verify.py`, the receipt as `data/07-affine-inputs-verification.json` and `SOURCE_AUDIT.md` as `07-affine-inputs-SOURCE_AUDIT.md`; source 07's `article.tex`, `article.pdf` and `README.md` are not shipped. Without `-output` the program writes `verification.json` in the current directory, overwriting a file of that name, and it never compares its output with the bundled receipt; compare explicitly, as the README shows. When the manuscript was placed, the command above, run on a copy, passed all 91,142 cases in 7 seconds, and its output equalled the receipt apart from line endings.

59 Further research questions and concrete continuation targets

The following problems are ordered approximately from immediately implementable to more exploratory. Their status is relative to the work and sources audited here; additional literature review is needed before claiming any is globally open.

1. **Formalize the exact affine evaluation lattice.** A focused Lean development should first prove Lemma 45.1 and Theorem 45.2, then the coset cancellation and moment-curve rank bound. This isolates the main result from the much larger group-embedding machinery. A useful final interface would take C, D, K and return the finite-or-periodic alternative without claiming computable extraction of the intersection lattice.
2. **Locate the strongest existing polynomial-sequence theorem containing the classification.** Compare the result carefully with the group-polynomial literature of Leibman and Hu and the integer-valued-polynomial literature [40, 41, 39]. The questions are not only whether finite-or-periodic behavior is implicit there, but whether the exact basis $U(1), \dots, U(k)$, the prescribed finite-set realization, and the quotient-exponent period $P_k(e)$ are stated in an equivalent form. This is the principal remaining publication-priority audit.
3. **Compute intersections for tractable subgroup classes.** Identify hypotheses on a finitely generated K that make $K \cap A_N$ effectively computable, with complexity bounds. Congruence-defined groups, explicitly abelian groups, and supplied polycyclic normal forms are natural starting points. Any proposed unrestricted algorithm must confront Proposition 50.1; merely proving abstract finite generation is insufficient.
4. **Classify the periodic languages at fixed nilpotency index.** For $k = 1$ only one coset occurs; for $k = 2$ the set of two residues $\{0, 1\} \pmod{4}$ already occurs. Determine exactly which periodic sets arise as preimages of lattice cosets under $\theta(n)$ for a fixed k . A first concrete target is a complete local prime-power description for $k = 2$, including which unions of roots of quadratic congruences are attainable simultaneously.
5. **Extend the polynomial theorem beyond commuting coefficients.** The quadratic universal example shows that no unrestricted extension is possible. There may nevertheless be a larger tractable class in which all evaluations lie in a specified finitely generated nilpotent group with effective Malcev coordinates. The task is to formulate hypotheses that give polynomial coordinate functions and then prove explicit finite-hit and period bounds, rather than assume them from pointwise unipotence.

6. **Optimize the affine multiplication gadget.** Can a jointly affine unimodular multiplication interface use a smaller matrix dimension while retaining uniquely determined added coordinates and an elementary fixed subgroup? Within the present independent cyclic-block model, study whether two three-dimensional blocks are necessary. A meaningful lower bound must state which coordinate sharing, subgroup coupling, and integer-domain constraints are permitted.
7. **Share coordinates and blocks across an entire circuit.** The $6g + 2$ dimension and $2g$ added-coordinate bounds are compositional, not optimized. Repeated affine operands, squarings, and shared sums may permit reuse of triangular-number coordinates or cyclic tests. The challenge is to retain the exact witness bijection and to count actual affine-form evaluation costs instead of only multiplication gates.
8. **Find the least universal existential budget in the block-cyclic model.** Proposition 56.1 rules out one quantified coordinate; MRDP plus the compiler gives some finite budget. Determine substantially stronger lower or upper bounds after fixing a precise universal source relation. This problem must count all original Diophantine witnesses, not just the compiler’s uniquely determined additions.
9. **Replace subgroups by semigroups or constrained word languages.** The rigidity proof uses inverses in the step from $L(n)$ to $L(n_0)^{-1}L(n)$ and in the lattice structure of $K \cap A_N$. It therefore does not automatically apply to positive-word semigroups or rational subsets. Determine which affine-input classifications survive with these different acceptance semantics, and which counterexamples can be constructed without importing a hidden arbitrary computation into the input map. [write 78] For rational subsets the negative half is answered by Part II: by Remark 46.3, products of three subgroups, or of three free commutative submonoids, of H^d can have nonrecursive hit sets along an affine line. Products of two factors, positive-word semigroups and other constrained word languages remain open. [write 82] (3 October 2026.) Part V does not answer the semigroup case: its fixed positive semigroup is universal, but its target map $E(\ell, \rho)$ is not an affine curve (its entries grow exponentially with the tape length, Section 71), so it gives no affine-input classification and no counterexample to one.
10. **Charge a complete universal matrix certificate.** Instantiate a concrete finite presentation and its fixed matrix generators from the quadratic construction, then give a fully charged unbounded Diophantine word checker. Only after these steps is a numerical comparison with the repository’s fixed universal-polynomial benchmark meaningful. The present lower bound can guide loader choice, but it does not pay the cost of certifying an arbitrarily long product. [write 78] Part of this is done in the repository: the research note [18] gives a fully charged positive Diophantine certificate, with literal operation counts, for membership of $\text{diag}(L_r, L_r)$ in a fixed subgroup of $\text{SL}_2(\mathbb{Z}) \times \text{SL}_2(\mathbb{Z})$ given by a supplied finite macro table. What remains is the numerically instantiated universal alphabet. [write 82] (3 October 2026.) Answered in part, for a different membership problem, by Part V: it instantiates a universal alphabet numerically, 229 literal matrices of $\text{SL}_4(\mathbb{Z})$ generating a positive semigroup whose membership is many-one r.e.-complete, conditionally on the Neary–Woods simulation chain (Theorem 61.1); the research programme has reduced it to 193 generators (Section 75). Two parts of the question remain open. The semigroup is positive, so neither the subgroup of Theorem 52.1 nor the interface of [18] is instantiated; and its Diophantine certificates (Theorem 61.2) are bounded-length certificates bought with $130r$ witnesses for length r , not a fully charged unbounded word checker of fixed arity. The numerical comparison with the repository’s fixed universal-polynomial benchmark is therefore still not meaningful. [write 91] (4 October 2026.) The fixed-arity unbounded checker now exists in part, for the semigroup that succeeds Part V’s. Part VI gives one fully expanded

polynomial of fixed arity, with 41,309 positive witnesses, exact degree 12 and 184,016 gates, that has a positive zero at an ordinary input x exactly when the target $T(x)$ is a product of the 195 fixed singular 7×7 generators, under inherited interfaces and for one pair of fixed contexts (Theorem 77.2). The research programme’s uniform compilers of the same night pay the same histories for every valid program recipe, in 1,393 operations at exact degree 71,105 (note in Section 78). Both certify a semigroup, not the subgroup of Theorem 52.1 or the interface of [18], so the subgroup half of the question remains open. For the programme’s uniform compilers the comparison the question asks for can now be made, and it is 1,393 operations against the 84-operation bound; Part VI’s polynomial, for one fixture, is not a universal construction and gives no such comparison.

60 Conclusion

The obstruction left implicit by the higher-dimensional counterexample has a clean resolution: affine ordinary-input curves into unimodular matrix groups remain computationally rigid in every dimension. What fails beyond square-zero normalization is the single-coset description, not decidability. The correct language class is finite-or-exactly-periodic, with a sharp $d - 1$ finite bound and a canonical nilpotent-lattice period formula.

Together with the existing quadratic group construction, this gives an exact degree threshold for the specified ordinary-input interface. At the same time, the affine multiplication gadget shows why existentially quantified coordinates must not be folded into that theorem. A fixed abelian unipotent subgroup with easy direct membership can support arbitrary Diophantine computation through a jointly affine pencil once enough coordinates are projected away.

The resulting contribution is structural and quantitative at the input and witness interfaces. It does not lower a universal polynomial’s arithmetic-operation count, instantiate the large universal subgroup, settle finite-fold MRDP, or carry a new proof-assistant certificate. Those boundaries are part of the result: they distinguish a proved extension from an uncharged claim of universality.

Part V

A fixed universal matrix semigroup (source 08)

[write 82] Part V prints batch-82 manuscript 22, “Research Report 32” of the pipeline that delivered it, *A fixed universal matrix semigroup and bounded length Diophantine certificates*, dated 3 October 2026. Its author line reads “Research construction and reproducibility report”; it names no author and is AI-assisted. It is called *source 08* after the file prefix of its shipped programs and data, which continues this report’s own sequence and is not a batch-82 manuscript number. Its labels carry the prefix `gts:um:.` Its Section n is Section $n + 60$ here and its Theorem $n.m$ is Theorem $(n + 60).m$; its Appendices A and B are Appendices J and K, placed after the existing appendices so that those keep their letters. Its equations are numbered (V. n), where n is the source’s own equation number, so that the equation numbers of the appendices do not move. Its matrix shorthand and its norm are printed with this report’s macros, its bibliography keys carry the prefix `um-`, and its two appendix titles begin with “Part V”; nothing else in its text was changed, apart from the [write 82] notes. Its title page and table of contents are replaced by this opening. It names no ProveIt commit as a snapshot and cites no repository file and no other report; only its shipped dependency audit

(08-matrix-semigroup-core-loader-audit-U15_DEPENDENCY_AUDIT.md) mentions two other reports of its pipeline, as evidence for the finite-tape input convention: Report 16 (Part II of the collection report *five-particle-binary-automata*, placed in the same batch; the audit pins its loader and its machine table, which is the table file of *quadratic-orthant-certificates* named below) and Report 23 (Part I of the collection report *fixed-universal-polynomials*, placed in the same batch). Its abstract and scope statement follow.

Abstract (source 08). One fixed list of 229 explicitly expanded matrices in $SL_4(\mathbb{Z})$ has a many-one recursively enumerable complete membership problem. The local reduction is implemented from every finite binary initialization of the Neary–Woods 15-state machine: a directed rewriting system with 93 rules becomes 114 correspondence tiles, then an unrestricted positive matrix semigroup. A free-group marker in the lower block enforces the required paired word form. The arbitrary-machine-to-finite-tape stage remains a precisely identified published effective construction.

For each numerical inner length $r \geq 0$, a second compiler gives a literal sum-of-squares integer polynomial with four signed external target entries, exactly $130r$ natural auxiliary variables, and $17r + 4$ residual squares. Its roots correspond bijectively to matching length- r tile sequences. Its joint degree is four for $r \geq 1$ and two for $r = 0$. Canonical signed parts remove representation multiplicity, but distinct factor sequences can share a target. A checked length-two collision and an actual 94-tile accepting certificate make this distinction concrete. For accepted encoded inputs, an exact rational generating function proves infinite unbounded fibers and polynomial factorization growth. Complete local proofs, scalar arithmetic and bit-size ledgers, executable scope boundaries, and portable replay instructions accompany the numerical data.

Scope. This report makes no novelty or optimality claim. It proves a fixed semigroup theorem and a bounded-length polynomial-family theorem. It does not assert inverse closure, an implemented arbitrary-program loader, a fixed-arity encoding of unbounded length, or an unbounded finite-fold representation. Accepted encoded targets are proved to have infinite fibers across lengths in Section 73.

[write 82] **What is proved, and what is conditional.** The abstract states many-one r.e.-completeness before its qualifier. The local equivalence of Theorem 61.1, that U halts on (ℓ, ρ) if and only if $E(\ell, \rho) \in S$, is proved in full (Lemmas 63.1, 64.1, 65.1 and 66.1 and Section 67) and implemented. The many-one r.e.-completeness of the one fixed semigroup is *conditional* on the published Neary–Woods simulation chain of Section 62, which source 08 cites, does not reprove and does not implement, as Theorem 61.1, Section 62 and the conclusion say. The 229 matrices, Theorem 61.2 and the fibre formula do not depend on that chain. The abstract’s sentence that a generating function “proves infinite unbounded fibers” joins two results: the infinite unbounded fibre of every accepted encoded target is proved by the stutter argument at the start of Section 73, which needs no generating function, and Theorem 73.1 counts the fibre of a halting *live* input exactly.

[write 82] The rewriting and tile stages re-derive a construction of the research programme for the same machine, and the machine table is the collection’s third printing of it (notes after Table 4 and Lemma 64.1); the first half of Lemma 65.1 is part of Part I’s Sanov theorem. The programme reviewed the archive on the day it arrived and reduced the generator count to 197 and then 193 (note after question 4 of Section 75). Section 1.4 collects these relations; Table 3 lists the letters of this Part that mean something else in Parts I–IV.

61 The two results and their exact interfaces

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$, all matrix products are ordered left to right as written, and semigroup membership permits nonempty positive words in the listed generators. An inverse occurring inside the formula for a fixed numerical generator does not authorize adjoining that generator's inverse to the semigroup.

Let $\mathcal{G} = (A_1, \dots, A_{114}, B_1, \dots, B_{114}, C)$ be the literal ordered list in the accompanying core/dat a/semigroup.json. Section 65 gives its full finite mathematical specification. Every matrix entry is expanded as an integer in that file and in core/data/matrices.txt. Write

$$S = \{G_1 \cdots G_m : m \geq 1, G_j \in \mathcal{G}\} \subseteq \mathrm{SL}_4(\mathbb{Z}).$$

For finite words $\ell, \rho \in \{0, 1\}^*$, initialize the fixed machine U in state A scanning 0 at coordinate zero, with cell $-i - 1$ containing ℓ_i and cell $i + 1$ containing ρ_i . All unlisted cells are zero. Thus both halves are nearest-head-first; neither includes the scanned cell. We reserve r for a numerical certificate length and use ρ for the right tape half.

Theorem 61.1 (Fixed semigroup and executable tape interface). *There are 229 distinct literal matrices $\mathcal{G} \subset \mathrm{SL}_4(\mathbb{Z})$ and an explicitly implemented computable map E on finite binary tape pairs such that*

$$U \text{ halts on } (\ell, \rho) \iff E(\ell, \rho) \in S.$$

The membership problem of this one fixed S is many-one r.e.-complete. The latter conclusion uses the published effective finite-tape universality construction of Neary and Woods in addition to the local, implemented equivalence.

Set

$$D = \begin{pmatrix} 157 & 4 \\ -6712 & -171 \end{pmatrix}, \quad P = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}. \quad (\text{V.1})$$

These are the upper and lower blocks of C . A query to the polynomial interface consists of a numerical $r \geq 0$ and an arbitrary signed integer 2×2 matrix $T = (t_{ab})$.

Theorem 61.2 (Fully counted bounded-length certificate). *For every numerical $r \geq 0$, the companion compiler emits an integer polynomial*

$$F_r(T; x), \quad T \in \mathbb{Z}^{2 \times 2}, \quad x \in \mathbb{N}^{130r},$$

as a literal sum of $17r + 4$ squares of explicit sparse residual polynomials. For each fixed T , its natural roots are in bijection with sequences $(i_1, \dots, i_r) \in \{1, \dots, 114\}^r$ satisfying

$$A_{i_1} \cdots A_{i_r} C B_{i_r} \cdots B_{i_1} = \mathrm{diag}(T, P). \quad (\text{V.2})$$

The joint total degree before external specialization is 2 for $r = 0$ and exactly 4 for $r \geq 1$. Hence the number of roots is at most 114^r . The generator length in (V.2) is exactly $2r + 1$.

The word “joint” counts external and auxiliary variables together. At $r = 0$, specializing the four external entries leaves a constant and zero auxiliary slots. One sequence has exactly one certificate, but one target need not have only one sequence.

The lower-marker theorem proved below implies

$$\mathrm{diag}(T, P) \in S \iff \exists r \geq 0 \exists x \in \mathbb{N}^{130r} : F_r(T; x) = 0. \quad (\text{V.3})$$

This is a countable union of growing-arity polynomial families. An existential statement about a variable r in prose is not a polynomial encoding of that variable at fixed arity. For a bound $L \geq 1$ on generator length, membership within that bound is the finite disjunction of the separate instances with $0 \leq r \leq \lfloor (L-1)/2 \rfloor$. No uncharged selector for that disjunction is being used.

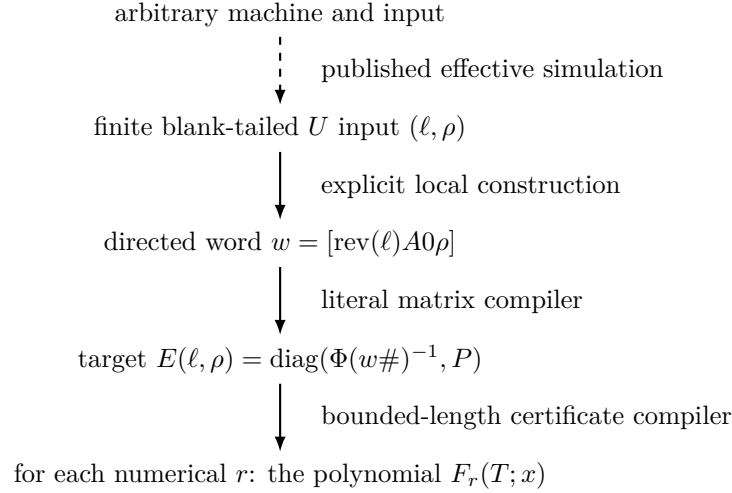


Figure 1: Dependency and implementation boundaries. The dashed stage is a mathematical source dependency; the solid stages are delivered algorithms. The bottom family has arity depending on r .

62 The fixed machine and the published dependency

62.1 The complete transition table

Table 4 is the 30-cell finite source data. The ordered states $A, \dots, O$ correspond to $u_1, \dots, u_{15}$ of Neary–Woods, and $0 = c$, $1 = b$. A defined entry lists the written symbol, movement, and next state. The only undefined cell is $J1 = (u_{10}, b)$, which is the halting event.

State	Read 0	Read 1	State	Read 0	Read 1
A	$0, R, B$	$1, R, A$	I	$0, R, A$	$1, L, J$
B	$1, R, C$	$1, R, A$	J	$1, L, K$	halt
C	$0, L, G$	$0, L, E$	K	$0, R, L$	$1, R, N$
D	$0, L, F$	$1, L, E$	L	$0, R, M$	$1, R, L$
E	$1, R, A$	$1, L, D$	M	$0, L, B$	$1, R, L$
F	$1, L, D$	$1, L, D$	N	$0, L, C$	$0, R, O$
G	$0, L, H$	$1, L, G$	O	$0, R, N$	$1, R, N$
H	$1, L, I$	$1, L, G$			

Table 4: The fixed $U_{15,2}$ table. Source: Neary–Woods, Table 16 [47]; factual transcription and relabeling.

The source prose elsewhere names (u_{10}, c) as the halting cell, whereas Table 16 defines that cell as b, L, u_{11} . This release follows the unambiguous table: $J1$ is undefined and $J0$ is defined. The issue is recorded rather than silently changing a transition.

[write 82] Table 4 is the third printing of this table in the collection. It agrees entry by entry (states $A, \dots, O$ for $u_1, \dots, u_{15}$, $0 = c$, $1 = b$, $J1$ undefined) with Tables `qoc:wf:tab:tm` and `qoc:um:tab:tm` of the report *quadratic-orthant-certificates* [21] (its Parts III and V) and with the transcription in the research programme’s note [22], added on 1 October 2026. The inconsistency between Table 16 and the prose of Neary and Woods recorded in the preceding paragraph is also recorded in that note; *quadratic-orthant-certificates* prints it in full after its Table `qoc:wf:tab:tm`. Source 08 cites neither, found the inconsistency independently and resolves it in the same way. Its serialized table `core/data/u15_table.json` is byte-identical to `data/16-universal-membrane-tm_table.json` of *quadratic-orthant-certificates* and is not shipped here (the README restores it for reruns); all 30 cells of that file were compared with Table 4 at this write.

62.2 Exactly what universality supplies

Let H_U be the set of finite tape pairs from Section 61 on which this fixed table reaches $J1$. All such pairs are legitimate inputs, even when they do not encode a meaningful simulated program. Hardness requires only a well-formed subfamily.

The published reference is *Fundamenta Informaticae* 91(1) (2009), pages 123–144, DOI [10.3233/FI-2009-0036](https://doi.org/10.3233/FI-2009-0036) [47]. The author-hosted PDF used for detailed inspection has draft pagination 105–126 and different draft DOI metadata. Therefore a locator such as “Table 16, printed page 121” below refers to that draft PDF, not to page 121 of the published article.

The effective source chain is as follows.

1. Lemma 2.1, draft pages 108–109, constructs a clockwise machine from an ordinary deterministic one-tape machine. Input conversion is a finite-state transduction; finite boundary markers represent the two infinite blank tails.
2. Lemma 2.2, draft pages 110–111, converts the clockwise machine to a bi-tag system while preserving the simulated halting event. Theorem 2.1 composes these stages.
3. Definition 3.1, equations (3)–(4), and Tables 1–3, draft pages 112–113, specify finite program/data encodings for the universal machines.
4. Theorem 3.1 proves the simulation for $U_{9,3}$; the paragraph after that proof extends the correctness argument to the other machines. Section 3.5 and Table 16 give the $U_{15,2}$ implementation and halting mechanism, draft pages 120–123.

Theorem 3.1 alone is not a separately stated $U_{15,2}$ r.e.-completeness theorem. The cited chain yields that conclusion by effective composition.

For the properly encoded bi-tag configurations produced by these constructions, the finite physical tape has the form $P_{\text{prog}} b[c] D_{\text{data}}$, where brackets here designate the scanned cell, not tape characters. Taking no optional padding, our two halves are

$$\ell = \text{rev}(\text{binary}(P_{\text{prog}}b)), \quad \rho = \text{binary}(D_{\text{data}}),$$

with $b \mapsto 1$ and $c \mapsto 0$. Both tails are blank. The construction supplies a total effective halting-preserving map from valid machine/input codes into this family. Standard effective coding can assign malformed codes to a fixed nonhalting computation. Consequently the ordinary halting set reduces many-one to H_U . Direct simulation makes H_U r.e., hence it is many-one r.e.-complete.

This report proves the downstream local construction in full. It does not reprove every step of the published universal simulation, does not materialize its complete arbitrary-TM-to-bi-tag

compiler, and does not infer a source-program complexity bound from the finite-tape bound below. An executable finite-tape encoder is a narrower deliverable than an executable arbitrary-program encoder.

63 A directed rewriting system with 93 rules

Use the ordered alphabet

$$\Sigma = (0, 1, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, [,], X).$$

The brackets are actual letters in this alphabet. The fresh letter X is a cleanup marker and the one-letter terminal word. A live finite configuration is $[l q a v]$, with l, v binary words, q a state, and a the scanned bit. Unrepresented cells beyond the endpoints are zero. The initial word is

$$w(\ell, \rho) = [\text{rev}(\ell) A 0 \rho]. \quad (\text{V.4})$$

For each defined transition $(q, a) \mapsto (b, R, p)$, include

$$qa0 \longrightarrow bp0, \quad qa1 \longrightarrow bp1, \quad qa] \longrightarrow bp0]. \quad (\text{V.5})$$

For each defined transition $(q, a) \mapsto (b, L, p)$, include

$$0qa \longrightarrow p0b, \quad 1qa \longrightarrow p1b, \quad [qa \longrightarrow [p0b. \quad (\text{V.6})$$

The boundary cases insert one newly visited blank. Finally include

$$J1 \longrightarrow X, \quad 0X \longrightarrow X, \quad X0 \longrightarrow X, \quad 1X \longrightarrow X, \quad X1 \longrightarrow X, \quad [X] \longrightarrow X. \quad (\text{V.7})$$

There are $29 \cdot 3 + 1 + 4 + 1 = 93$ rules, all with nonempty sides. Their deterministic enumeration scans transition cells $A0, A1, B0, B1, \dots, O1$, skips $J1$, and lists the two bit contexts then the boundary context for each cell. It then appends the six rules in (V.7). This convention and Table 4 specify every rule without an implicit choice.

Lemma 63.1 (Exact simulation and cleanup). *For every finite binary pair (ℓ, ρ) ,*

$$U \text{ halts on } (\ell, \rho) \iff w(\ell, \rho) \longrightarrow^* X.$$

Proof. A live configuration contains one state letter. If the cell is defined, its state and scanned bit select one transition, and the symbol on the movement side selects exactly one of its three context rules. No rule of another cell can match. No cleanup rule can match because X is absent. The resulting word is the next live configuration, with exactly the prescribed tape write and head move. At an endpoint the matching boundary rule adds precisely the blank cell the infinite-tape machine next visits.

At the undefined cell $J1$, the only applicable rule involving a state is $J1 \longrightarrow X$. The result has the form $[Xv]$. The four erasures can delete all binary letters on either side, after which $[X] \longrightarrow X$ finishes. Each cleanup step shortens the word. Conversely, the only way to create X is to apply $J1 \longrightarrow X$, so any derivation to X must first reach the machine's halting cell. Before this event every rewrite was the unique correct simulation step. There is no reverse rewrite, extra rejecting halt, or boundary erasure that could bypass it. $\square$

64 From rewriting to 114 fixed tiles

Introduce a fresh separator $\#$. The ordered tile list (h_i, g_i) consists of 20 copy tiles (a, a) in alphabet order, 93 tiles (v, u) corresponding in order to the directed rules $u \rightarrow v$, and the final separator tile $(\#, \#)$ at index 114. Every side is nonempty; only tile 114 contains $\#$.

For a sequence $s = i_1 \cdots i_m$, let $h(s) = h_{i_1} \cdots h_{i_m}$ and $g(s) = g_{i_1} \cdots g_{i_m}$. Empty sequences give empty words.

Lemma 64.1 (Exact correspondence equation). *For every word $w \in \Sigma^*$, including malformed configuration words,*

$$w \rightarrow^* X \iff \exists s : w\#h(s) = g(s)X\#. \quad (\text{V.8})$$

Proof. Suppose $w = w_0 \rightarrow w_1 \rightarrow \cdots \rightarrow w_d = X$. A step $\alpha u \beta \rightarrow \alpha v \beta$ gives a tile block: the copies of α , the tile (v, u) , the copies of β , and the separator. Concatenating these blocks yields

$$h(s) = w_1\# \cdots w_d\#, \quad g(s) = w_0\# \cdots w_{d-1}\#,$$

so (V.8) holds. For the zero-step derivation $w = X$, choose $s = \varepsilon$.

Conversely split the sequence at its k occurrences of tile 114: $s = s_0 114 s_1 114 \cdots 114 s_k$, where no s_j contains that tile. Comparing the final $\#$ -separated segments of the equation forces $h(s_k) = \varepsilon$. Nonempty tile sides imply $s_k = \varepsilon$, and hence $g(s_k) = \varepsilon$. If $k = 0$, the entire sequence is empty and $w = X$. If $k > 0$, the remaining segment equalities are

$$w = g(s_0), \quad h(s_0) = g(s_1), \quad \dots, \quad h(s_{k-2}) = g(s_{k-1}), \quad h(s_{k-1}) = X.$$

For each j , the tiles of s_j partition $g(s_j)$ into disjoint, successive nonempty pieces. Copy tiles preserve their pieces; every other tile replaces one rule's left side by its right side. These disjoint replacements can be performed from right to left, preserving the original positions of all remaining pieces, and give $g(s_j) \rightarrow^* h(s_j)$. Concatenation gives $w \rightarrow^* X$.

This direction allows several rule tiles in one segment and copy-only segments. It does not assume that segment words are machine configurations. For well-formed initial words, Lemma 63.1 supplies that additional invariant. $\square$

[write 82] **Credit.** Lemmas 63.1 and 64.1 re-derive, for the same machine, a construction already in the research programme, which source 08 does not cite. The note [22] (its Section 1) compiles $U_{15,2}$ into a bounded-tape rewriting system with $29(2+1) + 1 + 2 \cdot 2 = 92$ rules and 113 full-copy tiles (97 after deleting the copies of state letters), and the notes [23] prove the fresh-delimiter equivalence $\exists w : \sigma(w)\#v = u\#\tau(w) \iff u \Rightarrow^* v$, the mirror image of (V.8) (tile sides interchanged, delimiter at the other end). The two constructions differ only at the end of a run: the note keeps a halting state and erases the tape on both sides down to the word $[halt]$, whereas source 08 turns $J1$ into a fresh letter X and also erases the brackets, by $[X] \rightarrow X$, which accounts for its 93 rules and 114 tiles. No novelty is claimed here for the two lemmas; they are printed as source 08's own route. The matrix stage of Sections 65–67 has no counterpart in those notes.

65 The numerical matrices and free embeddings

65.1 A faithful integer encoding

Define

$$P = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad Q = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}, \quad E_j = Q^{-j} P Q^j = \begin{pmatrix} 1+4j & 2 \\ -8j^2 & 1-4j \end{pmatrix}. \quad (\text{V.9})$$

Every determinant is one.

Lemma 65.1. *The matrices P, Q freely generate a rank-two free group. Any finite set of distinct E_j , including $E_0 = P$ if desired, is freely independent.*

Proof. Use the real projective line with the disjoint sets $\mathcal{A} = \{|z| > 1\}$ (including infinity) and $\mathcal{B} = \{|z| < 1\}$. For every nonzero integer n ,

$$P^n z = z + 2n \in \mathcal{A} \quad (z \in \mathcal{B}), \quad Q^n z = \frac{z}{2nz + 1} \in \mathcal{B} \quad (z \in \mathcal{A}).$$

The inequalities are strict. In the second case $|2nz + 1| > |z|$ for $|z| > 1$, and infinity maps to $1/(2n)$. Each nonidentity power is nonidentity, so both cyclic subgroups are infinite.

For clarity, the ping-pong conclusion can be seen directly. A reduced alternating word with the same factor type at both ends sends the opposite set into a disjoint set, so is nonidentity. For an alternating word with different end types, conjugate by a suitable nonzero power of its final factor type. Because that cyclic group is infinite, the conjugating exponent can avoid cancellation at that end. The conjugated reduced word then has the same type at both ends and is nonidentity. Pure powers are already nonidentity. Thus no nonempty reduced P, Q word acts trivially; in particular it cannot be the identity matrix.

Now collect any nonempty reduced word in distinct E_j as $E_{j_1}^{n_1} \cdots E_{j_k}^{n_k}$, with nonzero n_i and adjacent indices unequal. Its expansion is

$$Q^{-j_1} P^{n_1} Q^{j_1-j_2} P^{n_2} \cdots Q^{j_{k-1}-j_k} P^{n_k} Q^{j_k}.$$

Every interior Q exponent and every P exponent is nonzero. Deleting any empty endpoint powers leaves a nonempty reduced word in the free generators P, Q . This proves the second assertion, not merely abstract freeness of their generated subgroup. $\square$

[write 82] P and Q are the Sanov shears A and B of Part I (10). The first assertion of Lemma 65.1 is therefore the injectivity half of Theorem 4.1, a classical theorem of Sanov [1]; source 08 plays ping-pong on the real projective line and Part I on vectors of $\mathbb{R}^2$, so the proof above is a second proof (source 08). The second assertion, the free independence of distinct conjugates $E_j = Q^{-j} P Q^j$, is not in Part I. Each E_j has the congruence form of Part I's image (13), with chart coordinates $(j, 1, -4j^2, -j)$. In this Part the letters A_i, B_i, C name 4×4 generators, never Part I's shears (Table 3).

Assign the letters of Σ in their displayed order to $E_1, \dots, E_{20}$, and $\#$ to E_{21} . Let Φ multiply these matrices in word order. Lemma 65.1 makes Φ injective even on the free group, hence on positive words. In a separate block put $t = E_0 = P$ and $x_i = E_i$ for $1 \leq i \leq 114$.

65.2 All generators and all target entries are numerical

For the already fixed 114 tiles define

$$A_i = \text{diag}(\Phi(h_i), x_i), \quad (\text{V.10})$$

$$B_i = \text{diag}(\Phi(g_i)^{-1}, t^{-1}x_i^{-1}t), \quad (\text{V.11})$$

$$C = \text{diag}(\Phi(X\#)^{-1}, t). \quad (\text{V.12})$$

The inverse in (V.11) is the inverse of the whole product. For any determinant-one block,

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}.$$

Thus the generator list contains only explicit integers, with no unspecified presentations, existentially chosen matrices, or input-dependent entries. For example,

$$A_1 = \begin{pmatrix} 5 & 2 & 0 & 0 \\ -8 & -3 & 0 & 0 \\ 0 & 0 & 5 & 2 \\ 0 & 0 & -8 & -3 \end{pmatrix}, \quad C = \begin{pmatrix} 157 & 4 & 0 & 0 \\ -6712 & -171 & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

The lower free-group words x_i , $t^{-1}x_j^{-1}t$, and t are pairwise distinct. Hence all $2 \cdot 114 + 1 = 229$ generators are distinct even if some upper blocks coincide. They lie in $\text{SL}_4(\mathbb{Z})$ because both blocks have determinant one.

For any query word $w \in \Sigma^*$ define

$$\mathcal{T}(w) = \text{diag}(\Phi(w\#)^{-1}, t), \quad E(\ell, \rho) = \mathcal{T}(w(\ell, \rho)). \quad (\text{V.13})$$

This computes the target without simulating U or deciding membership.

66 Why unrestricted semigroup words have the exact paired form

The correspondence equation is constrained, whereas membership in S quantifies over arbitrary positive generator words. The next lemma is the bridge. It rules out spurious orders at every length, rather than checking a bounded sample.

Lemma 66.1 (Marker normal form). *A positive word in $\mathcal{G}$ has lower block t if and only if it is*

$$A_{i_1} \cdots A_{i_r} C B_{i_r} \cdots B_{i_1} \quad (\text{V.14})$$

for some $r \geq 0$ and indices $i_s \in \{1, \dots, 114\}$.

Proof. Work in the free group on $t, x_1, \dots, x_{114}$, faithfully represented by the lower blocks. The exponent homomorphism η defined by $\eta(t) = 1$, $\eta(x_i) = 0$ sends each A_i, B_i lower block to zero and C to one. Since the word is positive, a value equal to t forces exactly one occurrence of C .

The kernel of η has free basis

$$y_{i,h} = t^h x_i t^{-h}, \quad 1 \leq i \leq 114, \quad h \in \mathbb{Z}.$$

Here is a direct justification. In a reduced word in these symbols, group consecutive letters of equal height h into maximal runs. The x word within a run remains nonempty and reduced; consecutive runs are separated on expansion by a nonzero power of t . The expansion is therefore nontrivial in the free t, x group. Conversely, moving every t letter rightward and recording the height at each x letter expresses every total-height-zero word in the displayed generators.

Write the lower product as Kt , pushing the final t to the right. Before the unique C , the height is zero; after it, the height is one. The contributions to K are

Position	Generator	Contribution
before C	A_i	$y_{i,0}$
before C	B_i	$y_{i,-1}^{-1}$
after C	A_i	$y_{i,1}$
after C	B_i	$y_{i,0}^{-1}$

Equality with t means $K = 1$. Every height -1 letter occurs only negatively, so no inverse occurrence can cancel it; every height $+1$ letter occurs only positively. No such letter can disappear in free reduction. Thus no B precedes C , and no A follows C . The remaining kernel word is a positive word in the $y_{i,0}$ followed by a negative word. It cancels exactly when the latter indices reverse the former. This gives (V.14). Conversely, a word of that form cancels directly to lower block t . $\square$

The exponent used here is the t exponent in the free basis $t, x_1, \dots, x_{114}$, not the P exponent in the ambient P, Q group. No cyclic rotation, chosen start marker, inverse-generator closure, or external regular-language restriction is part of the argument.

67 The complete membership reduction

For a normal-form word with sequence $s = i_1 \cdots i_r$, its upper block is

$$\Phi(h(s)) \Phi(X\#)^{-1} \Phi(g(s))^{-1}.$$

Indeed the reversed B order is $\Phi(g_{i_r})^{-1} \cdots \Phi(g_{i_1})^{-1} = \Phi(g(s))^{-1}$. Equality with the upper block of $\mathcal{T}(w)$ is equivalent, multiplying on the left by $\Phi(w\#)$ and on the right by $\Phi(g(s))\Phi(X\#)$, to

$$\Phi(w\#h(s)) = \Phi(g(s)X\#).$$

Both arguments are positive words, so injectivity gives precisely the literal equation of Lemma 64.1. Combining that lemma with the marker normal form proves

$$\mathcal{T}(w) \in S \iff w \longrightarrow^* X.$$

For $w = X$, the empty inner sequence is realized by the nonempty generator product C . No missing identity convention is required. Main-interface inputs in (V.4) contain state A and are never the word X .

Applying Lemma 63.1 proves the executable equivalence in Theorem 61.1. Section 62 supplies a computable many-one reduction from ordinary halting to the finite-tape source language, and the implemented target map composes with it. Finally enumerate all nonempty words in the 229 generators, compute their exact integer products, and compare them with the query. This

semidecides membership. Hence the fixed semigroup membership language is r.e. and many-one hard for every r.e. language, proving its many-one r.e.-completeness.

This mechanism is situated within the matrix undecidability literature, including Halava–Harju–Hirvensalo’s correspondence-based constructions [48]. The present proof does not import an optimized Claus-instance generator count. Its 229-generator figure is a verified count of this specified construction, not a bound claimed to improve the literature.

[write 82] The problem is positive-semigroup membership: no inverse generator is adjoined, and Lemma 66.1 uses positivity (a lower block t forces exactly one occurrence of C). It differs from the subgroup membership of Part IV’s fixed subgroups of $\mathrm{SL}_4(\mathbb{Z})$ (Theorem 52.1, Proposition 50.1) and of Part VI of [13]; neither kind of result implies the other. This write did not survey the matrix-semigroup literature, and no priority is certified.

68 A literal quartic certificate for each numerical length

Let U_i be the upper block of A_i , and let V_i be the inverse of the entire upper block of B_i . Then $U_i = \Phi(h_i)$ and $V_i = \Phi(g_i)$, all fixed integral determinant-one matrices. Keep D from (V.1). A normal-form product has upper block

$$H_r D G_r^{-1}, \quad H_r = U_{i_1} \cdots U_{i_r}, \quad G_r = V_{i_1} \cdots V_{i_r}.$$

Its equality to T can therefore be tested without auxiliary inverse entries by $H_r D = T G_r$.

68.1 Every variable is allocated

For every $s = 1, \dots, r$, allocate the following natural variables:

- 114 selectors $e_{s,i}$, $1 \leq i \leq 114$
- 16 signed-part variables $h_{s,a,b}^+, h_{s,a,b}^-, g_{s,a,b}^+, g_{s,a,b}^-$, $a, b \in \{0, 1\}$

There are exactly $130r$ natural auxiliary slots. Define the linear expressions

$$H_{s,a,b} = h_{s,a,b}^+ - h_{s,a,b}^-, \quad G_{s,a,b} = g_{s,a,b}^+ - g_{s,a,b}^-.$$

These expressions are abbreviations, not additional variables. Set $H_0 = G_0 = I_2$ as literal constants. The four t_{ab} are signed external parameters, not quantified auxiliaries.

68.2 Every residual is specified

There is one selection residual per step:

$$S_s = \sum_{i=1}^{114} e_{s,i} - 1. \tag{V.15}$$

There are eight update residuals per step:

$$U_{s,a,b}^H = H_{s,a,b} - \sum_{i=1}^{114} e_{s,i} \sum_{k=0}^1 H_{s-1,a,k} (U_i)_{k,b}, \tag{V.16}$$

$$U_{s,a,b}^G = G_{s,a,b} - \sum_{i=1}^{114} e_{s,i} \sum_{k=0}^1 G_{s-1,a,k} (V_i)_{k,b}. \tag{V.17}$$

There are eight canonicity residuals per step:

$$K_{s,a,b}^H = h_{s,a,b}^+ h_{s,a,b}^-, \quad K_{s,a,b}^G = g_{s,a,b}^+ g_{s,a,b}^-. \quad (\text{V.18})$$

Finally there are four terminal residuals:

$$L_{a,b} = \sum_{k=0}^1 H_{r,a,k} D_{k,b} - \sum_{k=0}^1 t_{a,k} G_{r,k,b}. \quad (\text{V.19})$$

The literal polynomial is

$$\begin{aligned} F_r(T; x) = & \sum_{s=1}^r \left[S_s^2 + \sum_{a,b=0}^1 \left((U_{s,a,b}^H)^2 + (U_{s,a,b}^G)^2 + (K_{s,a,b}^H)^2 + (K_{s,a,b}^G)^2 \right) \right] \\ & + \sum_{a,b=0}^1 L_{a,b}^2. \end{aligned} \quad (\text{V.20})$$

There are $(1 + 8 + 8)r + 4 = 17r + 4$ squared residuals. The natural domain alone makes the selectors one-hot; separate Booleanity equations would be redundant and are not counted. There are no hidden arithmetic-gate, determinant, quotient, inverse, or exponentiation witnesses.

69 Soundness completeness and the precise multiplicity theorem

Lemma 69.1 (Unique canonical signed parts). *For each $z \in \mathbb{Z}$ there is exactly one $(p, n) \in \mathbb{N}^2$ with $p - n = z$ and $pn = 0$, namely $(\max(z, 0), \max(-z, 0))$.*

Proof. If $z > 0$, then $p = n + z > 0$, so $pn = 0$ forces $n = 0$, $p = z$. If $z < 0$, interchange the roles. If $z = 0$, then $p = n$ and $p^2 = 0$, so both are zero. Each resulting pair satisfies the two equations. $\square$

Proof of Theorem 61.2 apart from degree. Since all values are integers, a sum of squares vanishes if and only if every residual vanishes. For a natural root, (V.15) says nonnegative integers sum to one, so exactly one selector e_{s,i_s} equals one at each step and the others equal zero. The updates reduce to

$$H_s = H_{s-1} U_{i_s}, \quad G_s = G_{s-1} V_{i_s}.$$

Starting at the constant identities, induction forces all signed prefix entries. The complement residuals and Lemma 69.1 then force their natural parts uniquely. The terminal residuals give $H_r D = T G_r$. Every G_r is unimodular, so multiplication by G_r^{-1} gives $T = H_r D G_r^{-1}$, exactly the upper block of (V.2). The lower block is P by the marker lemma. This proves soundness.

Conversely, for any sequence satisfying (V.2), set its one-hot selectors, multiply the two prefix sequences, and use each entry's unique canonical signed parts. Selection, update, and complement residuals all vanish. Its target equality is $H_r D = T G_r$, so the terminal residuals vanish too. This proves completeness. The preceding forced induction proves that this is the only full auxiliary tuple for that specified sequence. Reading the unique selected index at each step is the inverse map. Therefore

$$\#\{x \in \mathbb{N}^{130r} : F_r(T; x) = 0\} = \#\{(i_1, \dots, i_r) : (\text{V.2})\} \leq 114^r. \quad (\text{V.21})$$

$\square$

The assumption $x \in \mathbb{N}^{130r}$ is essential. Signed or real selectors summing to one are not automatically one-hot. The theorem does not silently replace this domain by $\mathbb{Z}$ or $\mathbb{R}$.

69.1 Zero length and automatically excluded targets

At $r = 0$ the unique possible auxiliary tuple is empty. The formula becomes

$$F_0(T) = (157 - t_{00})^2 + (4 - t_{01})^2 + (-6712 - t_{10})^2 + (-171 - t_{11})^2. \quad (\text{V.22})$$

It has that single empty root precisely at $T = D$. Its positive generator witness is C , of length one, not the empty product.

Every satisfying T has determinant one, since $T = H_r D G_r^{-1}$ is a product of determinant-one matrices. No determinant residual is needed. The mathematical external domain is nevertheless all of $\mathbb{Z}^4$, including zero and negative entries; determinant-invalid targets simply have empty fibers. For a fixed sequence, changing T cannot preserve the terminal equations because G_r is invertible. A changed target might of course be realized by another sequence.

The implementation rejects floats, booleans, malformed matrix shapes, negative auxiliary values, and missing or extra variable names. Its full- 4×4 helper accepts only inputs of the form $\text{diag}(T, P)$. A rejection for wrong off-diagonal or lower blocks means the input lies outside this helper's interface; it is not a theorem of nonmembership in the original semigroup.

69.2 Optional all-natural external inputs

An independently charged variant uses eight external natural parameters t_{ab}^+, t_{ab}^- , replaces t_{ab} by their difference, and adds the four squared residuals $(t_{ab}^+ t_{ab}^-)^2$. It still uses exactly $130r$ quantified auxiliaries and now has $17r + 8$ residuals. Canonical external pairs represent signed targets uniquely. Noncanonical pairs have a positive added square and no roots even if their differences are otherwise valid. External slots remain parameters; they do not increase the quantified auxiliary count.

70 Degrees coefficients and literal sparse size

The polynomial exporter lists every residual as integer-coefficient monomials and declares their sum of squares. This is an explicit polynomial representation: no numerical oracle or unevaluated matrix-word coefficient remains. Fully expanding the outer squares is unnecessary. The release includes both external modes at $r = 0, 1, 2$ with all names and coefficients materialized.

70.1 Exact degree and fixed coefficient data

At $s = 1$ the update equations are linear because $H_0 = G_0 = I_2$. Later updates are bilinear in selectors and previous state parts. Complements are quadratic; terminal residuals are bilinear for $r \geq 1$ and linear at $r = 0$. Thus $\deg F_r \leq 4$. For $r \geq 1$, a state complement has a nonzero quadratic homogeneous part. The degree-four component of a sum of real polynomial squares is a sum of squares of the quadratic leading parts; it cannot vanish identically when one of those parts is nonzero. Therefore $\deg F_r = 4$. Equation (V.22) gives degree two at $r = 0$. In the all-natural external variant, the external complements supply degree four even at $r = 0$.

The independently audited literal source has 456 nonzero entries among the U_i and 456 among the V_i . Their maximum absolute values are respectively

$$63,038,000 \quad \text{and} \quad 11,924,776.$$

The maximum absolute D entry is 6,712. Hence the largest residual coefficient for $r \geq 1$ is exactly 63,038,000, with 26 magnitude bits; at $r = 0$ it is 6,712, with 13 magnitude bits. No coefficient-size contribution is concealed by the shorthand U_i, V_i .

70.2 Monomial census with a declared convention

Let N_d count degree- d monomial occurrences across the residuals, combining equal monomials within each residual but not across different residual squares. Let R be the number of residuals. For signed external targets,

$$r = 0 : \quad (N_0, N_1, N_2) = (4, 4, 0), \quad R = 4, \quad (\text{V.23})$$

$$r \geq 1 : \quad (N_0, N_1, N_2) = (r, 130r + 928, 3656r - 3632), \quad R = 17r + 4. \quad (\text{V.24})$$

Here is the complete accounting. Each of the r selector residuals contributes one constant and 114 linear terms. Each of the eight first-step updates has 116 linear terms: two signed parts and 114 selected fixed coefficients. Each of the $8(r - 1)$ later updates has two linear and 456 quadratic terms, because each of 114 selectors multiplies two matrix entries, each represented by two signed parts. The $8r$ complements each contribute one quadratic monomial. Each of four terminal residuals contributes four linear and four quadratic terms. Consequently

$$N_1 = 114r + 8 \cdot 116 + 16(r - 1) + 16 = 130r + 928,$$

$$N_2 = 8 \cdot 456(r - 1) + 8r + 16 = 3656r - 3632.$$

All stated coefficients are nonzero, and distinct indices separate the counted monomials. Total residual size is therefore $3787r - 2704$ monomials for $r \geq 1$.

For natural external targets the triples are $(4, 8, 4)$ at $r = 0$, and

$$(N_0, N_1, N_2) = (r, 130r + 928, 3656r - 3612) \quad (r \geq 1).$$

The increase is 16 quadratic terms from splitting the signed terminal inputs and four external complement terms. Here $R = 17r + 8$, and the total for $r \geq 1$ is $3787r - 2684$.

No residual has more than 458 monomials. If the SOS is fully expanded, a safe coefficient bound for either mode and every r is

$$R(458 \cdot 63,038,000)^2. \quad (\text{V.25})$$

Indeed each residual's coefficient ℓ^1 norm is at most the quantity in parentheses, the norm is submultiplicative under squaring, and the R squares add. This conservative bound has bit length $O(\log(r + 1))$. It is not a claim that the expanded polynomial is an efficient storage format.

71 Resource ledgers with arithmetic and bit costs separated

71.1 Fixed matrices and finite-tape target encoding

The numerical source census is

Object	Exact count
Machine states and tape symbols	15 and 2
Defined transitions	29
Rewrite alphabet letters and directed rules	20 and 93
Copy tiles, rewrite tiles, separator tiles	20, 93, 1
Inner tiles and distinct 4×4 generators	114 and 229
All matrix-entry slots	3664
Nonzero matrix entries	1831
Sum of matrix-entry magnitude bit lengths	21372
Largest absolute generator entry	63038000

Magnitude bit length means $\text{bit_length}(|z|)$, with zero counted as zero. The 21,372 total is a census of mathematical values, not a file-size or compression theorem.

Let $n = |\ell| + |\rho|$. Then w has $n + 4$ letters, and $w\#$ has $n + 5$. The target encoder makes $n + 5$ exact 2×2 multiplications by fixed matrices. Since

$$\max_{1 \leq j \leq 21} \|E_j\|_\infty = 3611 < 2^{12},$$

every target entry has at most $12(n + 5) + 1$ magnitude bits, a deliberately conservative bound also covering the fixed lower block. The inverse formula changes two signs and swaps the diagonal entries once. The implementation additionally verifies determinant one with two variable-integer multiplications and one subtraction.

Each fixed-matrix multiplication uses eight scalar fixed-integer multiplications and four additions. The k th prefix contains $O(k)$ bits, so elementary fixed-integer arithmetic costs $O(k)$; summing gives $O(n^2)$ bit operations. The final determinant check also costs at most $O(n^2)$ with schoolbook multiplication. Working values use $O(n)$ bits, apart from stored input/output. This is complexity in the supplied finite tape length, not arbitrary source-program length or halting time. Far-end zero padding can change the finite word and target while describing the same infinite tape; the reduction holds for either spelling and needs no semantic-tape injectivity.

71.2 Exact generic sparse evaluation charges

The companion evaluator deliberately uses an unoptimized convention. For each monomial it starts at 1, multiplies once for every variable occurrence, multiplies by its coefficient even when that coefficient is 1 or -1 , and adds the result into an initially zero residual accumulator. It then squares every residual and adds the square into an initially zero SOS accumulator. Substituting (V.23)–(V.24) gives

$$M = N_1 + 2N_2 + (N_0 + N_1 + N_2) + R, \quad A = N_0 + N_1 + N_2 + R.$$

Thus the exact signed-interface census is

Numerical length	Scalar multiplications M	Scalar additions A
$r = 0$	16	12
$r \geq 1$	$11246r - 9036$	$3804r - 2700$

Natural external inputs cost 40 multiplications and 24 additions at $r = 0$; for $r \geq 1$, add 64 multiplications and 24 additions to the signed figures. These counts belong to this precise evaluator, not to an optimal arithmetic circuit. They exclude parsing, comparisons, variable lookup, hashing, validation, loop control, source loading, and polynomial construction. Those operations remain finite implementation work and are separately bounded or documented; they are not free witnesses hidden in the polynomial.

The fixed loader calls its determinant-one 2×2 inverse routine 572 times: 229 upper blocks, 229 lower blocks, and 114 extracted V_i blocks. Its arithmetic is 1,144 multiplications, 572 subtractions, and 1,144 negations, apart from parsing, hashing, shape/domain checks, swaps, and control. The literal coefficient JSON has 117,288 bytes. None of these arithmetic counts is the matrix-generator count, and no earlier constant-operation claim is inherited.

71.3 Certificate construction and value growth

Direct auditing gives

$$K_H = \max_i \|U_i\|_\infty = 64,675,047 < 2^{26}, \quad K_G = \max_i \|V_i\|_\infty = 12,234,453 < 2^{24}.$$

The maximum absolute row-sum norm is submultiplicative, so at prefix $s \geq 1$,

$$\|H_s\|_\infty \leq K_H^s < 2^{26s}, \quad \|G_s\|_\infty \leq K_G^s < 2^{24s}.$$

Each canonical H part needs at most $26s$ magnitude bits and each G part at most $24s$. Only one part of each pair is nonzero. Four entries of each kind plus the one selected bit per step give the total bound

$$\sum_{s=1}^r (4 \cdot 26s + 4 \cdot 24s + 1) = 100r(r+1) + r \quad (\text{V.26})$$

on auxiliary magnitude bits, counting zero as zero. A storage convention assigning at least one bit to every slot adds $O(130r)$; names and explicit zero slots further affect text size.

Since $\|D\|_\infty = 6883 < 2^{13}$ and for determinant-one 2×2 matrices $\|G^{-1}\|_\infty \leq 2\|G\|_\infty$, any realized target entry has at most $50r + 14$ magnitude bits. This upper bound is not a claim that every sequence achieves exponential growth.

Constructing the two prefixes takes two fixed-factor 2×2 multiplications per step, hence $16r$ scalar multiplications and $8r$ additions, plus comparisons, canonical splitting, and selector output. Streaming prefixes uses $O(r)$ working bits and $O(r^2)$ elementary bit operations. Retaining the entire certificate uses $O(r^2)$ value bits; explicit indexed names add $O(r \log(r+1))$ textual overhead. A supplied target adds its encoded length to reading/checking cost. If the target is computed from the sequence, two terminal 2×2 products add 16 multiplications and eight additions, and the determinant-one inverse check adds two multiplications, one subtraction, two negations, and entry swaps. The final nonconstant product costs $O(r^2)$ schoolbook bit operations. This work is not included in the polynomial's auxiliary count.

71.4 Evaluating arbitrary assignments and constructing the family

Suppose every signed external entry and natural auxiliary entry has absolute value less than 2^B , $B \geq 1$. Each residual has at most 458 monomials, degree at most two, and coefficient magnitude

less than 2^{26} . Therefore each residual has magnitude less than 2^{2B+35} . The SOS has magnitude-bit length at most

$$4B + 70 + \lceil \log_2 R \rceil.$$

The evaluator has $O(r + 1)$ counted arithmetic operations, on integers of $O(B + \log(r + 1))$ bits, and hence the conservative schoolbook upper bound

$$O((r + 1)(B + \log(r + 1))^2)$$

bit operations. Variable/domain validation and parsing add their ordinary finite costs.

A streaming residual generator need not store the expanded SOS. The compiler emits $O(r + 1)$ residual monomials with the explicit constants of Section 70, and $O((r + 1)\log(r + 2))$ name characters. Its work is linear in the numerical value of r at the symbolic output level, not polynomial in the binary encoding length of r . This output-sensitive accounting is essential when a family index is passed as an integer.

72 Two concrete examples and what their certificates mean

72.1 An actual accepting finite tape

The accepted input $\ell = 011$, $\rho = \varepsilon$ has initial word $[110A0]$ and target

$$E(011, \varepsilon) = \begin{pmatrix} 1347733333 & 37956152 & 0 & 0 \\ -57391245432 & -1616307011 & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Its full derivation is

$$\begin{aligned} [110A0] &\longrightarrow [1100B0] \longrightarrow [11001C0] \longrightarrow [1100G10] \\ &\longrightarrow [110G010] \longrightarrow [11H0010] \longrightarrow [1I11010] \longrightarrow [J111010] \\ &\longrightarrow [X11010] \longrightarrow [X1010] \longrightarrow [X010] \longrightarrow [X10] \\ &\longrightarrow [X0] \longrightarrow [X] \longrightarrow X. \end{aligned}$$

The first seven steps are machine transitions, the eighth is the halting rewrite, and the remaining six erase the finite tape and finish. The tile compiler encodes these 14 rewrites using 94 tiles. The corresponding A – C – B word has $2 \cdot 94 + 1 = 189$ generator factors, whose generic integer 4×4 product equals the displayed target in all sixteen entries.

The signed-target certificate at $r = 94$ has

$$130r = 12,220 \quad \text{natural auxiliaries}, \quad 17r + 4 = 1,602 \quad \text{residuals}.$$

The saved assignment has SOS value exactly zero, largest auxiliary magnitude 409 bits, and total auxiliary magnitude-bit census 154,084. The generic evaluator charges are exactly 1,048,088 multiplications and 354,876 additions. The complete assignment is included as paired/examples/accepting-94-certificate.json; no huge expanded quartic is needed to verify it.

72.2 A length two target with two distinct roots

The distinct sequences (20, 109) and (110, 20) both give

$$T_* = \begin{pmatrix} 22474169 & 568874 \\ -892774960 & -22598231 \end{pmatrix}. \quad (\text{V.27})$$

The mechanism is visible at the word level. Tile 20 is (X, X) , tile 109 is $(X, 0X)$, and tile 110 is $(X, X0)$. Both sequences therefore have

$$h(s) = XX, \quad g(s) = X0X.$$

They give identical H_2, G_2 and thus the same target $H_2 D G_2^{-1}$, but their selector choices, first prefixes, and complete auxiliary tuples differ. Each tuple has 260 natural slots and makes all 38 residuals vanish. The saved paired/examples/target-collision.json binds these sequences to the literal numerical target.

An exhaustive enumeration of the $114^2 = 12,996$ length-two sequences found 12,994 distinct targets and two excess sequence collisions. This is a finite diagnostic, not a classification of all multiplicities at greater lengths. Equation (V.21) concerns the number of sequence witnesses at one fixed r ; it does not imply a unique target root or a finite number of roots after allowing all lengths.

[write 82] The second excess collision is (20, 111) and (112, 20), by the same mechanism with the bit 1: tile 111 is $(X, 1X)$ and tile 112 is $(X, X1)$, so both sequences have $h(s) = XX$ and $g(s) = X1X$. Source 08 names only the first. Both were found by an independent enumeration of the 12,996 products from the literal generators at placement, and again at this write from the shipped data/08-matrix-semigroup-core-data-semigroup.json.

73 Exact factorization fibers and their asymptotic growth

The bounded-family theorem can be sharpened substantially for targets that encode actual accepted configurations. This section counts *factorization words*, equivalently tile sequences or canonical polynomial roots, not distinct matrices. All the words counted in a fiber have the same resulting matrix.

For $w \in \Sigma^*$, set $T_w = \Phi(w\#)^{-1}$ and define

$$N_w(r) = \#\{s : |s| = r, w\#h(s) = g(s)X\# \}.$$

By the marker lemma and the certificate bijection, this is simultaneously the number of generator words for $\text{diag}(T_w, P)$ of length $2r + 1$ and the number of roots of $F_r(T_w; x)$.

73.1 Every accepted encoded word has an infinite unbounded fiber

Write $c(v\#)$ for the copy tiles of v followed by tile 114. Both its tile-side words equal $v\#$. If s is a witness for w , then

$$s_k = c(w\#)^k s \quad (k \geq 0)$$

is another witness: both sides of the original word equality acquire the same prefix $(w\#)^k$. Its length is $|s| + k(|w| + 1)$. Thus every represented encoded word target has infinitely many distinct factorization words at unbounded lengths, and infinitely many canonical roots in the disjoint union

over r . Every fixed- r fiber remains finite. This is a proved obstruction to unbounded finite-foldness of this particular family, rather than merely a missing finite-fold proof.

One may also append any number of $c(X\#)$ blocks. For the special terminal input $w = X$, these are exactly all witnesses, so

$$\sum_{r \geq 0} N_X(r) z^r = \frac{1}{1 - z^2}.$$

The empty inner word at $r = 0$ represents the one-generator word C .

73.2 An exact rational generating function for every halting live input

Suppose w is a live configuration that halts. Let its deterministic machine run make H moves before reaching the undefined cell. Write its live words as

$$v_0 = w, v_1, \dots, v_H = [l J 1 v], \quad n_j = |v_j|, \quad m = |l| + |v|, \quad B = \begin{pmatrix} m \\ |l| \end{pmatrix}.$$

Here H excludes the halting rewrite and all cleanup. Define

$$r_* = \sum_{j=0}^{H-1} (n_j - 1) + (m + 4) + \sum_{a=1}^m (a + 3) + 2, \quad (\text{V.28})$$

$$d = H + m + 2. \quad (\text{V.29})$$

Empty sums are zero. There are d genuine rewrites from w through cleanup to X .

Theorem 73.1 (Exact accepted-input fiber). *For a halting live input as above,*

$$\sum_{r \geq 0} N_w(r) z^r = \frac{B z^{r_*}}{\left(\prod_{j=0}^H (1 - z^{n_j+1}) \right) \left(\prod_{q=4}^{m+4} (1 - z^q) \right) (1 - z^2)}. \quad (\text{V.30})$$

Repeated weights are retained. There are $d + 1$ denominator factors.

Proof. Separator parsing from Lemma 64.1 uniquely splits every witness into complete separator-terminated blocks, without a trailing nonseparator suffix. Each rule left side contains exactly one state or X . Every reachable word contains exactly one such marker. Since a block partitions its bottom word into disjoint tile sides, it can contain at most one rule tile.

A block with no rule tile is exactly the unique all-copy block $c(v\#)$ for its current word. A block with one rule tile is exactly the copies of its left context, that rule tile, its right context copies, and a separator. Thus every witness has a uniquely determined genuine rewrite history interspersed with uniquely counted copy loops.

Before the halting rewrite, the history is the deterministic machine run. It has no repeated live word, since such a repetition would force a deterministic cycle and prevent halting. After $J1 \rightarrow X$, the word is $[lXv]$. Cleanup removes the rightmost remaining left symbol or the leftmost remaining right symbol, then performs $[X] \rightarrow X$. The m erasure schedules are exactly the B choices of the $|l|$ left deletions among m positions. They yield distinct sequences of rule blocks even when some tape bits coincide, because left and right erasure rules are distinct. Later merging of intermediate words does not identify those block sequences.

For a rule $u \rightarrow u'$ in a source word of length n , its block uses $n - |u|$ context-copy tiles, one rule tile, and one separator, hence $n - |u| + 2$ tiles. Machine rules have $|u| = 3$, giving costs $n_j - 1$. The halt rule has $|u| = 2$ in a word of length $m + 4$, giving cost $m + 4$. The tape erasures have source lengths $m + 3, m + 2, \dots, 4$ and $|u| = 2$, hence those same costs. The final three-letter rule costs two. Summing gives the common genuine-history cost r_* in (V.28).

At any visited word of length n , one copy loop costs $n + 1$, and any number of such loops is possible. The live words give weights $n_0 + 1, \dots, n_H + 1$. The cleanup words after the halt have lengths $m + 3, m + 2, \dots, 3, 1$, giving weights $m + 4, m + 3, \dots, 4, 2$. Every one of the B genuine histories has exactly this same multiset of weights.

Copy-loop multiplicities at each history position are independent. Separator parsing recovers the rule blocks and all intervening loop counts, so different choices never collide. Multiplying the geometric series for the loop counts and summing over the B disjoint histories yields (V.30). $\square$

73.3 Support quasipolynomials and inversion

Let $a_1, \dots, a_{d+1}$ be the denominator weights of (V.30), and put

$$A = \prod_{i=1}^{d+1} a_i, \quad L = \text{lcm}(a_1, \dots, a_{d+1}).$$

Every such weight list contains 2, 4, and 5. Weight 2 occurs at X , weight 4 at $[X]$, and weight 5 occurs at a length-four cleanup word if $m \geq 1$, or at the halt word if $m = 0$. No reachable word has length zero or two, so there are no loop weights 1 or 3. Nonnegative combinations of 2 and 5 are exactly $\{0, 2\} \cup \{n : n \geq 4\}$, and every other weight belongs to that same set. Therefore

$$N_w(r) > 0 \iff r \in \{r_*, r_* + 2\} \text{ or } r \geq r_* + 4. \quad (\text{V.31})$$

In particular r_* is the minimum inner length and $N_w(r_*) = B$.

All poles of (V.30) are roots of unity with orders dividing L . Moreover direct subtraction of (V.28) from the sum of the loop weights gives

$$\sum_i a_i - r_* = 2H + m + 5 > 0.$$

Thus the rational function is proper. Partial fractions express its coefficients for every $r \geq 0$ as sums of polynomials in r times L th roots of unity to the r th power. Hence $N_w(r)$ is an exact quasipolynomial of period dividing L : on each residue class modulo L it agrees with a polynomial. Properness means no extra polynomial-in- z part is needed to correct the initial coefficient range.

Only $z = 1$ has pole order $d + 1$, since a root common to every denominator factor must have order dividing both 4 and 5. At $z = 1$, the leading pole coefficient is B/A , using $(1 - z^a)/(1 - z) \rightarrow a$. All other poles have order at most d . Coefficient extraction gives

$$N_w(r) = \frac{B}{d!A} r^d + O(r^{d-1}). \quad (\text{V.32})$$

Every residue class has the same positive leading coefficient. The fixed accepted target therefore has polynomial, rather than exponential, factorization growth, with exponent depending on its halting history.

For the cumulative count and its generalized inverse,

$$C_w(R) = \sum_{0 \leq r \leq R} N_w(r), \quad R_w(M) = \min\{R \in \mathbb{N} : C_w(R) \geq M\},$$

summation and monotone inversion give

$$C_w(R) = \frac{B}{(d+1)!A} R^{d+1} + O(R^d), \quad (V.33)$$

$$R_w(M) = \left(\frac{(d+1)!A}{B} M \right)^{1/(d+1)} + O(1). \quad (V.34)$$

For the inverse error, shifting the leading-order root by a sufficiently large constant changes the main cumulative term by a constant multiple of R^d , which bounds the remainder on both sides. Monotonicity then brackets the least admissible integer R . The generator-length threshold is $2R_w(M) + 1$.

These are conditional formulas for an already accepted input. Obtaining H and the live history requires reaching its halt; the formulas give no total decision procedure for whether an arbitrary finite input halts.

73.4 Exact growth for the released 94 tile example

For $w = [110A0]$, one has $H = 7$, $m = 5$, $|l| = 0$, $B = 1$, $d = 14$, and $r_* = 94$. Therefore

$$\sum_{r \geq 0} N_w(r) z^r = \frac{z^{94}}{(1-z^2)(1-z^4)(1-z^5)(1-z^6)(1-z^7)(1-z^8)^2(1-z^9)^2(1-z^{10})^6}. \quad (V.35)$$

Its weight product is 8,709,120,000,000 and its period divides 2520. In particular

$$N_w(r) = \frac{r^{14}}{759,246,199,455,744,000,000} + O(r^{13}).$$

The counts at $r = 94, 95, \dots, 106$ are

$$1, 0, 1, 0, 2, 1, 3, 2, 6, 5, 14, 7, 19.$$

At $z = -1$ the pole order is 11, and every other nontrivial root of unity has pole order at most 7. Thus all periodic coefficient terms have degree at most ten: coefficients of degrees 14 through 11 in the quasipolynomial are independent of residue class. No complete list of its 2520 residue polynomials is claimed to have been tabulated.

The focused fiber/ packet [51] contains the independent literal-tile graph checks and proof. Its checker reads numerical JSON and the saved witness without importing either compiler; finite stutter and coefficient tests support, but do not substitute for, the all-length arguments above.

74 Evidence architecture and portable reproduction

74.1 What has been checked and what each check establishes

The local construction is proved above. Testing checks the implementation against independent arithmetic and bounded semantics; it is not a replacement for universality or an unbounded non-halting argument. Normal Python and optimized `python -O` runs are both supplied, so correctness does not depend on removable assertions.

The core suite checks all 229 determinants and distinct generators, 6,750 local configuration contexts, all cleanup cases from 225 short binary contexts, and 64,036 matched machine/rewrite transitions over 225 bounded runs. Its exhaustive marker sample covers all 488,281 words of length at most eight over A_1, A_2, B_1, B_2, C ; exactly 15 have lower block P , all in the proved form. It also verifies the actual 189-factor accepting word, 100 randomized target encodings, malformed inputs, the zero-step word X , and an 8,192-bit command-line input beyond Python’s usual decimal conversion limit.

The companion suite checks six literal small- r polynomial/count/degree cases, 162 zero-length target-cube cases, 228 length-one certificate evaluations, all 12,996 length-two products, 1,544 length-two certificate evaluations, 860 selector/state/target mutations, 18 invalid-input rejections, and determinant-invalid targets. It checks the complete long accepting certificate and its measured arithmetic charges. Separate audits read only the pinned literal numerical JSON rather than importing the upstream compiler, reconstruct the matrix formulas, inspect the interfaces, and instrument the arithmetic census.

The core mathematical review separately inspected the arbitrary-length marker proof, separator edge cases, upper-block order, finite-tape simulation, and published dependency. The companion mathematical review separately inspected domain restrictions, canonicity, bijection, degree, exact count formulas, invalid targets, and multiplicity. These are independent human-readable mathematical and programmatic reviews, not proof-assistant certificates.

The fiber checker enumerates 33,544 distinct bounded stutter witnesses and independently checks 12 full matrix/SOS samples. A separate literal-tile graph dynamic program compares 31,366 coefficients over 226 fixtures, with identical normal and optimized receipts. At the release level, an additional JSON-only audit reconstructs all six exported polynomials (11,970 residual monomials), 14 small roots, the full 94-step auxiliary assignment, and both collision roots without importing either compiler.

Evidence map. The following relative paths identify the principal review and receipt artifacts in the portable package. The release inventory binds their bytes; each review also states its mathematical scope.

- `core/audit/REVIEW.md` and `core/loader-audit/U15_DEPENDENCY_AUDIT.md`: local proof review and precise published input dependency
- `core/evidence/tests.json`, `core/evidence/tests-optimized.json`, and `core/audit/independent-results.json`: machine, matrix, marker, and witness checks
- `paired/audit/REVIEW.md` and `paired/audit/REVIEW_BINDING.json`: companion proof review and exact reviewed-file identities
- `paired/evidence/tests.json`, `paired/evidence/tests-optimized.json`, and the JSON receipts in `paired/audit/`: authored and independent polynomial/interface checks
- `fiber/PROOF.md`, `fiber/CHECKS.json`, and `fiber/audit/`: exact-fiber proof, stutter/sample checks, and independent graph-coefficient review
- `verification/literal-polynomials.json` and `verification/expected-receipts.json`: independent release-level reconstruction and the strictly typed replay expectations

74.2 Release contents and identity boundaries

The portable release keeps the two source packets under `core/` and `paired/`, and the focused multiplicity analysis under `fiber/`, with authored code and numerical data retained byte-for-byte. Documentation whose original provenance contained machine-specific paths is accompanied by explicit sanitized provenance and fresh release identities. Earlier releases are unchanged. Third-party primary-PDF caches, interpreter caches, private machine paths, and transient compilation files are not part of this distribution. The bibliography links the primary sources instead.

The central numerical source is identical in both packets and has SHA-256

```
506144b361b2bea634d468a94921644d91868
dc089a257fe289a0bf51b74cec9
```

(the line break is typographical only). Its source accepting-witness JSON has SHA-256

```
13a3857d28b0207d9baa83facac5b2e67bb
aeb858d00b82ef9a91c4ab38df890
```

These content identities distinguish a replay of this numerical construction from a successful test of some edited construction. Checksums are integrity bindings, not mathematical proofs or digital signatures.

The release-level verifier first checks the complete expected path inventory and file identities before executing copied code. Scientific replay runs in fresh isolated copies, so `build/test` commands cannot overwrite the distributed evidence or original packets. Receipt comparisons validate exact key sets, values, and types; a boolean is not accepted in place of an integer merely because Python equates `True` and `1`. The normal and optimized modes have explicit mode fields. Release tests include tampered contents, inventory changes, unexpected paths, and receipt type mutations.

The executable instructions and exact current entry points are in `RELEASE_README.md`. From the release directory, the commands below check identity and then perform numerical replay:

```
python3 -I -B verify_release.py -verify-only
python3 -I -B verify_release.py -replay
```

Save its redirected receipt outside the sealed release tree. `build_report.sh` rebuilds the PDF in an external output directory. The ordinary workflow is an integrity check, a complete replay, and optionally a PDF rebuild. The release records expected semantics as well as byte hashes: rebuilt matrix JSON, ledger, and literal list must match the pinned outputs; shared source JSON and witness identities must agree; saved normal/optimized and independent receipts must have the expected typed content. Relocation and extracted-archive replay exercise the same procedures without the original workspace.

74.3 Rebuilding and reading the artifacts

Only Python 3.10 or later and the standard library are required for numerical replay. A standard LaTeX installation with the packages listed at the start of `report32.tex` is needed to rebuild the PDF. The report source is an ordinary self-contained LaTeX file. The release README describes an isolated rebuild rather than overwriting the sealed report.

Useful direct scientific commands from the appropriate packet directory include

```
python compiler.py target -left 011 -right "
python paired_compiler.py ledger 94
python paired_compiler.py export 1
python paired_compiler.py verify examples/accepting-94-certificate.json
```

The first command belongs to `core/`; the other three to `paired/`. A `target` command computes a matrix and is not a membership solver. Certificate verification checks a proposed bounded witness, not absence of witnesses. The polynomial exporter is explicit for any resource-feasible numerical r ; actual execution remains subject to ordinary time and memory limits.

[write 82] **Shipped names and the Windows replay.** In this report the release is shipped flattened under the prefix `08-matrix-semigroup-`: programs in `code/`, numerical data and receipts in `data/`, and the proofs, reviews, packet READMEs, dependency audit and sanitized provenance at the report root; for example `core/compiler.py` is `code/08-matrix-semigroup-core-compiler.py` and `core/data/semigroup.json` is `data/08-matrix-semigroup-core-data-semigroup.json`. The README maps every name. Not shipped: `report32.tex` and `report32.pdf` (this Part is the article), `RELEASE_README.md`, the seals `MANIFEST.json`, `SHA256SUMS` and `fiber/SHA256SUMS`, the two $r = 2$ exports (1.27 MB, which the shipped compiler regenerates byte for byte in under a second), twelve byte copies inside the release (among them the second and third copies of `semigroup.json` and of `accepting-witness.json`), and `core/data/u15_table.json`. The verifier’s identity gate requires the complete delivered inventory, so `verify_release.py` cannot pass on the shipped files alone; the README gives a rerun from the arrival archive and a reconstruction of the delivered layout. As delivered, `verify_release.py -replay` fails on Windows: `core/compiler.py`, `core/audit/independent_check.py`, `fiber/check_fibers.py`, `paired/tests/check_all.py` and `tamper_regression.py` write text with the platform newline, so the rebuilt `semigroup.json`, `ledger.json` and `matrices.txt` are CRLF files and the byte comparison stops the replay after about four seconds; after CRLF→LF conversion they equal the delivered bytes. At placement every replay component was rerun on copies with LF output forced, in ordinary mode, and all but the deterministic export and certificate commands also in optimized mode; every receipt and rebuilt file matched; `tamper_regression.py` passed its 18 cases unmodified; the end-to-end replay was not rerun. The “portable replay” of this section is portable to POSIX systems. Several of these programs also overwrite recorded outputs next to themselves; run them only on copies.

75 Consequences limitations and further questions

The report separates three notions of finiteness. The generator list is fixed once and for all. A bounded certificate at numerical r is a finite polynomial with a finite root set. The unbounded membership problem ranges over all r , and that union recovers r.e.-complete behavior. Confusing these levels would turn a correct bounded theorem into an unsupported uniform one.

The following questions are substantive next steps, not conclusions already obtained.

1. **Materialize the upstream compiler.** Implement the published ordinary-machine to clockwise-machine to bi-tag to finite- U -tape transformation with explicit input conventions, proper finite boundary symbols, and checked halting correspondence. This would strengthen the executable endpoint without changing the mathematical dependency already sufficient for Theorem 61.1.
2. **Study target multiplicity.** Beyond the accepted-input fibers classified in Section 73, characterize when different tile sequences have the same $(h(s), g(s))$, and when distinct such

pairs still have the same $H_r DG_r^{-1}$. The explicit collision shows that canonical signed states alone cannot make target factorization unique. Any proposed canonical factorization selector needs its own soundness, completeness, and resource accounting.

3. **Reduce paid certificate size.** The one-hot selectors are transparent but account for 114 of 130 variables per step. Binary encodings or interpolation may trade variables against degree, coefficient height, and gate witnesses. A genuine improvement should give a full ledger in the same natural domain and preserve the exact root-sequence correspondence or state explicitly how it changes.

4. **Improve generator or dimension counts.** The local construction prioritizes a clean positive-word marker proof. The broader literature has other correspondence encodings. Any comparison should use the same fixed-versus-variable semigroup convention, coefficient domain, dimension, inverse policy, and input interface; the present count is not an optimality claim.

[write 82] (3 October 2026.) Question 4 is answered in part in the repository, on the day the archive arrived. The research programme’s review [24] reconstructed all 229 matrices and derived a subset of 197: delete the copy tiles of the fifteen state letters and of X , with their paired generators A_i, B_i . On a valid input every copied context consists of $0, 1, [,]$, so the forward direction of Lemma 64.1 survives, and the subset semigroup has the same target map and the same halting equivalence on valid inputs, hence the same conditional r.e.-completeness; the all-word equivalence of Lemma 64.1 is not preserved. The successor note [25] uses the terminal word $[J1]$ in place of X , with the four cleanup rules $0J1 \rightarrow J1$, $J10 \rightarrow J1$, $1J1 \rightarrow J1$, $J11 \rightarrow J1$ and the 87 machine rules (91 rewrites and 96 tiles), and obtains 193 literal generators in $SL_4(\mathbb{Z})$; the accepting example of Section 72 then needs 12 rewrites, 83 tiles and 167 generator factors. Its largest entry grows from 26 to 31 magnitude bits, while the total of entry bit lengths falls from 21,372 to 19,321. Neither note claims minimality, an inverse-closed alphabet or a Diophantine gate bound, and both keep the Neary–Woods dependency. They are not printed here, and the shipped files are source 08’s.

5. **Relate bounded search to source computation.** A particular machine derivation produces a tile witness by copying contexts. Section 73 gives the exact minimal tile length and growth for an accepted live input. Further bounds could express its formula in terms of source-machine time and space through the published upstream simulation, while charging any implemented input compiler separately.
6. **Formalize the proof interfaces.** A proof assistant could check the rewriting simulation, separator lemma, free-group marker lemma, and signed-part bijection, with data extraction tied to the source table. Existing exact arithmetic checks do not supply this formal derivation. [write 82] The repository has no Lean or Rocq module for any statement of this Part.
7. **Investigate a uniform unbounded polynomial.** Any encoding of all lengths into one fixed-arity equation must explicitly pay for sequences, arithmetic, and canonicity. The present family is not such a construction. The explicit stuttering theorem proves that this family has infinite unbounded fibers for every accepted encoded target. A different uniform representation would need a separate multiplicity analysis. [write 82] This is the part of Part IV’s question 10 (Section 59) that this Part leaves open; the note there records what Part V supplies. [write 91] (4 October 2026.) Answered in part by Part VI, for a successor of this Part’s semigroup and one pair of fixed contexts. Its Theorem 77.2 is one positive-integer polynomial of fixed arity, with 41,309 witnesses and exact degree 12, whose zeros at an ordinary input encode accepting histories of every length, with sequences and arithmetic paid

explicitly. It does not pay for canonicity: offsets and congruence quotients vary, it states no finite-fold or unique-witness representation (Section 86), and it gives no multiplicity analysis. Its semigroup is the research programme’s 195-generator singular 7×7 successor, reached through the 193-generator semigroup of question 4, not the 229 matrices of this Part. The programme’s uniform compilers of the same night do the same uniformly over program recipes, at much lower cost and much higher degree (note in Section 78).

76 Conclusion

One literal positive semigroup in $\mathrm{SL}_4(\mathbb{Z})$ supports an exact finite-tape halting reduction. Its lower free-group block enforces the required paired factors inside ordinary unrestricted semigroup membership, while its upper block faithfully encodes a directed correspondence equation. The published effective universal-machine construction supplies the only nonmaterialized reduction stage needed for the many-one r.e.-completeness conclusion.

At every numerical inner length, the companion polynomial gives a fully charged quartic natural-witness interface with $130r$ auxiliaries and $17r + 4$ residuals. Its roots encode sequences canonically, not targets uniquely. For accepted inputs the exact rational fiber formula also determines minimal length, all supported lengths, quasipolynomial multiplicities, and polynomial asymptotic growth. The numerical examples, arithmetic ledgers, and identity-gated replay make these results and their limits inspectable.

Part VI

Chronological Diophantine matrix certificates (source 09)

[write 91] Part VI prints batch-91 manuscript 07, “Research Report 55” of the pipeline that delivered it, *Chronological Diophantine Matrix Certificates*, subtitled *Fixed arity with an ordinary positive input*, dated 4 October 2026. Its author line reads “Report55”; it names no author and is AI-assisted. It is called *source 09* after the file prefix of its shipped programs and data, which continues this report’s own sequence and is not a batch-91 manuscript number. Its labels carry the prefix `gts:cm:`. Its Section n is Section $n + 76$ here and its Theorem $n.m$ is Theorem $(n + 76).m$; its Appendices A and B are Appendices L and M, placed after all existing appendices. Its equations are numbered (VI. n), where n is the source’s own equation number. Two symbols are printed differently: its digest macro `\sha` is `\cmsha` here, because `\sha` is source 03’s snapshot macro, and its selector streams E_i are written Λ_i (note in Section 83), because the source also writes E_i for the inherited lower-marker matrices of Section 79, where E_0 is the marker itself. Its bibliography keys carry the prefix `cm-`, with the entries unchanged, and its two appendix titles begin with “Part VI”; nothing else in its text was changed, apart from the [write 91] notes. Its title page and table of contents are replaced by this opening, and its reading guide below counts its own Sections 1–14. It names two research-programme commits of the evening of 3 October 2026 as its pins, 750aeb4f7 (the countdown receipt and five matrix notes) and d31e29030 (the counted-suffix proof), and cites no other report and none of Parts I–V. Its abstract and reading guide follow.

Abstract (source 09). We give a fully expanded integer polynomial that recognizes finite

accepting paths of the actual pinned five-register matrix countdown, with its ordinary positive input x unchanged. A single common offset encodes signed row coordinates; explicitly paid binary-containment clauses select one of 97 synchronized branches at every time; exact shift equalities enforce the initial frame, every internal join, and the last frame. A fixed coefficient bound makes the signed affine updates carry-free. The counter forces exactly x initial loader steps followed by a tile suffix.

The polynomial has 41 309 strictly positive integer witnesses, 23 618 equality residuals, 184 016 binary arithmetic gates, and exact total degree 12. Every POWER and containment call is expanded into polynomial equations. Conditional on the explicitly inherited constructive Pell and matrix-interface theorems, its positive zeros are equivalent to membership of the direct-input target $T(x)$ in the associated directed semigroup generated by 195 fixed singular integer 7×7 matrices. An independent exact-source audit reconstructs every residual, macro argument, port, matrix map, and gate count; a separate interface audit reconstructs all 195 matrices and checks the proof boundaries.

The concrete contexts are [110 and A0]. No arbitrary-program compiler is implemented, no universality of those particular contexts is inferred, and no improvement to the repository’s prior 84-operation benchmark is claimed.

Reading guide. Sections 1–3 state the exact language and dependencies. Sections 4–10 prove the positive-integer equivalence. Sections 11–14 give the literal ledger, independent evidence, reproducibility contract, and remaining questions. The appendix supplies notation and a complete outer-clause checklist.

[write 91] **What is proved, and what is conditional.** Theorem 77.2 has three layers, which the source keeps apart (Section 78). The numerical theorem, that the literal polynomial recognizes the accepted inputs $\mathcal{A}$ of the pinned countdown, is proved here relative to the two pinned mathlib theorems `Pell.matiyasevic` and `Pell.eq_pow_of_pell`, which it specializes and does not re-prove. Its equivalence with membership of $T(x)$ in the 195-generator semigroup rests in addition on the research programme’s faithful-group, lower-marker, context-transfer and synchronized-row theorems, which Section 79 summarizes and does not re-prove. The program-family reading needs further the Neary–Woods simulation premise and an arbitrary-program compiler, which is not implemented. The polynomial is for one fixture, the contexts [110 and A0]; it is not a universal polynomial.

[write 91] **Relation to Part V.** The semigroup of this Part succeeds Part V’s. The research programme reduced Part V’s 229 generators to 193 (note at Part V’s question 4); this Part uses that 193-generator directed semigroup recoded faithfully in $\Gamma_1(5)$, with the two contexts absorbed into the matrices and a counted suffix that makes the input ordinary, which gives 195 singular integer 7×7 matrices (Section 79). Part V certifies bounded lengths, one polynomial for each length; this Part gives one polynomial of fixed arity for histories of every length, and so answers Part V’s question 7 in part (note there). Section 1.4 lists its other relations, and Table 5, below, the letters that mean something else in Parts I–V; it is printed here rather than in Section 1.3 so that no earlier table number moves.

Table 5: [write 91] Letters of Part VI that are used differently elsewhere in the report.

Letter	Part VI (source 09)	Parts I–V
$x, \mathcal{A}$	the ordinary positive input; the set $\mathcal{A}$ of accepted inputs	II: the phase image $\mathcal{A}$; II and V: x a vector of unknowns

Letter	Part VI (source 09)	Parts I–V
$T(x), T$	the direct-input 7×7 target (VI.12); T also a generator of $\Gamma_1(5)$	II: the third phase; IV: the pencil; V: a target upper block
B	the fixed 2×2 loader matrix (VI.2)	I: a Sanov shear; V: the generators B_i
A_i, B_i, C E_i, E_{ij}, Λ_i	the 193 context-transferred generators (VI.7) lower-marker matrices $E_i = Q^{-i}P_0Q^i$ (Part V's E_j); 3×3 matrix units E_{ij} ; selector streams Λ_i (source: E_i)	V: the 229 generators A_i, B_i, C I: the chart equation E ; IV: the shear $E(t)$, matrix units E_{ij} ; V: E_j as here, and the target map $E(\ell, \rho)$
P_0, Q, t	the lower marker (Part V's P) and Q ; $t = E_0 = P_0$	III: the quartic P ; IV: $P = h(1, 1, 0)$
K_i, G_i, H_i, H	tile matrices; H the common offset 2^ℓ ; $H(s)$, $G(s)$ products	I, II: height bounds H ; V: machine moves H , prefix products H_r, G_r
C_0, C_*, D, D_c	the central matrix; the radix factor 2^{104} ; the digit limit $D = 2H$; the count-control matrix	I: a relator matrix C , a DAG D ; V: the generator C and its block D
U, V, U_s, V_r	the context words [110, A0]; pre-row expressions and post streams; U_0, V_0 generators of $\Gamma_1(5)$	I: the chart matrix U ; V: the machine U , upper blocks U_i, V_i
R, W, h, b	the repetition stream, the top power $W = b^h$, the duration, the packing radix	I: h conjugation nodes; II, III: h Heisenberg elements; V: r the certificate length
Ψ, H'	the inherited faithful twenty-letter encoding and its image subgroup	V: the word encoding Φ
POWER, Sub	the expanded power and binary-containment modules	—

77 The exact represented language

We write $\mathbb{N} = \{0, 1, 2, \dots\}$ and $\mathbb{Z}_{>0} = \{1, 2, 3, \dots\}$. The supplied input is $x \in \mathbb{Z}_{>0}$. It is neither a shifted code $x - 1$ nor a freely supplied Pell index. A countdown state has two signed row vectors and one signed counter,

$$(X, Y, n) \in \mathbb{Z}^2 \times \mathbb{Z}^2 \times \mathbb{Z}.$$

Its fixed initial state is

$$X = (35\ 426\ 321, -19\ 628\ 667), \quad Y = (1, 0), \quad n = x. \quad (\text{VI.1})$$

The fixed matrix used by the loader is

$$B = \begin{pmatrix} -52109 & 29036 \\ -94920 & 52891 \end{pmatrix}, \quad B^{-1} = \begin{pmatrix} 52891 & -29036 \\ 94920 & -52109 \end{pmatrix}. \quad (\text{VI.2})$$

There are exactly 97 branch choices:

$$\text{LOAD} : X' = X, \quad Y' = YB^{-1}, \quad n' = n - 1, \quad (\text{VI.3})$$

$$\text{TILE}_i : X' = XK_i, \quad Y' = YG_i, \quad n = n' = 0. \quad (\text{VI.4})$$

The matrices (K_i, G_i) are the actual 96 pairs in the pinned countdown receipt [55]. Row-vector multiplication is on the right. The accepting endpoint is

$$X = Y, \quad n = 0. \quad (\text{VI.5})$$

No independent choice is permitted for the two row lanes. The common i is essential to the matrix equivalence.

Definition 77.1. Let $\mathcal{A} \subseteq \mathbb{Z}_{>0}$ contain precisely those inputs for which the state (VI.1) has a finite chronological path under (VI.3)–(VI.4) reaching (VI.5). A path is a sequence of states with consecutive endpoints identified, not merely a multiset of locally valid edges.

The retained original tile identifiers are

$$\{1, 2, 18, 19\} \cup \{21, \dots, 107\} \cup \{109, 110, 111, 112, 114\}.$$

The source numbers branch zero as **LOAD** and its other 96 branches according to the receipt's retained tile order. The complete numerical arrays, not small substitutes, are included in the release. They are data for a fixed polynomial; neither the input nor a witness selects a previously unpaid coefficient table.

Theorem 77.2 (Literal fixed-arity certificate). *There is the explicitly supplied polynomial*

$$\mathcal{P}(x, w_1, \dots, w_{41309}) \in \mathbb{Z}[x, w_1, \dots, w_{41309}]$$

with exact total degree 12 such that, for every $x \in \mathbb{Z}_{>0}$,

$$x \in \mathcal{A} \iff \exists (w_1, \dots, w_{41309}) \in \mathbb{Z}_{>0}^{41309} \quad \mathcal{P}(x, w) = 0. \quad (\text{VI.6})$$

The declared literal implementation has 184 016 binary gates: 72 093 multiplications, 64 111 additions, and 47 812 subtractions. Fixed integer literals are free, while nontrivial fixed-coefficient multiplications occurring in the arithmetic are charged. The output is exactly the unweighted sum of the squares of 23 618 residuals.

The arithmetic semantics use the pinned constructive Pell theorems stated in Section 80. Under the inherited faithful-group, lower-marker, and synchronized-row interfaces in Section 79, (VI.6) is equivalent to membership of $T(x)$ in the associated fixed 195-generator singular 7×7 directed semigroup.

This is an implementation theorem. It asserts neither a shortest arithmetic circuit nor a bound on the bit size of witnesses. Its positive-integer domain is part of the theorem. In particular, degree twelve alone supplies no corresponding real-witness equivalence.

78 What is fixed and what was previously unpaid

The fixed contexts in this exact numerical instance are

$$U = [110, \quad V = \mathbf{A0}].$$

They determine the context-transferred matrix array. The finite word that motivates the ordinary-input family is $U(W_0^2)^x V$, with $W_0 = 01010111$. This is an actual fixed-context family; it is not an implemented compiler from arbitrary programs.

The predecessor countdown source has a complete local transition polynomial of exact degree 194 and 2,065 gates, together with a seven-gate endpoint polynomial [55]. For fixed duration h , copying that local schedule gives a $2066h + 7$ gate upper bound with $5h$ signed trajectory witnesses. These

are valid length-dependent ledgers, but existentially naming h does not turn $5h$ coordinates into a fixed list of variables.

Report55 supplies the missing arbitrary-duration packing. It does not insert the degree-194 polynomial as one free gate. Instead it certifies every selected affine branch through paid digit slices and four packed signed row updates. This different representation has a larger fixed circuit and a lower exact degree. The two resource comparisons must not be confused.

[write 91] (4 October 2026.) The two preceding paragraphs describe the research programme at the source’s pin, commit 750aeb4f7 of 3 October 2026 (21:11; times are those of the commits, UTC−7), whose account of the countdown [55] ends: “Unbounded fixed-arity packing remains unpaid.” The packing was missing then. The programme supplied it by another route before this archive arrived: a marked-loader packing (b388dc588, 02:34 on 4 October), a fully emitted uniform grammar of 176,586 operations at exact degree 2,360,653 (bbb73e0c7, 03:10), an atomic packing of 4,155 operations at exact degree 34,045 (7f7b87500, 03:30) and a chain of reductions ending in the shared-action fusion [27] (a32e1b139, 10:12), whose four variants cost 1,399, 1,396, 1,396 and 1,393 operations, with 141, 140, 140 and 139 positive witnesses and exact degrees 35,587, 53,345, 53,347 and 71,105. The archive arrived four minutes later (0d7f51c44, 10:16). Those sources are uniform over valid program recipes: the program contexts enter as eight fixed coefficient ports and a paid matrix switch, where this Part fixes the contexts [110 and A0] and absorbs them into the matrices. Both have the ordinary input x , and both count every binary multiplication, addition and subtraction with fixed numerals free, so the figures compare directly. This Part is the low-degree end of the trade-off: 184,016 operations and 41,309 witnesses at degree 12, against 1,393 operations and 139 witnesses at degree 71,105, or 1,399 at degree 35,587. The programme’s review of the archive [26] calls it “a useful paid low-degree reference, with much larger cost and witness count than the current work”. Neither construction is below the repository’s 84-operation universal bound (commit 20aafb9a5). The local figures quoted from the countdown source (2,065 gates, $2066h + 7$ for duration h) were also lowered after the pin: the programme’s integer selector `matrix193_centered_crt_selector.md` (47b11748c, 01:43) pays the whole countdown step in 133 operations at exact degree 10 over the integers, and $165h + 7$ operations with $31h + 1$ positive witnesses for a fixed duration h .

78.1 Three separate levels of assertion

1. **Exact numerical theorem.** The literal graph and matrix arrays in this release give Theorem 77.2 for this fixed fixture.
2. **Matrix-interface equivalence.** The synchronized-row and counted-suffix arguments identify the same acceptance set with the specified 195-generator target predicate. Their group and lower-marker premises remain explicit inherited mathematical dependencies.
3. **Program-family interpretation.** The inherited effective initialization theorem supplies program-dependent contexts for any chosen c.e. language, conditional on its published simulation premise [62, 63]. Applying this architecture then requires those actual matrices and a newly computed sufficient fixed radix factor. That recipe has not been implemented or audited here as an arbitrary-program compiler.

The present exact numerical gate ledger belongs to the first level. Coefficient simplifications and shared schedules for one fixture cannot silently be claimed for every program-dependent array. The current result does not improve the repository’s prior 84-operation benchmark [59], and is not a 193-generator externally supplied Pell391-loader result, a mortality theorem, or an inverse-closed

group-membership theorem.

79 The inherited matrix interfaces and their exact use

We distinguish the central 2×2 matrix C_0 below from the scalar radix factor C_* used later. Put

$$P_0 = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad e_1 = (1, 0).$$

The context-transferred 193-generator array consists of

$$A_i = \text{diag}(H_i, E_i), \quad B_i = \text{diag}(G_i^{-1}, P_0^{-1} E_i^{-1} P_0), \quad C = \text{diag}(C_0, P_0), \quad (\text{VI.7})$$

where

$$E_i = \begin{pmatrix} 4i+1 & 2 \\ -8i^2 & 1-4i \end{pmatrix}, \quad K_i = C_0^{-1} H_i C_0.$$

All relevant 2×2 matrices are integral with determinant one. The independently reconstructed initial row is $e_1 C_0 = (35\,426\,321, -19\,628\,667)$.

79.1 Faithful alphabet and first-row injectivity

The inherited faithful twenty-letter encoding Ψ has image in the subgroup H' of $\Gamma_1(5)$ described in [59]. That source supplies an explicit finite-graph basis proof, rather than an inference from short-word testing. Its ambient generators are

$$T = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad U_0 = \begin{pmatrix} 1 & 0 \\ 5 & 1 \end{pmatrix}, \quad V_0 = \begin{pmatrix} -9 & 5 \\ -20 & 11 \end{pmatrix}.$$

An immersed rank-twenty graph, based at a vertex with no U_0 edge in either direction, gives

$$H' \cap \langle U_0 \rangle = \{I_2\}. \quad (\text{VI.8})$$

The actual tape-letter choices and eighteen remaining letters are the inherited faithful basis changes in that source. The earlier kernel-row construction [61] explains the same structural issue in a different subgroup; it is not a replacement coefficient array for this report.

The implication we use is elementary once (VI.8) and membership in $\Gamma_1(5)$ are supplied. If $M, N \in H'$ have equal first rows, then NM^{-1} has first row $(1, 0)$ and determinant one, so it is a lower unipotent. Its lower-left entry is divisible by five, making it a power of U_0 . Equation (VI.8) forces that power to be the identity. Therefore

$$M, N \in H', \quad e_1 M = e_1 N \implies M = N. \quad (\text{VI.9})$$

This is not a claim that one row determines an arbitrary determinant-one matrix. Every matrix to which we apply it lies in H' inductively from the actual generators and initial matrices.

79.2 Context transfer and the lower marker

For fixed words U, V , the context construction sets

$$L = \Psi(V\#)^{-1}, \quad R = \Psi(U)^{-1}.$$

It replaces the old upper tile blocks by the conjugates $L^{-1}\Psi(h_i)L$ and $R\Psi(g_i)^{-1}R^{-1}$, and the central block by $L^{-1}\Psi([J1]\#)^{-1}R^{-1}$. The unchanged lower blocks force exactly the word shape

$$A_{i_1} \cdots A_{i_d} C B_{i_d} \cdots B_{i_1}, \quad d \geq 0. \quad (\text{VI.10})$$

On these words the conjugations telescope; applying L^{-1} and R^{-1} to arbitrary unshaped products is not a homomorphism claim [57].

For clarity, the inherited lower-marker proof [60] uses the free conjugate letters $E_i = Q^{-i}P_0Q^i$ in the free group generated by P_0 and $Q = \begin{pmatrix} 1 & 0 \\ 2 & 1 \end{pmatrix}$. In the free family consisting of the marker $t = E_0 = P_0$ and the retained E_i , the marker exponent homomorphism sends t to one and each E_i to zero. It therefore forces exactly one central factor; this is not the ambient P_0 exponent in $\langle P_0, Q \rangle$. In its kernel basis $y_{i,j} = t^j E_i t^{-j}$ indexed by marker height j , a B before that factor has an uncancellable negative height-1 letter, and an A after it has an uncancellable positive height-1 letter. At height zero the remaining letters cancel precisely in reversed order. This yields (VI.10) for every positive product with lower block P_0 , including $d = 0$.

[write 91] The free conjugates $E_i = Q^{-i}P_0Q^i$ of this paragraph are Part V's $E_j = Q^{-j}PQ^j$ of (V.9), with $P_0 = P$ and the same Q , and expanding the product gives the matrices E_i of (VI.7); the two uses of E_i in this section are one family. The inherited lower-marker theorem [60] is the research programme's 193-generator successor [25] of Part V's semigroup, whose marker argument is Lemma 66.1; the successor is summarized at Part V's question 4 (Section 75). The selector streams of Sections 83–86, which the source also writes E_i , are written Λ_i here.

This summary states the precise use of the inherited theorem. The release checks the actual fixed blocks and their interfaces; it does not rerun the upstream machine programs or claim a new proof-assistant formalization of the free-group foundations.

79.3 Synchronized rows recover the old product

For a tile suffix $s = (i_1, \dots, i_d)$, write $H(s) = H_{i_1} \cdots H_{i_d}$ and $G(s) = G_{i_1} \cdots G_{i_d}$. If the counter has first performed x loads, the two rows begin the suffix at $e_1 C_0$ and $e_1 B^{-x}$. Right multiplication gives

$$X_d = e_1 H(s) C_0, \quad Y_d = e_1 B^{-x} G(s).$$

By (VI.9), their equality is equivalent to

$$H(s) C_0 G(s)^{-1} = B^{-x}. \quad (\text{VI.11})$$

The upper block of (VI.10) is exactly the left side. The reverse order of the B factors is indispensable. The lower-marker theorem gives the converse, so the row relation omits no accepting old product. The empty suffix $d = 0$ corresponds to the single old central factor C, not an empty old word [56, 55].

79.4 The counted suffix and the direct input target

Let E_{ij} be 3×3 matrix units with indices $0, 1, 2$. Define

$$O = E_{00}, \quad F = E_{01}, \quad D_c = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix}.$$

Lift each old generator M_j to $L_j = \text{diag}(M_j, O)$, and add

$$\text{END} = \text{diag}(I_4, F), \quad \text{COUNT} = \text{diag}(B, I_2, D_c).$$

The target is

$$T(x) = \text{diag}(I_2, P_0, F + xE_{02}) = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 2 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & x \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}. \quad (\text{VI.12})$$

Its only varying entry is a direct copy of x . That zero-gate target-assembly observation is not the ledger of the full polynomial.

Lemma 79.1 (Control language). *A word in O, F, D_c has entry $(0, 1)$ equal to one if and only if it has spelling $O^n F D_c^k$ for $n, k \geq 0$. Its product then equals $F + kE_{02}$.*

Proof. O is idempotent on coordinate zero, whereas D_c is supported on coordinates one and two. Their mixed products vanish. The only link from zero to the latter coordinates is F . A D_c before F , an O after F , or a second F kills the relevant product. Words with no F have zero in entry $(0, 1)$. Finally the lower-right block of D_c^k is $\begin{pmatrix} 1 & k \\ 0 & 1 \end{pmatrix}$, which gives the stated entire product. The empty word is I_3 and fails the entry test. $\square$

Thus a matching 7×7 product must be

$$L_{j_1} \cdots L_{j_n} \text{ENDCOUNT}^x.$$

The old prefix is nonempty: otherwise the lower physical block is I_2 , whereas the target requires P_0 . Its physical block is the old product multiplied on the right by $\text{diag}(B^x, I_2)$. Since B is invertible, matching (VI.12) is equivalent to the old product being $\text{diag}(B^{-x}, P_0)$ [58]. This proves the desired equivalence in both directions.

The old lifts have rank five, **END** has rank five, and **COUNT** has rank six. All are singular integral 7×7 matrices. Malformed words may have zero control, which cannot match the target's nonzero control entry. That argument would not prove a mortality statement with zero target.

[write 91] Lemma 79.1 and the argument after it re-prove the research programme's counted-suffix note [58], and the row implication (VI.9) re-proves the first-row injectivity of [59]; the source cites both, and both are printed as its own route. The context transfer, the synchronized rows and the lower-marker shape are summarized, not re-proved, as the source says.

80 The explicit constructive exponentiation module

The certificate contains no exponentiation primitive. We use $\text{POWER}(b, e)$ only as a name for the following fixed list of polynomial equations. Its semantic contract is $b \geq 2$, $e \in \mathbb{N}$, and output $o = b^e$. A separate fresh copy of its witnesses is used in every call.

The external number-theoretic source is `Mathlib/NumberTheory/PellMatiyasevic.lean` at `mathlib4` commit `ac77769fabe23cb237559e7f56578dbead91499f`, specifically `matiyasevic` and `eq_pow_of_pell` [54]. The exact file is bundled as inert source. No Lean process or dependency rebuild is part of this report.

80.1 Positive leaves and complete equations

Every independent leaf is strictly positive. A mathematical natural q is represented by a private positive leaf q^+ through $q = q^+ - 1$; this subtraction is emitted and charged. The variables $a, \beta \geq 2$ are positive leaves plus one. The positive variables are

$$o, w, M, g, \xi, \eta, u, v, s, t, q_b, q_v, S,$$

and the eleven natural auxiliaries are

$$d_{wb}, d_{wk}, d_{\eta k}, \alpha_1, \alpha_2, \sigma_1, \sigma_2, \tau_1, \tau_2, \rho_1, \rho_2.$$

Together with the two shifted leaves for a, β , there are $13 + 11 + 2 = 26$ positive leaves. Put $k = e + 1$ and $m = bo$. The complete fifteen equalities are

$$\xi^2 = 1 + (a^2 - 1)\eta^2, \tag{VI.13}$$

$$u^2 = 1 + (a^2 - 1)v^2, \tag{VI.14}$$

$$s^2 = 1 + (\beta^2 - 1)t^2, \tag{VI.15}$$

$$\beta = 1 + 4\eta q_b, \tag{VI.16}$$

$$\beta + u\alpha_1 = a + u\alpha_2, \tag{VI.17}$$

$$v = \eta^2 q_v, \tag{VI.18}$$

$$s + u\sigma_1 = \xi + u\sigma_2, \tag{VI.19}$$

$$t + 4\eta\tau_1 = k + 4\eta\tau_2, \tag{VI.20}$$

$$\eta = k + d_{\eta k}, \tag{VI.21}$$

$$w = b + d_{wb}, \tag{VI.22}$$

$$w = k + d_{wk}, \tag{VI.23}$$

$$M = m + S, \tag{VI.24}$$

$$a^2 = 1 + ((w + 1)^2 - 1)(wg)^2, \tag{VI.25}$$

$$2ab = M + (b^2 + 1), \tag{VI.26}$$

$$\xi + M\rho_1 = \eta(a - b) + m + M\rho_2. \tag{VI.27}$$

Paired natural quotients encode a congruence with either sign. Replacing such a pair by one nonnegative quotient would in general lose valid witnesses. The strict slack S is likewise substantive: the theorem requires $m < M$.

80.2 Why these clauses select the correct Pell index

For $a > 1$, let $(X_a(j), Y_a(j))$ be the Pell sequence in the pinned source. The positive-index branch of `matiyasevic` says that the first nine conditions above characterize

$$(\xi, \eta) = (X_a(k), Y_a(k)).$$

The first three equations imply the ordinary natural-subtraction Pell forms in the theorem. The other clauses impose $\beta \equiv 1 \pmod{4\eta}$, $\beta \equiv a \pmod{u}$, $v > 0$, $\eta^2 \mid v$, $s \equiv \xi \pmod{u}$, $t \equiv k \pmod{4\eta}$, and $k \leq \eta$. Since $k = e + 1 > 0$, the zero-index alternative is excluded. This step cannot be replaced by one unconstrained Pell norm.

Equations (VI.22)–(VI.27) are the positive-base, positive-index case of `eq_pow_of_pell`. In particular $w \geq b, k, M > m$, and the auxiliary Pell equation holds. Its use of natural subtraction $a - b$ agrees with the displayed integer polynomial: $w \geq b \geq 2$ and $g > 0$ in (VI.25) imply

$$a^2 \geq 1 + ((w + 1)^2 - 1)w^2 \geq (w + 1)^2,$$

so $a \geq w + 1 > b$. The theorem therefore gives $b^k = m = bo$. Cancellation of the positive factor b yields $o = b^e$, including $e = 0$.

80.3 Completeness with strictly positive leaves

Conversely, start with $o = b^e$ and $k = e + 1 > 0$. The constructive direction of `eq_pow_of_pell` chooses $w \geq b, k$, an auxiliary positive-index Pell solution a , a modulus $M > m$, and a quotient g . In that construction $w \geq 2$ and the auxiliary Pell ordinate is positive, so $g > 0$. The constructive direction of `matiyasevic` supplies positive Pell coordinates and $v > 0$. Since $\beta > 1$ and $\beta \equiv 1 \pmod{4\eta}$, the quotient q_b is positive; since $v > 0$ and $\eta > 0$, the quotient q_v is positive. All nonnegative differences in (VI.21)–(VI.23) are admissible natural auxiliaries. Every signed congruence quotient is a difference of two naturals.

Thus the positive-leaf adapters impose no extra exclusion. Their semantics are both sound and complete, relative to the explicit constructive theorem source. The implementation does not introduce a general MRDP representation oracle or silently appeal to an unexpanded exponential relation.

[write 91] **Second route.** This module is the polynomial power macro of the report *periodic-turmite-first-revisits* [28], Lemma `ptr:ai:lem:exp` (its Report 42): the same fifteen equations specializing the same two pinned theorems, up to the names of variables (that report's x, y, p, q, Θ are ξ, η, u, v, M here). That report counts 25 positive internal witnesses, leaving out the output, where this section counts 26 leaves including o ; it charges the module 70 operations, 31 multiplications and 39 additions or subtractions. Source 09 does not cite it. The section is printed as source 09's own route to that lemma, with no novelty claimed for it. The builder and source notes of this Part are in turn the origin of the power module of Part X of *signal-machine-collision-certificates* [29] (the pipeline's Report 58, batch 91), which bundles byte copies of them (Section 1.4).

81 Binary containment with complete extraction bounds

For $M, V \in \mathbb{N}$, write $V \preceq M$ when every binary one-bit of V is a one-bit of M . The symbol $\text{Sub}(M, V)$ abbreviates three expanded POWER calls giving

$$R_s = 2^{M+1}, \quad Y_s = R_s^V, \quad Z_s = (R_s + 1)^M, \quad (\text{VI.28})$$

and three more equations

$$Z_s = (qR_s + 2j + 1)Y_s + r, \quad (\text{VI.29})$$

$$2j + 1 + g_c = R_s, \quad (\text{VI.30})$$

$$r + g_r = Y_s. \quad (\text{VI.31})$$

Here q, j, r are naturals, and g_c, g_r are positive. Thus the additional cost is five positive leaves and three residuals beyond the three POWER modules.

Lemma 81.1 (Exact containment). *For natural M, V , these equations have positive auxiliary witnesses if and only if $V \preceq M$.*

Proof. Every binomial coefficient in $(1 + z)^M$ is at most $2^M < R_s$. Consequently

$$(R_s + 1)^M = \sum_{a=0}^M \binom{M}{a} R_s^a$$

is its canonical radix- R_s expansion. Equations (VI.30) and (VI.31) give $0 < 2j + 1 < R_s$ and $0 \leq r < Y_s$. Equation (VI.29) therefore says that the digit at position V is odd. Over $\mathbb{F}_2$, if $M = \sum_a \epsilon_a 2^a$, then

$$(1 + z)^M = \prod_{a:\epsilon_a=1} (1 + z^{2^a}).$$

The coefficient of z^V is one precisely when the binary digits of V select a subset of those positions. This proves soundness.

If the coefficient is odd, choose its half-minus-one parameter j , the Euclidean high quotient q , and low remainder r . The strict gaps are positive; completeness of the three POWER calls supplies all their leaves. This proves the converse. $\square$

The boundary $M = V = 0$ has digit one and succeeds. When $V > M$, the relevant binomial digit is zero and cannot satisfy the positive odd-digit clause. No hidden inequality $V \leq M$ is needed. The proof also covers $V = 0$ for every M .

This extraction is a mathematical explanation of polynomial clauses. No bitwise instruction, radix-digit lookup, comparison, or binomial-coefficient primitive survives in the graph.

[write 91] The parity fact behind Lemma 81.1, that $\binom{M}{V}$ is odd exactly when $V \preceq M$, is Lucas's theorem modulo 2; it is the classical masking relation of the register-machine proof of exponential Diophantine representation (J. P. Jones and Yu. V. Matiyasevich, J. Symbolic Logic **49** (1984), 818–829). Source 09 proves it without naming it. The same expanded containment masks are used, for sandpile certificates, in Part XXI of *canonical-diophantine-certificates* (the pipeline's Reports 50 and 52–54, batch 91, written at the same time as this Part); this Part's packing review (09-chrono-matrix-science-review-PACKING_REVIEW.md) records reading, as inert data, the source notes of a packet named `sandpile-repeated-target-20261004`, by that name Report 53's.

82 The signed radix and well-founded macro domains

Store the four row coordinates in a column

$$z = (X_0, X_1, Y_0, Y_1)^T, \quad a = (35\,426\,321, -19\,628\,667, 1, 0)^T.$$

For each branch $i \in \{0, \dots, 96\}$ define its fixed column map

$$A_i = \text{diag}(K_i^T, G_i^T), \quad c_{ir} = 1 - \sum_{s=0}^3 A_i[r, s],$$

with $A_0 = \text{diag}(I_2, (B^{-1})^T)$ for the loader. The transposes are forced by the row-right-action convention; reversing them would change the language.

Choose the fixed scalar

$$C_* = 2^{104} = 20\,282\,409\,603\,651\,670\,423\,947\,251\,286\,016. \quad (\text{VI.32})$$

For the actual 97 maps, exact reconstruction gives

$$\max \left(128, \max_{i,r} \left(2 + \sum_s |A_i[r, s]| + |c_{ir}| \right) \right) = 18\,510\,406\,623\,962\,009\,412\,894\,903\,228\,521 < C_*. \quad (\text{VI.33})$$

The large numeral is a coefficient bound chosen once for this fixed array. Its size is not a witness or a hidden function of the input.

Introduce positive ℓ, h , two positive gaps g_H, g_x , and expanded powers

$$H = 2^\ell, \quad D = 2H, \quad b = C_*D, \quad W = b^h. \quad (\text{VI.34})$$

The two bound clauses are

$$H = 35\,426\,321 + g_H, \quad x + g_x = D. \quad (\text{VI.35})$$

Thus $H > \max_r |a_r|$, $0 < x < D$, and D, b are powers of two with $D < b$ and $b > 97$. A signed coordinate is represented by an offset digit

$$q = z + H, \quad 0 \leq q < D. \quad (\text{VI.36})$$

The same H is used for all rows and all times. Zero therefore has one common representation, and comparing final offset digits compares the actual signed values.

A natural R satisfies

$$(b-1)R + 1 = W. \quad (\text{VI.37})$$

Since $W = b^h$ and $h > 0$, this determines

$$R = 1 + b + \dots + b^{h-1}.$$

The repetition stream is produced by a polynomial equality and an expanded power, not by an unbounded sum in the source.

82.1 No circular type reasoning

The arithmetic modules are applied in a fixed logical order. The initial call has constant base two and positive exponent ℓ . The bound clauses then give $H, D, b > 0$. The second direct POWER call has $b \geq 2$ and positive h . Natural adapters make R , selectors, slices, post streams, the pre-counter, and final digits nonnegative before any mask theorem is used. The equation for R identifies the nonnegative geometric sum.

Every containment mask below is a product of quantities already known to be nonnegative: R , $D - 1$, and natural selectors. Inside each Sub module, the first power has base two and natural exponent $M + 1$; the next two have bases at least two and natural exponents V, M . Hence every invocation meets its contract without presuming the transitions it will later certify. The carry bound is established only after mask semantics have bounded the digits, and before the update equations are read digitwise.

83 One common selector and paid whole-digit slices

For every branch introduce a natural selector stream Λ_i and impose

$$\text{Sub}(R, \Lambda_i) \quad (0 \leq i \leq 96), \quad \sum_{i=0}^{96} \Lambda_i = R. \quad (\text{VI.38})$$

Because b is a power of two, R has one binary low bit at each of the h time positions, so each Λ_i has radix- b digits in $\{0, 1\}$ at those positions and zero elsewhere. The digit sum across all 97 branches is at most 97, strictly below b . Thus the selector sum has no carries, and its equality to R forces exactly one branch at each time.

[write 91] The source writes these selector streams E_i (and the loader's E_0); they are Λ_i and Λ_0 throughout this Part, because E_i is the lower-marker matrix of Section 79 and $E_0 = P_0$ is the marker itself. No normalization changed. The shipped notes and review (09-chrono-matrix-science-ARCHITECTURE.md, 09-chrono-matrix-science-review-PACKING_REVIEW.md and the others) keep the source's E_i .

For each branch i and row coordinate s , introduce a natural slice Q_{is} and impose

$$\text{Sub}((D - 1)\Lambda_i, Q_{is}), \quad U_s = \sum_{i=0}^{96} Q_{is}. \quad (\text{VI.39})$$

The latter equality is a definition by emitted additions, not an extra equality residual. There are $97 \cdot 4 = 388$ containment calls in (VI.39).

Lemma 83.1 (Paid slice partition). *For each s , the stream U_s has exactly the allowed h digit positions, every digit lies in $[0, D)$, and Q_{is} is precisely the whole-digit portion of U_s at positions selected by Λ_i . Conversely every bounded stream U_s and one-hot selector partition has such slices.*

Proof. Since D is a power of two, $D - 1$ is a block of low binary ones. Multiplying by a selector with digits zero or one places disjoint copies of that block at selected time positions. There is no overlap because $D < b$. Binary containment permits any digit in $[0, D)$ inside a selected block and forces zero outside. At a given position exactly one selector is active, so the sum of the Q_{is} contains at most one nonzero digit there. It has neither digit overflow nor carrying. The selected slice therefore equals the entire U_s digit, not a fragment or an independently chosen row value.

Conversely, assign each given digit to its unique selected branch, setting all other slices there to zero. Every containment relation holds, and the sums recover the original streams. $\square$

The same selectors serve all four coordinates. Four independent selector families would allow incompatible row transitions. The present partition avoids generic AND modules: once one-hotness is proved, containment by disjoint masks suffices. It does not make variable selection free; all 388 masks, their POWER expansions, and their assembly arithmetic are counted.

84 Chronology with all three boundaries

Introduce five natural post streams V_0, V_1, V_2, V_3, V_n . They satisfy

$$\text{Sub}((D-1)R, V_r) \ (0 \leq r \leq 3), \quad \text{Sub}((D-1)R, V_n). \quad (\text{VI.40})$$

A natural counter pre-stream N has the stronger loader-only mask

$$\text{Sub}((D-1)\Lambda_0, N). \quad (\text{VI.41})$$

It therefore vanishes at all tile positions, while every stream digit remains in $[0, D)$.

For each row introduce a natural final digit f_r and a positive gap g_r with

$$f_r + g_r = D. \quad (\text{VI.42})$$

The chronology clauses are the exact integer equalities

$$bV_r + (H + a_r) = U_r + Wf_r \quad (0 \leq r \leq 3), \quad (\text{VI.43})$$

$$bV_n + x = N. \quad (\text{VI.44})$$

Lemma 84.1 (Complete chronological joins). *The row clauses are equivalent to an initial digit $U_r[0] = H + a_r$, all joins $U_r[t+1] = V_r[t]$ for $0 \leq t < h-1$, and final digit $V_r[h-1] = f_r$. The counter clause gives $N[0] = x$, all joins $N[t+1] = V_n[t]$, and last post digit $V_n[h-1] = 0$.*

Proof. Write brackets for radix- b digits. Every stream has no positions outside $0, \dots, h-1$ and digits below $D < b$. The initial digit $H + a_r$ lies strictly between zero and D ; the final digit f_r lies in $[0, D)$. Multiplication by b shifts the post stream by one, leaving its bottom position free. Multiplication by $W = b^h$ places f_r in position h , above every pre-stream digit. Thus both sides of (VI.43) are already canonical expansions. Matching positions zero, $1, \dots, h-1$, and h gives exactly the three asserted conditions.

In (VI.44), $x < D < b$ fills the free bottom position and the right side has no digit at position h . Matching that position forces the terminal post-counter to be zero. Every lower position gives the initial value or an internal join. There is no reduction modulo W and no discarded top coefficient. $\square$

Consequently the certificate stores actual chronological pre- and post-states. It does not admit a permutation of independently valid edges, a cyclic tableau without initialization, disconnected row histories, or a final frame hidden beyond the packing boundary. The horizon h is shared by every stream through the same R, W .

85 Signed affine updates and a noncircular carry proof

For each output row r , the source splits its fixed signed coefficients once and imposes the following equality with nonnegative sides:

$$\begin{aligned} V_r + \sum_{\substack{i,s \\ A_i[r,s] < 0}} (-A_i[r,s])Q_{is} + H \sum_{\substack{i \\ c_{ir} < 0}} (-c_{ir})\Lambda_i \\ = \sum_{\substack{i,s \\ A_i[r,s] > 0}} A_i[r,s]Q_{is} + H \sum_{\substack{i \\ c_{ir} > 0}} c_{ir}\Lambda_i. \end{aligned} \quad (\text{VI.45})$$

These are four equalities, one per coordinate. Every nontrivial coefficient multiplication and offset product in this fixed schedule is an ordinary paid multiplication gate. Comparing a *fixed* coefficient with zero when emitting the source is compile-time organization, not an input-dependent comparison primitive.

Lemma 85.1 (Both sides are carry-free). *The already established masks and selector partition imply that every radix- b coefficient on either side of (VI.45) is nonnegative and strictly less than b , before using the transition equality itself.*

Proof. At a time position only one branch i contributes. Put

$$J_{ir} = \sum_s |A_i[r, s]| + |c_{ir}|.$$

All selected row digits and post digits are at most $D - 1$. Also $H \leq D - 1$, since $D = 2H$ and $H \geq 1$. Therefore each side's raw coefficient is at most

$$(1 + J_{ir})(D - 1) < (2 + J_{ir})D \leq C_*D = b.$$

Every summand on a given side is nonnegative. The finite source indices do not create shifts, and the mask supports are confined to the same h positions. This proves the claimed bound without assuming that any proposed output is a correct matrix image. $\square$

Uniqueness of radix expansion now reads (VI.45) digitwise. If branch i is selected at time t , the result is

$$q_r(t + 1) = \sum_s A_i[r, s]q_s(t) + c_{ir}H. \quad (\text{VI.46})$$

Substitution of $q_s(t) = z_s(t) + H$ and $c_{ir} = 1 - \sum_s A_i[r, s]$ yields

$$z_r(t + 1) = \sum_s A_i[r, s]z_s(t).$$

Every coordinate has the same selected i , so this is the actual synchronized row update. The representation uses a single common offset instead of an independently chosen sign decomposition. It introduces neither a sign oracle nor a variable inverse.

The fixed numeral C_* is intentionally conservative. Its correctness is checked against the actual 97 maps, including all large positive and negative entries. Replacing those maps by a small toy alphabet would not validate the present carry theorem or its source ledger.

86 Counter guards and the equivalence in both directions

The remaining counter update is

$$N = V_n + \Lambda_0. \quad (\text{VI.47})$$

Its right-side digits are at most D , which is strictly below b , so the equality is digitwise. At a load position it says $n' = n - 1$ and natural post digits force $n \geq 1$. At a tile position, the loader-only mask (VI.41) first gives $n = 0$; equation (VI.47) then gives $n' = 0$. Both tile guards are paid.

Lemma 86.1 (Exact phase and input count). *A packed accepting branch word is exactly $\text{LOAD}^x\text{TILE}^*$, with x initial loads and any finite tile suffix. Restricting packed counters to naturals loses no accepting path of the original signed-counter system.*

Proof. Chronology starts the counter at x and ends it at zero. Each load decrements it by one and every tile keeps it zero, so telescoping counts exactly x loads. Before zero is reached a tile is disallowed; at zero a later load is impossible in the packed natural representation. This proves the stated shape.

For the original signed system, the same telescoping identity already counts exactly x loads in any accepting path. The first tile can occur only after x loads, because its pre-counter must be zero. Any later load would make the total exceed x , and there is no increasing transition to repair it. Thus every original accepting counter lies in $[0, x]$ and has the same shape. Locally legal signed loads below zero never belong to an accepting path. $\square$

Finally impose the two row endpoint equalities

$$f_0 = f_2, \quad f_1 = f_3. \quad (\text{VI.48})$$

Because the offset is common, these give $X = Y$. The final counter zero is already enforced by (VI.44); it does not need an omitted extra residual.

86.1 Soundness of the positive polynomial zero

Let x and all independent witness leaves be positive and suppose $\mathcal{P}(x, w) = 0$. The output is a sum of squares of integer residuals, so every residual vanishes. Apply the POWER and Sub semantics in the well-founded order of Section 82. Equations (VI.38) give a unique branch at every one of h positions. The paid slice lemma provides one bounded pre-state in all four row coordinates. The post masks and chronology lemma identify their consecutive states, the specified initial row, and the final row.

The carry lemma turns the four packed affine identities into the actual selected matrix transitions. The loader-only mask and counter update supply both tile guards and exact decrementing loads. The counter chronology supplies its input x and terminal zero. Finally (VI.48) supplies $X = Y$. We have reconstructed a genuine finite accepting chronological path. Section 79 then gives the exact 195-generator target membership under the named inherited interfaces.

86.2 Completeness from a genuine accepting path

Conversely, take a finite accepting signed path on $x > 0$. Lemma 86.1 gives $h \geq x \geq 1$ and natural counters in $[0, x]$. Its finitely many signed row values have a finite maximum absolute value. Choose a power of two $H = 2^\ell$ strictly larger than that maximum and 35 426 321, with $x < 2H$. Such an $\ell > 0$ exists. Then every row offset and every counter fits its stated digit range.

Set $D = 2H$, $b = C_*D$, $W = b^h$, and $R = \sum_{t < h} b^t$. Pack the genuine pre-states, post-states, endpoints, and selectors. Put each pre-row digit into the slice of its selected branch. All masks hold by binary containment; all bound gaps are strictly positive. The actual chronological sequence gives the shift equalities, and its true local updates give (VI.45) and (VI.47). The true endpoint gives (VI.48). The completeness parts of POWER and Sub supply every remaining strictly positive leaf. Therefore every residual, and hence $\mathcal{P}$, is zero.

86.3 Boundary cases and nonclaims

A zero-length tile suffix is allowed throughout. Under the row theorem it corresponds to the one-factor old word C; the counted-suffix word still has END and x copies of COUNT. Positive

input forces positive total path duration, so demanding $h > 0$ loses no case in the theorem. No unimplemented zero-duration disjunction for $x = 0$ is used.

The converse chooses a sufficiently large offset after seeing a finite accepting path. It does not promise a computable bound on that offset from the input alone. Likewise, no finite-fold or unique-witness statement follows: offsets and auxiliary congruence quotients may vary. The proof is existential over positive integers and does not justify the same semantics for arbitrary real leaves.

87 The fully expanded source and exact resource ledger

The authoritative graph is the inert JSON file `science/evidence/polynomial-dag.json`. Its SHA256 is

95e2563fcfcaecfdc5918ffd6dd7421896350df8969a38f08cbb5f5060d80034

Its only leaf classes are the one literal input `input:x`, the named independent positive witnesses, and fixed integer literals. Every gate is a binary addition, subtraction, or multiplication whose operands are earlier gates or leaves. Macro records and 506 named ports describe already emitted expressions; they are not additional primitive operations.

[write 91] This file, 11,069,996 bytes, is not shipped in this report. Its builder, shipped as `code/09-chrono-matrix-science-build_certificate.py`, regenerates it byte for byte, with the digest above, in a few seconds from the research programme’s countdown receipt (README, “Reconstructing the excluded data”). The other files named in this Part are shipped under the prefix `09-chrono-matrix-`, or are repository files; the README maps every name.

The source has fixed shape. All source-generating loops run over fixed lists: 97 branches, four rows, five post streams, fifteen POWER equations, or the final fixed residual list. No input x , horizon h , offset H , radix, or numerical witness value changes the number of variables or source instructions.

87.1 Arithmetic gates

Region	Multiplication	Addition	Subtraction	Total
Body	48,475	40,494	24,194	113,163
Residuals and squares and sum	23,618	23,617	23,618	70,853
Full polynomial	72,093	64,111	47,812	184,016

The tail forms each residual by one subtraction, squares it by one multiplication, and sums all 23 618 squares by 23 617 additions. Thus it costs $3 \cdot 23\,618 - 1 = 70\,853$ gates. The combined addition/subtraction convention gives

$$72\,093\,M + 111\,923\,A = 184\,016.$$

Fixed numerals themselves are free. Multiplications by nontrivial coefficients in the emitted arithmetic are charged. Omitting zero summands or multiplication by one is ordinary fixed-source simplification, not a free selected matrix operation.

87.2 Module and witness ledger

There are exactly

$$97 + 388 + 5 + 1 = 491$$

Sub calls: selector masks, row slices, post masks, and the loader-only counter mask. Each contains three POWER calls; the two direct powers for H and W give

$$3 \cdot 491 + 2 = 1\,475$$

POWER calls in all. There is no AND or SPREAD call.

The witness ledger is

$$1\,475 \cdot 26 + 491 \cdot 5 + 504 = 41\,309. \quad (\text{VI.49})$$

The 504 outer leaves are tabulated here; a natural variable always means a positive leaf minus one.

Outer witness class	Count
Positive h, ℓ	2
Positive initial-offset and input gaps	2
Natural repetition R	1
Natural selectors Λ_i	97
Natural slices Q_{is}	388
Natural row and counter post streams	5
Natural counter pre-stream N	1
Natural final row digits f_r	4
Positive final row gaps	4
Total	504

The power output leaves for H and W are already counted inside their modules; they are not repeated in this outer total.

87.3 Residual ledger

The equality count is

$$1\,475 \cdot 15 + 491 \cdot 3 + 20 = 23\,618. \quad (\text{VI.50})$$

The twenty outer clauses are two initial/input bounds, repetition, selector partition, four final bounds, four row chronology equations, counter chronology, two row endpoint equalities, four affine row updates, and the counter update. The tile guards arise jointly from the paid counter mask and update. The terminal counter zero arises from chronology. Counting them again as absent explicit equalities would misread how the certificate enforces them.

87.4 Exact total degree twelve

Sparse independent normalization gives the following residual-degree distribution:

Degree	1	2	3	4	6
Residual count	4,825	10,927	1,966	4,425	1,475

Every residual has degree at most six, so the sum of their squares has degree at most twelve. This is the degree in x and all independent positive leaves. Fresh module outputs remain independent leaves linked by equations; one must not substitute their semantic exponential meanings to compute polynomial degree.

The thirteenth POWER residual is

$$a^2 - 1 - ((w + 1)^2 - 1)(wg)^2.$$

Its degree-six homogeneous term is $-w^4g^2$, using that call's private positive leaves w, g . The sum of all squared highest homogeneous parts is a sum of real polynomial squares and cannot vanish identically if one summand is nonzero. More specifically, the independent source checker extracts the coefficient of

$$\text{bounds.offset.w}^8 \text{bounds.offset.g}^4$$

from the squares of *all* residual leading parts and obtains exactly one. Thus degree twelve is attained in the literal polynomial. This argument uses formal polynomial identities, not relations valid only on the zero set.

[write 91] The research programme's review [26] checked the degree independently: restricting the thirteenth residual of the first POWER call (`bounds.offset.eq13`) to its leaves w, g , all other leaves zero, gives $-w^4g^2 - 2w^3g^2$, whose degree-six part is the $-w^4g^2$ printed above.

88 Independent verification and limits of the evidence

The independent audit [53] reports PASS for the stated positive-integer theorem and exact source, conditional on the named inherited Pell and matrix-interface theorems. It did not import or execute the submitted builder, submitted semantic checker, predecessor programs, saved accepting schedules, or Lean. It read fixed matrices, text, JSON, and the literal polynomial graph. Its newly written standard-library checkers are separate from the scientific emitter.

88.1 Complete symbolic reconstruction

The exact-source checker normalizes every one of the 113 163 body gates as a sparse polynomial over $\mathbb{Z}$. It independently specifies the intended clauses from named positive leaves and matrices. In particular, it reconstructs the four transition equations directly in signed affine form rather than copying the emitter's sign-split gate order. It checks:

- All 23 618 residuals, with no missing or extra clauses, and all arguments and expanded equations of 1 475 POWER and 491 Sub calls
- The exact set of 41 309 distinct positive leaves, all adapters, and all 506 exposed ports
- The literal unshifted input, all topological references, and the allowed binary gate vocabulary
- Every one of the 70 853 finalizer gates, including all residual squares and their unweighted sum
- All 97 actual matrix maps and offset corrections, the exact coefficient bound, full liveness, and every declared resource count
- The exact degree distribution and the attaining degree-twelve monomial coefficient

A gate-by-gate proof of this finite source does not materialize enormous numerical values of nested powers. Its output is an exact symbolic certificate audit, not a large-witness numerical evaluation.

88.2 Matrix and provenance reconstruction

The independent interface checker obtains L, R and the central block from actual fixed letter products. It reconstructs $B = \Psi((01010111)^2)$ and its inverse. For each retained tile it reconstructs the complete context-transferred A_i, B_i , both lower marker blocks, $H_i, K_i = C_0^{-1}H_iC_0$, and G_i with the correct inverse convention. It then reconstructs all 195 lifted matrices, checking all 9 555 entries and ranks, and the literal target template.

Seven local Git blob pins are recomputed from exact bytes. The original audit also records independent read-only GitHub confirmations of the specified proof commits, receipt blobs, and the current-main blobs for the two requested matrix notes. Current-main equality is a statement about that recorded retrieval, not a promise that upstream files will never change.

The author source and graph were frozen before exact audit. Explanatory and supplemental evidence were completed during it and then frozen separately. The final eighteen-file author manifest was verified in full. The release preserves all of those bytes, including historical prose and programs as inert provenance.

88.3 Fresh finite challenges

The interface audit’s generated probes are:

Check family	Cases
Control words of length at most nine	29,524
Fresh signed-counter candidates	14,322
Arbitrary bounded chronology tableaux	4,352
Exact binomial and containment parity cases	1,280
Actual-matrix affine carry-bound corners	37,248
Four-coordinate slice partitions	112

Exactly 80 of the chronology candidates pass, all with the correct initial, internal, and final joins. The counter probes include negative initial values and empty paths. The binomial checks include zero masks, zero indices, and out-of-range indices. Carry tests cover all 97 actual maps; slice cases range across the branch family. These are fresh finite checks, not executions of saved accepting traces.

The scientific author’s separate semantic receipt records its own earlier probes, including fresh chronological prefixes with row magnitudes up to 147 bits. That receipt is preserved as historical evidence, not rerun by this release. Neither it nor the independent audit constructs a full gigantic Pell witness for an accepting positive input. The unrestricted mathematical proofs, not finite enumerations, establish the equivalence.

[write 91] **The research programme’s review.** The audits above were produced by the pipeline that wrote the manuscript. The one check from outside it is the research programme’s intake review [26] (commit [a9ab9a698](#), 4 October 2026, 10:45, half an hour after the archive arrived), which read the delivered README, ARCHITECTURE.md and SOURCE_NOTES.md completely and parsed the entire DAG as inert data. It confirms 184,016 gates (72,093 multiplications, 64,111 additions, 47,812

subtractions; 111,923 additions and subtractions together), all gate operands and the topological order, complete backward liveness of all gates, of the 41,309 witnesses and of the input x , the 113,163-gate body and the 70,853-gate finalizer that squares and sums all 23,618 residuals, the witness and residual ledgers (VI.49) and (VI.50), and exact degree 12 (note in Section 87). It reports no defect. It executed no archived program, and it did not recertify the inherited Pell and group semantics or the chronological proof; its finding is about the arithmetic of the DAG. At placement (on an extraction of the archive) and at this write (from the shipped files), the two independent checkers of Section 88 were rerun from the shipped files on a reconstructed science tree; both receipts equal the shipped ones after CRLF→LF (README, “Rerunning the programs”).

89 Reproducibility and the immutable release

The archive separates mathematical inputs, inherited dependencies, independent audit evidence, and release tooling. Its principal entries are:

- `article/Report55.tex` and `article/Report55.pdf`: this editable article and its rendered version
- `science/`: byte-preserved scientific proof, source graph, fixed arrays, source notes, original code, and frozen manifest
- `independent_audit/`: unchanged independent checkers, report, pins, and original receipts
- `dependencies/`: the pinned Pell theorem source and license, and the deeper inherited matrix proof notes
- `audit_replay/`: the separate portable adapter for isolated replay of only the inspected independent checkers
- `verification/`: release-time preservation, build, layout, and review evidence

Historical commands and absolute paths inside preserved source packets are provenance, not the supported release entry points. Do not run the submitted scientific builder or checker, historical upstream programs, saved schedules, or Lean to reproduce this release.

The portable replay takes explicit science and independent-audit directory paths and a fresh output directory outside both inputs. It stages unchanged independent checker bytes into that output, supplies the bundled science path explicitly, and checks regenerated receipts against their pins. Its own documentation gives the exact supported invocation and safety guards. Normal and optimized Python replays are separately checkable; optimization must not disable scientific assertions.

The release utility verifies the exact manifest inventory and file hashes, builds the PDF in a fresh external directory with writable TeX caches there, and emits a deterministic ZIP. No-shell-escape pdfLaTeX runs three passes with fixed timestamp inputs. Sealed PDF reproduction verifies byte equality with the packaged PDF in the same installed TeX/font environment; cross-version binary PDF identity is not promised. The ZIP uses sorted entries, fixed timestamps, fixed POSIX metadata, and uncompressed stored members.

The manifest excludes itself and must be authenticated by a trusted accompanying SHA256. Archive and final manifest hashes remain outside the archive to avoid self-reference. Verification,

replay, PDF building, and ZIP creation preserve release input bytes and metadata. The README contains the short commands and distinguishes author-time preparation from sealed reproduction.

89.1 What a successful replay establishes

A successful exact replay establishes that the supplied literal graph still reconstructs to the stated clauses, matrices, ledger, and degree, and that the finite challenges still produce the frozen receipts. Manifest verification establishes that the inspected files are the delivered files. Rebuilding the PDF checks document reproducibility, not a new mathematical theorem. These are complementary checks; none is a substitute for the stated inherited mathematical premises.

[write 91] **Shipped names and the Windows replay.** In this report the release is flattened under the prefix `09-chrono-matrix-`: programs in `code/`, receipts, manifests and logs in `data/`, and the Markdown notes, reviews and audits at the report root, with `dependencies/` shortened to `deps-`; for example `science/build_certificate.py` is `code/09-chrono-matrix-science-build_certificate.py` and `independent_audit/AUDIT.md` is `09-chrono-matrix-independent_audit-AUDIT.md`. Not shipped: `article/Report55.tex` and `Report55.pdf` (this Part is the article), the delivery README, the DAG (regenerable; note in Section 87), the mathlib Pell source and its licence (third-party; cited by commit and digest in Appendix M), eleven byte copies of research-programme files in `science/sources/` and `dependencies/` (cited by path), one checksum list and six byte copies of other files of the release. The release tool's `verify` step needs the complete delivered inventory with its POSIX file modes and refuses to run on NTFS, and the replay adapter needs the complete frozen science and audit trees; on Windows the adapter's byte gate also fails, because the checkers write their receipts with CRLF line endings. The README gives two routes: the delivered tools on an extraction of the arrival archive on a POSIX system, and a direct run of the builder and both independent checkers on a reconstruction in a scratch directory.

90 Future questions and present boundaries

The construction leaves several focused research questions.

1. **Reduce the paid selection cost.** The 388 row-slice predicates dominate the module count. Can a smaller polynomial system preserve common branch choice and arbitrary signed row digits without introducing an uncharged selected-table primitive? Any proposed reduction needs both a digit-level proof and a literal new ledger.
2. **Share range structure safely.** Every Sub currently expands three independent POWER modules. Sharing may reduce operations, but one must specify exactly which outputs and witnesses can be reused without changing existence or invalidating a module's domain. A semantic identity alone does not identify source gates.
3. **Tighten the coefficient bound.** The current C_* is deliberately conservative. A smaller fixed numeral would reduce digit and witness sizes, but would not necessarily change gate count or total degree. It must bound both nonnegative sides on arbitrary masked candidate digits, not merely on valid trajectories.
4. **Make the program-family compiler explicit.** The inherited initialization theorem supplies mathematical recipes for program-dependent contexts. A separate artifact could implement

that path, freeze its actual matrices, and recompute the coefficient bound. The present fixed fixture cannot serve as evidence that this work is already done.

5. **Formalize the interface chain.** The Pell source is a pinned formal theorem, while the exact adapter arguments, signed packing, lower-marker use, and faithful-group interface are written proofs with exact finite audits. A single end-to-end mechanization would be valuable, but is not claimed by this release.
6. **Compare alternative ledgers honestly.** Replacing the countdown by a direct seven-dimensional product history might trade selector count for matrix dimension. The direct-input target has zero assembly gates, but any comparison must include arbitrary-duration history, positivity, bounds, chronology, and endpoint arithmetic.

No practical numerical witness-size bound, uniqueness, finite-fold representation, real-witness characterization, or arithmetic optimality follows here. The repository’s prior 84-operation benchmark [59] is unchanged. The established result is narrower and fully explicit: fixed-arity positive-integer recognition of the actual ordinary-input chronological countdown and its pinned directed-semigroup target.

[write 91] (4 October 2026.) Questions 1, 2, 4 and 6 bear on the research programme’s construction of the same night (note in Section 78). It makes the program-family compiler explicit for valid program recipes, with the program contexts as fixed coefficient ports and a paid matrix switch instead of absorbed contexts (question 4, in part: the coefficient recipe is not an implemented compiler from arbitrary programs). Its sharing of paid powers, selector groups and packing sums reaches 1,393 operations (questions 1 and 2), but at exact degree at least 35,587, so it trades degree for operations rather than reducing this Part’s selection cost at degree 12; the two ledgers are the comparison that question 6 asks for. Question 3 is answered in part by the source’s own packing review (09-chrono-matrix-science-review-PACKING_REVIEW.md, Section 3): any fixed power of two $C \geq \max(128, \max_{i,r} M_{ir})$, with $M_{ir} = \max(P, 1 + N)$, where P and N are the sums of the positive coefficients and of the absolute values of the negative ones in row r of the matrix A_i , already suffices, which for the actual maps is 2^{103} ; the article keeps $C_* = 2^{104}$. On question 5: the repository has no Lean or Rocq module for any statement of this Part. Its Lean development uses mathlib’s `Pell.matiyasevic` (Computability/HilbertTenthProblem/Lean/Diophantine/Paper1976/Cor 26.lean), but for a different four-equation system, not this Part’s fifteen-equation module.

[write 91] **Claims of this Part that remain open here.** Three statements of source 09 are used, not proved, in this report; each is kept where the source states it, and is listed here as open with what is missing. (a) *The matrix-interface equivalence* of Theorem 77.2: positive zeros $\Leftrightarrow T(x)$ lies in the 195-generator semigroup. Source 09’s argument (Section 79) combines the faithful twenty-letter encoding with $H' \cap \langle U_0 \rangle = \{I_2\}$ (VI.8), the lower-marker shape (VI.10), context transfer and the synchronized rows; it proves the row implication (VI.9) and the control-language lemma, but takes the finite-graph basis proof of (VI.8), the free-group marker argument and the context telescoping from the research programme’s notes [59, 60, 57, 56]. Missing: a proof of those inputs in this report, or a formal one (question 5); the programme’s notes give conventional proofs, which source 09 and this write did not re-check. (b) *The program-family interpretation* (level 3 of Section 78): that the same architecture recognizes every c.e. language once program-dependent contexts are supplied. Source 09’s sketch is the inherited effective initialization [62] composed with this Part’s packing and a recomputed radix factor. Missing: the Neary–Woods simulation premise [63], which is cited and not re-proved, and an implemented and audited compiler with its per-program coefficient bound (question 4). (c) *The arithmetic semantics of the POWER module* rests on the two pinned mathlib

theorems, which are formal in mathlib but were not re-run here (Lean was not executed); this is a stated dependency rather than a gap, and is listed for completeness. No statement of source 09 was found false, and none was dropped.

A Part I: Scalar templates and an independent specification

All matrix residuals in the report can be expanded without any matrix library. For $X = \begin{pmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{pmatrix}$

and $Y = \begin{pmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \end{pmatrix}$,

$$XY = \begin{pmatrix} x_{11}y_{11} + x_{12}y_{21} & x_{11}y_{12} + x_{12}y_{22} \\ x_{21}y_{11} + x_{22}y_{21} & x_{21}y_{12} + x_{22}y_{22} \end{pmatrix}. \quad (157)$$

For a free-conjugation gate with child X , output Z , and chart coordinates (x, y, z, t) , the four residuals $ZU - UX$ are

$$\begin{aligned} (1 + 4x)Z_{11} + 2zZ_{12} - (1 + 4x)X_{11} - 2yX_{21}, \\ 2yZ_{11} + (1 + 4t)Z_{12} - (1 + 4x)X_{12} - 2yX_{22}, \\ (1 + 4x)Z_{21} + 2zZ_{22} - 2zX_{11} - (1 + 4t)X_{21}, \\ 2yZ_{21} + (1 + 4t)Z_{22} - 2zX_{12} - (1 + 4t)X_{22}. \end{aligned}$$

Together with $x + t + 4xt - yz$, these are the complete local scalar specification. The inverse residuals are

$$Z_{11} - X_{22}, \quad Z_{12} + X_{12}, \quad Z_{21} + X_{21}, \quad Z_{22} - X_{11}.$$

These templates show that no hidden division, exponentiation of a variable, or nonpolynomial matrix inverse appears in the final compiler.

B Part I: A compact theorem-dependency map

Endpoint	Dependencies beyond elementary arithmetic and definitions
Sanov chart	Ping-pong and the explicitly proved integer descent
Fixed-itinerary quartic	Sanov chart, sum-of-squares zero criterion
Budget quartic	Fixed-itinerary induction, integer Boolean selectors
Height bound	Classical bounded-conjugator lemma, matrix norm estimate
Native arity identity	Budget equivalence and literal allocation count
Computable cutoff equivalence	Height-bound decision procedure, finite enumeration
DAG quartic	Sanov chart and topological induction
Dyadic product optimum	Heisenberg homomorphism, commutator identities, doubling bound
Addition-chain sandwich	Central-coordinate evaluation and commutator identities
Noncomputability examples	A classical undecidable finite presentation and two-generator embedding
Separate fixed-arity existence	An enumerable-predicate adapter and MRDP

C Part I: Source and novelty audit

The repository review was restricted to the relevant Hilbert-tenth material and the identified comparison note, rather than a complete certification of every result in ProveIt. The snapshot used in the citations is `c58206ca101d4744a015a0f0104646109357d943`. The source list records the public MRDP documentation and the group-projective controller-lane note that was inspected. The formal builds reported by the repository were not rerun as part of this work.

Sanov’s free subgroup and arithmetic image are classical. The bounded-conjugator estimate is imported from an identified lemma. Undecidable finite presentations, effective two-generator embeddings, and the general MRDP consequence are also classical. The report’s proposed contribution is the explicit composition of those tools into the area-native and DAG-native quartic compilers, their literal ledgers and semantic bijections, and the sharp product-count analysis in the specified calculus. The elementary area proof and the basic Dehn/decidability equivalence are not presented as newly discovered group theory.

The literature check supports the dependency attributions, but does not establish priority for the compiler formulas or compression theorem. These are proved constructions whose historical novelty still requires specialist review.

D Part II: A compact independent proof capsule

This appendix collects the essential identities in a form suitable for a separate review. Let $L : \mathbb{Z}^n \rightarrow \mathbb{Z}^N$ be injective, and let

$$F(x) = C\mathbf{b}(Lx) + Dx + c.$$

The three maps in $\mathbf{H}^N \times \mathbb{Z}^m$ are

$$\begin{aligned} A(x, u) &= ((Lx, 0, -u), Dx + Cu), \\ B(y) &= ((0, Ly, 0), 0), \\ T(r) &= ((-r, -r, \mathbf{b}(r)), 0), \end{aligned}$$

where the triples denote componentwise Heisenberg coordinates. The first two maps are homomorphisms because their within-phase cross terms vanish. The third is a homomorphism because $\mathbf{b}(r + r') = \mathbf{b}(r) + \mathbf{b}(r') + r \odot r'$. All are injective. Direct multiplication gives

$$A(x, u)B(y)T(r) = ((Lx - r, Ly - r, -u + (Lx) \odot (Ly - r) + \mathbf{b}(r)), Dx + Cu).$$

Equality to $(0, 0, 0, t - c)$ forces, in order,

$$r = Lx = Ly, \quad y = x, \quad u = \mathbf{b}(Lx), \quad F(x) = t.$$

Conversely these choices give equality. Every integer quadratic map has such a normal form because

$$x_i^2 = 2\mathbf{b}(x_i) + x_i, \quad x_i x_j = \mathbf{b}(x_i + x_j) - \mathbf{b}(x_i) - \mathbf{b}(x_j),$$

and adding the coordinate forms makes L injective. General polynomial systems flatten into quadratic ones with unique gate outputs. A fixed MRDP polynomial therefore gives a fixed triple of subgroups, with the external input entering only the target. This is the entire dependency chain for the main undecidability theorem.

E Part II: Domain and claim audit

Potential ambiguity	Resolution in this manuscript
Negative powers	$\mathbf{b}(r) = r(r - 1)/2$ is defined on all integers; the power and homomorphism identities are proved there.
Natural exponents	The chosen L has nonnegative entries, so the canonical r and u are natural. Negative matrix entries are permitted.
Generator dependence	All three displayed maps are injective; their images have the stated free ranks.
Spurious witnesses	Horizontal coordinates first force $r = Lx = Ly$; central coordinates then force $u = \mathbf{b}(Lx)$.
Input-dependent subgroups	The input is a pinned wire on the right-hand side. The quadratic map and generator lists are fixed.
Pure Heisenberg group	A central embedding into $\mathbf{H}^{N+m}$ is explicit. The smaller packing is not claimed to preserve ambient centrality.
Counting	The exact objects counted are ordered subgroup-element triples or unique basis-exponent tuples, not arbitrary words.
Finite-fold MRDP	No finite-fold hypothesis is derived from ordinary MRDP. The four-square preliminary conversion can change multiplicities.
Universal numerical record	No optimized numerical ambient dimension or fully expanded universal matrix table is claimed.

Potential ambiguity	Resolution in this manuscript
Verification	190,826 finite exact-integer assertions supplement proofs; there is no Lean build or independent referee report.
Literature priority	The identified sources state the three-subgroup problem. A limited search is not proof of worldwide novelty.

F Part II: Source audit and bibliography notes

The König–Lohrey–Zetsche text was inspected in its accessible arXiv version, including Theorem 6.6, Remark 6.7 and Theorem 6.8. Roman’kov’s institutional publication record supplied the abstract, bibliographic metadata, and explicit open question; the full journal paper was not accessed. That record gives a 2024 online date and a 2025 print date, which accounts for differing year labels in search results. The reference below uses the print year.

The ProveIt core source was read through the connected GitHub interface at the confirmed commit cited below. Its comments and source were inspected; the entire formal development was not rebuilt. The proof of the new compiler depends only on ordinary exact algebra plus the classical MRDP statement, so repository-specific performance claims play no role.

The external-source search was performed on October 2, 2026. No later resolution was identified in that search. This last sentence reports the scope of the search, not an assertion that none exists.

G Provenance of this report

[write] The report was built from two manuscripts that arrived together in the archives of commit `6914ccca6` and were placed by commit `2a04b60f2`, which staged their programs and data byte-identical to the delivery. [write 78] Two of those programs, `code/03-van-kampen-van_kampen.py` and `code/05-three-phases-heisenberg_compiler.py`, were then patched in place by commit `db3b377f0` (input-guard repairs reviewed in the research programme), before Parts I and II were written; the README records this. Parts III and IV were added from two manuscripts of batch 78 that arrived in the archives of commits `808b53ed8` (manuscript 09) and `48ee077c7` (manuscript 13) and were placed by commit `41e7f1189`, which staged their programs, data and source audit byte-identical to the delivery. [write 82] Part V was added from batch-82 manuscript 22, which arrived in the archive of commit `db37d18c8` (3 October 2026) and was placed by commit `2f58ab4e9`, which staged 59 of its 80 files byte-identical to the delivery: 16 programs, 32 data files and receipts, and 11 proofs, reviews, packet READMEs and provenance texts. [write 91] Part VI was added from batch-91 manuscript 07, which arrived in the archive of commit `0d7f51c44` (4 October 2026) and was placed by commit `22a8ca89e`, which staged 44 of its 68 files byte-identical to the delivery: 11 programs, 25 data files, receipts, manifests and logs, and 8 notes, reviews and audits.

Source	Archive	Pin	Contributed	Printed as
03 (base)	arithmetic_van_kampen. zip, batch 76 manuscript 03; main file arithmetic_van_kampen. tex, 29-page PDF	c58206ca1	the area and proof-DAG compilers, the dyadic optimum, the addition-chain sandwich, 12 research questions, its audits	Part I with Appendices A–C
05	three_commutative_ph ases_research.zip, batch 76 manuscript 05; main file article.tex, 22-page PDF	433df1be3	the three-phase compiler, the three-subgroup theorem, two-phase decidability, the monoid and quartic versions, the nonclosed product, 9 research directions, its audits	Part II with Appendices D–F
06	Order_Is_Not_a_Moment. zip, batch 78 manuscript 09 (arrival 808b53ed8); main file article.tex, 26-page PDF	4cccfa068	ternary pair-count intervals, the gap and canonical quartic certificates, four-letter holes, the Heisenberg lift, supplied-multiplicity decidability, 12 research questions, its provenance audit	Part III with Appendices H–I
07	ProveIt_Affine_Matri x_Diophantine_Resear ch.zip, batch 78 manuscript 13 (arrival 48ee077c7); main file article.tex, 25-page PDF	4cccfa068	affine rigidity in every dimension, the free basis and sharp period, the reconstructed quadratic loader and degree threshold, the multiplication gadget and affine abelian compiler, 10 research questions, its source audit	Part IV
08	Universal_Matrix_Sem igroup_and_Diophanti ne_Certificates_Pack age.zip, batch 82 manuscript 22 (“Research Report 32”; arrival db37d18c8); main file report32.tex, 26-page PDF	none named	the fixed 229-matrix semigroup and its marker lemma, the bounded-length quartic family with exact ledgers, the collision and the 94-tile certificate, the exact fibre generating function, 7 research questions, its proofs, reviews and audits	Part V with Appendices J–K

Source	Archive	Pin	Contributed	Printed as
09	Chronological_Diophantine_Matrix_Certificates_Package.zip, batch 91 manuscript 07 ("Research Report 55"; arrival 0d7f51c44); main file article/Report55.tex, 21-page PDF	750aeb4f7 and d31e29030	the fixed-arity degree-12 countdown certificate with its 184,016-gate ledger, the expanded POWER and containment modules, the chronological packing lemmas, 6 research questions, its audits and reviews	Part VI with Appendices L–M

The full pins are

c58206ca101d4744a015a0f0104646109357d943 (source 03),
 433df1be37188224dd00d0561bfb157949cd6320 (source 05),
 4cccfa06866b6b81b2467e1cf7514ea85ed0216d (sources 06 and 07),
 750aeb4f7834332deb2bec5f32fb470ff8254431 (source 09, countdown receipt and notes),
 d31e29030214c35083b0c2e052c8a3a5d0409eb0 (source 09, counted suffix).

Each pin is the commit that its source names as its repository snapshot: source 03 on its title page and in its bibliography, source 05 in Section 29 and its bibliography. Both are ancestors of the placement commit. The repository files the sources cite (`Computability/HilbertTenthProblem/Lean/MRDP.md`, `Lean/Diophantine/Common/MRDPCore.lean` and the note `group_projective_label_aligned_lanes.md`) are unchanged from the pins to the write, so the sources' descriptions of them remain accurate. [write 78] Sources 06 and 07 name commit `4cccfa068` as their snapshot: source 06 in its provenance appendix (Appendix I) and bibliography, source 07 in Section 43, its bibliography and its source audit. It is an ancestor of the placement commit. The repository files they cite (the research notes `heisenberg_two_generator_membership.md`, `group_affine_input_obstruction.md`, `group_commutator_universal_substrate.md` and `imported_substrate_review_20261002.md`, and `Lean/Diophantine/Common/MRDPCore.lean`) have the same blobs at the pin and at this write. [write 82] Source 08 names no repository commit and cites no repository file. It identifies its numerical data by the digest of its literal coefficient file, printed in Section 74; the shipped `data/08-matrix-semigroup-core-data-semigroup.json` is that file, byte for byte. [write 91] Source 09 names the research-programme commits `750aeb4f7` (its countdown receipt and five matrix notes) and `d31e29030` (its counted-suffix proof) as pins, in Appendix M and in its bibliography. Both are ancestors of the placement commit, and every repository file it pins or copies has the same blob at the pins and at this write (note at the end of Appendix M).

Where the merge had to choose.

- *Base and order.* Source 03 is the base (Part I): it is the broader framework (an arbitrary finitely presented group as substrate) and the larger manuscript. Source 05 follows as Part II. Every result, proof, example, remark, research question, audit and limitation of both sources is printed, in the sources' order and with their numbering inside each Part.

- *Shared setup.* The Heisenberg group, its matrices, multiplication, inverse and power formulas, and the associativity and power-law proofs, which source 05 prints in Sections 18 and 20 and source 03 in Section 11, are printed once, in Section 1.2, with source 05’s equation labels; both places now point there. Source 03’s letter H for this group is written $\mathbb{H}$ (Table 1). Nothing else was renamed, and no normalization changed.
- *Front matter.* The two title pages are replaced by one; each source’s title, abstract and status statement open its Part verbatim. Section 1 is new.
- *Appendices and bibliography.* Each source’s appendices are printed after both Parts, those of source 03 first, with “Part I” or “Part II” added to their titles. The two bibliographies are merged without change of entries; entries [13, 14, 15, 16] were added for the cross-references of Section 1.4.
- *Added text.* Every sentence written in the merge is marked [write]: Section 1; the openings of the two Parts; notes in Part I in Sections 2 (repository relationship), 11 (the shared setup) and 14 (shipped file names); notes in Part II in Sections 18, 20 (the shared setup), 22 (the Heisenberg note), 23 (Remark 23.2 and the phase threshold), 25 and 29 (shipped file names and the repository relationship); and this appendix. Labels were added to the research questions of both Parts, to the first section and the software section of Part I, and to the source-audit appendix of Part II.
- [write 78] *Parts III and IV (batch 78).* Sources 06 and 07 follow Part II in batch-78 manuscript order, as additions to this report rather than a new report: their subject, Heisenberg submonoid membership and the loading of inputs into subgroup membership, is this report’s. Their file prefixes 06– and 07– continue the report’s sequence. Every result, proof, example, remark, question, limitation and audit of both sources is printed, in their order and with their numbering inside each Part; source 06’s two appendices are Appendices H and I, after this one, so that Appendices A–G keep their letters. Each title page is replaced by a Part opening that quotes the source’s abstract (and source 06’s status box); the delivered author line, “Research report prepared for Vladimir Reshetnikov”, is kept, with “AI-assisted” added.
- [write 78] *Renaming and conversion.* Source 06’s $H(\alpha, \beta, \gamma)$ and $\mathbb{H}$ are written $h(\alpha, \beta, \gamma)$ and $\mathbb{H}$; source 07’s λ_i and λ are written θ_i and θ ; one proof step of source 06 called “the second phase” is “the second stage”. Source 07’s `cleverref` references are printed as “Lemma ...”, “Section ...” and so on. The labels of source 06 (55) carry the prefix `gts:po:` and those of source 07 (69) `gts:ai:`; their bibliography keys carry `po-` and `ai-`, with the entries unchanged. Nothing else in the two sources was changed, apart from the [write 78] notes.
- [write 78] *Duplicates.* The Heisenberg multiplication law is printed both in Section 1.2 and, in Greek coordinates, in Part III (119), which the Part refers to. Theorem 52.1 is an attributed reconstruction of the research note [14]; its Step 3 is printed as a second route to `ppc:dr:prop:mihailova` of [13]. No theorem of Parts I and II is re-proved in Parts III and IV.
- [write 78] *Added text.* Section 1 was extended (Sections 1.1–1.6, Table 2); one note in Part II (end of the section on the three-subgroup question); the two Part openings; in Part III, notes on Parikh matrices, the review’s finalizer, the multiplication law, the recording map, the logarithmic map, the research note’s open point, the shipped files, two questions and Appendix I; in Part IV, notes on the repository material, Remarks 46.3 and 53.2, the Mihailova pointer, the reconstruction, the curve $Q(n)$, the review’s square-gate reduction, the shipped

files and questions 9 and 10; this appendix and the bibliography entries [17, 18, 19, 20]. New labels: `gts:part:po`, `gts:part:ai`, `gts:tab:letters78`, `gts:po:sec:litterature`, `gts:po:sec:projection`, `gts:po:sec:software`, `gts:po:sec:questions`, `gts:po:app:names`, `gts:po:app:provenance`, `gts:ai:rem:subgroup` and `gts:ai:rem:binomial`.

- [write 82] *Part V (batch 82)*. Source 08 follows Part IV as an addition rather than a new report: fixed matrix groups and semigroups as Diophantine substrates are this report's subject, and Part IV's question 10 asks for the universal alphabet that source 08 supplies in part. Its file prefix `08-` continues the report's sequence. Every result, proof, example, remark, question and limitation of source 08 is printed, in its order and with its numbering; its title page is replaced by a Part opening that quotes its abstract and scope statement, and its two appendices are Appendices J and K, after all existing ones. Its 61 labels carry the prefix `gts:um:` and its bibliography keys `um-`, with the entries unchanged. Its machine table is kept although it is the collection's third printing, because its rewriting rules and Appendix J are defined from it; the note after it points to the earlier printings and to the repository's other records of the erratum. Its rewriting and tile lemmas re-derive research-programme constructions and are kept as its own route with a credit note; the first half of Lemma 65.1 is kept as a second proof of part of Part I's Sanov theorem. The research programme's 197- and 193-generator reductions are summarized, not printed. Added text: Section 1 was extended (Sections 1.1–1.6, Table 3); the title page and abstract; notes at Part IV's questions 9 and 10; the Part opening and notes after Table 4, Lemmas 64.1 and 65.1, Section 67, the collision, Section 74, questions 4, 6 and 7 and Appendix K; this appendix; and the bibliography entries [21, 22, 23, 24, 25]. New labels besides the 61: `gts:part:um`, `gts:um:tab:letters` and `gts:um:app:sequence`.
- [write 91] *Part VI (batch 91)*. Source 09 follows Part V as an addition rather than a new report: its semigroup is the research programme's successor of Part V's, and it answers in part Part V's question 7 and the fixed-arity half of Part IV's question 10. Its file prefix `09-` continues the report's sequence. Every result, proof, remark, question and limitation of source 09 is printed, in its order and with its numbering; its title page is replaced by a Part opening that quotes its abstract and reading guide, and its two appendices are Appendices L and M, after all existing ones. Its 73 labels carry the prefix `gts:cm:` and its bibliography keys `cm-`, with the entries unchanged. Its selector streams E_i are written Λ_i , and its macro `\sha` is `\cmsha`. Its POWER module re-derives Lemma `ptr:ai:lem:exp` of *periodic-turmite-first-revisits* and is kept as a marked second route; its control-language lemma and its row implication re-prove research-programme notes that it cites and are kept as its own route. The programme's later uniform compilers and its review of the archive are summarized in dated notes, not printed. Added text: Section 1 was extended (Sections 1.1–1.6, Table 5); the title page and abstract; notes at Part IV's question 10 and Part V's question 7; the Part opening and notes in Sections 78, 79, 80, 81, 83, 87, 88, 89 and 90 and at the end of Appendix M; this appendix; and the bibliography entries [26, 27, 28, 29]. New labels besides the 73: `gts:part:cm` and `gts:cm:tab:letters`.

Files. The report directory holds this text (`article.tex`, `article.pdf`), a README, source 03's `SOURCES.md` (as `03-van-kampen-SOURCES.md`), and the two sources' programs in `code/` and recorded outputs in `data/`, under the prefixes `03-van-kampen-` and `05-three-phases-`. The README maps each delivered name to its shipped name and gives rerun instructions that do not overwrite the shipped files. The manuscripts, their PDFs, the delivered READMEs and source 03's checksum ledger (verified, 13 of 13 files, at placement) are not shipped; they survive in

the archives of commit 6914ccca6. [write 78] Parts III and IV add source 07's `SOURCE_AUDIT.md` (as `07-affine-inputs-SOURCE_AUDIT.md`) and the two sources' programs and data under the prefixes `06-pair-order-` and `07-affine-inputs-`; source 06's `Makefile` is shipped as `code/06-pair-order-Makefile`. Not shipped: both manuscripts and PDFs, both delivered READMEs, and source 06's checksum ledger `SHA256SUMS` (verified, 9 of 9 files, at placement); source 07 shipped none. They survive in the archives of commits 808b53ed8 and 48ee077c7. [write 82] Part V adds source 08's programs and data under the prefix `08-matrix-semigroup-` and its proofs, reviews, packet READMEs, dependency audit and sanitized provenance at the root. Not shipped: its manuscript, PDF and release README, its seals (`MANIFEST.json`, `SHA256SUMS`, `fiber/SHA256SUMS`, verified at placement), the two $r = 2$ polynomial exports (regenerable; the README gives the command), twelve byte copies inside the release, and its copy of the $U_{15,2}$ table, a byte copy of a data file of *quadratic-orthant-certificates*. They survive in the archive of commit db37d18c8. [write 91] Part VI adds source 09's programs and data under the prefix `09-chrono-matrix-` and its notes, reviews and audits at the root. Not shipped: its manuscript and PDF, its delivery README, the polynomial DAG (11.1 MB, regenerable; the README gives the command), the mathlib Pell source and its licence, eleven byte copies of research-programme files, one checksum list (verified at placement) and six byte copies inside the release. They survive in the archive of commit 0d7f51c44.

H Part III: Canonical witness names and a complete sample

The machine-readable export uses the following witness names. Their order fixes the literal residual ordering and makes the polynomial reproducible.

Names	Meaning
xlower_plus, xlower_minus	Positive and negative parts of $p - a$.
ylower_plus, ylower_minus	Positive and negative parts of $q - b$.
p_floor, p_rem	Quotient and remainder on division of p by two.
q_floor, q_rem	Quotient and remainder on division of q by two.
p_bound, q_bound	$2a - p$, $2b - q$.
min1_plus, min1_minus, etc.	The three minimum-comparison sign splits.
max1_plus, max1_minus, etc.	The three maximum-comparison sign splits.
lower_gap, upper_gap	$k - \mathcal{L}_2$, $\mathcal{U}_2 - k$.

The minimum and maximum comparisons are interleaved by stage: min1, max1, min2, max2, min3, max3.

For $(a, b, p, q, k) = (3, 2, 2, 1, 4)$, the unique witness values are

Witness	Value	Witness	Value
xlower_plus	0	xlower_minus	1
ylower_plus	0	ylower_minus	1
p_floor	1	p_rem	0
q_floor	0	q_rem	1
p_bound	4	q_bound	3
min1_plus	0	min1_minus	0
max1_plus	0	max1_minus	0
min2_plus	0	min2_minus	0
max2_plus	1	max2_minus	0
min3_plus	0	min3_minus	0
max3_plus	1	max3_minus	0
lower_gap	2	upper_gap	1

Here the lower endpoint is 2 and the upper endpoint is 5. Two repeated corner values explain the zero sign splits; they do not create alternative witnesses.

I Part III: Repository provenance and dependency audit

The immediate repository source was inspected at commit

4cccf06866b6b81b2467e1cf7514ea85ed0216d.

The relevant path is

Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/heisenberg_two_generator_membership.md

Its inspected blob identifier was

f155e77516db512d3434c0077cf50bff10ecbed1.

The proof dependency in this report is limited: the binary base construction is reproduced in full, the repository’s ternary obstruction is used as a test family, and the Heisenberg multiplication law is derived directly. No uninspected universal compiler or unverified external certificate is used as a theorem premise. The repository was not modified.

The logical dependencies of the principal results are

binary realization $\implies$ gap compiler,
 paired crossings + balancing $\implies$ ternary interval $\implies$ four-corner criterion,
 four-corner criterion + unique gadgets $\implies$ canonical quartic,
 profile realization + collection identity $\implies$ matrix membership lift.

The four-letter obstruction is proved independently by a forced separator skeleton. The supplied-multiplicity decision theorem uses the collection identity, moment-fibre nonemptiness, and the elementary linear-feasibility lemma; it does not depend on numerical tests or on the canonical quartic.

[write 78] The note is unchanged from that commit to this report’s write (same blob). The repository has no Lean module for any statement of Part III; the module names of Section 42.2’s formalization route are suggestions.

J Part V: A compact deterministic specification of the entire literal list

The following prescription is finite and fixes all 229 numerical generators without a symbolic parameter depending on the query. It is repeated here so that the literal data can be regenerated without interpreting prose about a universal compiler.

1. Use Table 4 exactly. Number the 20 alphabet letters in Section 63 from 1 to 20 and # as 21.
2. Scan the 29 defined transition cells in state/read order. For each right move, list the three rules in (V.5); for each left move, list those in (V.6). Append (V.7) in its displayed order. This numbers rules 1 through 93.
3. Number copy tiles 1 through 20 in alphabet order. For rule $j : u \rightarrow v$, let tile $20 + j$ be (v, u) . Tile 114 is $(\#, \#)$.
4. For a tile side $a_1 \cdots a_k$, multiply the explicit matrices $E_{\text{code}(a_1)} \cdots E_{\text{code}(a_k)}$ using (V.9). Substitute into (V.10)–(V.12), evaluating inverses by the determinant-one two-by-two formula.
5. Output the 114 A matrices, the 114 B matrices, and C , in that order. Query words never enter this process.

The numerical data file is the expanded result, rather than code that defers the above products to a future user. The independent literal audit repeats these formula checks from JSON without importing the source compiler. The plain-text matrix file supplies a directly readable full 229-matrix listing; its exact equality with the JSON is part of rebuild verification.

K Part V: The 94 tile accepting sequence

The sequence below is the entire inner witness in order. The full generator word is its A version, followed by C , followed by its reversed B version. Line breaks do not indicate separators; index 114 does.

```

18, 2, 2, 1, 23, 114, 18, 2, 2, 1, 1, 29, 114,
18, 2, 2, 1, 1, 34, 19, 114, 18, 2, 2, 1, 60, 1, 19, 114,
18, 2, 2, 57, 2, 1, 19, 114, 18, 2, 64, 1, 2, 1, 19, 114,
18, 73, 2, 1, 2, 1, 19, 114, 18, 108, 2, 2, 1, 2, 1, 19, 114,
18, 112, 2, 1, 2, 1, 19, 114, 18, 112, 1, 2, 1, 19, 114,
18, 110, 2, 1, 19, 114, 18, 112, 1, 19, 114,
18, 110, 19, 114, 113, 114

```

The exact rule/position trace and all intermediate configuration words are stored in `core/evidence/accepting-witness.json`; the copy in `paired/data/accepting-witness.json` is byte-identical. Positions in that JSON are zero-based, while rule and tile identifiers are one-based. The certificate compiler uses the same one-based tile identifiers and zero-based 2×2 row and column indices.

[write 82] In this report these two files are one shipped file, `data/08-matrix-semigroup-core-evidence-accepting-witness.json`; the copies in `paired/data/` and `fiber/data/` are not shipped.

L Part VI: Notation and a complete outer-clause checklist

The main symbols separate matrix and packing roles as follows.

Symbol	Meaning
B	Fixed 2×2 input-block matrix
P_0	Fixed 2×2 lower marker
C_0	Fixed central upper matrix
D_c	Fixed 3×3 count-control matrix
A_i	Fixed 4×4 column-action branch map
C_*	Fixed scalar coefficient bound 2^{104}
H, D, b	Common offset, digit limit, packing radix
h, W, R	Duration, top-position power, repetition stream
Λ_i, Q_{is}	Selector and selected pre-row slice
U_r, V_r	Pre-row expression and post-row stream
N, V_n	Pre-counter and post-counter streams
f_r	Final offset row digit

Here is the complete outer system, with definitions separated from equality residuals.

1. Two direct expanded POWER calls: $H = \text{POWER}(2, \ell)$ and $W = \text{POWER}(b, h)$; source expressions $D = 2H$, $b = C_*D$
2. Two bound equalities: $H = 35\,426\,321 + g_H$ and $x + g_x = D$
3. One repetition equality: $(b - 1)R + 1 = W$
4. Ninety-seven selector masks $\text{Sub}(R, \Lambda_i)$ and one partition equality $\sum_i \Lambda_i = R$
5. Three hundred eighty-eight slice masks $\text{Sub}((D - 1)\Lambda_i, Q_{is})$; source expressions $U_s = \sum_i Q_{is}$
6. Five post masks $\text{Sub}((D - 1)R, V)$ and one loader-only mask $\text{Sub}((D - 1)\Lambda_0, N)$
7. Four final bound equalities $f_r + g_r = D$
8. Four row chronology equalities $bV_r + H + a_r = U_r + Wf_r$
9. One counter chronology equality $bV_n + x = N$
10. Two final row equalities $f_0 = f_2$, $f_1 = f_3$
11. Four sign-split affine equalities (VI.45)
12. One counter update equality $N = V_n + \Lambda_0$

The outer equality counts in this order are $2 + 1 + 1 + 4 + 4 + 1 + 2 + 4 + 1 = 20$. Module expansions account for everything else. This checklist can be compared directly with the named residuals and ports in the graph.

M Part VI: Principal source pins and dependency roles

The following pins distinguish the authoritative finite objects.

Countdown receipt. The actual fixed transition array is from ProveIt commit 750aeb4f7834332deb2bec5f32fb470ff8254431.

Its Git blob is d8cdf1509237575bfce3b9c96faf3c883c998412. Its SHA256 is f365eb9b62242b395b33766d00a246f0ebcd1a28d867cbcf1173037552b916e0

Counted-suffix proof. The requested proof is pinned at commit d31e29030214c35083b0c2e052c8a3a5d0409eb0.

Its Git blob is 5b0af4b4c3a6c61f5eee75beed2c1d586b86c519. The complete 195-matrix receipt has SHA256

3c803a9a219eebf299a40dcece8958e905b3c59f8ba8b5cb1e60d9c52812c5bd

Context-transferred array. Its SHA256 is

73f8cae212c6c1917e73cc76c7bf3c43b8c587748bbf4327eddf4c82e726a436

Constructive Pell source. The pinned file at the mathlib commit in Section 80 has SHA256 993760c797ad0ff66fa77064a616fce779550bc745cbf6c44e04f392fd0bed0a

The source carries its original Apache 2.0 license notice, preserved with the license text. Its import graph is not rebuilt here.

The separately included deeper matrix notes are exact inert copies at the countdown proof commit and match the hashes already recorded in the context proof. They explain faithful alphabet construction, first-row injectivity, lower-marker shape, and effective initialization. Their inclusion makes the dependency chain inspectable; it does not upgrade inherited results into newly executed compiler or theorem-prover checks.

[write 91] At this write every repository file that source 09 pins or copies, the eleven notes and receipts under Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/, has the same blob at 750aeb4f7 (and, for the counted suffix, at d31e29030) as at this write, and the three SHA-256 digests above match those files. The copies that the bibliography places in science/sources/ and dependencies/ are not shipped here; the repository files are the same bytes. The mathlib file is not shipped either; it is cited by commit and digest.

References

- [1] I. N. Sanov, *A property of a representation of a free group*, Doklady Akademii Nauk SSSR (N.S.) **57** (1947), 657–659. Classical source for the faithful shear representation and its arithmetic image; the statements are reproduced in [2].
- [2] A. Chorna, K. Geller, and V. Shpilrain, *On two-generator subgroups in $SL_2(\mathbb{Z})$, $SL_2(\mathbb{Q})$, and $SL_2(\mathbb{R})$* , arXiv:1605.05226v4, February 5, 2017. <https://arxiv.org/abs/1605.05226>. Theorems 1–2 are the exact classical inputs used here; Theorem 3 warns against extending the same membership shortcut to larger integral shear parameters.
- [3] Y. Cornulier and R. Tessera, *Geometric presentations of Lie groups and their Dehn functions*, arXiv:1310.5373v2, November 26, 2016. <https://arxiv.org/abs/1310.5373>. Lemma 2.D.2, page 28, supplies the bound $|g_i| \leq n + L\alpha$ on factor conjugators.
- [4] W. W. Boone, *The word problem*, Annals of Mathematics (2) **70** (1959), 207–265. <https://www.jstor.org/stable/1970103>. Used only for the classical existence of finitely presented groups with undecidable word problem, not for any explicit numerical presentation in the software.
- [5] G. Higman, B. H. Neumann, and H. Neumann, *Embedding theorems for groups*, Journal of the London Mathematical Society **24** (1949), 247–254. Theorem IV is the classical two-generator embedding result. A detailed account and effective substitution are given in [6].
- [6] M. R. Bridson and C.-F. Nyberg-Brodda, *HNN extensions and embedding theorems for groups*, arXiv:2512.10800. <https://arxiv.org/abs/2512.10800>. The public version consulted on October 2, 2026 contains the explicit finite-presentation discussion in Section 2.3 and the word-problem history in Section 3.1. No later development is needed for the compiler proofs.
- [7] V. Reshetnikov, *ProveIt: MRDP—recursively enumerable if and only if Diophantine*, repository documentation, snapshot c58206ca101d4744a015a0f0104646109357d943. <https://github.com/VladimirReshetnikov/ProveIt/blob/c58206ca101d4744a015a0f0104646109357d943/Computability/HilbertTenthProblem/Lean/MRDP.md>. The public endpoints and their claimed formal boundary are documented here; no repository build is claimed in this report.
- [8] V. Reshetnikov, *ProveIt: Label-aligned controller lanes within the existing geometry*, research note, snapshot c58206ca101d4744a015a0f0104646109357d943. https://github.com/VladimirReshetnikov/ProveIt/blob/c58206ca101d4744a015a0f0104646109357d943/Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_projective_label_aligned_lanes.md. Inspected to distinguish the present area/DAG compiler from the repository’s packed group-projective history work.
- [9] Daniel König, Markus Lohrey, and Georg Zetsche. *Knapsack and subset sum problems in nilpotent, polycyclic, and co-context-free groups*. arXiv:1507.05145, 2015. See Theorem 6.6, Remark 6.7 and Theorem 6.8. <https://arxiv.org/abs/1507.05145>.
- [10] Vitaly Roman’kov. *On decidability of the product of subgroups membership problem for nilpotent groups*. Journal of Group Theory **28**(2) (2025), 463–473. Published online September 20, 2024. DOI: [10.1515/jgth-2023-0119](https://doi.org/10.1515/jgth-2023-0119). Abstract and dates checked at the author’s institutional record: <https://sciact.math.nsc.ru/public/article/5627/en>.

- [11] Jonas Bayer, Marco David, Benedikt Stock, Abhik Pal, Yuri Matiyasevich, and Dierk Schleicher. *Diophantine Equations and the DPRM Theorem*. Archive of Formal Proofs, June 6, 2022. https://isa-afp.org/entries/DPRM_Theorem.html.
- [12] Vladimir Reshetnikov and contributors. *ProveIt*, HilbertTenthProblem MRDP development. Source read: `Diophantine/Common/MRDPCore.lean`, under `Computability/HilbertTenthProblem/Lean/`. Confirmed commit: 433df1be37188224dd00d0561bfb157949cd6320, October 2, 2026. <https://github.com/VladimirReshetnikov/ProveIt>.
- [13] [write] ProveIt research report *Probabilistic, quantum and continuous computation*, collection `SetTheory/Cardinals/docs/reports/hilbert-tenth-problem/probabilistic-quantum-and-continuous-computation/`; Part VI (exact gates; its sources 11 and 16) and Part II (quantum substrates). Cited by label names, not by page.
- [14] [write] ProveIt research programme, *An ordinary-input universal substrate from group commutators*, research note `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_commutator_universal_substrate.md`, as at the write of this report.
- [15] [write] ProveIt research programme, *Faithful free-group matrix embeddings with five-operation ordinary-input loading*, research note `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_unipotent_input_loaders.md`, as at the write of this report.
- [16] [write] ProveIt research programme, *Heisenberg submonoids: a paid two-generator certificate and an order obstruction*, research note `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/heisenberg_two_generator_membership.md`, as at the write of this report; Section 4 decides two-generator submonoids.
- [17] [write 78] A. Mateescu, A. Salomaa, K. Salomaa and S. Yu, *A sharpening of the Parikh mapping*, RAIRO Theoretical Informatics and Applications **35** (2001), 551–564. The original source of Parikh matrices.
- [18] [write 78] ProveIt research programme, *A complete generic matrix-membership arithmetic compiler*, research note `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_complete_matrix_compiler.md`, as at the write of this report (present already at the pin of sources 06 and 07).
- [19] [write 78] ProveIt research programme, *Seven incoming computational-substrate reports*, review `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/incoming_substrate_review_808b53ed8.md`, with the detailed review `incoming_parallel_order_review_808b.md` in the same directory; commit 126028588, 2 October 2026. It reviews the delivered archive of source 06 (and of other manuscripts), not this report.
- [20] [write 78] ProveIt research programme, *Affine-matrix report review*, `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/affine_matrix_square_projection_review808b.md`; commit a1264b55c, 2 October 2026. It reviews the delivered archive of source 07, not this report.
- [21] [write 82] ProveIt research report *Quadratic orthant certificates*, collection `SetTheory/Cardinals/docs/reports/hilbert-tenth-problem/quadratic-orthant-certificates/`; Tables

- qoc:wf:tab:tm (Part III) and qoc:um:tab:tm (Part V), and its data file `data/16-universal-membrane-tm_table.json`. Cited by label names, not by page.
- [22] [write 82] ProveIt research programme, *An explicit universal TM table with a paid ordinary-input bridge*, research note `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/neary_woods_explicit_universal_tm.md`; commit `f2622a68c`, 1 October 2026.
 - [23] [write 82] ProveIt research programme, research notes `gpcp_complete_fixed_program.md` and `gpcp_fixed_program_input_bridge.md`, in `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/`, as at the write of Part V.
 - [24] [write 82] ProveIt research programme, *Bounded intake: literal universal matrices and a complete Grill polynomial*, review `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/review_incoming_matrix_grill.md`, with its helper and receipt; commit `32da04296`, 3 October 2026. It reviews the delivered archive of source 08 (and three other archives), not this report.
 - [25] [write 82] ProveIt research programme, *A fixed directed semigroup with 193 literal $SL_4(\mathbb{Z})$ generators*, research note `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_directed_semigroup193.md`, with its helper and receipt; commit `0f7d618f4`, 3 October 2026.
 - [26] [write 91] ProveIt research programme, *Scoped arithmetic/compiler intake at 0d7f51c44*, review `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/review_new_arithmetic_0d7f51c44.md`, with its manifest `review_new_arithmetic_0d7f51c44.manifest.json`; commit `a9ab9a698`, 4 October 2026. It reviews the delivered archive of source 09 (and seven other archives), not this report.
 - [27] [write 91] ProveIt research programme, *Shared matrix action reduces the complete matrix source to 1393 operations*, research note `Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/matrix193_shared_action_fusion.md`, with its review; commit `a32e1b139`, 4 October 2026. Its predecessors in the same directory include `matrix193_marked_loader_packing.md` (`b388dc588`), `matrix193_uniform_context_packing.md` (`bbb73e0c7`) and `matrix193_atomic_context_packing.md` (`7f7b87500`).
 - [28] [write 91] ProveIt research report *Periodic turmite first revisits*, collection `SetTheory/Cardinals/docs/reports/hilbert-tenth-problem/periodic-turmite-first-revisits/`; Lemma `ptr:ai:lem:exp` (the polynomial power macro). Cited by label names, not by page.
 - [29] [write 91] ProveIt research report *Signal-machine collision certificates*, collection `SetTheory/Cardinals/docs/reports/hilbert-tenth-problem/signal-machine-collision-certificates/`; Part X (the pipeline’s Report 58, batch 91). Cited by Part, not by label or page.
 - [30] Vladimir Reshetnikov, *ProveIt repository*, working note “Heisenberg submonoids: a paid two-generator certificate and an order obstruction,” inspected 2 October 2026 at commit `4cccfa06866b6b81b2467e1cf7514ea85ed0216d`. https://github.com/VladimirReshetnikov/ProveIt/blob/4cccfa06866b6b81b2467e1cf7514ea85ed0216d/Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/heisenberg_two_generator_membership.md.

- [31] Michel Rigo and Pavel Salimov, “Another generalization of abelian equivalence: binomial complexity of infinite words,” *Theoretical Computer Science* **601** (2015), 47–57. DOI: [10.1016/j.tcs.2015.07.025](https://doi.org/10.1016/j.tcs.2015.07.025). Author record and long version: <https://orbi.uliege.be/handle/2268/178703>.
- [32] Wen Chean Teh, “Parikh matrices and Parikh rewriting systems,” arXiv:1506.06476 (2015). <https://arxiv.org/abs/1506.06476>.
- [33] Vitaly Roman’kov, “Undecidability of the submonoid membership problem for a sufficiently large finite direct power of the Heisenberg group,” arXiv:2209.14786 (2022). <https://arxiv.org/abs/2209.14786>.
- [34] Doron Shafrir, “Bounded generation of submonoids of Heisenberg groups,” arXiv:2405.05939 (2024). <https://arxiv.org/abs/2405.05939>.
- [35] V. Reshetnikov and contributors, *Affine paired- SL_2 input cannot give universal subgroup membership*, ProveIt, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_affine_input_obstruction.md. Audited October 2, 2026. Pinned source: https://github.com/VladimirReshetnikov/ProveIt/blob/4cccfa06866b6b81b2467e1cf7514ea85ed0216d/Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_affine_input_obstruction.md.
- [36] V. Reshetnikov and contributors, *An ordinary-input universal substrate from group commutators*, ProveIt, Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_commutator_universal_substrate.md. Audited October 2, 2026. https://github.com/VladimirReshetnikov/ProveIt/blob/4cccfa06866b6b81b2467e1cf7514ea85ed0216d/Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/group_commutator_universal_substrate.md.
- [37] V. Reshetnikov and contributors, *Review of the imported computational-substrate reports, 2 October 2026*, ProveIt. https://github.com/VladimirReshetnikov/ProveIt/blob/4cccfa06866b6b81b2467e1cf7514ea85ed0216d/Computability/HilbertTenthProblem/Papers/research-wip/native-stream-queue/imported_substrate_review_20261002.md.
- [38] V. Reshetnikov and contributors, *MRDPCore.lean*, ProveIt, theorem `mrdp`: every recursively enumerable subset of the natural numbers has a fixed polynomial representation with finitely many natural witnesses. Source inspected near the final theorem. <https://github.com/VladimirReshetnikov/ProveIt/blob/4cccfa06866b6b81b2467e1cf7514ea85ed0216d/Computability/HilbertTenthProblem/Lean/Diophantine/Common/MRDPCore.lean>.
- [39] P.-J. Cahen and J.-L. Chabert, *Integer-Valued Polynomials*, Mathematical Surveys and Monographs, vol. 48, American Mathematical Society, 1997. Publisher record: <https://pubs.ams.org/view?ProductCode=SURV%2F48>.
- [40] A. Leibman, Polynomial sequences in groups, *Journal of Algebra* **201** (1998), 189–206. <https://www.sciencedirect.com/science/article/pii/S0021869397972690>.
- [41] Y.-Q. Hu, *Polynomial maps and polynomial sequences in groups*, arXiv:2105.08000. <https://arxiv.org/abs/2105.08000>. Related framework; not cited as establishing the exact sharp statements of this report.

- [42] G. Higman, Subgroups of finitely presented groups, *Proceedings of the Royal Society of London. Series A* **262** (1961), 455–475. <https://royalsocietypublishing.org/rspa/article/262/1311/455/11030/Subgroups-of-finitely-presented-groups>.
- [43] V. H. Mikaelian, *An explicit algorithm for the Higman Embedding Theorem*, arXiv:2507.04347, 2025, subsequently revised. <https://arxiv.org/abs/2507.04347>. The effective finite-presentation embedding is an external dependency; this report does not reproduce the algorithm or require its two-generator-output refinement.
- [44] O. Bogopolski and E. Ventura, *A recursive presentation for Mihailova’s subgroup*, arXiv:0810.0690, 2008. <https://arxiv.org/abs/0810.0690>. The introductory fibre-product finite-generation construction is the interface used here; extra hypotheses for later recursive-presentation results are not imported.
- [45] B. Chang, S. A. Jennings, and R. Ree, On certain pairs of matrices which generate free groups, *Canadian Journal of Mathematics* **10** (1958), 279–284. DOI: [10.4153/CJM-1958-029-2](https://doi.org/10.4153/CJM-1958-029-2). The elementary free-shear argument is also reconstructed in the text.
- [46] Yu. V. Matiyasevich, Enumerable sets are Diophantine, *Doklady Akademii Nauk SSSR* **191** (1970), 279–282; English translation, *Soviet Mathematics Doklady* **11** (1970), 354–358. Reprint publisher record: https://www.worldscientific.com/doi/abs/10.1142/9789812564894_0013.
- [47] Turlough Neary and Damien Woods. *Four Small Universal Turing Machines*. *Fundamenta Informaticae* **91**(1) (2009), 123–144. doi:10.3233/FI-2009-0036. Publisher record: <https://journals.sagepub.com/doi/10.3233/FI-2009-0036>. Author-hosted reading PDF: <https://dna.hamilton.ie/assets/dw/NearyWoods-FI09.pdf>. The latter uses draft pagination 105–126; detailed locators in this report use that draft pagination and not the published pagination.
- [48] Vesa Halava, Tero Harju, and Mika Hirvensalo. *Undecidability Bounds for Integer Matrices using Claus Instances*. *International Journal of Foundations of Computer Science* **18**(5) (2007), 931–948. See Section 2 and Section 7, especially Lemma 3 and Theorems 15–16. https://users.utu.fi/harju/articles/HHH_ClausInst.pdf.
- [49] *Fixed numerical universal matrix semigroup*. Authored source packet and independent audit, 3 October 2026. Released here under core/; numerical source identity specified in Section 74.
- [50] *Bounded-word matrix Diophantine compiler*. Authored companion source packet and independent audit, 3 October 2026. Released here under paired/; exact common coefficient source as in [49].
- [51] *Exact fibers of the fixed matrix construction*. Authored focused proof and independent literal-data audits, 3 October 2026. Released here under fiber/; both matrix and polynomial constructions are unchanged.
- [52] *Fixed-arity chronological certificate for the ordinary-input countdown*. Scientific source packet, 4 October 2026. Bundled proof `science/ARCHITECTURE.md`, complete source notes `science/SOURCE_NOTES.md`, and authoritative inert graph `science/evidence/polynomial-dag.json`.
- [53] *Independent audit of the chronological ordinary-input matrix certificate*. 4 October 2026. Bundled `independent_audit/AUDIT.md`, with separate exact-source and matrix-interface checkers and receipts. PASS relative to the explicit inherited arithmetic and matrix-interface theorems.

- [54] Mario Carneiro and the mathlib contributors. *Pell and Matiyasevic exponentiation theorems*. Mathlib/NumberTheory/PellMatiyasevic.lean, theorems matiyasevic and eq_pow_of_pell, commit ac77769fab23cb237559e7f56578dbead91499f. [Pinned source on GitHub](#). Exact inert copy and Apache 2.0 license included in dependencies/.
- [55] Vladimir Reshetnikov, ProveIt research sources. *Five countdown registers remove the external Pell-input interface*. matrix193_countdown_rows.md and its JSON receipt, commit 750aeb4f7834332deb2bec5f32fb470ff8254431. [Pinned countdown proof](#). Copies in science/sources/.
- [56] ProveIt research sources. *The directed193 product relation as a synchronized four-coordinate orbit*. matrix193_synchronized_rows.md, same pinned commit as [55]. [Pinned synchronized-row proof](#). Copy in science/sources/.
- [57] ProveIt research sources. *Absorbing fixed contexts into the directed193 generator phases*. matrix193_context_absorption.md and receipt, same pinned commit as [55]. [Pinned context-transfer proof](#). Copies in science/sources/.
- [58] ProveIt research sources. *A counted suffix gives an exact ordinary-input matrix target*. matrix195_counted_suffix.md, commit d31e29030214c35083b0c2e052c8a3a5d0409eb0. [Pinned counted-suffix proof](#). Proof and actual 195-matrix receipt in science/sources/.
- [59] ProveIt research sources. *A faithful 193-generator recoding with Pell parameter 391*. matrix193_gamma1_recode.md, same pinned commit as [55]. [Pinned faithful-group and first-row proof](#). Exact inert copy in dependencies/; its modular-group and graph foundations remain inherited.
- [60] ProveIt research sources. *A fixed directed semigroup with 193 literal $SL_4(\mathbb{Z})$ generators*. group_directed_semigroup193.md, same pinned commit as [55]. [Pinned lower-marker and finite-input proof](#). Exact inert copy in dependencies/.
- [61] ProveIt research sources. *Two target coordinates for a complete 193-generator recoding*. matrix193_kernel_row_projection.md, same pinned commit as [55]. [Pinned earlier first-row argument](#). Exact inert copy in dependencies/; a different coefficient frame from this report.
- [62] ProveIt research sources. *U15 has a universal repeated input block, but the block is hyperbolic*. u15_unary_block_interface.md, same pinned commit as [55]. [Pinned effective initialization argument](#). Exact inert copy in dependencies/; arbitrary-program compilation was not implemented here.
- [63] Turlough Neary and Damien Woods. *Four Small Universal Turing Machines*. Fundamenta Informaticae 91(1), 2009, pp. 123–144. [doi:10.3233/FI-2009-0036](https://doi.org/10.3233/FI-2009-0036). The effective finite-input simulation is the inherited premise used by [62]; the primary simulation and arbitrary-program compiler are not re-audited in this release.